Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»
МОСКОВСКИЙ ПРИБОРОСТРОИТЕЛЬНЫЙ ТЕХНИКУМ

специальность 09.02.07 «Информационные системы и программирование»
Квалификация: Программист

ПРАКТИЧЕСКИЕ РАБОТЫ
ПО МДК 04.02 «Обеспечение качества функционирования компьютерных систем»
И ИТОГОВЫЙ ИНДИВИДУАЛЬНЫЙ ПРОЕКТ
Тема проекта: Информационная система стоматологической клиники «Улыбка»

Выполнил студент
группы П-7-23
Веников Иван Николаевич

Проверил преподаватель
_____ М.В. Горбутова
«___» _____ 2026 года

Москва 2026

ОГЛАВЛЕНИЕ

ПРАКТИЧЕСКАЯ РАБОТА №1 3
ПРАКТИЧЕСКАЯ РАБОТА №2 26
ПРАКТИЧЕСКАЯ РАБОТА №3 60
ПРАКТИЧЕСКАЯ РАБОТА №4 91
ПРАКТИЧЕСКАЯ РАБОТА №5 110
ПРАКТИЧЕСКАЯ РАБОТА №6 133
ИТОГОВЫЙ ИНДИВИДУАЛЬНЫЙ ПРОЕКТ 157

ПРАКТИЧЕСКАЯ РАБОТА №1

Маршрутизация, формы и URL как состояние

Цель работы: создать первое web-приложение на Flutter из нескольких экранов и освоить маршрутизацию через адресную строку, передачу параметров через URL и обработку прямого перехода по ссылке. Научиться строить формы с проверкой ввода, сохранять настройки между перезагрузками страницы и покрывать вычислительную логику модульными тестами.

Ход работы:

1. Создаем проект командой flutter create calc_web и открываем его в Android Studio. В pubspec.yaml добавляем зависимость go_router для маршрутизации, shared_preferences для хранения настроек в браузере и flutter_web_plugins, которая поставляется вместе с SDK и нужна для адресов без решетки.

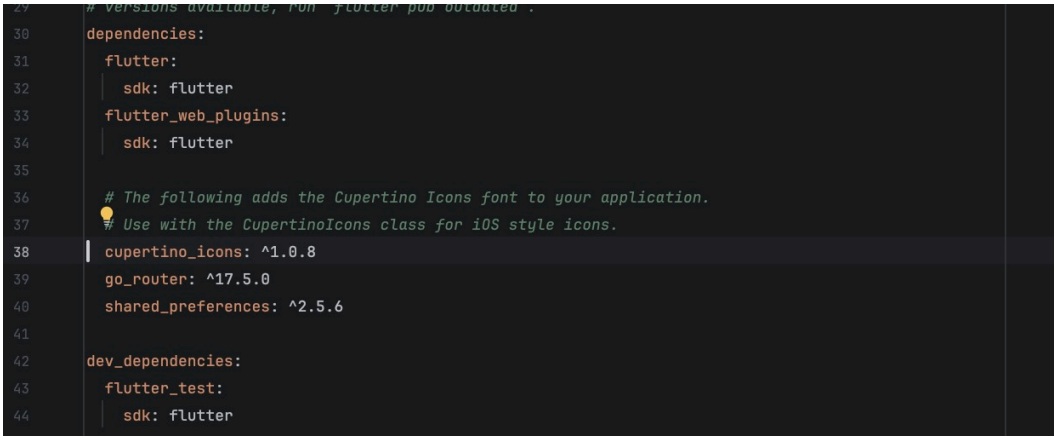


Рисунок 1 – Зависимости проекта в pubspec.yaml

2. Раскладываем код по каталогам: в lib/logic лежит вычислительная логика (calculator.dart, currency.dart), в lib/screens пять экранов (главная, калькулятор, конвертер, результат, 404), в lib/widgets общий каркас экрана, а маршруты, точка входа и хранилище настроек находятся в корне lib. Тесты лежат в каталоге test.

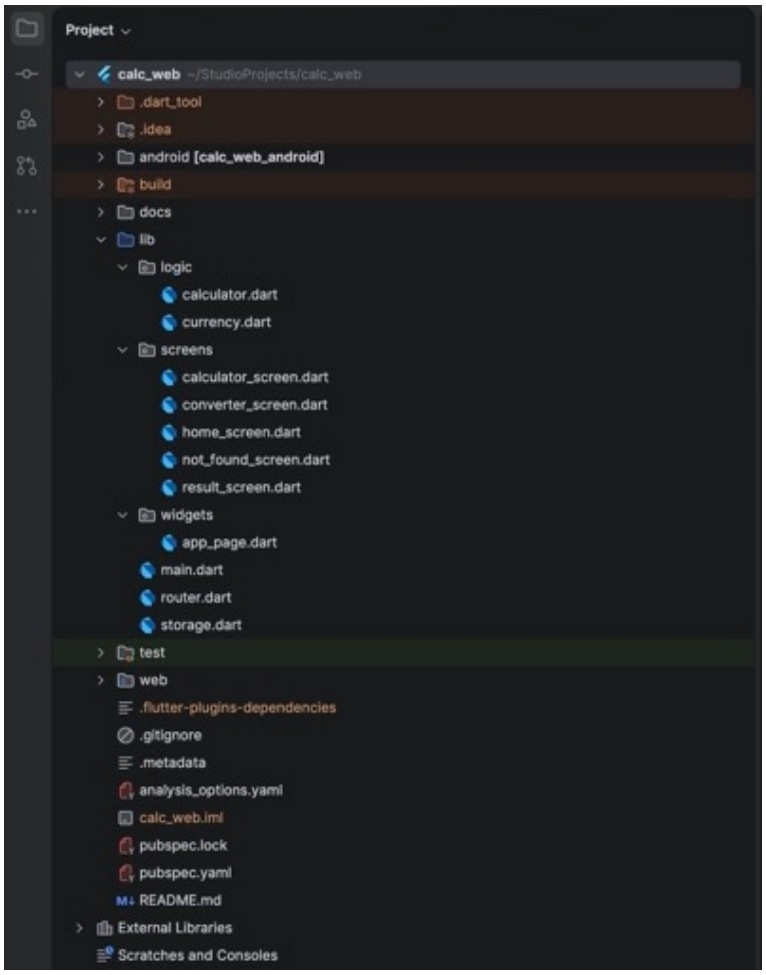
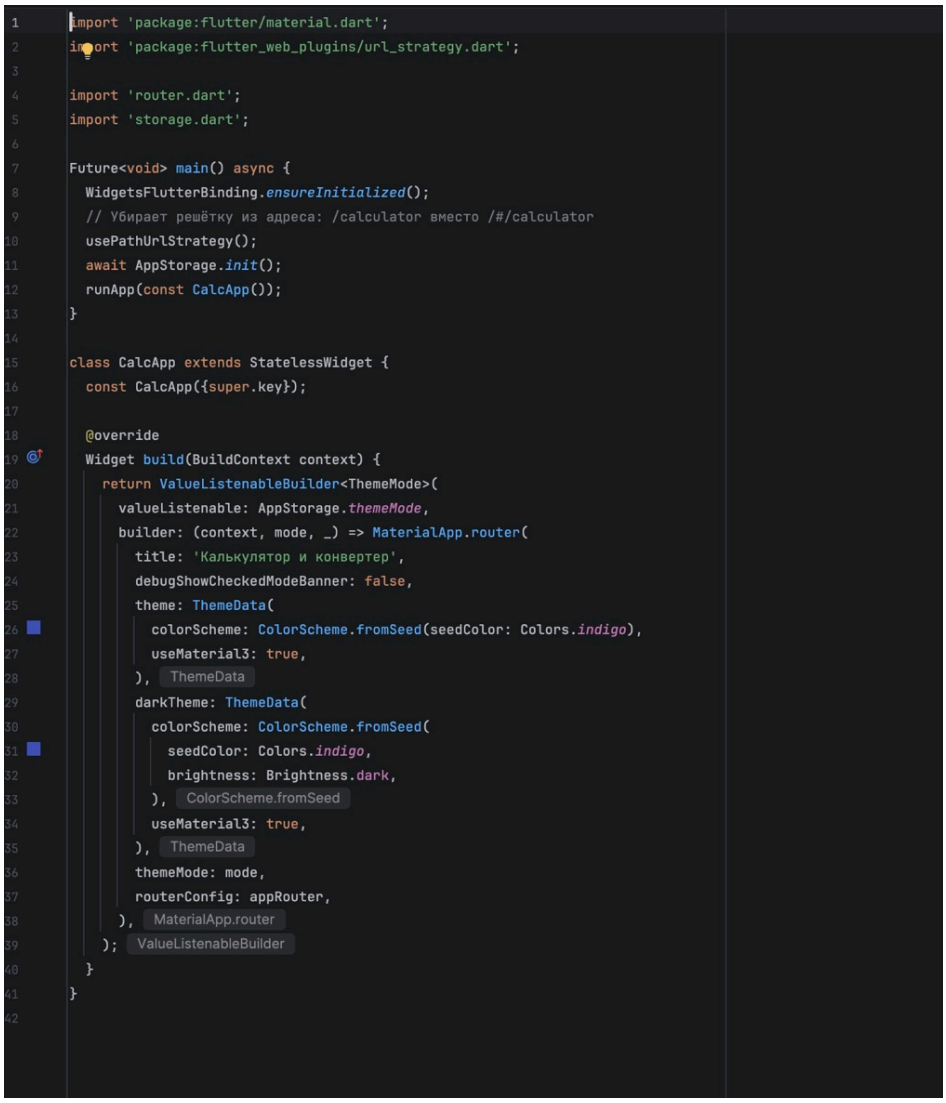


Рисунок 2 – Структура проекта в Android Studio

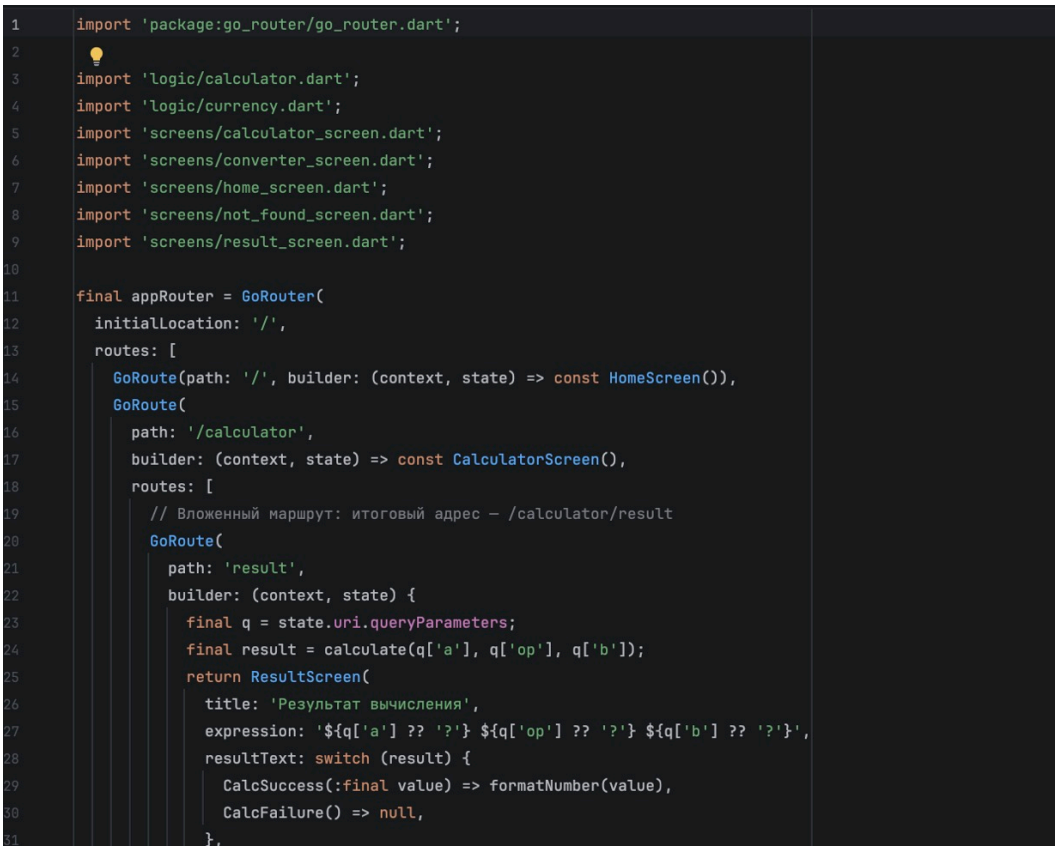
3. В main.dart вызываем usePathUrlStrategy(), чтобы адрес выглядел как /calculator, а не /#/calculator, и загружаем сохраненные настройки до запуска приложения. Приложение собираем через MaterialApp.router с параметром routerConfig: обычный конструктор MaterialApp с go_router не работает. Светлая и тёмная темы строятся от одного цвета indigo, выбранный режим берется из ValueNotifier в хранилище настроек.



```dart
import 'package:flutter/material.dart';
import 'package:flutter_web_plugins/url_strategy.dart';

import 'router.dart';
import 'storage.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  // Убирает решётку из адреса: /calculator вместо /#/calculator
  usePathUrlStrategy();
  await AppStorage.init();
  runApp(const CalcApp());
}

class CalcApp extends StatelessWidget {
  const CalcApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ValueListenableBuilder<ThemeMode>(
      valueListenable: AppStorage.themeMode,
      builder: (context, mode, _) => MaterialApp.router(
        title: 'Калькулятор и конвертер',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(
          colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
          useMaterial3: true,
        ), ThemeData
        darkTheme: ThemeData(
          colorScheme: ColorScheme.fromSeed(
            seedColor: Colors.indigo,
            brightness: Brightness.dark,
          ), ColorScheme.fromSeed
          useMaterial3: true,
        ), ThemeData
        themeMode: mode,
        routerConfig: appRouter,
      ), MaterialApp.router
    ); ValueListenableBuilder
  }
}
```

Рисунок 3 – Точка входа main.dart

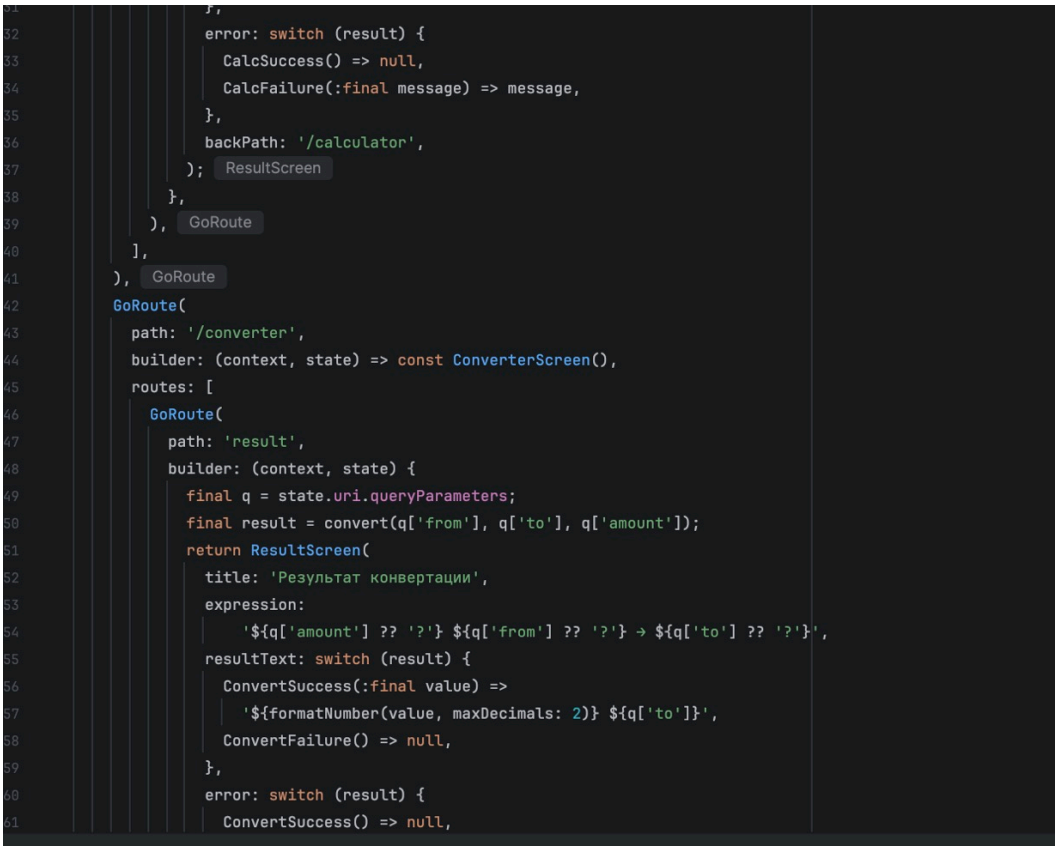
4. В router.dart описываем маршруты go_router. Корень / открывает главный экран, /calculator и /converter открывают формы. Маршруты результата объявлены вложенными: path: 'result' внутри /calculator дает итоговый адрес /calculator/result.



```dart
import 'package:go_router/go_router.dart';

import 'logic/calculator.dart';
import 'logic/currency.dart';
import 'screens/calculator_screen.dart';
import 'screens/converter_screen.dart';
import 'screens/home_screen.dart';
import 'screens/not_found_screen.dart';
import 'screens/result_screen.dart';

final appRouter = GoRouter(
  initialLocation: '/',
  routes: [
    GoRoute(path: '/', builder: (context, state) => const HomeScreen()),
    GoRoute(
      path: '/calculator',
      builder: (context, state) => const CalculatorScreen(),
      routes: [
        // Вложенный маршрут: итоговый адрес — /calculator/result
        GoRoute(
          path: 'result',
          builder: (context, state) {
            final q = state.uri.queryParameters;
            final result = calculate(q['a'], q['op'], q['b']);
            return ResultScreen(
              title: 'Результат вычисления',
              expression: '${q['a'] ?? '?'} ${q['op'] ?? '?'} ${q['b'] ?? '?'}',
              resultText: switch (result) {
                CalcSuccess(:final value) => formatNumber(value),
                CalcFailure() => null,
              },
```

Рисунок 4 – Маршруты главной страницы и калькулятора (router.dart, начало)

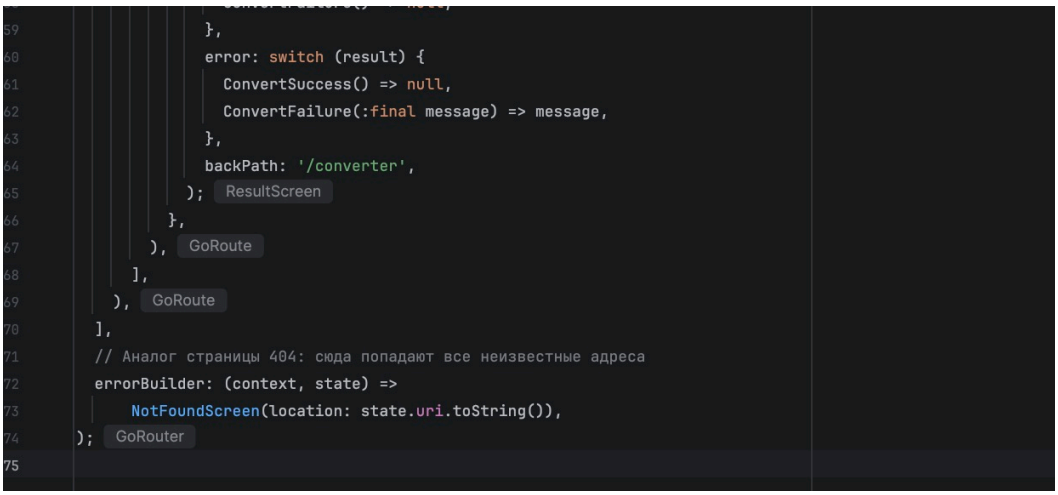
Builder маршрута результата читает параметры из state.uri.queryParameters (a, op и b), вызывает функцию calculate из каталога logic и передает на экран готовый текст результата или текст ошибки. Экран результата ничего не знает о форме и не получает данные через конструктор виджета или extra, поэтому работает и после перехода из формы, и при прямом открытии ссылки.



```
                },
                error: switch (result) {
                  CalcSuccess() => null,
                  CalcFailure(:final message) => message,
                },
                backPath: '/calculator',
              ); ResultScreen
            },
          ), GoRoute
        ],
      ), GoRoute
      GoRoute(
        path: '/converter',
        builder: (context, state) => const ConverterScreen(),
        routes: [
          GoRoute(
            path: 'result',
            builder: (context, state) {
              final q = state.uri.queryParameters;
              final result = convert(q['from'], q['to'], q['amount']);
              return ResultScreen(
                title: 'Результат конвертации',
                expression:
                    '${q['amount'] ?? '?'} ${q['from'] ?? '?'} → ${q['to'] ?? '?'}',
                resultText: switch (result) {
                  ConvertSuccess(:final value) =>
                    '${formatNumber(value, maxDecimals: 2)} ${q['to']}',
                  ConvertFailure() => null,
                },
                error: switch (result) {
                  ConvertSuccess() => null,
```

Рисунок 5 – Вложенный маршрут результата и маршрут конвертера (router.dart)

В конце списка маршрутов задаем errorBuilder: он принимает любой неизвестный адрес и показывает экран NotFoundScreen с самим адресом и кнопкой возврата на главную. Это аналог страницы 404 в серверных приложениях.

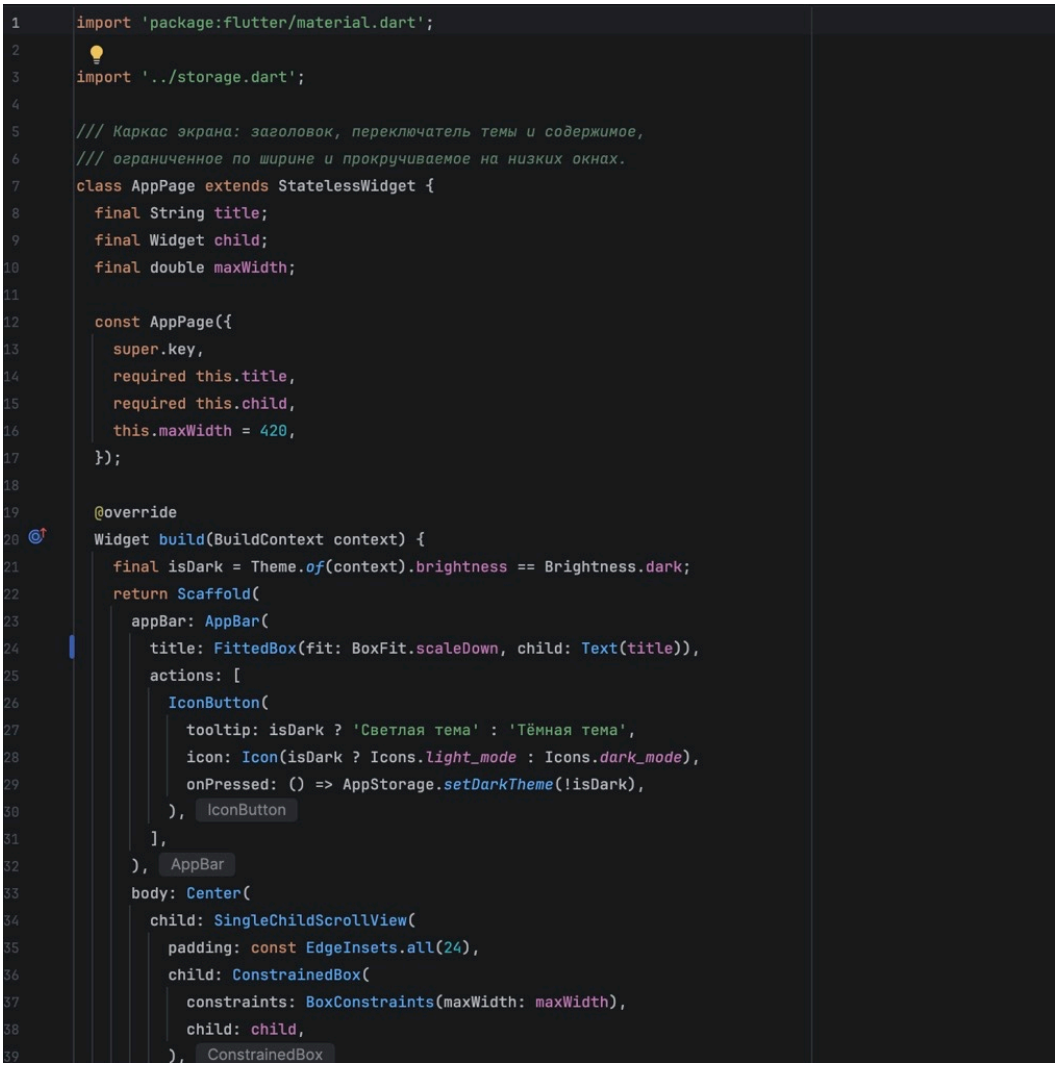


```
                },
                error: switch (result) {
                  ConvertSuccess() => null,
                  ConvertFailure(:final message) => message,
                },
                backPath: '/converter',
              ); ResultScreen
            },
          ), GoRoute
        ],
      ), GoRoute
    ],
    // Аналог страницы 404: сюда попадают все неизвестные адреса
    errorBuilder: (context, state) =>
        NotFoundScreen(location: state.uri.toString()),
  ); GoRouter

```

Рисунок 6 – Экран 404 через errorBuilder (router.dart, конец)

5. Общий каркас экрана выносим в виджет AppPage. Он содержит заголовок, кнопку переключения темы и содержимое, которое ограничено по ширине через ConstrainedBox (420 пикселей для форм) и обернуто в SingleChildScrollView.

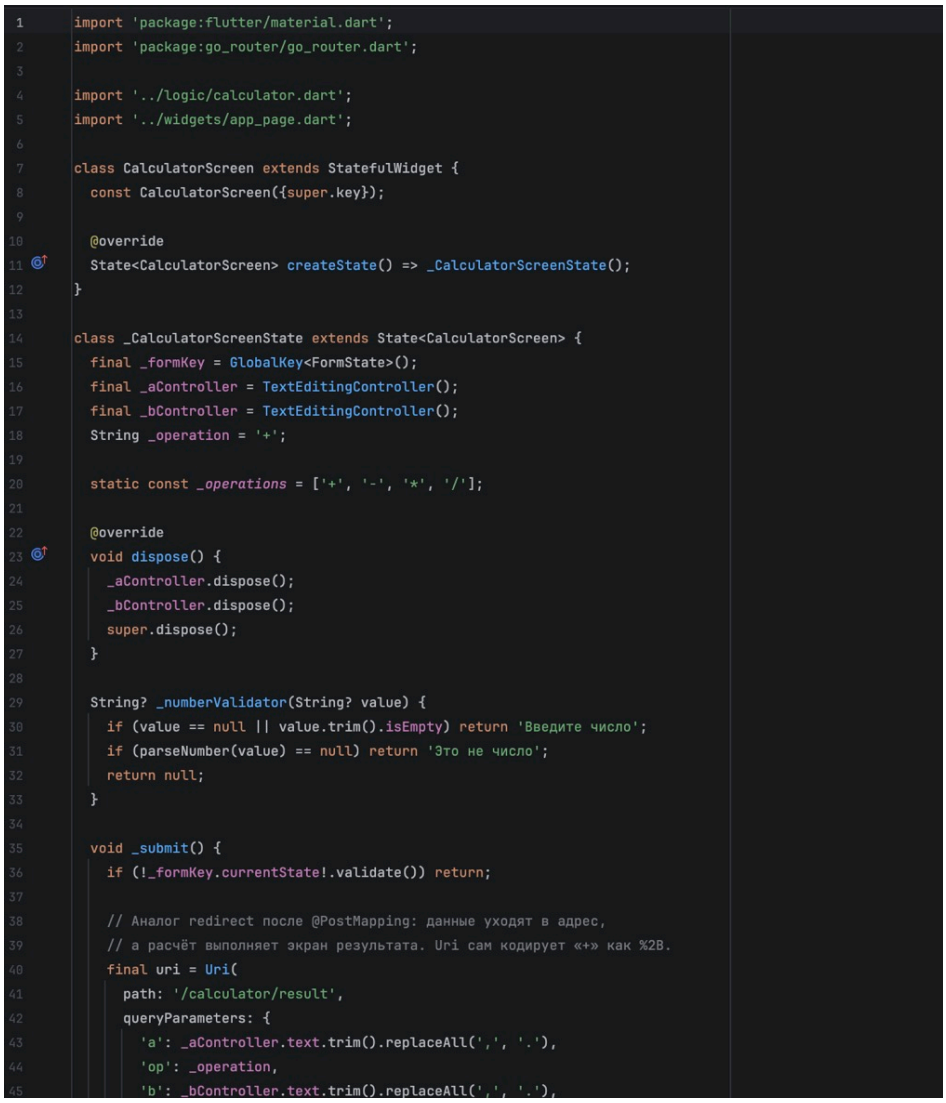
Благодаря этому формы не растягиваются на весь монитор при ширине 1920 пикселей и прокручиваются на узких и низких окнах.



```dart
import 'package:flutter/material.dart';

import '../storage.dart';

/// Каркас экрана: заголовок, переключатель темы и содержимое,
/// ограниченное по ширине и прокручиваемое на низких окнах.
class AppPage extends StatelessWidget {
  final String title;
  final Widget child;
  final double maxWidth;

  const AppPage({
    super.key,
    required this.title,
    required this.child,
    this.maxWidth = 420,
  });

  @override
  Widget build(BuildContext context) {
    final isDark = Theme.of(context).brightness == Brightness.dark;
    return Scaffold(
      appBar: AppBar(
        title: FittedBox(fit: BoxFit.scaleDown, child: Text(title)),
        actions: [
          IconButton(
            tooltip: isDark ? 'Светлая тема' : 'Тёмная тема',
            icon: Icon(isDark ? Icons.light_mode : Icons.dark_mode),
            onPressed: () => AppStorage.setDarkTheme(!isDark),
          ),
        ],
      ),
      body: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: ConstrainedBox(
            constraints: BoxConstraints(maxWidth: maxWidth),
            child: child,
          ),
```

Рисунок 7 – Каркас экрана AppPage

6. Форма калькулятора построена на Form с ключом GlobalKey<FormState> и режимом autovalidateMode: onUserInteraction, чтобы ошибки не показывались на пустой форме. Валидатор возвращает текст ошибки для пустого значения и для нечислового ввода, иначе null. Контроллеры освобождаются в dispose().



```dart
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';

import '../logic/calculator.dart';
import '../widgets/app_page.dart';

class CalculatorScreen extends StatefulWidget {
  const CalculatorScreen({super.key});

  @override
  State<CalculatorScreen> createState() => _CalculatorScreenState();
}

class _CalculatorScreenState extends State<CalculatorScreen> {
  final _formKey = GlobalKey<FormState>();
  final _aController = TextEditingController();
  final _bController = TextEditingController();
  String _operation = '+';

  static const _operations = ['+', '-', '*', '/'];

  @override
  void dispose() {
    _aController.dispose();
    _bController.dispose();
    super.dispose();
  }

  String? _numberValidator(String? value) {
    if (value == null || value.trim().isEmpty) return 'Введите число';
    if (parseNumber(value) == null) return 'Это не число';
    return null;
  }

  void _submit() {
    if (!_formKey.currentState!.validate()) return;

    // Аналог redirect после @PostMapping: данные уходят в адрес,
    // а расчёт выполняет экран результата. Uri сам кодирует «+» как %2B.
    final uri = Uri(
      path: '/calculator/result',
      queryParameters: {
        'a': _aController.text.trim().replaceAll(',', '.'),
        'op': _operation,
        'b': _bController.text.trim().replaceAll(',', '.'),
```

Рисунок 8 – Состояние формы калькулятора и валидатор (calculator_screen.dart)

Метод _submit сначала вызывает validate() и при ошибках прерывает переход. Если ошибок нет, адрес собирается через Uri с queryParameters и открывается командой context.go. Класс Uri сам кодирует спецсимволы, в том числе знак +, который в строке запроса иначе превратился бы в пробел. Запятая в числах заменяется точкой.

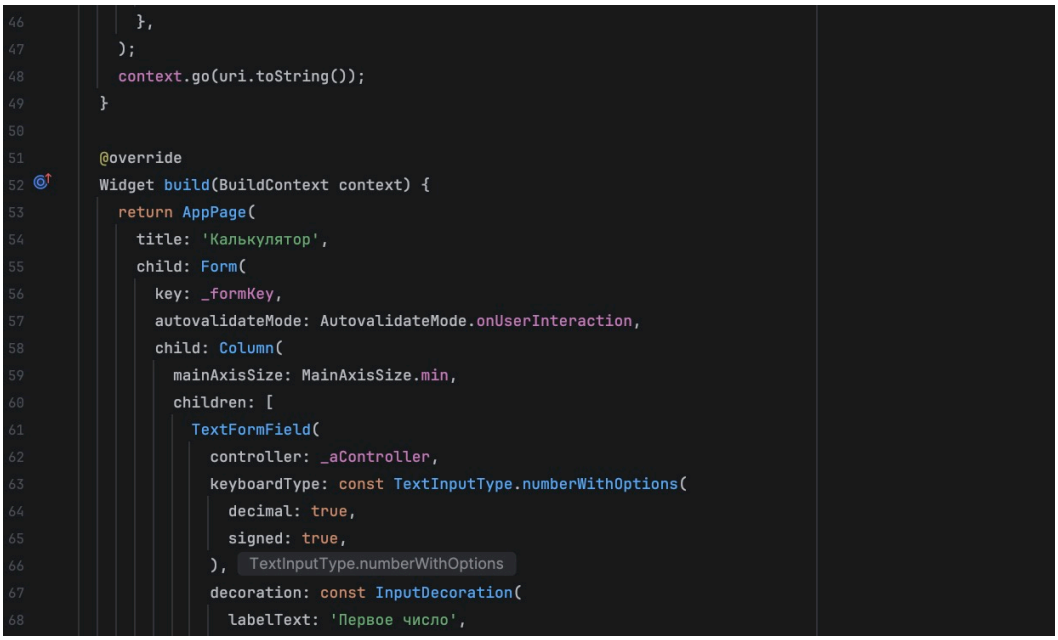
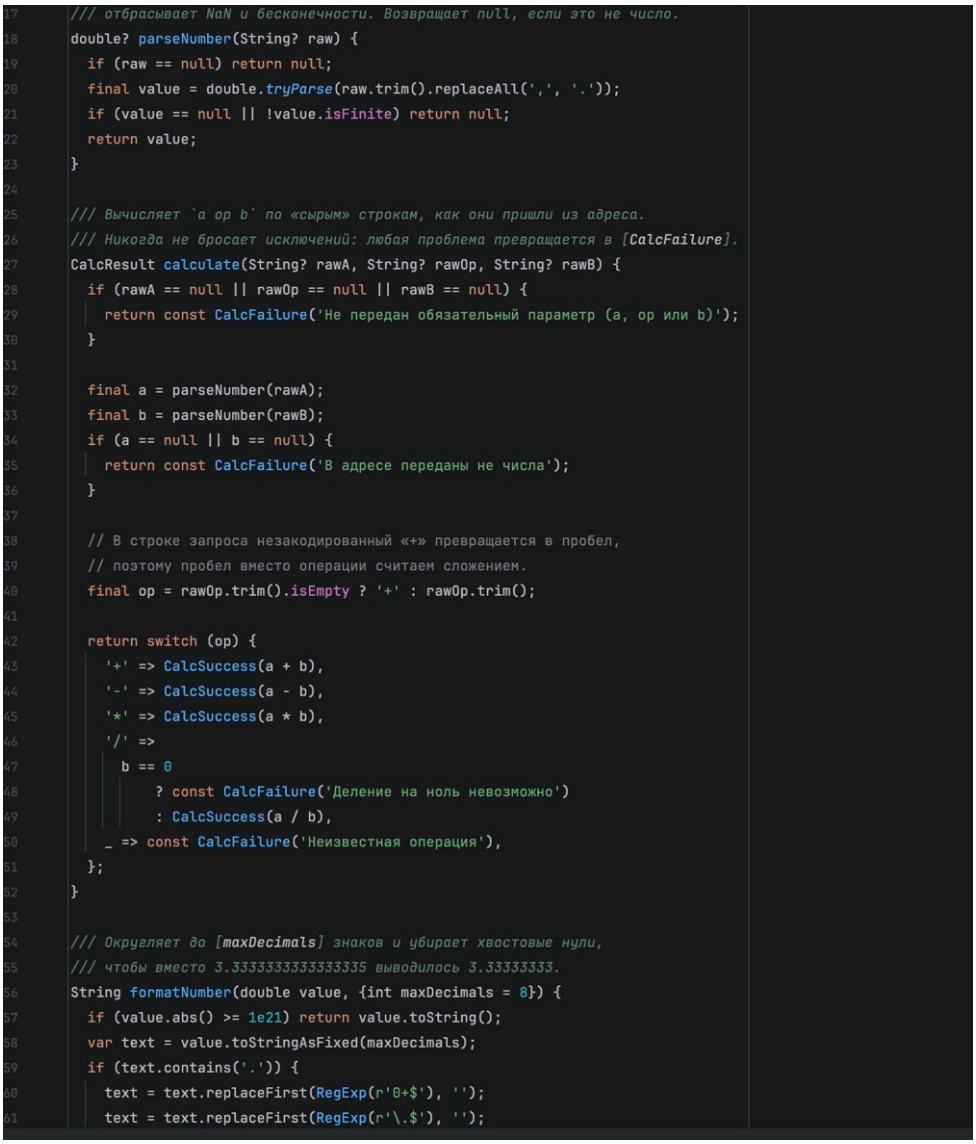


Рисунок 9 – Переход на экран результата через адрес (calculator_screen.dart)

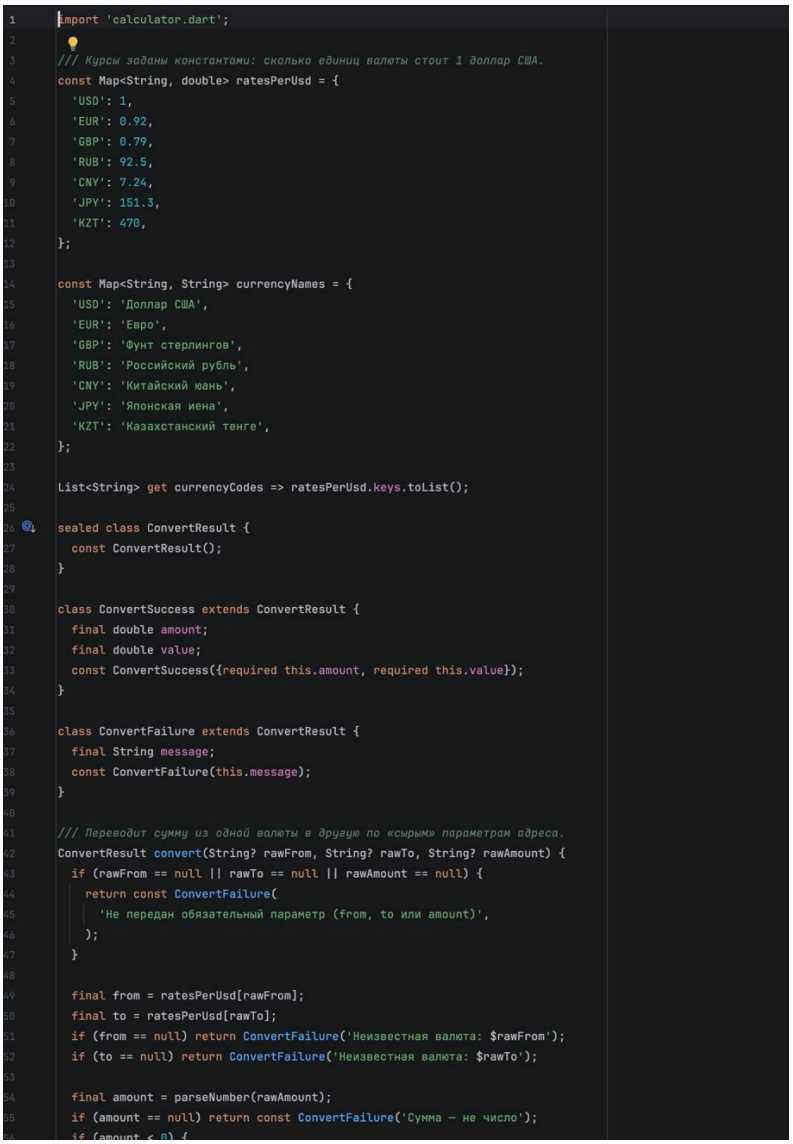
7. Вычисления выносим из виджетов в logic/calculator.dart. Функция calculate принимает «сырые» строки из адреса и всегда возвращает CalcSuccess или CalcFailure, а не бросает исключение. Обрабатываются отсутствие параметра, нечисловые значения (включая NaN и бесконечность), деление на ноль и неизвестная операция. Функция formatNumber округляет результат до 8 знаков и убирает хвостовые нули, поэтому вместо 3.3333333333333335 выводится 3.33333333.



```dart
/// отбрасывает NaN и бесконечности. Возвращает null, если это не число.
double? parseNumber(String? raw) {
  if (raw == null) return null;
  final value = double.tryParse(raw.trim().replaceAll(',', '.'));
  if (value == null || !value.isFinite) return null;
  return value;
}

/// Вычисляет `a op b` по «сырым» строкам, как они пришли из адреса.
/// Никогда не бросает исключений: любая проблема превращается в [CalcFailure].
CalcResult calculate(String? rawA, String? rawOp, String? rawB) {
  if (rawA == null || rawOp == null || rawB == null) {
    return const CalcFailure('Не передан обязательный параметр (a, op или b)');
  }

  final a = parseNumber(rawA);
  final b = parseNumber(rawB);
  if (a == null || b == null) {
    return const CalcFailure('В адресе переданы не числа');
  }

  // В строке запроса незакодированный «+» превращается в пробел,
  // поэтому пробел вместо операции считаем сложением.
  final op = rawOp.trim().isEmpty ? '+' : rawOp.trim();

  return switch (op) {
    '+' => CalcSuccess(a + b),
    '-' => CalcSuccess(a - b),
    '*' => CalcSuccess(a * b),
    '/' =>
      b == 0
          ? const CalcFailure('Деление на ноль невозможно')
          : CalcSuccess(a / b),
    _ => const CalcFailure('Неизвестная операция'),
  };
}

/// Округляет до [maxDecimals] знаков и убирает хвостовые нули,
/// чтобы вместо 3.3333333333333335 выводилось 3.33333333.
String formatNumber(double value, {int maxDecimals = 8}) {
  if (value.abs() >= 1e21) return value.toString();
  var text = value.toStringAsFixed(maxDecimals);
  if (text.contains('.')) {
    text = text.replaceFirst(RegExp(r'0+$'), '');
    text = text.replaceFirst(RegExp(r'\.$'), '');
```

Рисунок 10 – Логика калькулятора: calculate и formatNumber

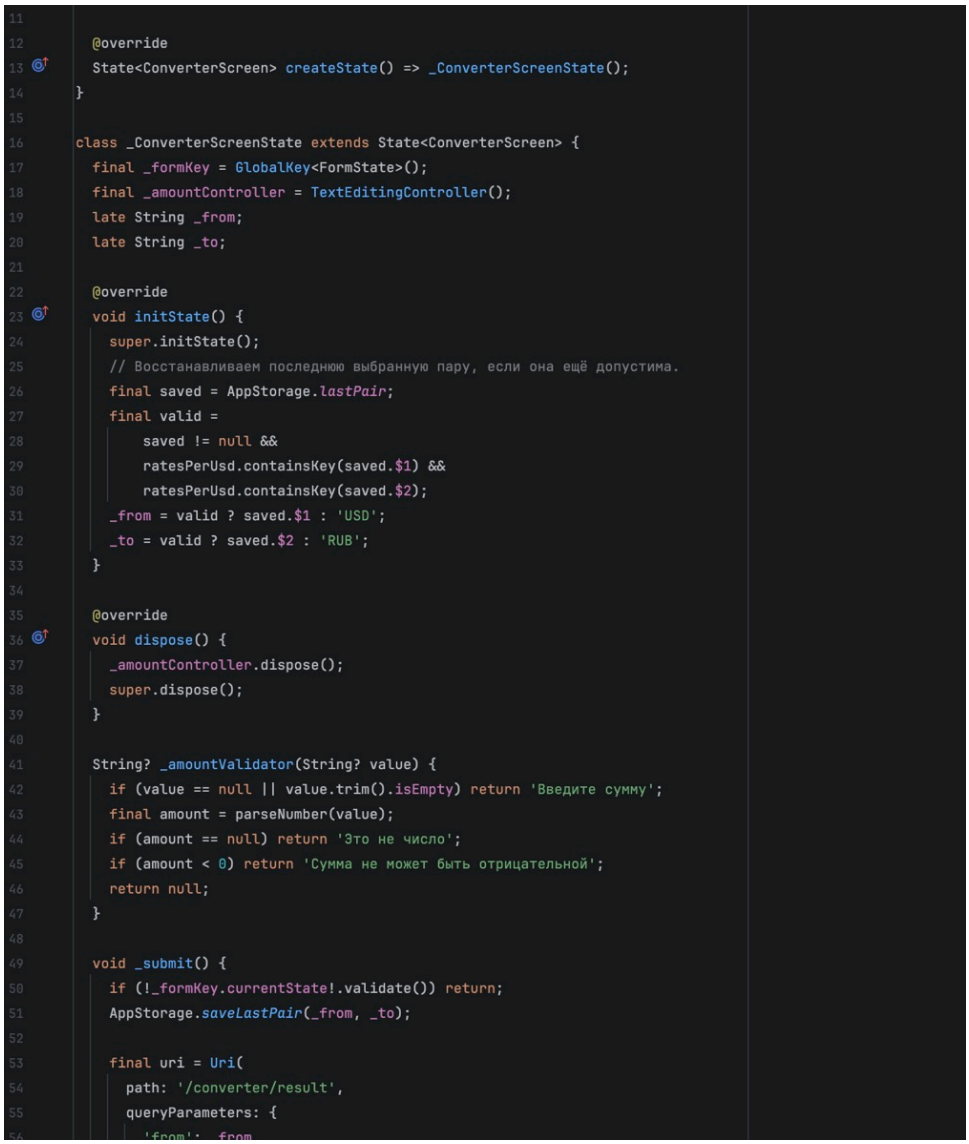
8. В logic/currency.dart задаем курсы семи валют константами (сколько единиц валюты стоит 1 доллар США) и функцию convert, которая пересчитывает сумму через доллар и проверяет параметры адреса: наличие, известность валют, число и неотрицательность суммы.



```dart
import 'calculator.dart';

/// Курсы заданы константами: сколько единиц валюты стоит 1 доллар США.
const Map<String, double> ratesPerUsd = {
  'USD': 1,
  'EUR': 0.92,
  'GBP': 0.79,
  'RUB': 92.5,
  'CNY': 7.24,
  'JPY': 151.3,
  'KZT': 470,
};

const Map<String, String> currencyNames = {
  'USD': 'Доллар США',
  'EUR': 'Евро',
  'GBP': 'Фунт стерлингов',
  'RUB': 'Российский рубль',
  'CNY': 'Китайский юань',
  'JPY': 'Японская иена',
  'KZT': 'Казахстанский тенге',
};

List<String> get currencyCodes => ratesPerUsd.keys.toList();

sealed class ConvertResult {
  const ConvertResult();
}

class ConvertSuccess extends ConvertResult {
  final double amount;
  final double value;
  const ConvertSuccess({required this.amount, required this.value});
}

class ConvertFailure extends ConvertResult {
  final String message;
  const ConvertFailure(this.message);
}

/// Переводит сумму из одной валюты в другую по «сырым» параметрам адреса.
ConvertResult convert(String? rawFrom, String? rawTo, String? rawAmount) {
  if (rawFrom == null || rawTo == null || rawAmount == null) {
    return const ConvertFailure(
      'Не передан обязательный параметр (from, to или amount)',
    );
  }

  final from = ratesPerUsd[rawFrom];
  final to = ratesPerUsd[rawTo];
  if (from == null) return ConvertFailure('Неизвестная валюта: $rawFrom');
  if (to == null) return ConvertFailure('Неизвестная валюта: $rawTo');

  final amount = parseNumber(rawAmount);
  if (amount == null) return const ConvertFailure('Сумма — не число');
  if (amount < 0) {
```

Рисунок 11 – Курсы валют и функция convert

9. Форма конвертера содержит два выпадающих списка валют и поле суммы. При открытии экрана в initState из хранилища восстанавливается последняя выбранная пара, а при отправке формы пара сохраняется, затем выполняется переход на /converter/result.



```dart
  @override
  State<ConverterScreen> createState() => _ConverterScreenState();
}

class _ConverterScreenState extends State<ConverterScreen> {
  final _formKey = GlobalKey<FormState>();
  final _amountController = TextEditingController();
  late String _from;
  late String _to;

  @override
  void initState() {
    super.initState();
    // Восстанавливаем последнюю выбранную пару, если она ещё допустима.
    final saved = AppStorage.lastPair;
    final valid =
        saved != null &&
        ratesPerUsd.containsKey(saved.$1) &&
        ratesPerUsd.containsKey(saved.$2);
    _from = valid ? saved.$1 : 'USD';
    _to = valid ? saved.$2 : 'RUB';
  }

  @override
  void dispose() {
    _amountController.dispose();
    super.dispose();
  }

  String? _amountValidator(String? value) {
    if (value == null || value.trim().isEmpty) return 'Введите сумму';
    final amount = parseNumber(value);
    if (amount == null) return 'Это не число';
    if (amount < 0) return 'Сумма не может быть отрицательной';
    return null;
  }

  void _submit() {
    if (!_formKey.currentState!.validate()) return;
    AppStorage.saveLastPair(_from, _to);

    final uri = Uri(
      path: '/converter/result',
      queryParameters: {
        'from': from
```

Рисунок 12 – Восстановление пары валют и переход к результату (converter_screen.dart)

10. Хранилище настроек storage.dart сохраняет выбранную тему и последнюю пару валют через shared_preferences (в браузере это localStorage). Метод init вызывается в main до запуска приложения, поэтому тема применяется сразу после перезагрузки страницы.

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

/// Настройки, которые переживают перезагрузку страницы
/// (в браузере shared_preferences хранит их в localStorage).
class AppStorage {
  AppStorage._();

  static const _themeKey = 'theme_mode';
  static const _fromKey = 'converter_from';
  static const _toKey = 'converter_to';

  static late SharedPreferences _prefs;

  /// Текущая тема; MaterialApp перестраивается при её изменении.
  static final ValueNotifier<ThemeMode> themeMode = ValueNotifier(
    ThemeMode.light,
  );

  static Future<void> init() async {
    _prefs = await SharedPreferences.getInstance();
    themeMode.value = _prefs.getString(_themeKey) == 'dark'
        ? ThemeMode.dark
        : ThemeMode.light;
  }

  static Future<void> setDarkTheme(bool dark) {
    themeMode.value = dark ? ThemeMode.dark : ThemeMode.light;
    return _prefs.setString(_themeKey, dark ? 'dark' : 'light');
  }

  /// Последняя выбранная пара валют или null, если её ещё не сохраняли.
  static (String, String)? get lastPair {
    final from = _prefs.getString(_fromKey);
    final to = _prefs.getString(_toKey);
    if (from == null || to == null) return null;
    return (from, to);
  }

  static Future<void> saveLastPair(String from, String to) async {
    await _prefs.setString(_fromKey, from);
    await _prefs.setString(_toKey, to);
  }
}
```

Рисунок 13 – Хранилище настроек (storage.dart)

11. Запускаем приложение из Android Studio: выбираем устройство Chrome (web) и конфигурацию main.dart. В консоли Run видно, что приложение запущено в браузере в режиме отладки.

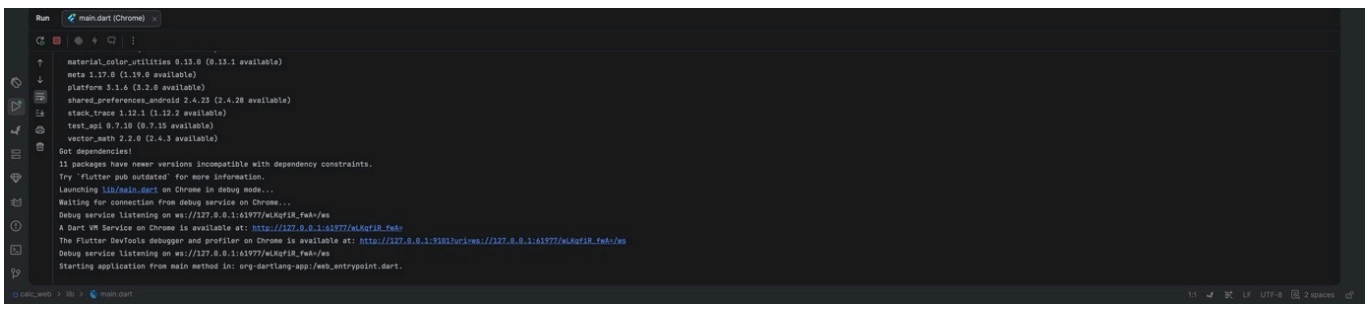


Рисунок 14 – Запуск приложения на Chrome (web) из Android Studio

12. Открываем главный экран. На нем две кнопки для перехода на калькулятор и конвертер валют, в заголовке находится переключатель темы.

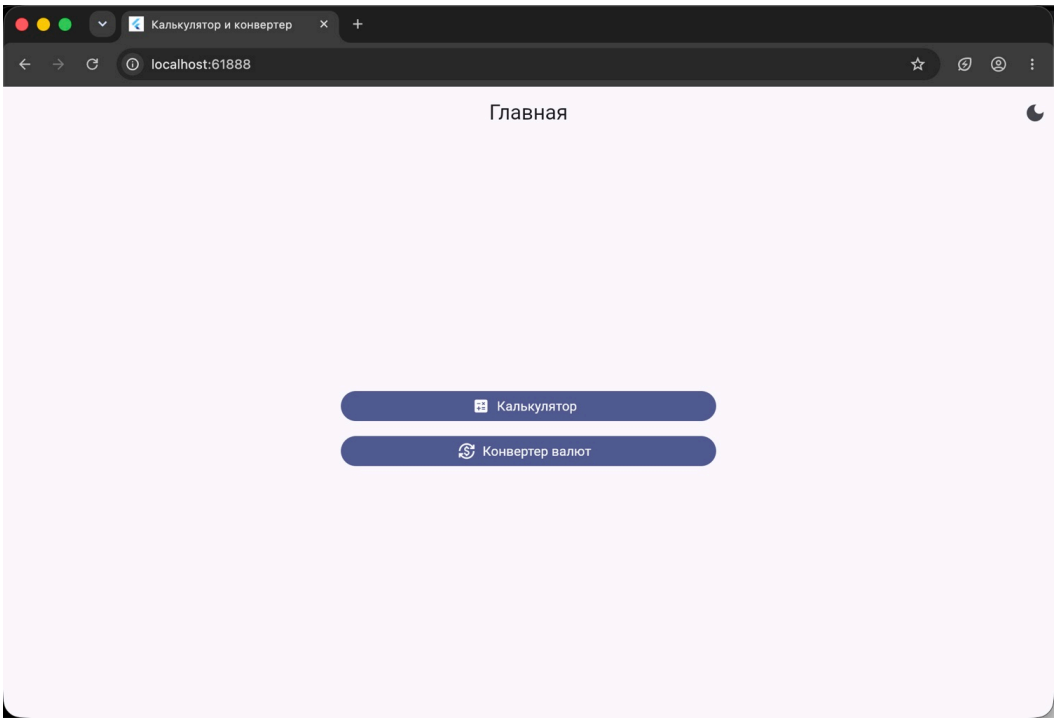


Рисунок 15 – Главный экран

13. Переходим на калькулятор. В адресной строке появился адрес /calculator, форма содержит два числовых поля и список операций.

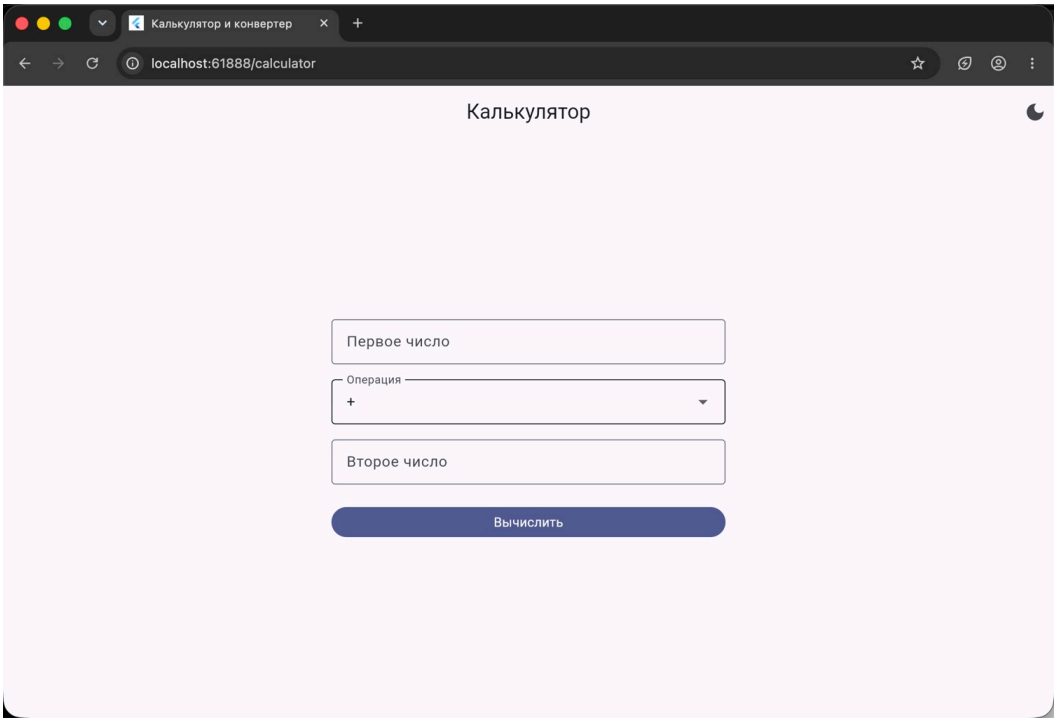


Рисунок 16 – Форма калькулятора

14. Нажимаем «Вычислить» с пустыми полями. Переход не выполняется, под полями появляются сообщения об ошибке.

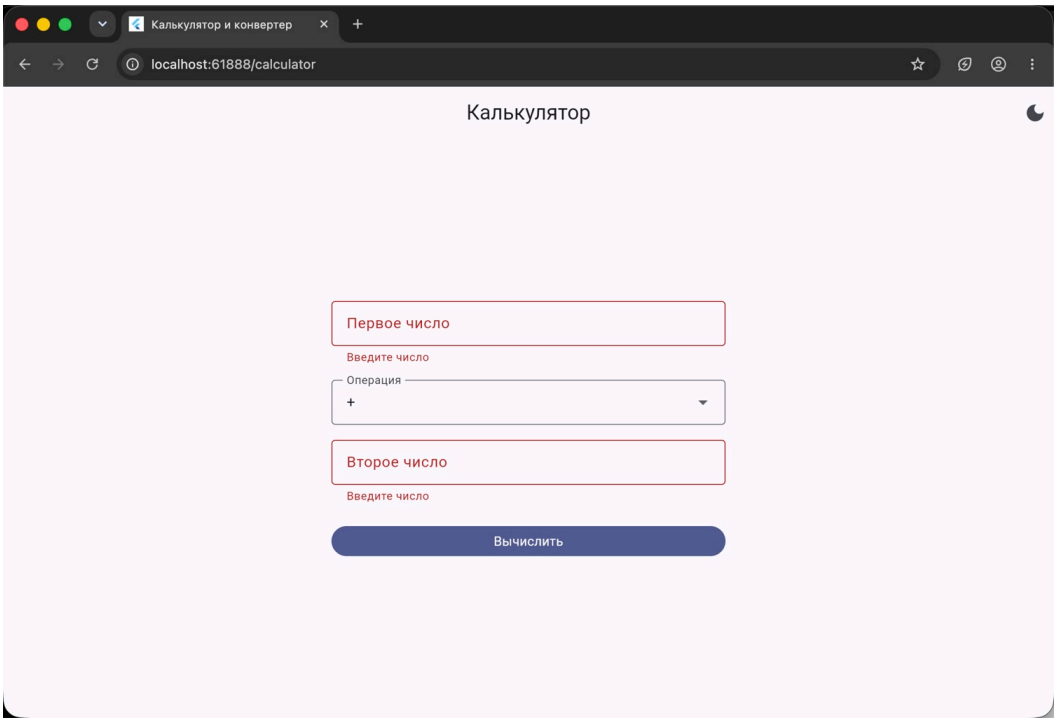


Рисунок 17 – Проверка пустых полей формы калькулятора

15. Вводим числа 10 и 3 и выбираем операцию деления. После нажатия на кнопку приложение переходит на /calculator/result, результат показан на отдельном экране (см. раздел «Результат работы»). Кнопка «назад» браузера возвращает на форму калькулятора, а не закрывает приложение.

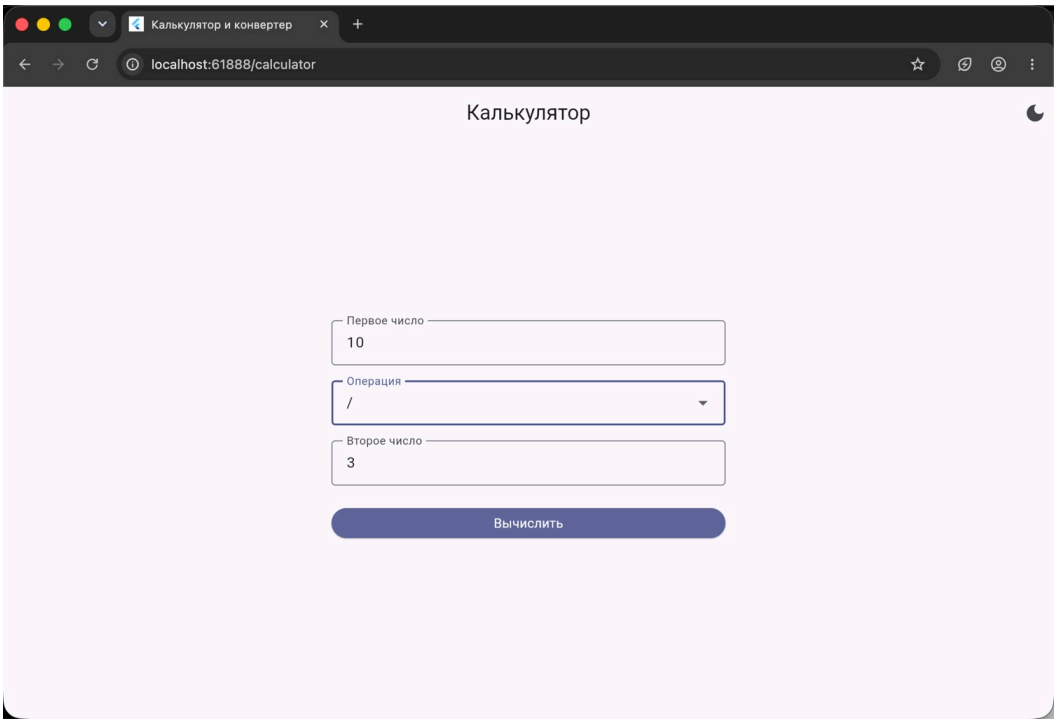


Рисунок 18 – Заполненная форма калькулятора

16. Проверяем обработку некорректных адресов. Открываем адрес результата напрямую в новой вкладке с параметрами, которые не получились бы из формы. Для деления на ноль экран показывает сообщение об ошибке.

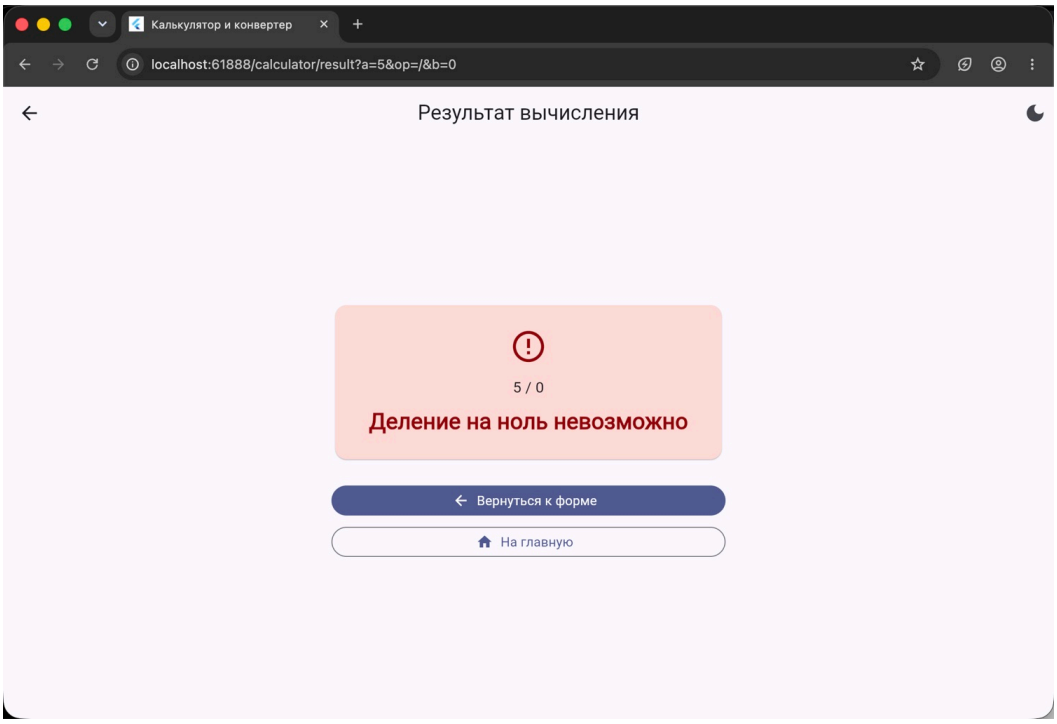


Рисунок 19 – Результат при делении на ноль (a=5&op=/&b=0)

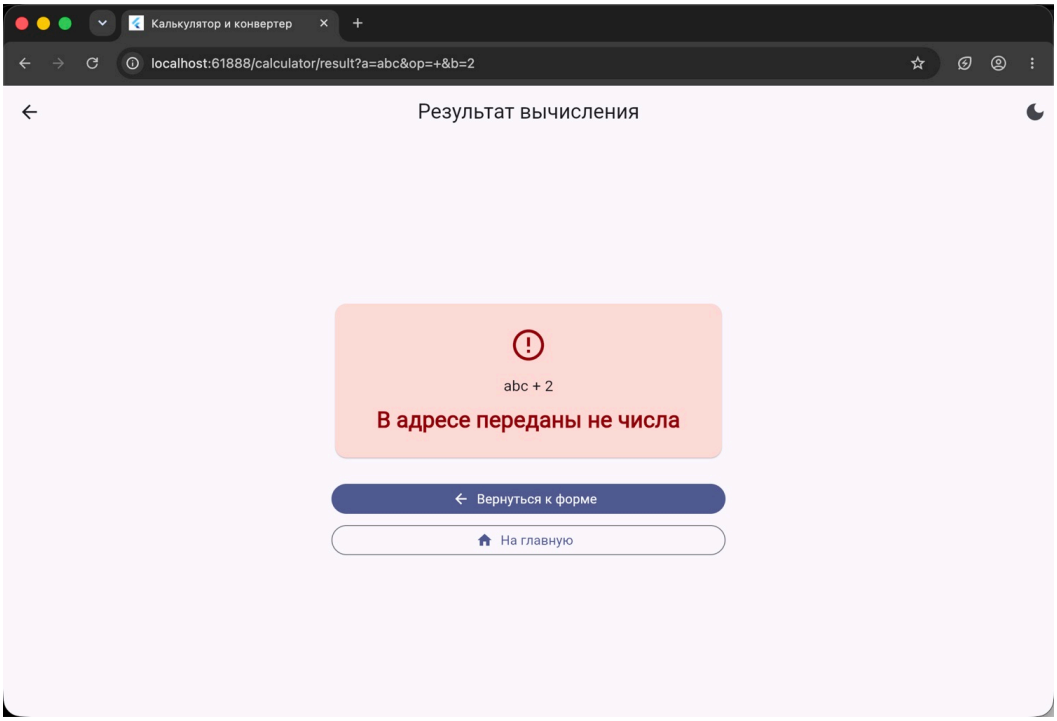


Рисунок 20 – Результат при нечисловом значении (a=abc)

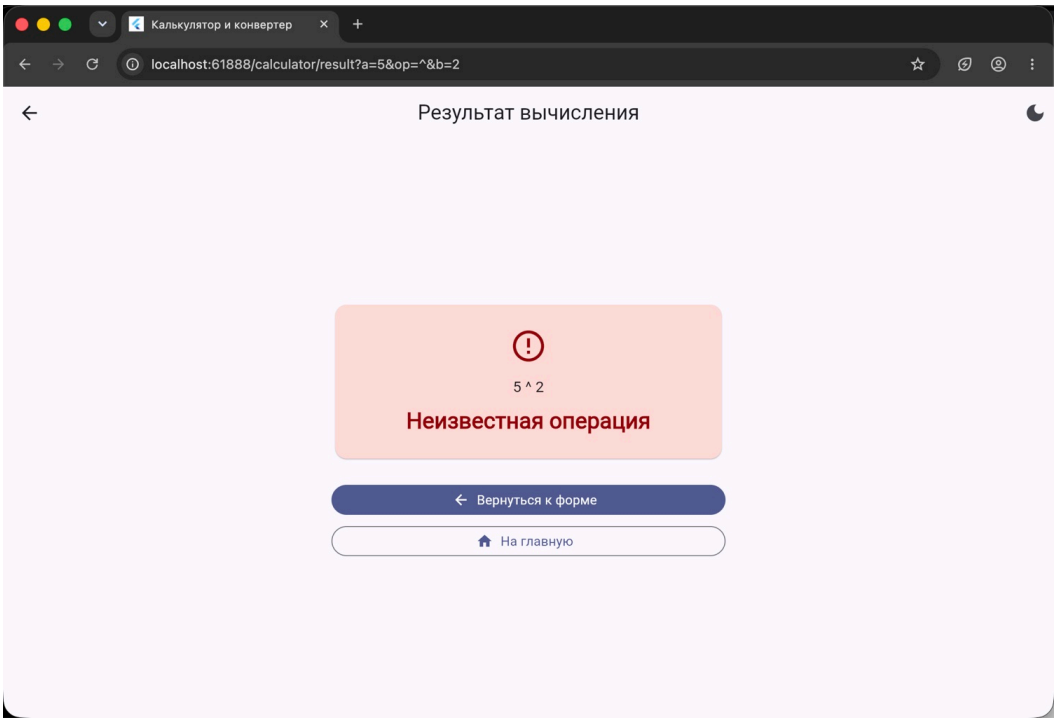


Рисунок 21 – Результат при неизвестной операции (op=^)

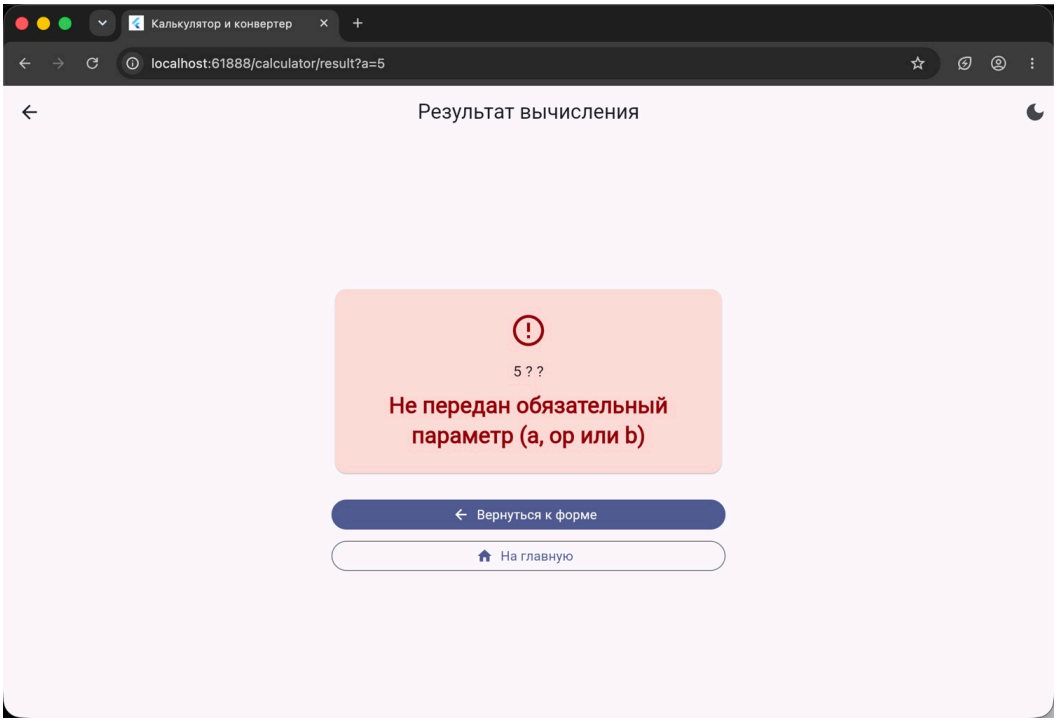


Рисунок 22 – Результат при отсутствии обязательных параметров (передан только a)

17. Открываем неизвестный адрес /qwerty. Срабатывает errorBuilder и показывается экран 404 с кнопкой возврата на главную.

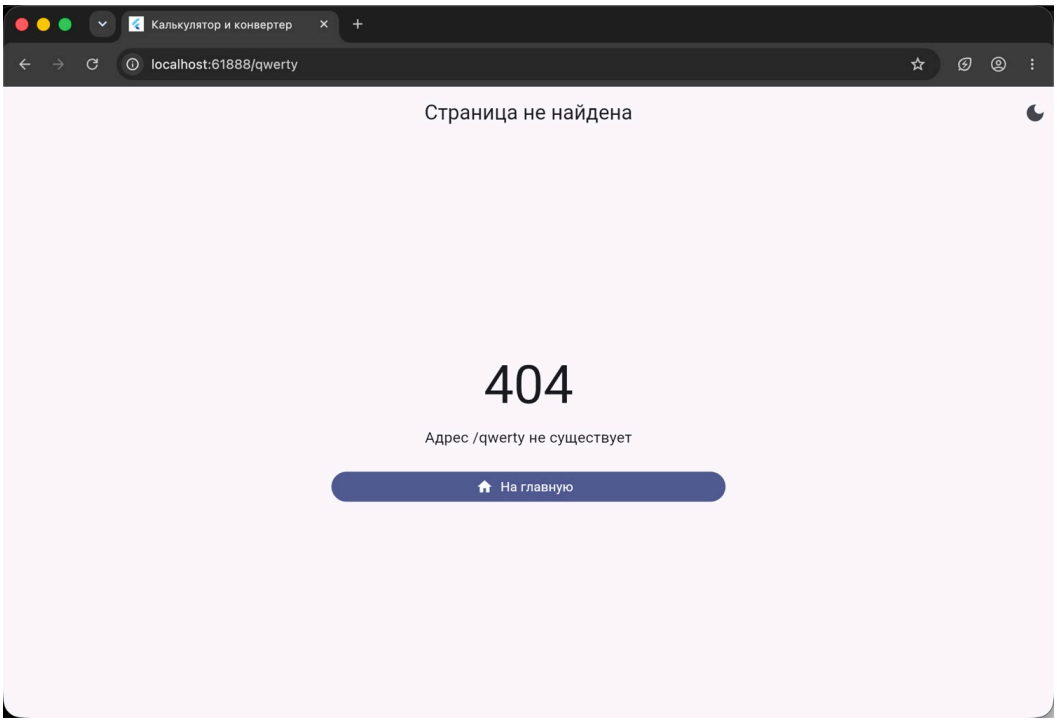


Рисунок 23 – Экран 404 для неизвестного адреса

18. Переходим на конвертер валют. По умолчанию выбрана пара USD и RUB.

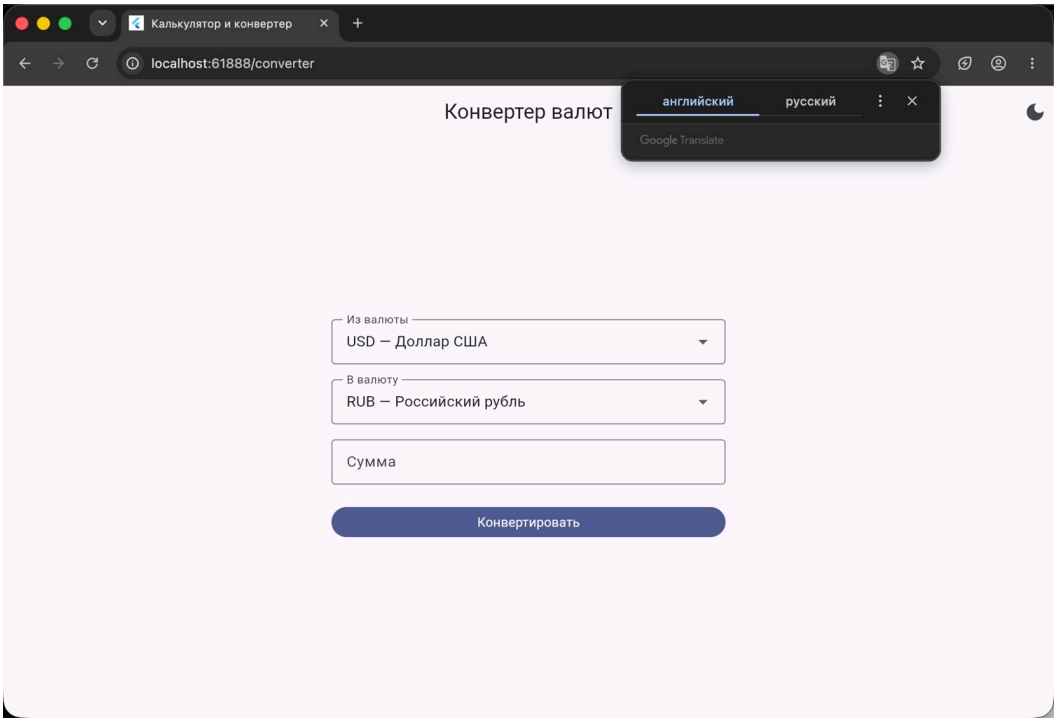


Рисунок 24 – Форма конвертера валют

19. Проверяем поле суммы: пустое значение и нечисловой ввод не пропускают дальше и показывают сообщения «Введите сумму» и «Это не число».

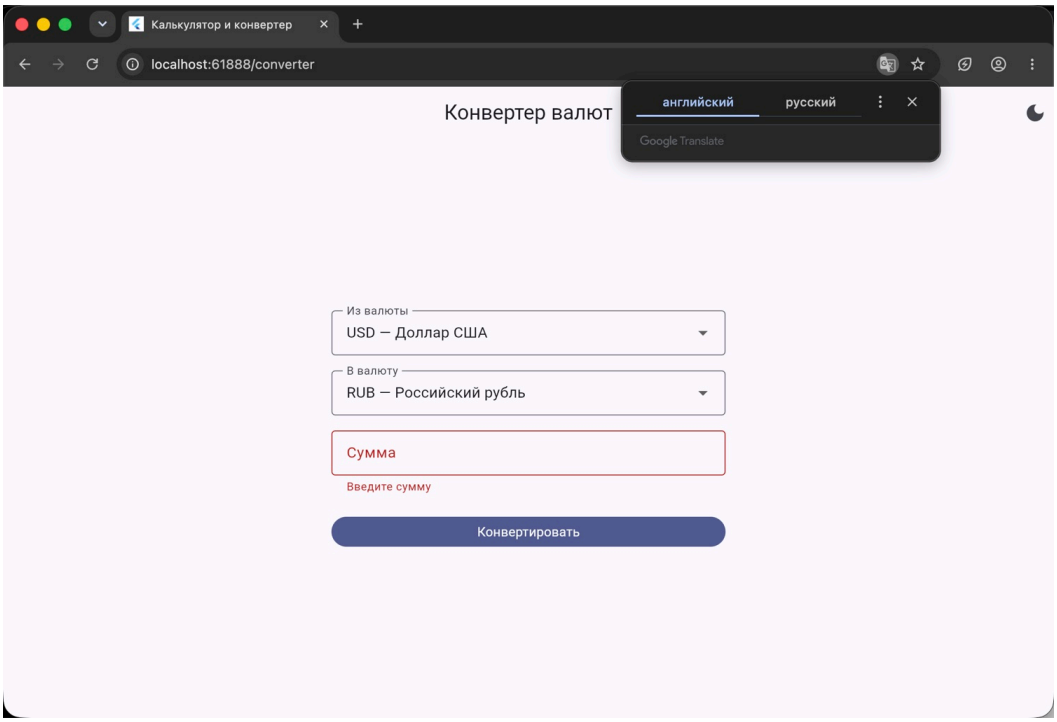


Рисунок 25 – Пустое поле суммы

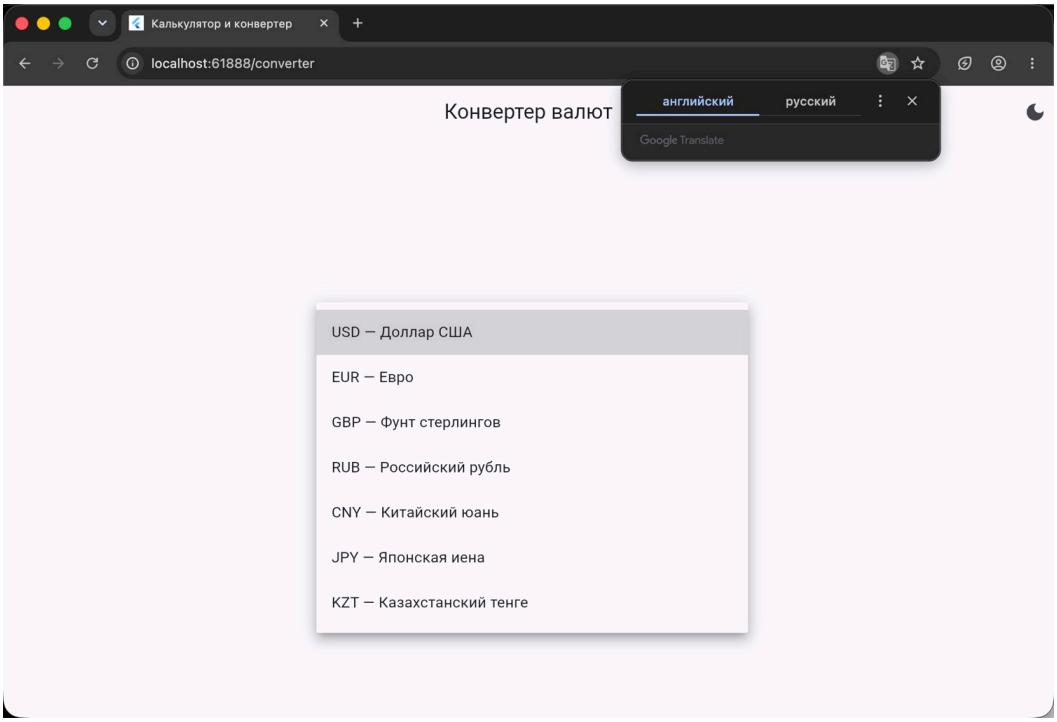


Рисунок 26 – Нечисловой ввод в поле суммы

20. Выбираем пару EUR и JPY, вводим сумму 250 и нажимаем «Конвертировать». Результат показан на экране /converter/result (см. раздел «Результат работы»).

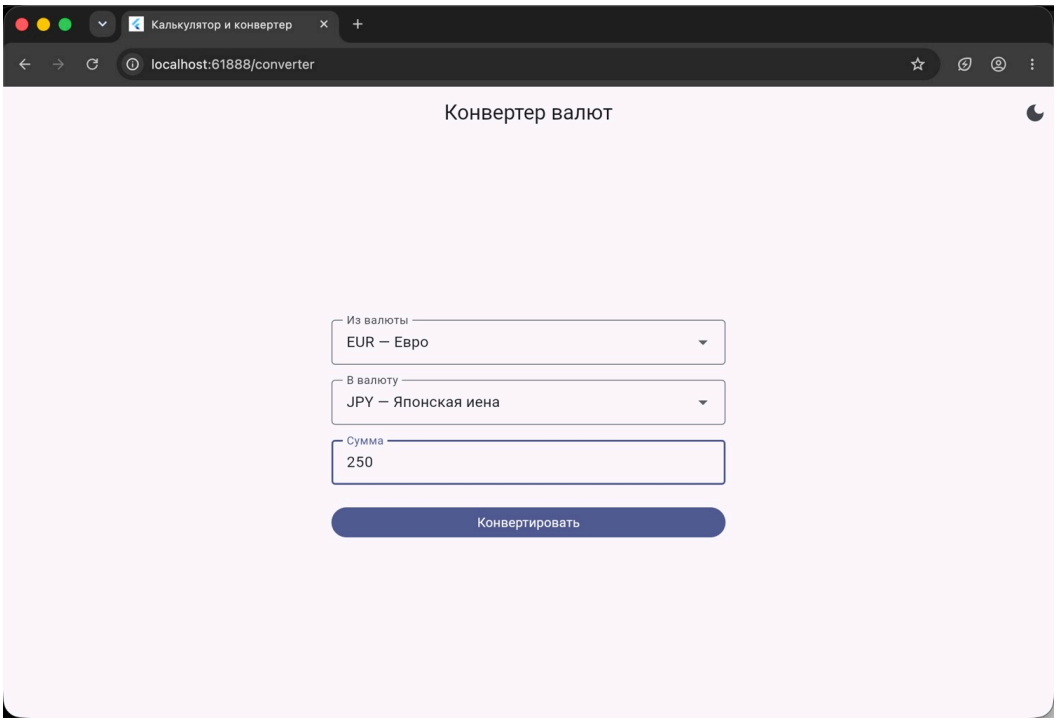


Рисунок 27 – Заполненная форма конвертера

21. Возвращаемся на конвертер. Приложение восстановило последнюю выбранную пару EUR и JPY, а не показывает пару по умолчанию.

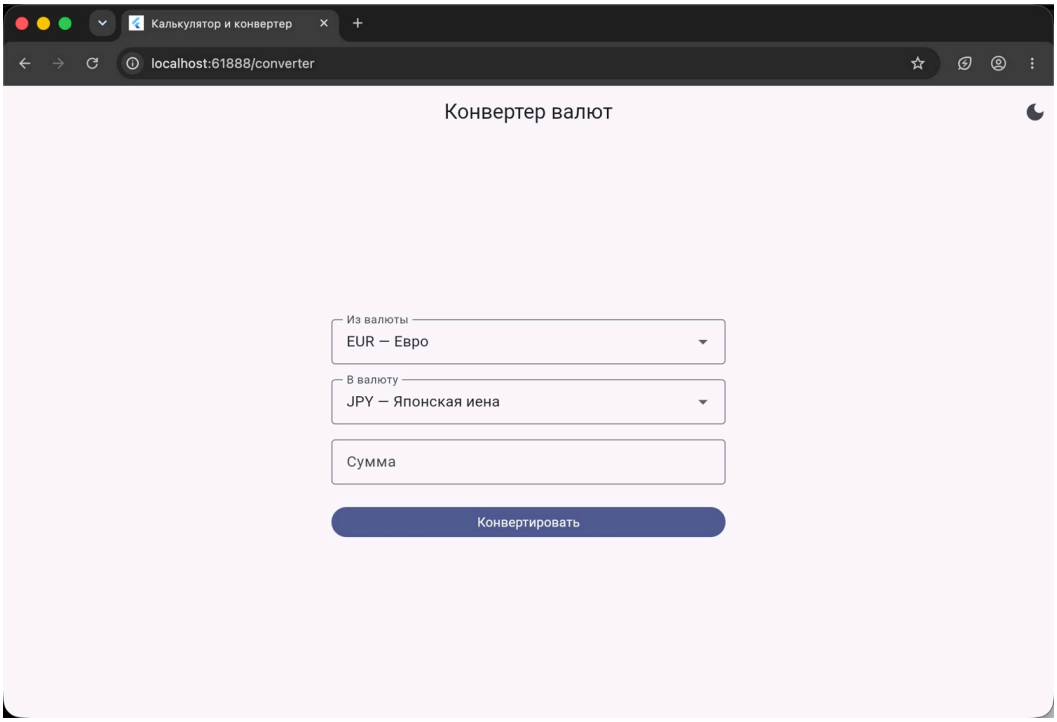


Рисунок 28 – Восстановленная пара валют

22. Нажимаем кнопку переключения темы в заголовке. Включается тёмная тема. После перезагрузки страницы выбор сохраняется.

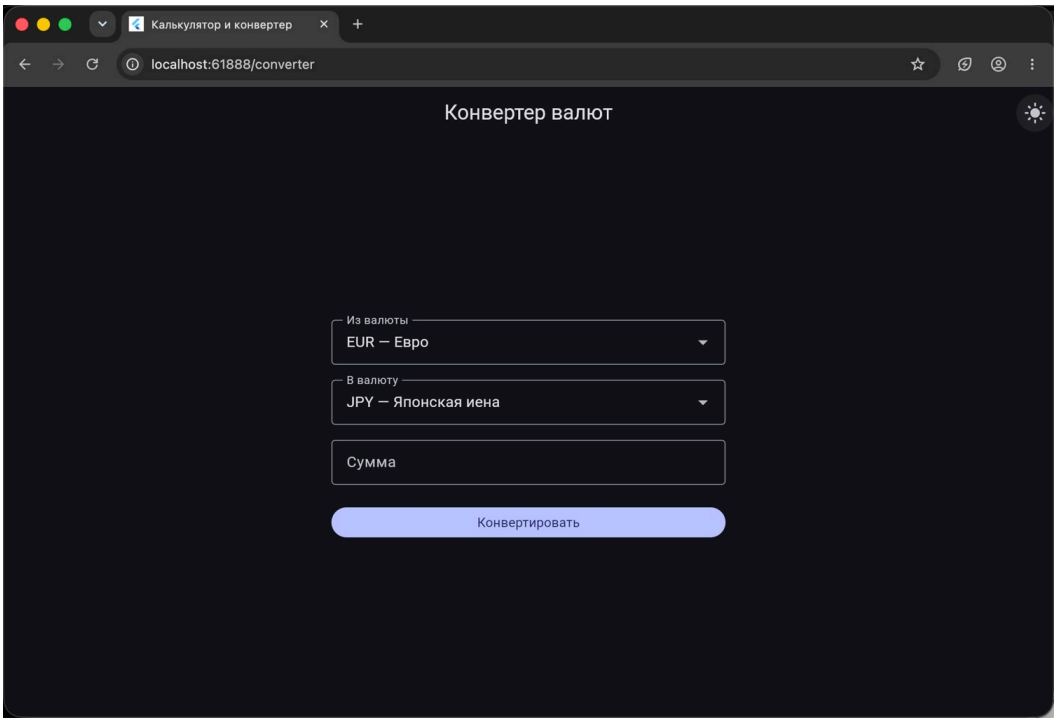


Рисунок 29 – Тёмная тема

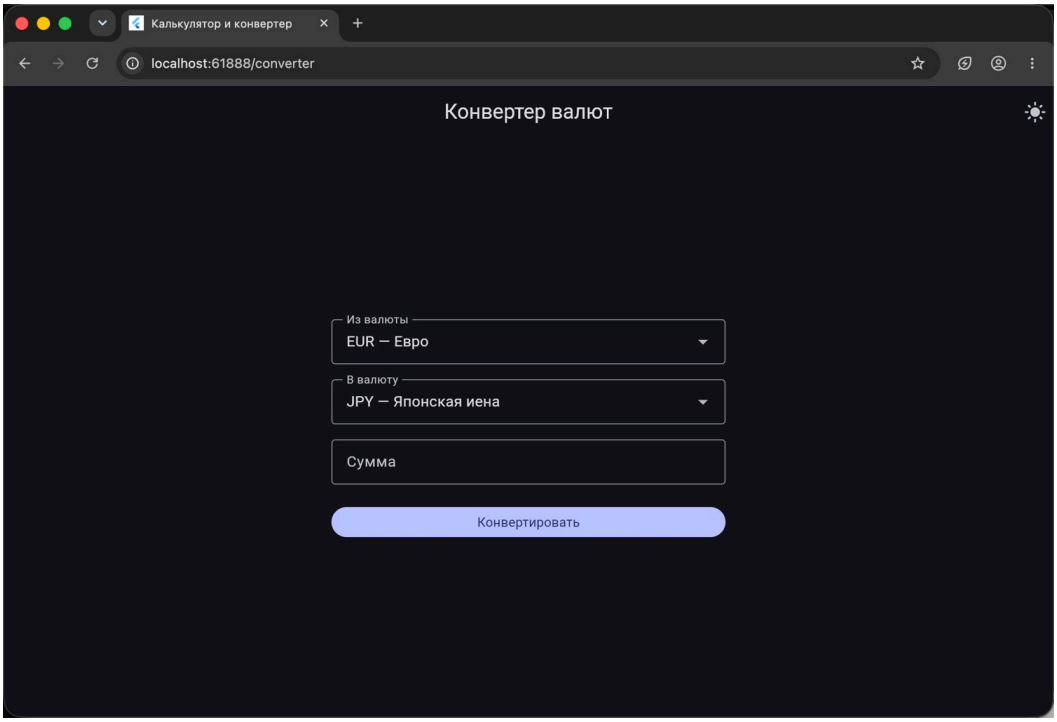


Рисунок 30 – Тёмная тема после перезагрузки страницы

23. Проверяем адаптивность. При ширине окна 360 пикселей формы занимают всю ширину и остаются работоспособными, содержимое прокручивается.

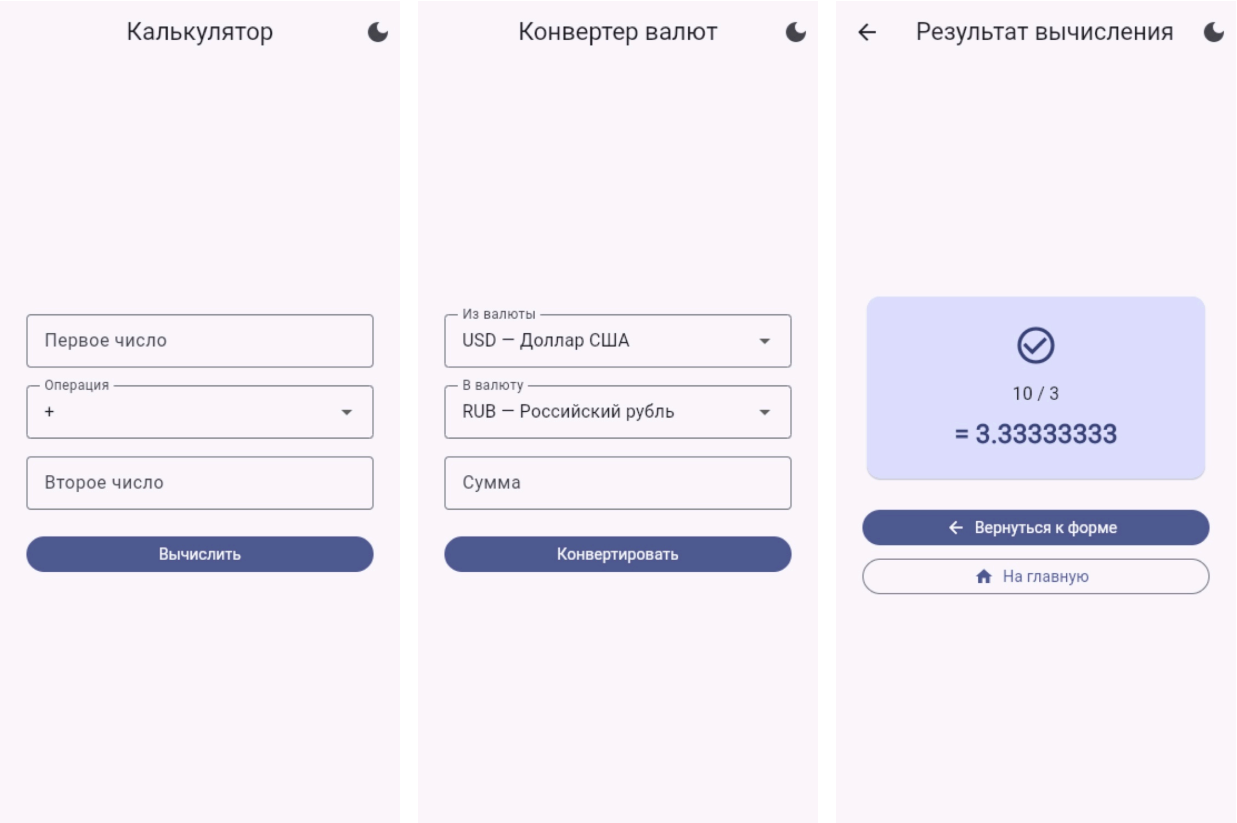


Рисунок 31 – Ширина окна 360 пикселей: калькулятор, конвертер, результат

При ширине 1920 пикселей форма не растягивается на весь монитор, а остается по центру с шириной 420 пикселей.

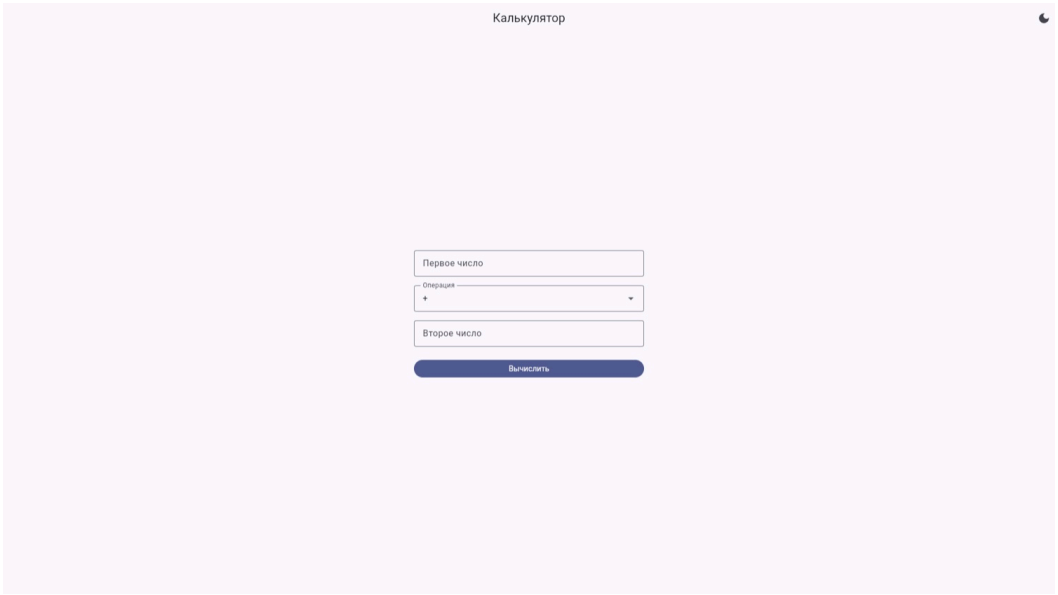


Рисунок 32 – Ширина окна 1920 пикселей

24. Покрываем логику тестами. В logic_test.dart проверяются четыре операции, деление на ноль, нечисловой ввод, неизвестная операция, отсутствующий параметр, округление и перевод валют. В app_test.dart проверяются экран 404, прямой

адрес результата с ошибкой и без параметров, валидация формы и сохранение темы. Запускаем все тесты из Android Studio: 19 тестов проходят.

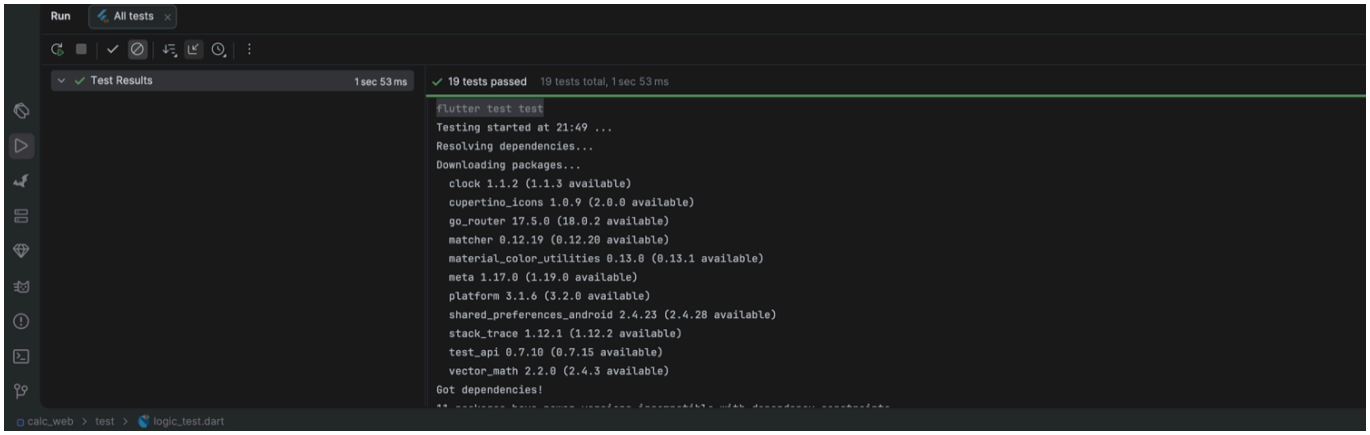


Рисунок 33 – Запуск тестов в Android Studio: 19 тестов пройдено

25. Проверяем сборку и анализ кода. Команда flutter analyze не выдает замечаний, команда flutter build web собирает приложение без ошибок.

Результат работы:

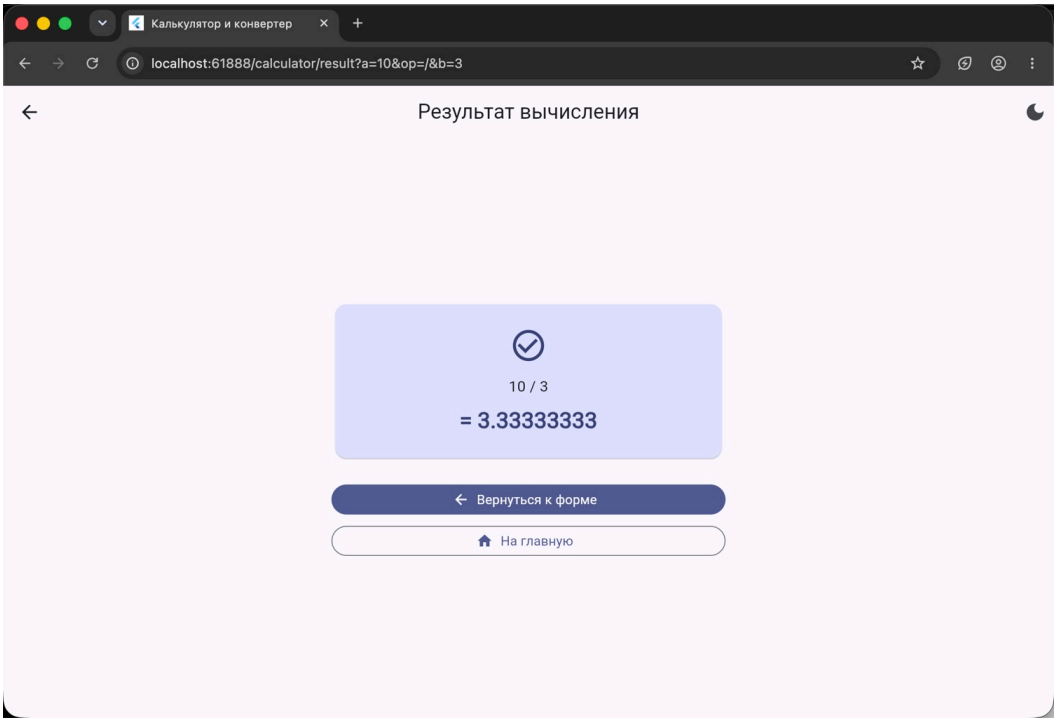


Рисунок 34 – Результат работы: экран результата калькулятора, параметры a, op и b видны в адресной строке

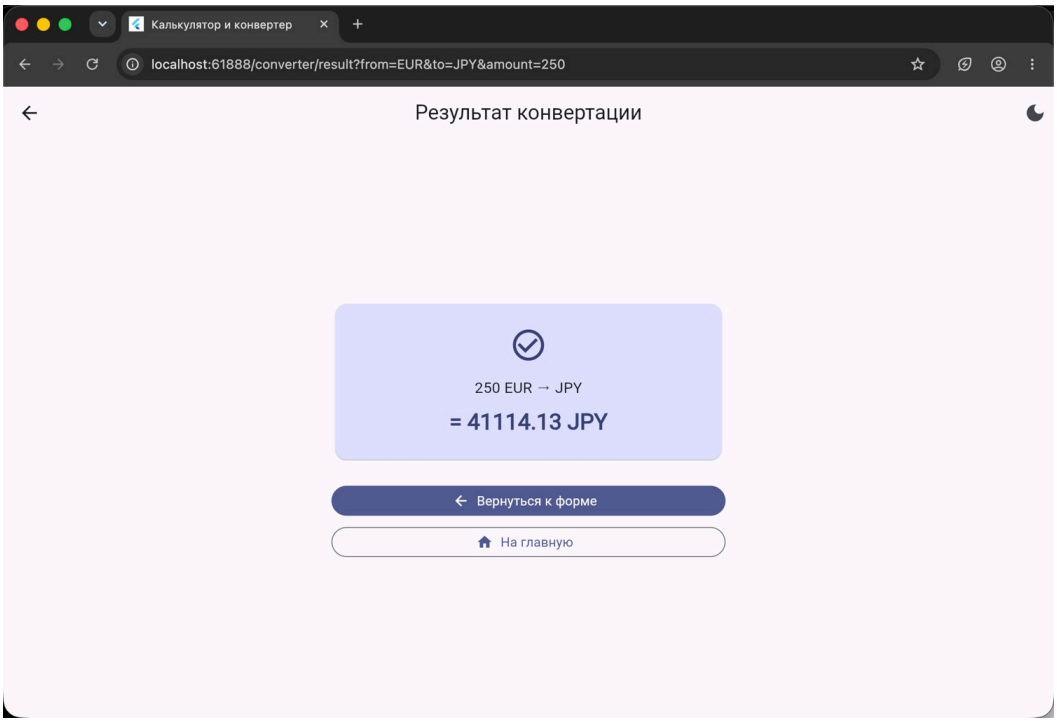


Рисунок 35 – Результат работы: экран результата конвертера, параметры from, to и amount видны в адресной строке

Вывод: создано web-приложение на Flutter с экранами «Главная», «Калькулятор» и «Конвертер валют», маршрутизацией на go_router и адресами без решетки. Освоена передача данных через параметры адреса: форма собирает адрес через Uri и переходит на него командой context.go, а экран результата восстанавливает состояние только по адресу, поэтому работает при прямом открытии ссылки. Реализованы проверка форм, обработка некорректных параметров, экран 404, сохранение темы и пары валют, адаптивная раскладка и 19 автоматических тестов.

ПРАКТИЧЕСКАЯ РАБОТА №2

Списки, поиск, фильтрация и пагинация

Цель работы: освоить управление состоянием через provider, разделение приложения на слои и построение списков с поиском, фильтрацией, сортировкой и постраничным выводом. Научиться выносить повторяющуюся таблицу в обобщённый виджет и хранить условия отбора в адресной строке, чтобы ссылку на отфильтрованный список можно было отправить коллеге. Работа выполняется на данных в памяти: сервер подключается позже.

Ход работы:

1. Создаем проект library_web командой flutter create и открываем его в Android Studio. В pubspec.yaml добавляем provider для управления состоянием, go_router для маршрутов и flutter_web_plugins для адресов без решетки. Это единое приложение «клиент библиотечной системы», которое дальше развивается до конца практики.

```
dependencies:
  flutter:
    sdk: flutter
  flutter_web_plugins:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.8
  provider: ^6.1.5+1
  go_router: ^17.5.0

dev_dependencies:
  flutter_test:
    sdk: flutter
```

Рисунок 1 – Зависимости проекта в pubspec.yaml

2. Делим приложение на слои: models, repositories, state, screens и widgets, а общие вещи (маршруты, точки перелома, задержка поиска, начальные данные) кладем в core и data. Структура каталогов в Android Studio показана ниже, схема зависимостей между слоями на следующем рисунке.

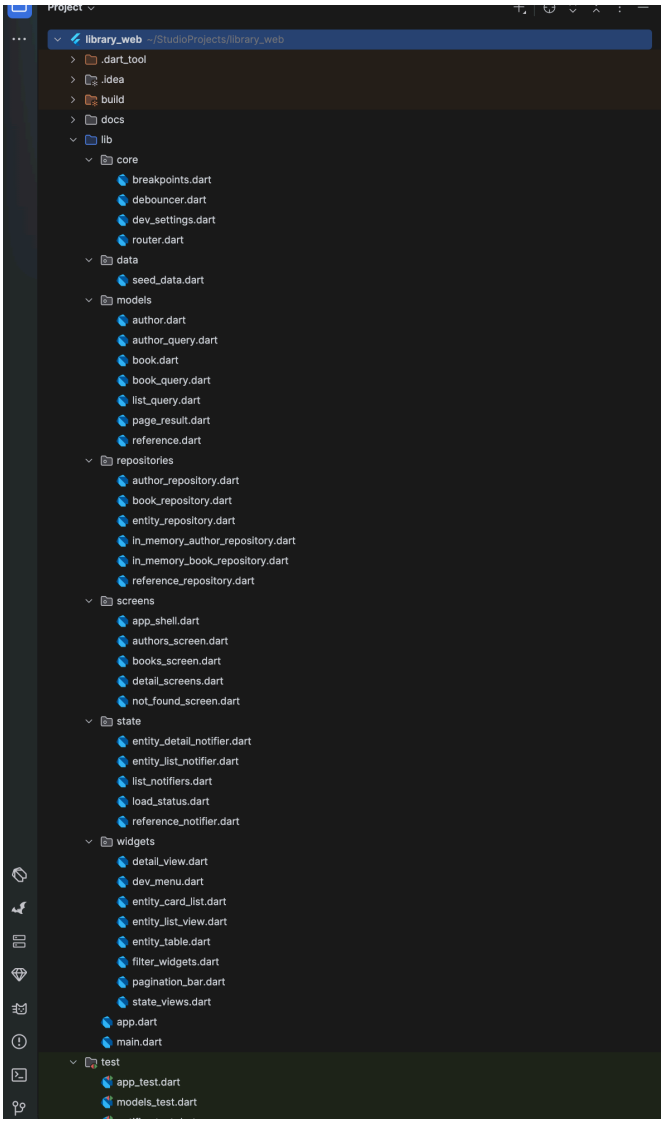


Рисунок 2 – Структура проекта в Android Studio

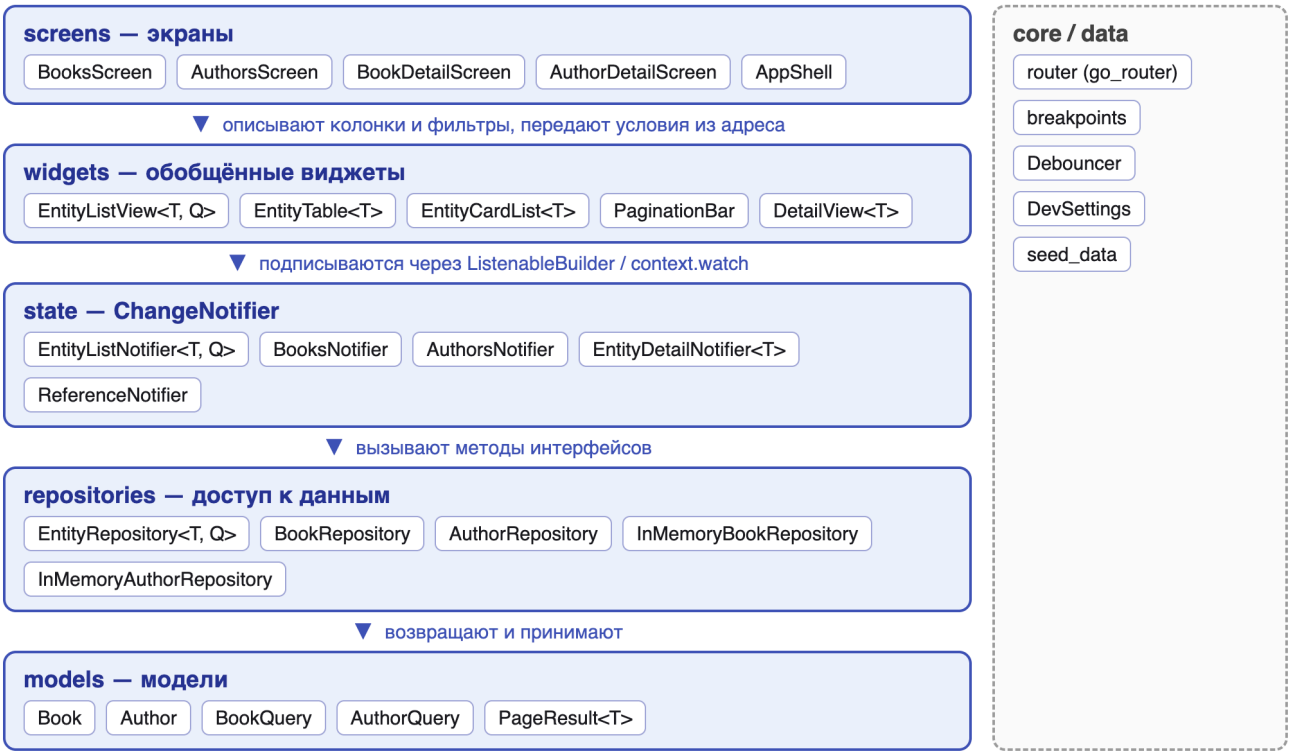
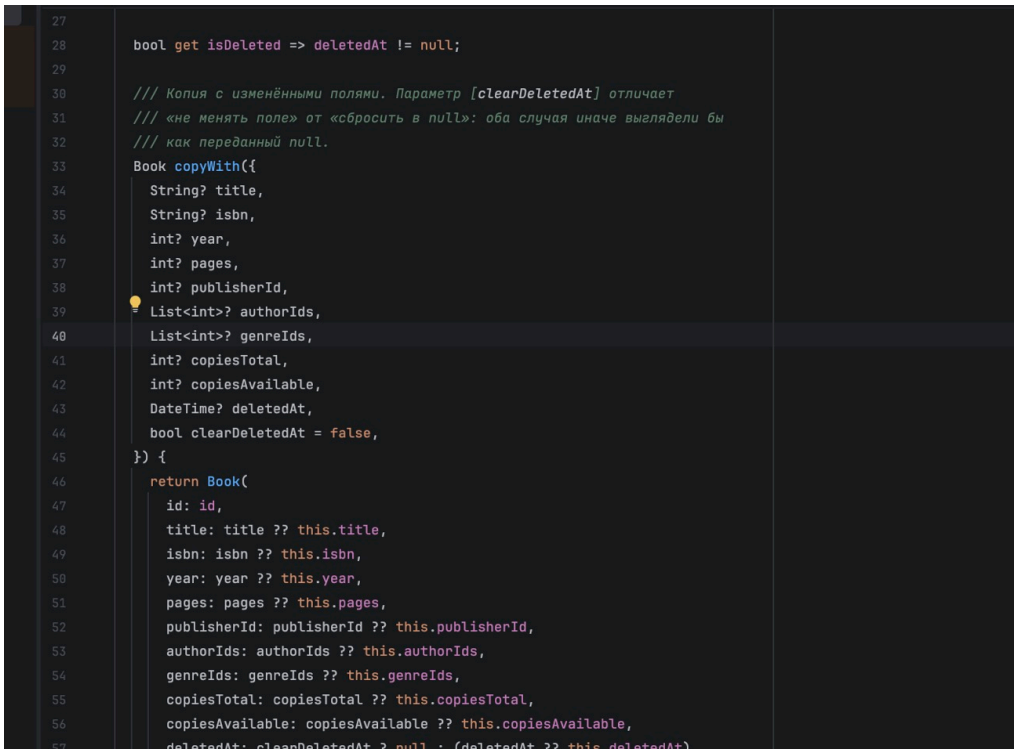


Рисунок 3 – Схема слоёв приложения

Зависимости направлены только вниз. Виджет не обращается к репозиторию напрямую, а получает данные через notifier. Notifier ничего не знает о виджетах и BuildContext. Модель не знает ни о ком. Благодаря этому при подключении сервера в ПР4 заменяется только реализация репозитория в одном месте, в провайдерах, а ни один экран не меняется.

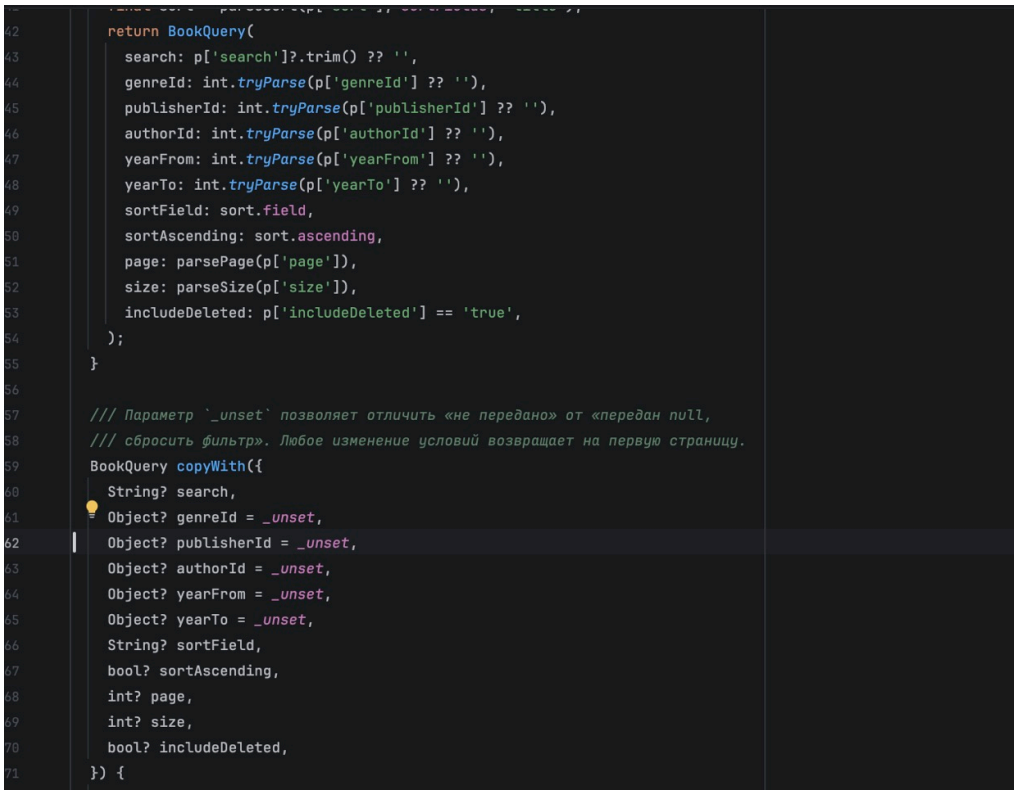
3. Описываем модель книги. В ней есть метод copyWith, который создает измененную копию без мутации исходного объекта. Параметр clearDeletedAt нужен, чтобы отличить «не менять поле» от «сбросить его в null»: без него оба случая выглядят как переданный null. Автор устроен так же, а общий класс PageResult<T> описывает страницу результатов (элементы, номер страницы, размер и общее число записей) и повторяет оболочку списка учебного API.



```dart
bool get isDeleted => deletedAt != null;

/// Копия с изменёнными полями. Параметр [clearDeletedAt] отличает
/// «не менять поле» от «сбросить в null»: оба случая иначе выглядели бы
/// как переданный null.
Book copyWith({
  String? title,
  String? isbn,
  int? year,
  int? pages,
  int? publisherId,
  List<int>? authorIds,
  List<int>? genreIds,
  int? copiesTotal,
  int? copiesAvailable,
  DateTime? deletedAt,
  bool clearDeletedAt = false,
}) {
  return Book(
    id: id,
    title: title ?? this.title,
    isbn: isbn ?? this.isbn,
    year: year ?? this.year,
    pages: pages ?? this.pages,
    publisherId: publisherId ?? this.publisherId,
    authorIds: authorIds ?? this.authorIds,
    genreIds: genreIds ?? this.genreIds,
    copiesTotal: copiesTotal ?? this.copiesTotal,
    copiesAvailable: copiesAvailable ?? this.copiesAvailable,
```

Рисунок 4 – Модель Book и метод copyWith (book.dart)

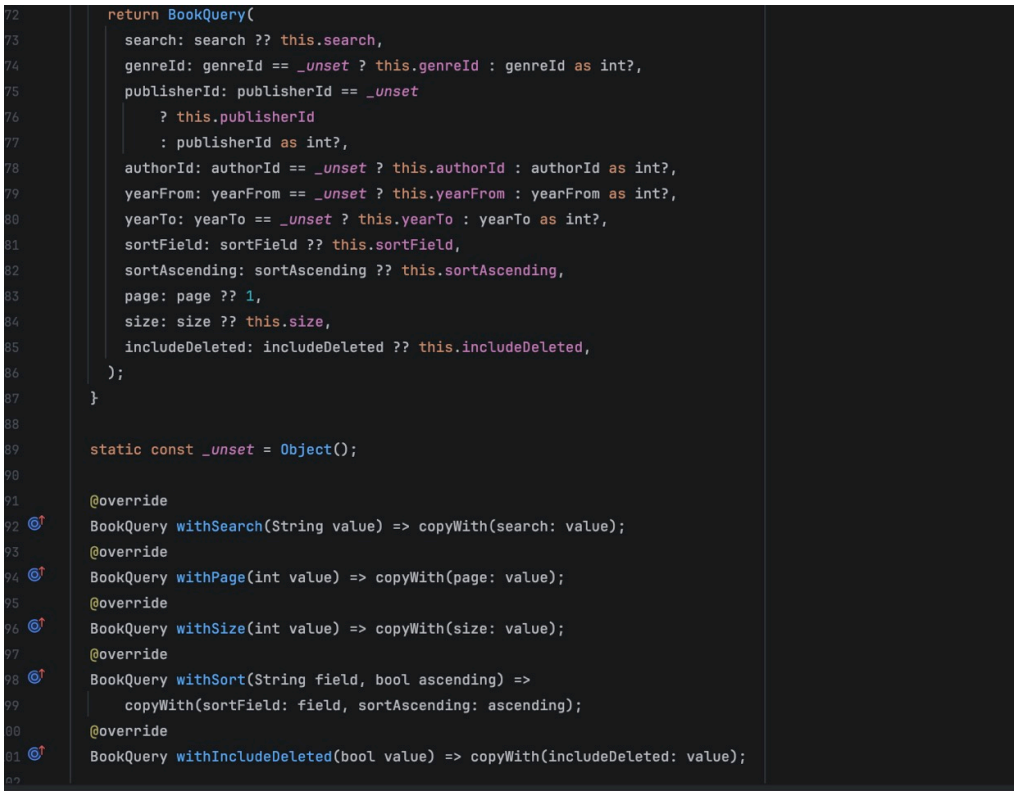
4. Все условия отбора собираем в один неизменяемый объект BookQuery: поиск, жанр, издательство, диапазон года, сортировка, страница, размер страницы и показ удалённых. Для автора есть такой же AuthorQuery. Конструктор fromParams восстанавливает условия из параметров адреса и заменяет любое некорректное значение значением по умолчанию, а toParams превращает условия обратно в параметры и не записывает значения по умолчанию.



```
    return BookQuery(
      search: p['search']?.trim() ?? '',
      genreId: int.tryParse(p['genreId'] ?? ''),
      publisherId: int.tryParse(p['publisherId'] ?? ''),
      authorId: int.tryParse(p['authorId'] ?? ''),
      yearFrom: int.tryParse(p['yearFrom'] ?? ''),
      yearTo: int.tryParse(p['yearTo'] ?? ''),
      sortField: sort.field,
      sortAscending: sort.ascending,
      page: parsePage(p['page']),
      size: parseSize(p['size']),
      includeDeleted: p['includeDeleted'] == 'true',
    );
  }

  /// Параметр `_unset` позволяет отличить «не передано» от «передан null,
  /// сбросить фильтр». Любое изменение условий возвращает на первую страницу.
  BookQuery copyWith({
    String? search,
    Object? genreId = _unset,
    Object? publisherId = _unset,
    Object? authorId = _unset,
    Object? yearFrom = _unset,
    Object? yearTo = _unset,
    String? sortField,
    bool? sortAscending,
    int? page,
    int? size,
    bool? includeDeleted,
  }) {
```

Рисунок 5 – BookQuery: разбор параметров адреса и начало copyWith

В copyWith константа _unset отличает «параметр не передан» от «передан null, сбросить фильтр». Строка page: page ?? 1 возвращает список на первую страницу при любом изменении условий. Иначе пользователь, стоящий на седьмой странице, после ввода поискового запроса увидел бы пустой экран.



```
    return BookQuery(
      search: search ?? this.search,
      genreId: genreId == _unset ? this.genreId : genreId as int?,
      publisherId: publisherId == _unset
          ? this.publisherId
          : publisherId as int?,
      authorId: authorId == _unset ? this.authorId : authorId as int?,
      yearFrom: yearFrom == _unset ? this.yearFrom : yearFrom as int?,
      yearTo: yearTo == _unset ? this.yearTo : yearTo as int?,
      sortField: sortField ?? this.sortField,
      sortAscending: sortAscending ?? this.sortAscending,
      page: page ?? 1,
      size: size ?? this.size,
      includeDeleted: includeDeleted ?? this.includeDeleted,
    );
  }

  static const _unset = Object();

  @override
  BookQuery withSearch(String value) => copyWith(search: value);
  @override
  BookQuery withPage(int value) => copyWith(page: value);
  @override
  BookQuery withSize(int value) => copyWith(size: value);
  @override
  BookQuery withSort(String field, bool ascending) =>
      copyWith(sortField: field, sortAscending: ascending);
  @override
  BookQuery withIncludeDeleted(bool value) => copyWith(includeDeleted: value);
```

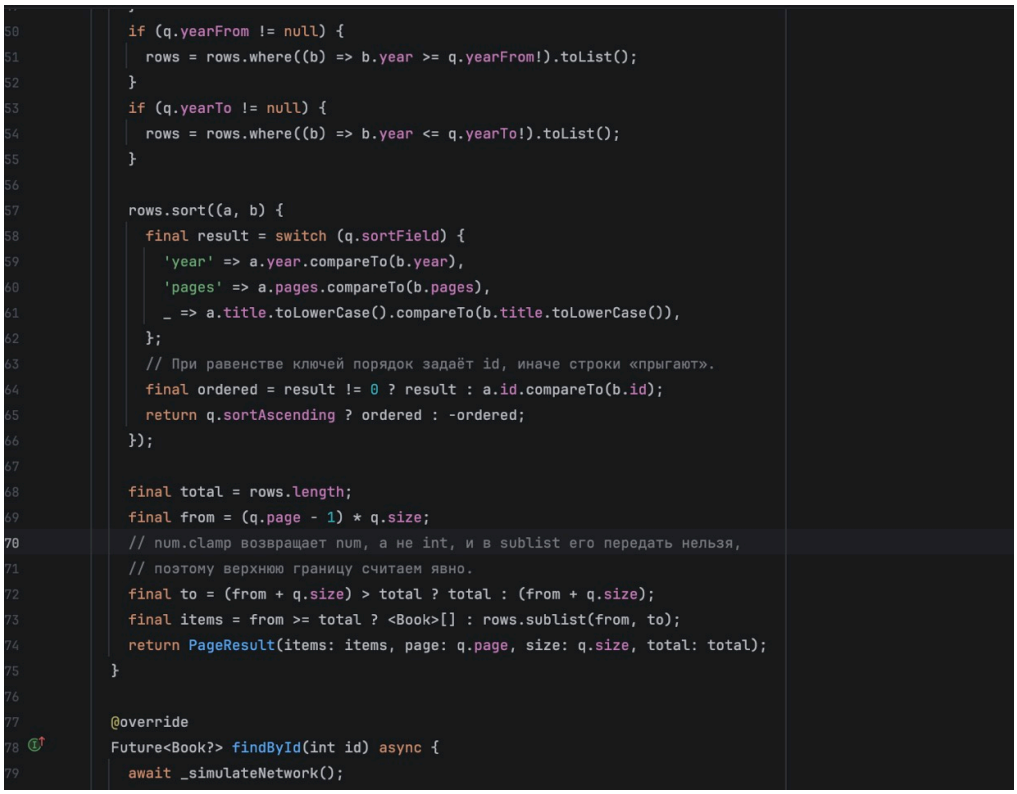
Рисунок 6 – BookQuery: copyWith и методы withSearch, withPage, withSort

5. Описываем репозиторий. Сначала интерфейс EntityRepository<T, Q> с операциями find, findById, softDelete, hardDelete, restore и deleteMany, затем BookRepository с create и update. Реализация InMemoryBookRepository хранит книги в списке, созданном из начального набора из 36 записей. Метод find имитирует сеть задержкой в 250 мс, фильтрует по поиску (название и ISBN без учета регистра), жанру, издательству и диапазону года.

```
  Future<void> _simulateNetwork() async {
    await Future.delayed(_dev?.delay ?? Duration.zero);
    if (_dev?.failing ?? false) {
      throw const RepositoryException('сервер недоступен (имитация сбоя)');
    }
  }

  @override
  Future<PageResult<Book>> find(BookQuery q) async {
    await _simulateNetwork();

    var rows = _books.where((b) => q.includeDeleted || !b.isDeleted).toList();

    if (q.search.trim().isNotEmpty) {
      final needle = q.search.trim().toLowerCase();
      rows = rows
          .where(
            (b) =>
                b.title.toLowerCase().contains(needle) ||
                b.isbn.toLowerCase().contains(needle),
          )
          .toList();
    }
    if (q.genreId != null) {
      rows = rows.where((b) => b.genreIds.contains(q.genreId)).toList();
    }
    if (q.publisherId != null) {
      rows = rows.where((b) => b.publisherId == q.publisherId).toList();
    }
    if (q.authorId != null) {
```

Рисунок 7 – InMemoryBookRepository.find: поиск и фильтры (in_memory_book_repository.dart)

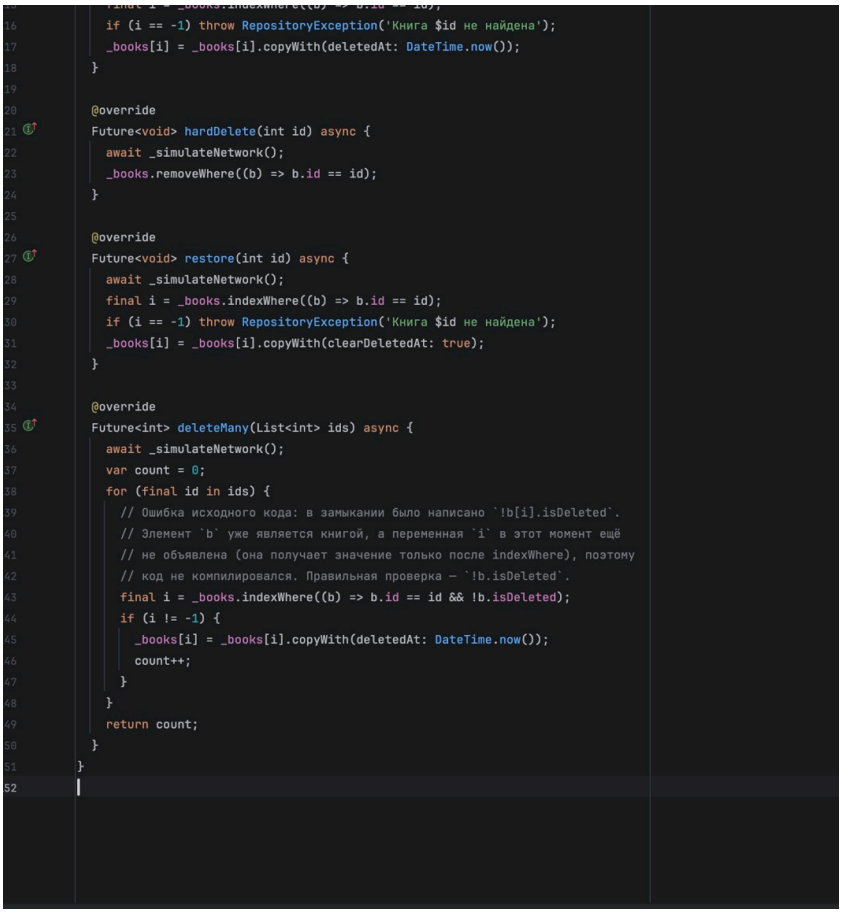
Затем список сортируется по выбранному полю. При равных ключах порядок задает идентификатор, иначе строки «прыгают». Страница вырезается из отфильтрованного списка; границу считаем явно, потому что num.clamp возвращает num, а не int, и в sublist его передать нельзя.



```
    if (q.yearFrom != null) {
      rows = rows.where((b) => b.year >= q.yearFrom!).toList();
    }
    if (q.yearTo != null) {
      rows = rows.where((b) => b.year <= q.yearTo!).toList();
    }

    rows.sort((a, b) {
      final result = switch (q.sortField) {
        'year' => a.year.compareTo(b.year),
        'pages' => a.pages.compareTo(b.pages),
        _ => a.title.toLowerCase().compareTo(b.title.toLowerCase()),
      };
      // При равенстве ключей порядок задаёт id, иначе строки «прыгают».
      final ordered = result != 0 ? result : a.id.compareTo(b.id);
      return q.sortAscending ? ordered : -ordered;
    });

    final total = rows.length;
    final from = (q.page - 1) * q.size;
    // num.clamp возвращает num, а не int, и в sublist его передать нельзя,
    // поэтому верхнюю границу считаем явно.
    final to = (from + q.size) > total ? total : (from + q.size);
    final items = from >= total ? <Book>[] : rows.sublist(from, to);
    return PageResult(items: items, page: q.page, size: q.size, total: total);
  }

  @override
  Future<Book?> findById(int id) async {
    await _simulateNetwork();
```

Рисунок 8 – InMemoryBookRepository.find: сортировка и постраничная выдача

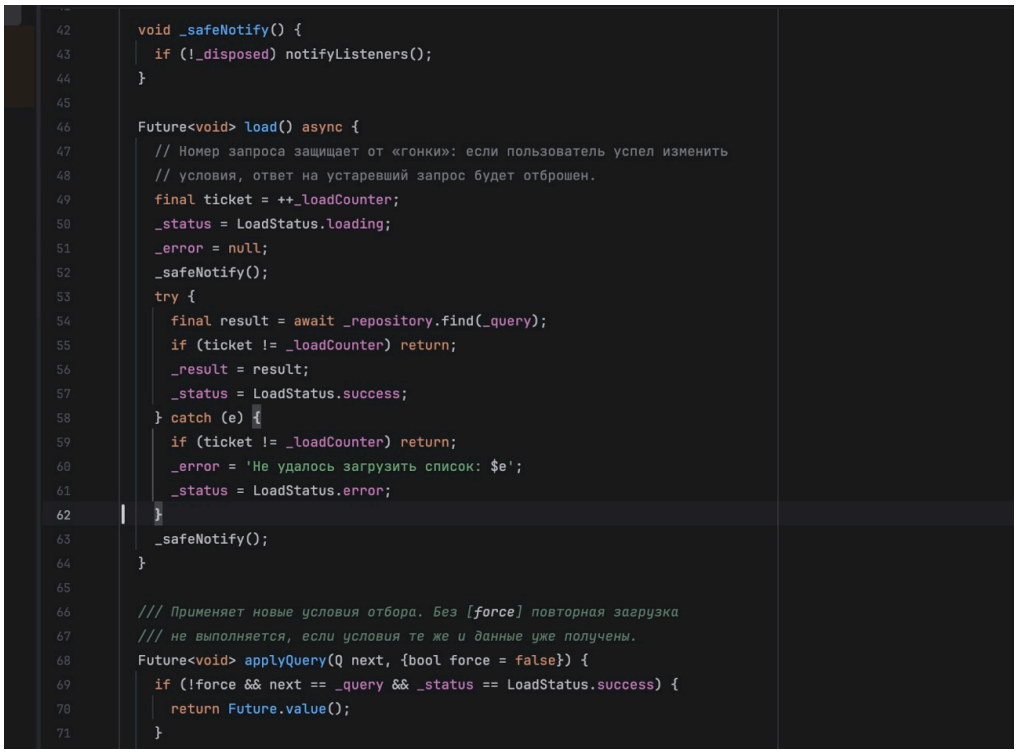
6. Исправляем ошибку в методе deleteMany. В исходном коде проверка записывалась как !b[i].isDeleted внутри замыкания, которое передается в indexWhere. Код не компилировался по двум причинам. Во-первых, b уже является книгой, а у класса Book нет оператора [], поэтому b[i] недопустимо. Во-вторых, переменная i объявляется этой же строкой (final i = …) и используется в собственном инициализаторе до получения значения, а анализатор запрещает обращаться к локальной переменной до ее объявления. Нужная проверка — !b.isDeleted: найти книгу с этим идентификатором, которая еще не удалена. Исправление покрыто тестом: метод не считает уже удалённые и несуществующие записи.



```
    if (i == -1) throw RepositoryException('Книга $id не найдена');
    _books[i] = _books[i].copyWith(deletedAt: DateTime.now());
  }

  @override
  Future<void> hardDelete(int id) async {
    await _simulateNetwork();
    _books.removeWhere((b) => b.id == id);
  }

  @override
  Future<void> restore(int id) async {
    await _simulateNetwork();
    final i = _books.indexWhere((b) => b.id == id);
    if (i == -1) throw RepositoryException('Книга $id не найдена');
    _books[i] = _books[i].copyWith(clearDeletedAt: true);
  }

  @override
  Future<int> deleteMany(List<int> ids) async {
    await _simulateNetwork();
    var count = 0;
    for (final id in ids) {
      // Ошибка исходного кода: в замыкании было написано `!b[i].isDeleted`.
      // Элемент `b` уже является книгой, а переменная `i` в этот момент ещё
      // не объявлена (она получает значение только после indexWhere), поэтому
      // код не компилировался. Правильная проверка — `!b.isDeleted`.
      final i = _books.indexWhere((b) => b.id == id && !b.isDeleted);
      if (i != -1) {
        _books[i] = _books[i].copyWith(deletedAt: DateTime.now());
        count++;
      }
    }
    return count;
  }
}
```

Рисунок 9 – Исправленный метод deleteMany

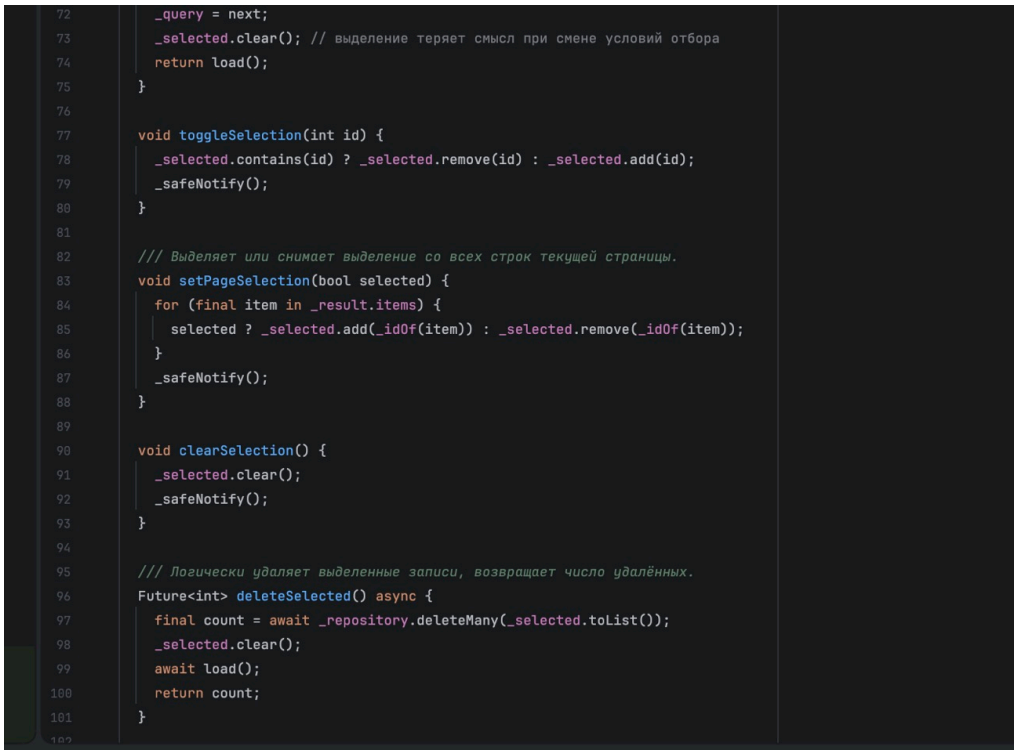
7. Описываем состояние экрана. Класс EntityListNotifier<T, Q> наследуется от ChangeNotifier и хранит текущие условия, страницу результатов, статус загрузки (idle, loading, success, error), сообщение об ошибке и множество выделенных строк. Метод load переводит состояние в loading, вызывает репозиторий и после ответа вызывает notifyListeners. Номер запроса защищает от «гонки»: если пользователь успел изменить условия, ответ на устаревший запрос отбрасывается. Пустой результат — это success со списком нулевой длины, а не отдельная ошибка.



```
  void _safeNotify() {
    if (!_disposed) notifyListeners();
  }

  Future<void> load() async {
    // Номер запроса защищает от «гонки»: если пользователь успел изменить
    // условия, ответ на устаревший запрос будет отброшен.
    final ticket = ++_loadCounter;
    _status = LoadStatus.loading;
    _error = null;
    _safeNotify();
    try {
      final result = await _repository.find(_query);
      if (ticket != _loadCounter) return;
      _result = result;
      _status = LoadStatus.success;
    } catch (e) {
      if (ticket != _loadCounter) return;
      _error = 'Не удалось загрузить список: $e';
      _status = LoadStatus.error;
    }
    _safeNotify();
  }

  /// Применяет новые условия отбора. Без [force] повторная загрузка
  /// не выполняется, если условия те же и данные уже получены.
  Future<void> applyQuery(Q next, {bool force = false}) {
    if (!force && next == _query && _status == LoadStatus.success) {
      return Future.value();
    }
```

Рисунок 10 – EntityListNotifier: load и applyQuery

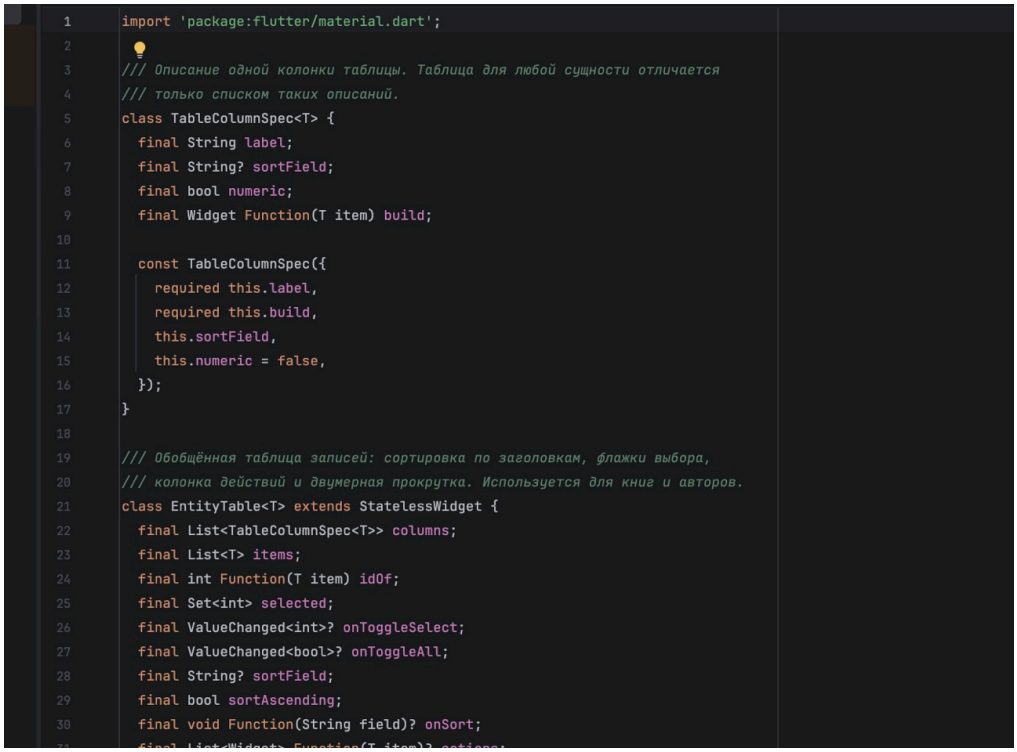
Метод applyQuery сбрасывает выделение при смене условий, иначе после смены фильтра удалялись бы не те записи. Остальные методы отвечают за выделение строк и удаление: логическое, физическое и множественное. После каждого действия список перезапрашивается.



```
    _query = next;
    _selected.clear(); // выделение теряет смысл при смене условий отбора
    return load();
  }

  void toggleSelection(int id) {
    _selected.contains(id) ? _selected.remove(id) : _selected.add(id);
    _safeNotify();
  }

  /// Выделяет или снимает выделение со всех строк текущей страницы.
  void setPageSelection(bool selected) {
    for (final item in _result.items) {
      selected ? _selected.add(_idOf(item)) : _selected.remove(_idOf(item));
    }
    _safeNotify();
  }

  void clearSelection() {
    _selected.clear();
    _safeNotify();
  }

  /// Логически удаляет выделенные записи, возвращает число удалённых.
  Future<int> deleteSelected() async {
    final count = await _repository.deleteMany(_selected.toList());
    _selected.clear();
    await load();
    return count;
  }
```

Рисунок 11 – EntityListNotifier: выделение и удаление выбранных

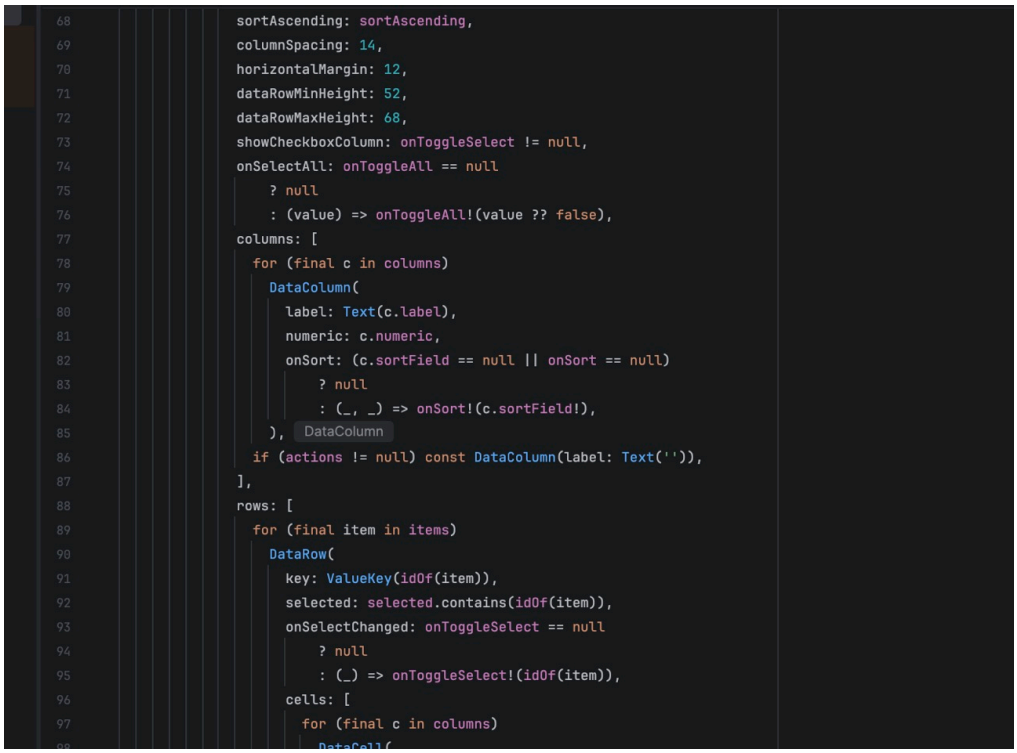
8. Выносим таблицу в обобщённый виджет EntityTable<T>. Колонка описывается объектом TableColumnSpec<T>: подпись, поле сортировки, признак числовой колонки и функция, строящая ячейку. Таблица принимает список колонок, список записей, функцию получения идентификатора, выделенные строки, текущую сортировку и функцию действий для строки.



```dart
import 'package:flutter/material.dart';

/// Описание одной колонки таблицы. Таблица для любой сущности отличается
/// только списком таких описаний.
class TableColumnSpec<T> {
  final String label;
  final String? sortField;
  final bool numeric;
  final Widget Function(T item) build;

  const TableColumnSpec({
    required this.label,
    required this.build,
    this.sortField,
    this.numeric = false,
  });
}

/// Обобщённая таблица записей: сортировка по заголовкам, флажки выбора,
/// колонка действий и двумерная прокрутка. Используется для книг и авторов.
class EntityTable<T> extends StatelessWidget {
  final List<TableColumnSpec<T>> columns;
  final List<T> items;
  final int Function(T item) idOf;
  final Set<int> selected;
  final ValueChanged<int>? onToggleSelect;
  final ValueChanged<bool>? onToggleAll;
  final String? sortField;
  final bool sortAscending;
  final void Function(String field)? onSort;
```

Рисунок 12 – TableColumnSpec и поля EntityTable (entity_table.dart)

Внутри используется DataTable в двумерной прокрутке с ограничением minWidth, чтобы на широком окне таблица занимала всю ширину, а на узком прокручивалась. Стрелка сортировки в активной колонке и флажки выбора строк обеспечиваются самим DataTable, нужно только передать sortColumnIndex, sortAscending и onSelectChanged.

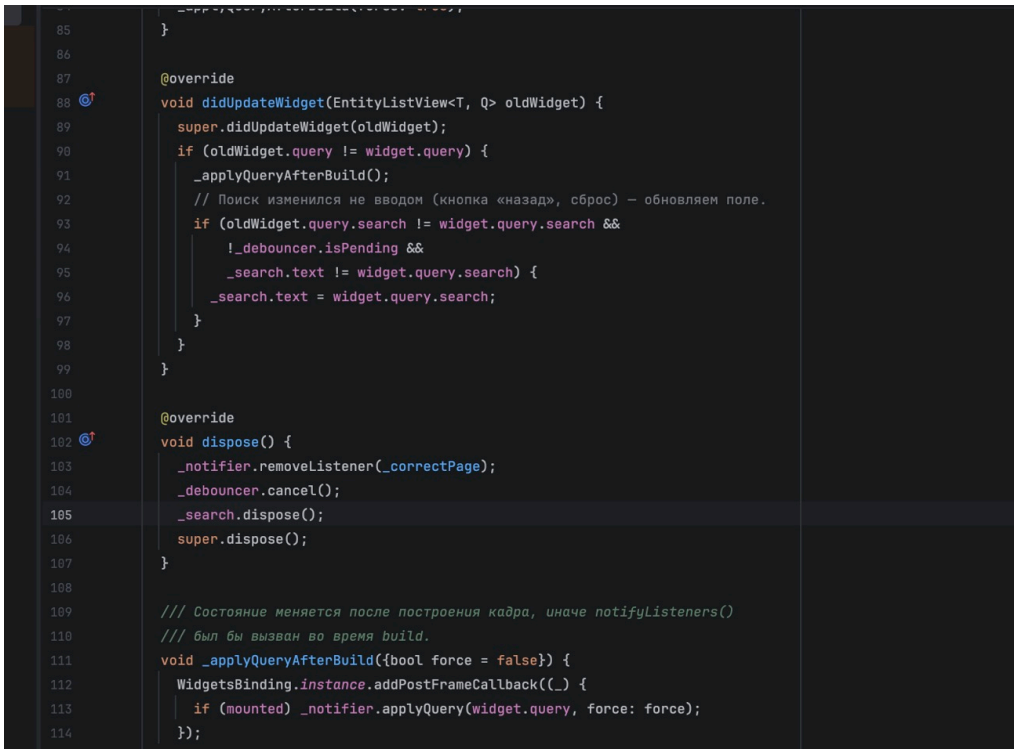


```
sortAscending: sortAscending,
columnSpacing: 14,
horizontalMargin: 12,
dataRowMinHeight: 52,
dataRowMaxHeight: 68,
showCheckboxColumn: onToggleSelect != null,
onSelectAll: onToggleAll == null
    ? null
    : (value) => onToggleAll!(value ?? false),
columns: [
  for (final c in columns)
    DataColumn(
      label: Text(c.label),
      numeric: c.numeric,
      onSort: (c.sortField == null || onSort == null)
          ? null
          : (_, _) => onSort!(c.sortField!),
    ), DataColumn
  if (actions != null) const DataColumn(label: Text('')),
],
rows: [
  for (final item in items)
    DataRow(
      key: ValueKey(idOf(item)),
      selected: selected.contains(idOf(item)),
      onSelectChanged: onToggleSelect == null
          ? null
          : (_) => onToggleSelect!(idOf(item)),
      cells: [
        for (final c in columns)
```

Рисунок 13 – Построение DataTable из описаний колонок

Обоснование обобщённой таблицы. В приложении будет много списков: книги, авторы, жанры, издательства, читатели, выдачи. Если писать для каждого свой экран со своей таблицей, поиском и пагинацией, получится шесть почти одинаковых файлов по триста строк, и любая правка (например, исправление страницы за пределами списка) пришлось бы вносить шесть раз. Единственное различие между списками книг и авторов — набор колонок и фильтров, поэтому оно вынесено в данные: описание TableColumnSpec<T>. Сортировка, выделение, действия, прокрутка, карточки на узком окне и пагинация написаны один раз и покрыты общими тестами, а новый список добавляется описанием колонок и маршрутом.

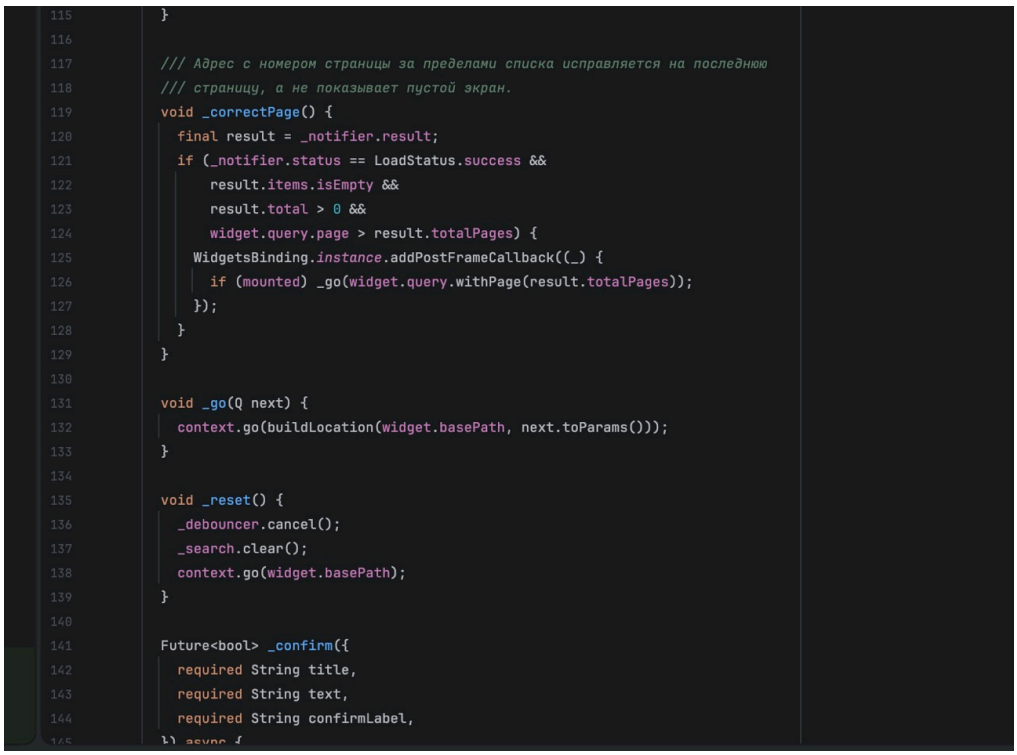
9. Весь экран списка делаем универсальным виджетом EntityListView<T, Q>: он содержит заголовок, поиск, фильтры, панель выбранных строк, таблицу или карточки, постраничную навигацию и все четыре состояния загрузки. Условия отбора приходят из адреса. В initState и в didUpdateWidget условия передаются в notifier после построения кадра (иначе notifyListeners вызывался бы во время build), а поле поиска обновляется, если адрес изменился не вводом, например кнопкой «назад».



```
  }

  @override
  void didUpdateWidget(EntityListView<T, Q> oldWidget) {
    super.didUpdateWidget(oldWidget);
    if (oldWidget.query != widget.query) {
      _applyQueryAfterBuild();
      // Поиск изменился не вводом (кнопка «назад», сброс) — обновляем поле.
      if (oldWidget.query.search != widget.query.search &&
          !_debouncer.isPending &&
          _search.text != widget.query.search) {
        _search.text = widget.query.search;
      }
    }
  }

  @override
  void dispose() {
    _notifier.removeListener(_correctPage);
    _debouncer.cancel();
    _search.dispose();
    super.dispose();
  }

  /// Состояние меняется после построения кадра, иначе notifyListeners()
  /// был бы вызван во время build.
  void _applyQueryAfterBuild({bool force = false}) {
    WidgetsBinding.instance.addPostFrameCallback((_) {
      if (mounted) _notifier.applyQuery(widget.query, force: force);
    });
```

Рисунок 14 – EntityListView: синхронизация условий с адресом

Номер страницы за пределами списка исправляется на последнюю, а любое изменение условий записывается обратно в адрес командой context.go. Благодаря этому каждое состояние списка получает собственную запись в истории браузера.



```
  }

  /// Адрес с номером страницы за пределами списка исправляется на последнюю
  /// страницу, а не показывает пустой экран.
  void _correctPage() {
    final result = _notifier.result;
    if (_notifier.status == LoadStatus.success &&
        result.items.isEmpty &&
        result.total > 0 &&
        widget.query.page > result.totalPages) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        if (mounted) _go(widget.query.withPage(result.totalPages));
      });
    }
  }

  void _go(Q next) {
    context.go(buildLocation(widget.basePath, next.toParams()));
  }

  void _reset() {
    _debouncer.cancel();
    _search.clear();
    context.go(widget.basePath);
  }

  Future<bool> _confirm({
    required String title,
    required String text,
    required String confirmLabel,
  }) async {
```

Рисунок 15 – EntityListView: исправление страницы и запись условий в адрес

10. Поле поиска не запускает выборку на каждую клавишу. Вспомогательный класс Debouncer откладывает действие на 400 мс после последнего нажатия: каждый новый вызов сбрасывает таймер. Те же условия применяются к полям года. Фильтры задаются на экране списком виджетов, а в адрес изменения попадают через FilterScope, который применяет их к актуальным условиям.

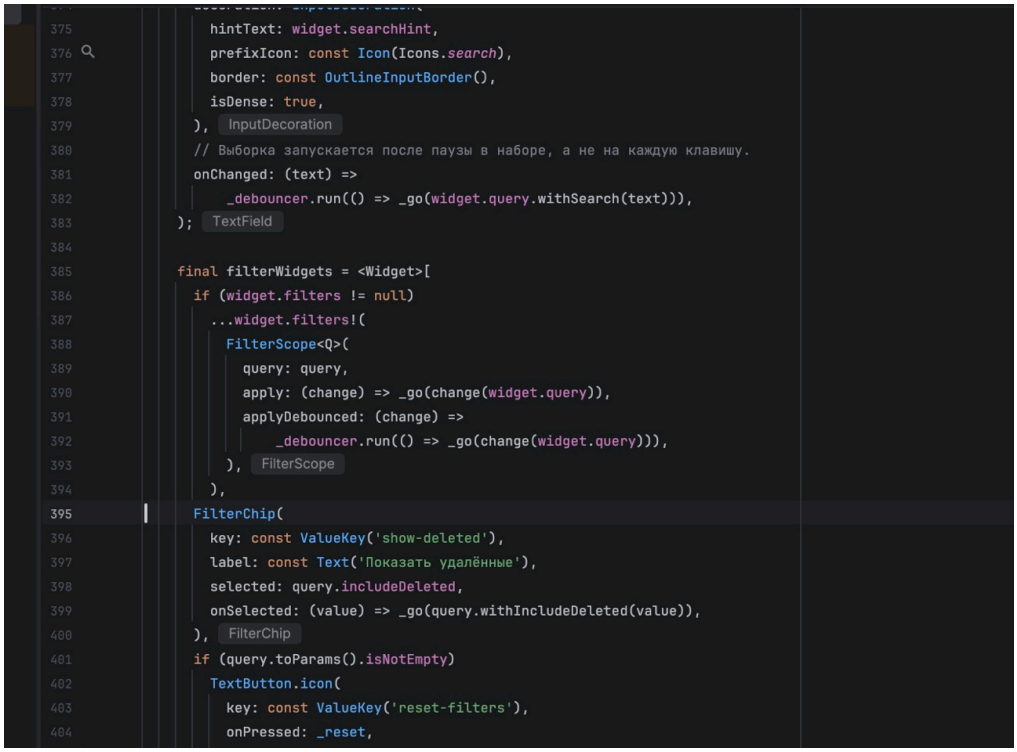


Рисунок 16 – Поле поиска с задержкой и построение фильтров (entity_list_view.dart)

11. Экран книг описывает только колонки и фильтры, остальное делает EntityListView. Для авторов используется тот же виджет с другим набором колонок (ФИО, страна, год рождения) и одним фильтром по стране.

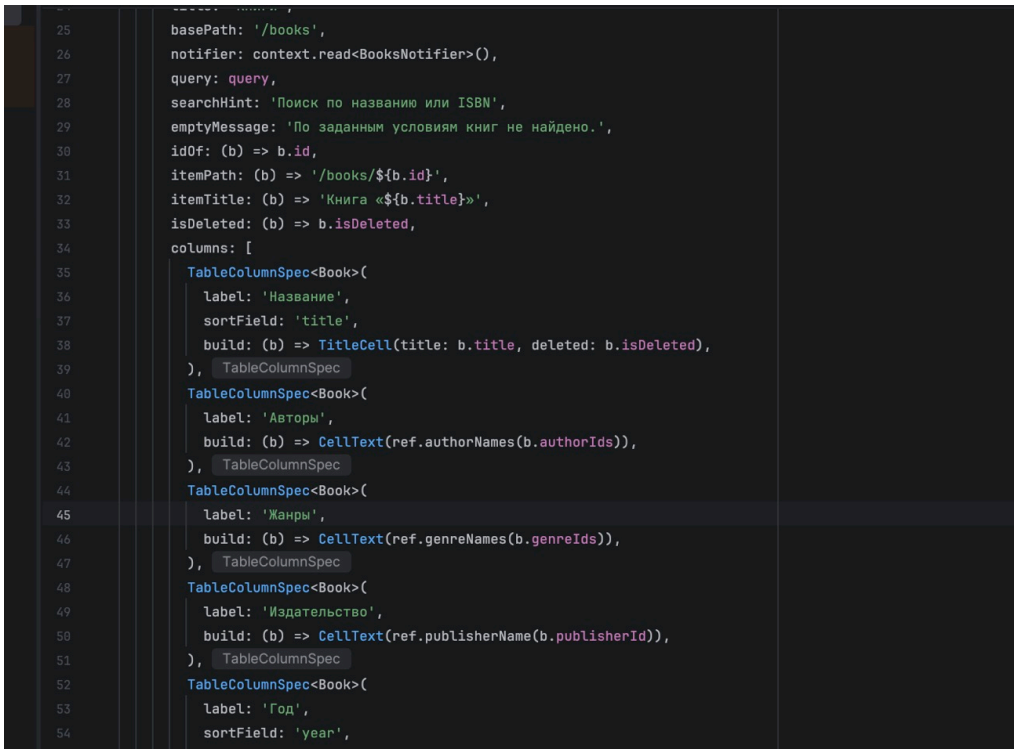


Рисунок 17 – BooksScreen: описание колонок

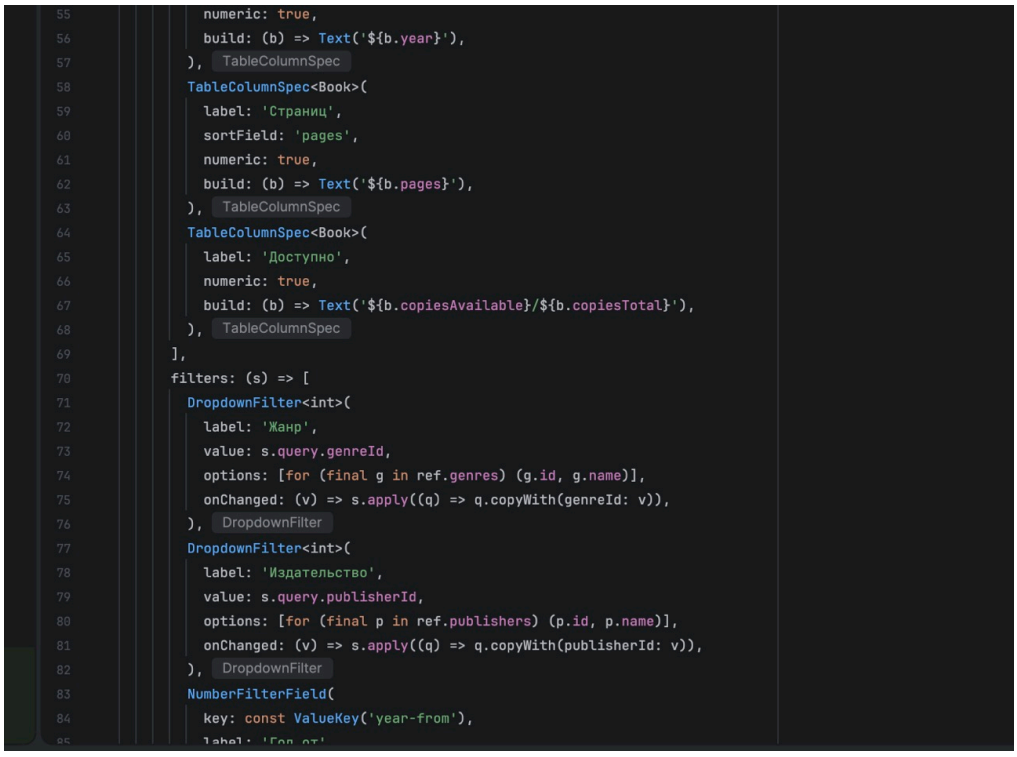
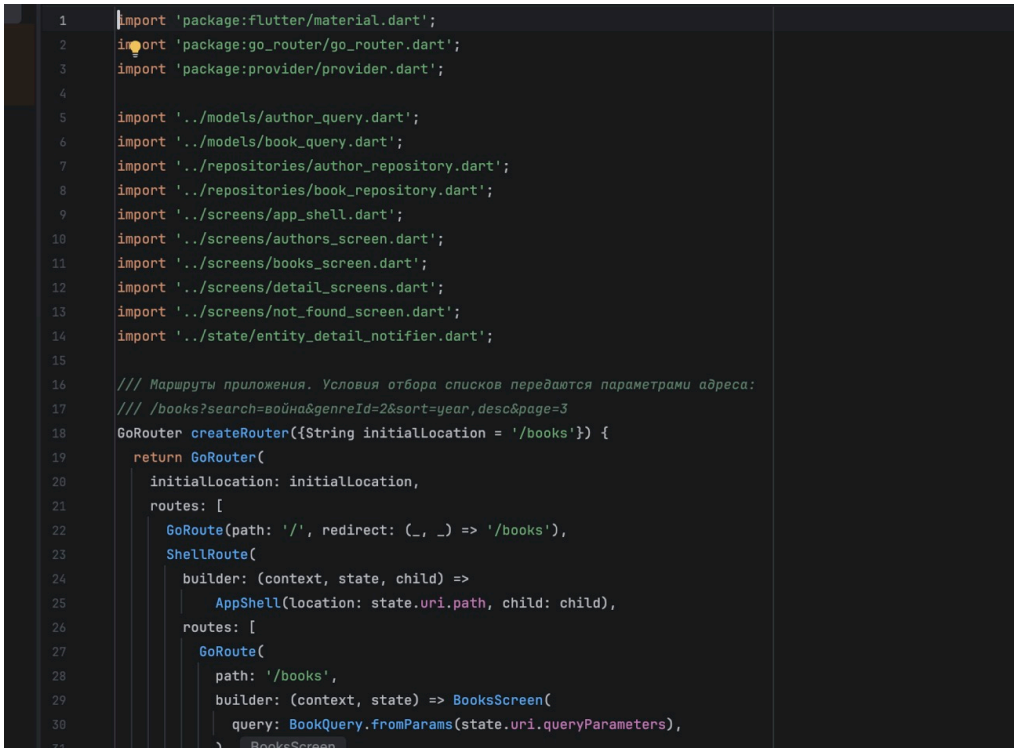


Рисунок 18 – BooksScreen: колонки и фильтры по жанру, издательству, году

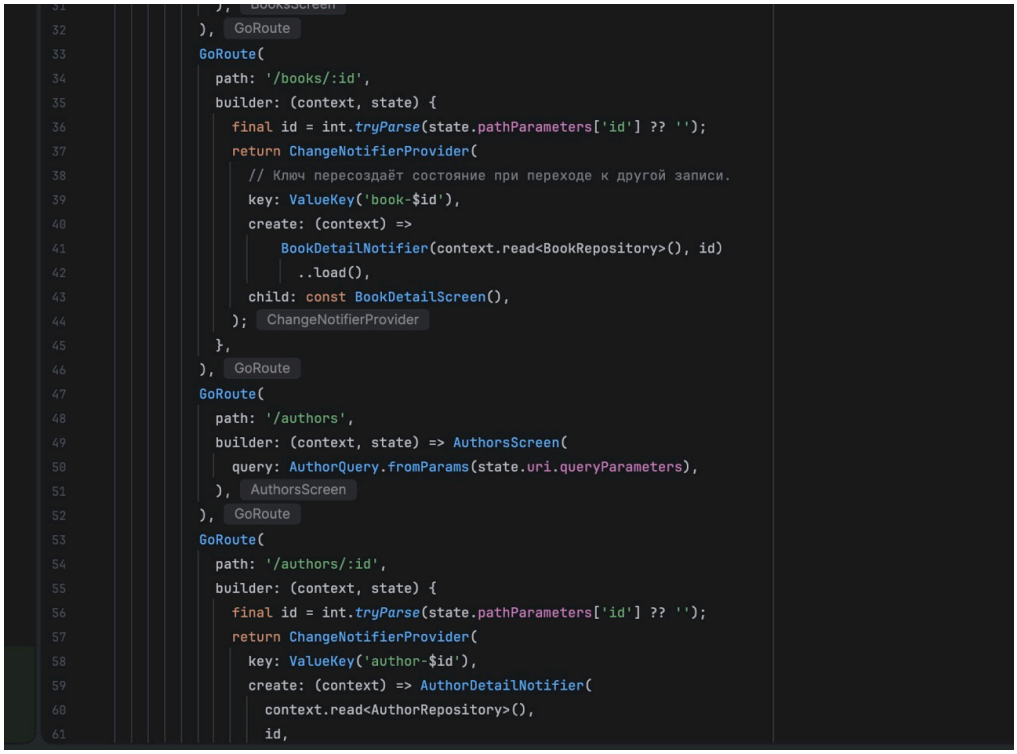
12. Подключаем маршруты. Для книг и авторов есть списки /books и /authors и карточки /books/:id и /authors/:id внутри ShellRoute с адаптивной навигацией (нижняя панель на узком окне, боковая полоса на широком). Builder списка превращает параметры адреса в объект условий методом fromParams. Для карточек

создается свой notifier, ключ ValueKey пересоздает его при переходе к другой записи. Неизвестный адрес открывает экран 404.



```dart
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
import 'package:provider/provider.dart';

import '../models/author_query.dart';
import '../models/book_query.dart';
import '../repositories/author_repository.dart';
import '../repositories/book_repository.dart';
import '../screens/app_shell.dart';
import '../screens/authors_screen.dart';
import '../screens/books_screen.dart';
import '../screens/detail_screens.dart';
import '../screens/not_found_screen.dart';
import '../state/entity_detail_notifier.dart';

/// Маршруты приложения. Условия отбора списков передаются параметрами адреса:
/// /books?search=война&genreId=2&sort=year,desc&page=3
GoRouter createRouter({String initialLocation = '/books'}) {
  return GoRouter(
    initialLocation: initialLocation,
    routes: [
      GoRoute(path: '/', redirect: (_, _) => '/books'),
      ShellRoute(
        builder: (context, state, child) =>
            AppShell(location: state.uri.path, child: child),
        routes: [
          GoRoute(
            path: '/books',
            builder: (context, state) => BooksScreen(
              query: BookQuery.fromParams(state.uri.queryParameters),
```

Рисунок 19 – Маршруты /books и /books/:id (router.dart)

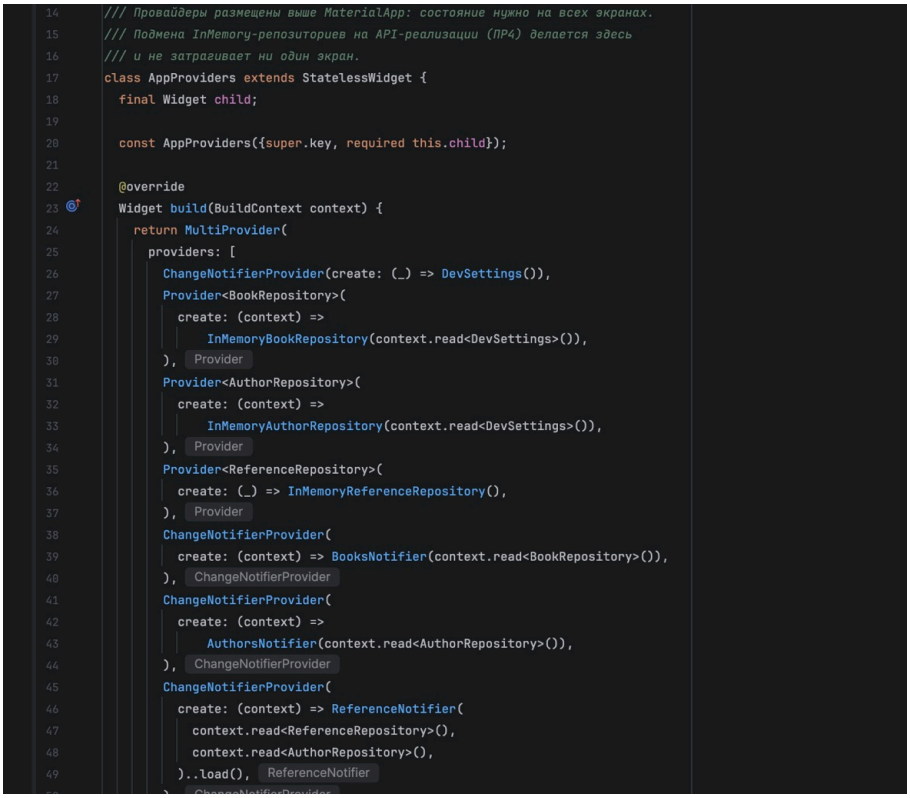


```dart
            ),
          ),
          GoRoute(
            path: '/books/:id',
            builder: (context, state) {
              final id = int.tryParse(state.pathParameters['id'] ?? '');
              return ChangeNotifierProvider(
                // Ключ пересоздаёт состояние при переходе к другой записи.
                key: ValueKey('book-$id'),
                create: (context) =>
                    BookDetailNotifier(context.read<BookRepository>(), id)
                      ..load(),
                child: const BookDetailScreen(),
              );
            },
          ),
          GoRoute(
            path: '/authors',
            builder: (context, state) => AuthorsScreen(
              query: AuthorQuery.fromParams(state.uri.queryParameters),
            ),
          ),
          GoRoute(
            path: '/authors/:id',
            builder: (context, state) {
              final id = int.tryParse(state.pathParameters['id'] ?? '');
              return ChangeNotifierProvider(
                key: ValueKey('author-$id'),
                create: (context) => AuthorDetailNotifier(
                  context.read<AuthorRepository>(),
                  id,
```

Рисунок 20 – Маршруты /authors и /authors/:id (router.dart)

13. Подключаем провайдеры выше MaterialApp, потому что состояние нужно на всех экранах. Репозитории объявлены через Provider, notifier через

ChangeNotifierProvider, а зависимости передаются через context.read. Подмена InMemory-репозиториев на API-реализации в ПР4 делается в этом месте.



```dart
/// Провайдеры размещены выше MaterialApp: состояние нужно на всех экранах.
/// Подмена InMemory-репозиториев на API-реализации (ПР4) делается здесь
/// и не затрагивает ни один экран.
class AppProviders extends StatelessWidget {
  final Widget child;

  const AppProviders({super.key, required this.child});

  @override
  Widget build(BuildContext context) {
    return MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => DevSettings()),
        Provider<BookRepository>(
          create: (context) =>
              InMemoryBookRepository(context.read<DevSettings>()),
        ), Provider
        Provider<AuthorRepository>(
          create: (context) =>
              InMemoryAuthorRepository(context.read<DevSettings>()),
        ), Provider
        Provider<ReferenceRepository>(
          create: (_) => InMemoryReferenceRepository(),
        ), Provider
        ChangeNotifierProvider(
          create: (context) => BooksNotifier(context.read<BookRepository>()),
        ), ChangeNotifierProvider
        ChangeNotifierProvider(
          create: (context) =>
              AuthorsNotifier(context.read<AuthorRepository>()),
        ), ChangeNotifierProvider
        ChangeNotifierProvider(
          create: (context) => ReferenceNotifier(
            context.read<ReferenceRepository>(),
            context.read<AuthorRepository>(),
          )..load(), ReferenceNotifier
```

Рисунок 21 – Провайдеры приложения (app.dart)

14. Запускаем приложение из Android Studio на устройстве Chrome (web). В консоли Run видно, что приложение запущено в браузере.

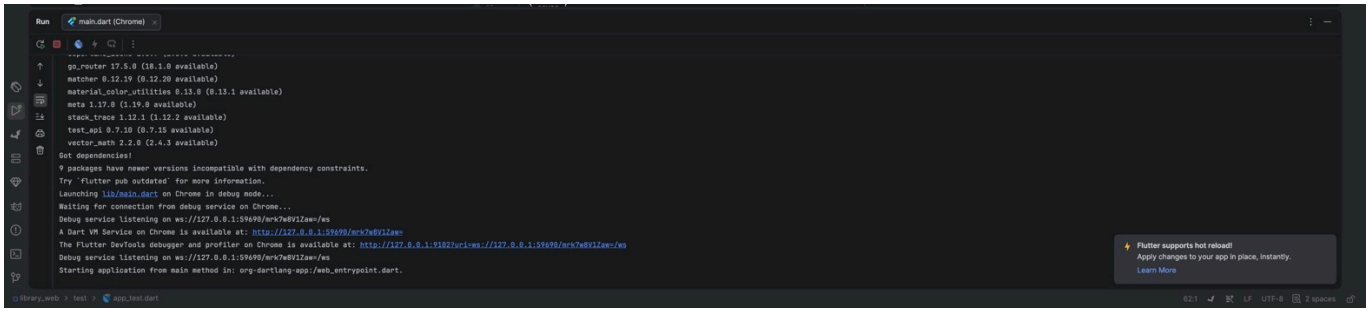


Рисунок 22 – Запуск приложения на Chrome (web) из Android Studio

15. Открываем каталог книг. Список выведен таблицей: 10 строк на странице, в заголовке колонки «Название» стрелка активной сортировки, под таблицей номер страницы, общее число записей и кнопки перехода на первую, предыдущую, следующую и последнюю страницы, а также выбор размера страницы.

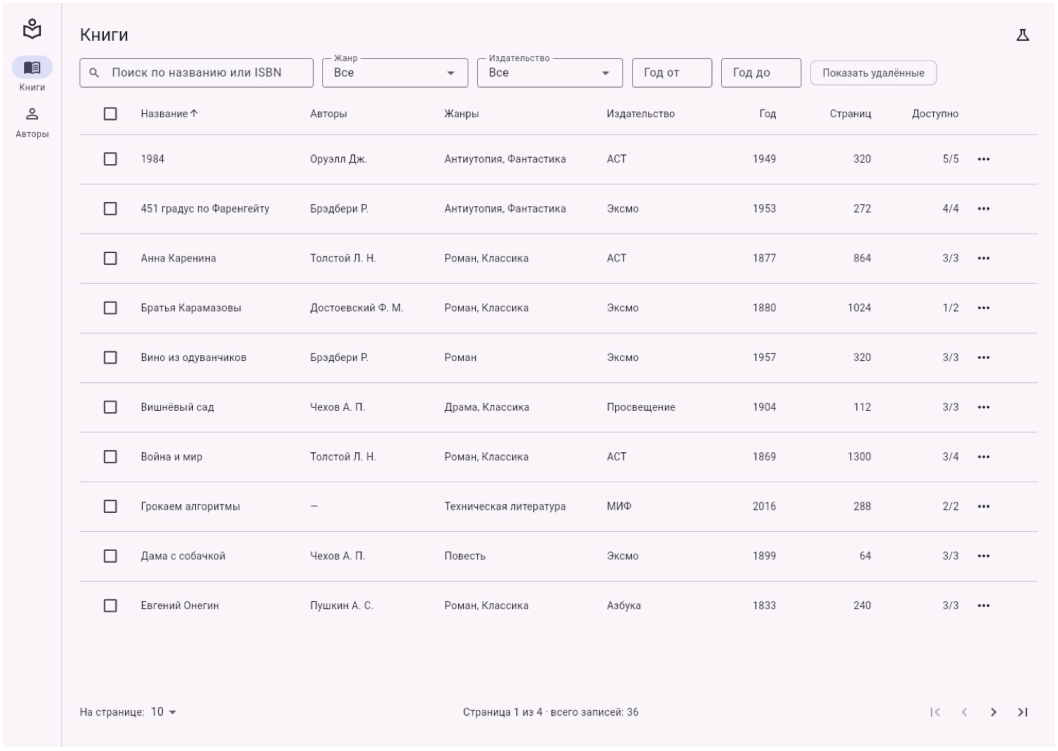


Рисунок 23 – Каталог книг в виде таблицы

16. Сортировка выполняется щелчком на заголовке колонки (название, год, число страниц). Повторный щелчок меняет направление. На рисунке книги отсортированы по году издания по убыванию, показана вторая страница.

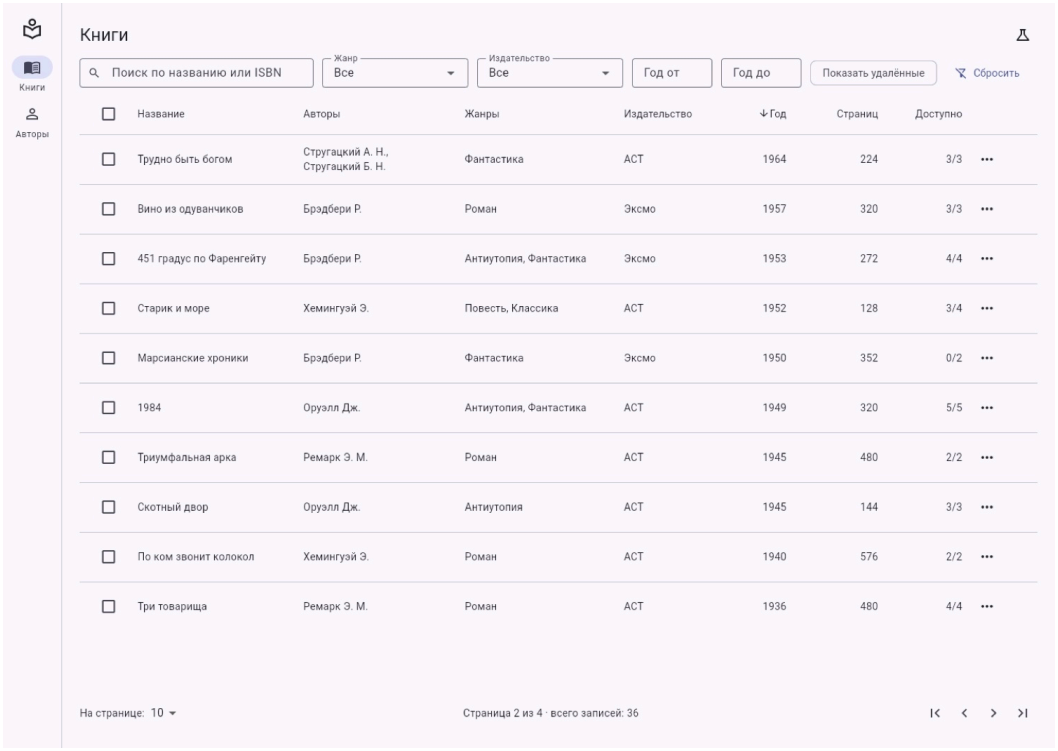


Рисунок 24 – Сортировка по году и вторая страница (sort=year,desc&page=2)

17. Размер страницы переключается значениями 10, 25 и 50. На рисунке выбрано 25 строк с сортировкой по числу страниц.

Рисунок 25 – Размер страницы 25 и сортировка по числу страниц

18. Проверяем поиск по названию и ISBN. Запрос «война» оставляет одну книгу, а адрес получает параметр search.

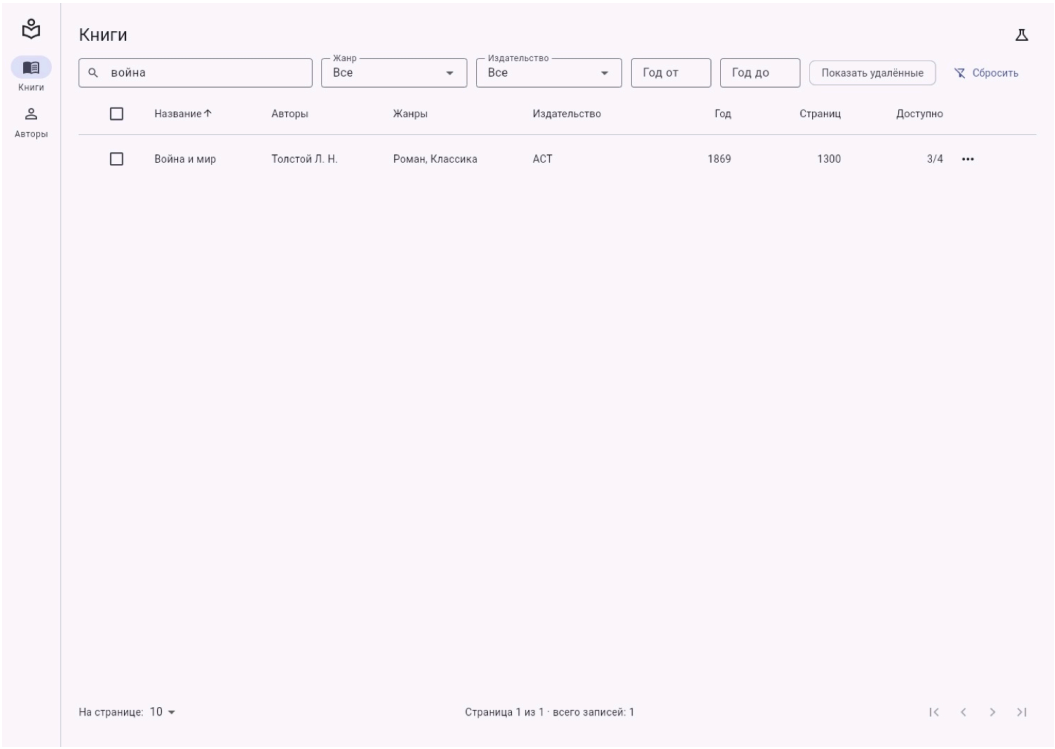


Рисунок 26 – Результат поиска по запросу «война»

19. Включаем фильтры по жанру, издательству и году издания одновременно. На рисунке выбраны жанр «Фантастика», издательство «АСТ» и год от 1960 с сортировкой по году. Фильтры комбинируются между собой и с поиском, найдено 4 книги.

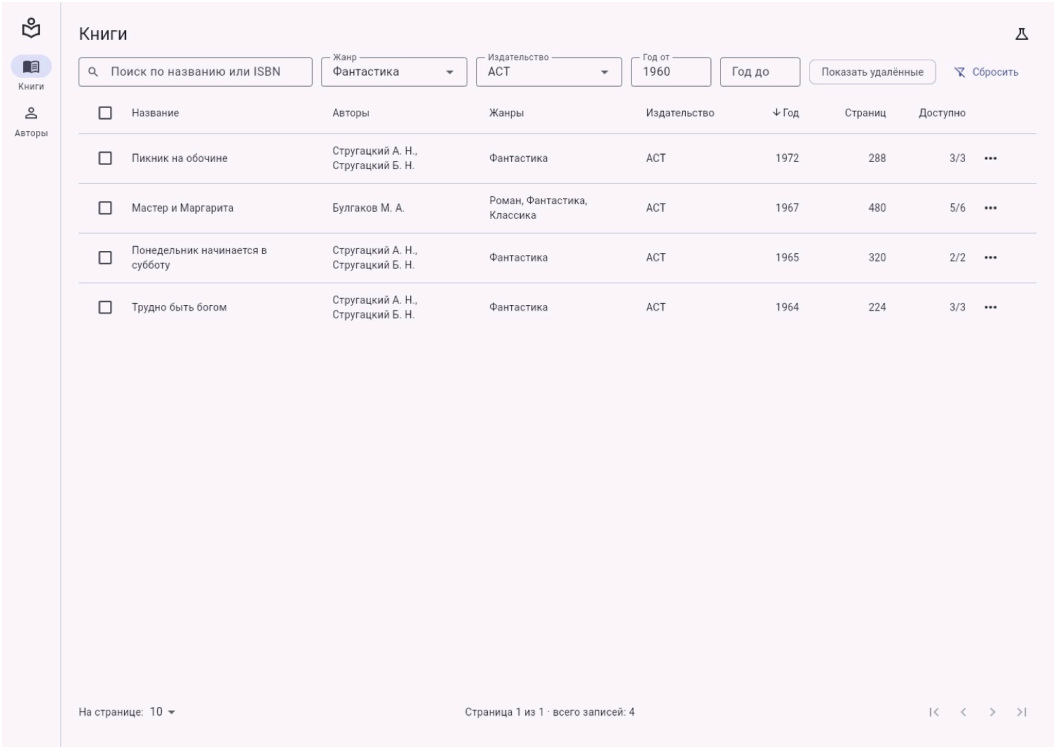


Рисунок 27 – Комбинация фильтров: жанр, издательство и год

20. Все условия отбора отражаются в адресе. На снимке окна браузера видны параметры genreId, publisherId, yearFrom и sort. Открытие такой ссылки в новой вкладке или после перезагрузки страницы восстанавливает тот же список с теми же фильтрами и теми же значениями в полях поиска и фильтров.

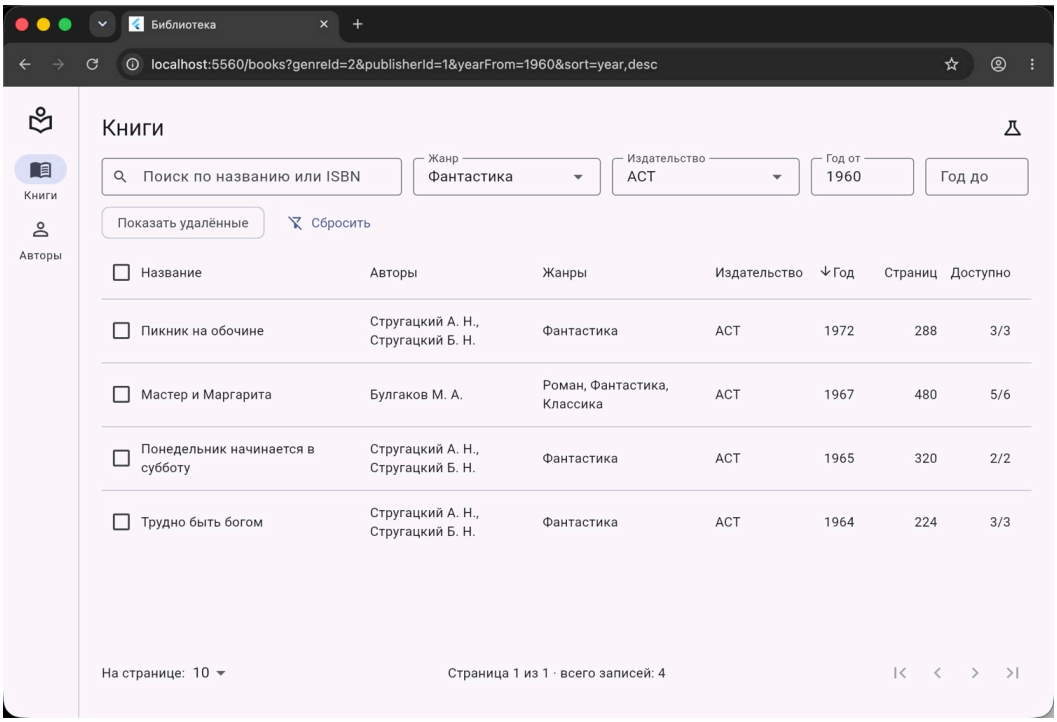


Рисунок 28 – Адресная строка с параметрами отбора

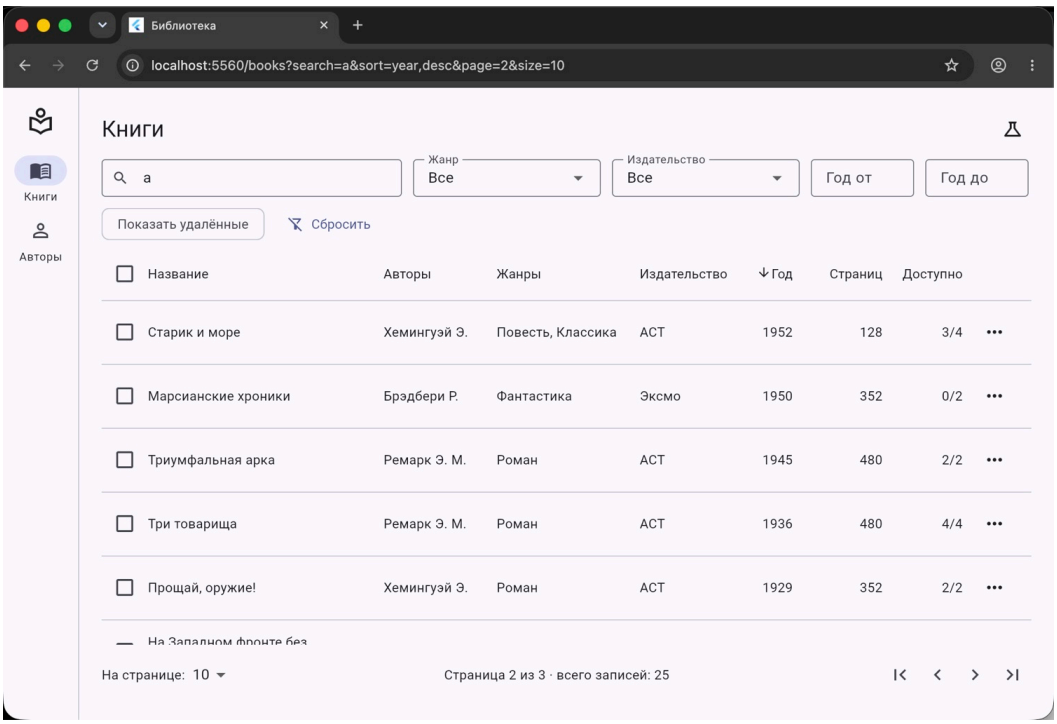


Рисунок 29 – Адрес с поиском, сортировкой и номером страницы

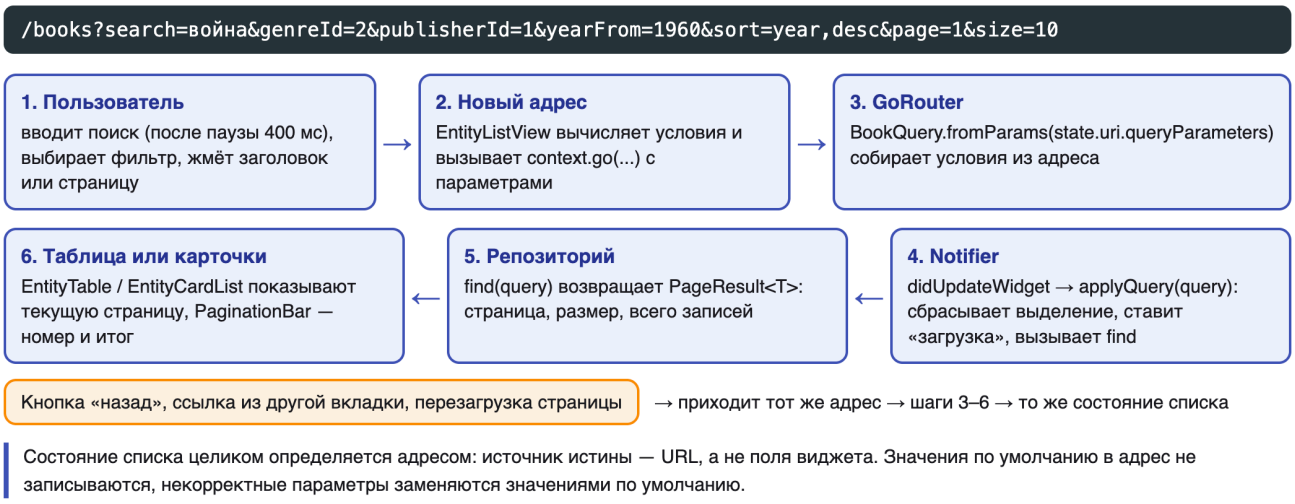


Рисунок 30 – Как условия списка превращаются в адрес и обратно

21. Проверяем кнопку «назад». Фильтр «Показать удалённые» добавляет в адрес параметр includeDeleted, а переход назад по истории браузера возвращает прежний адрес и прежний набор условий.

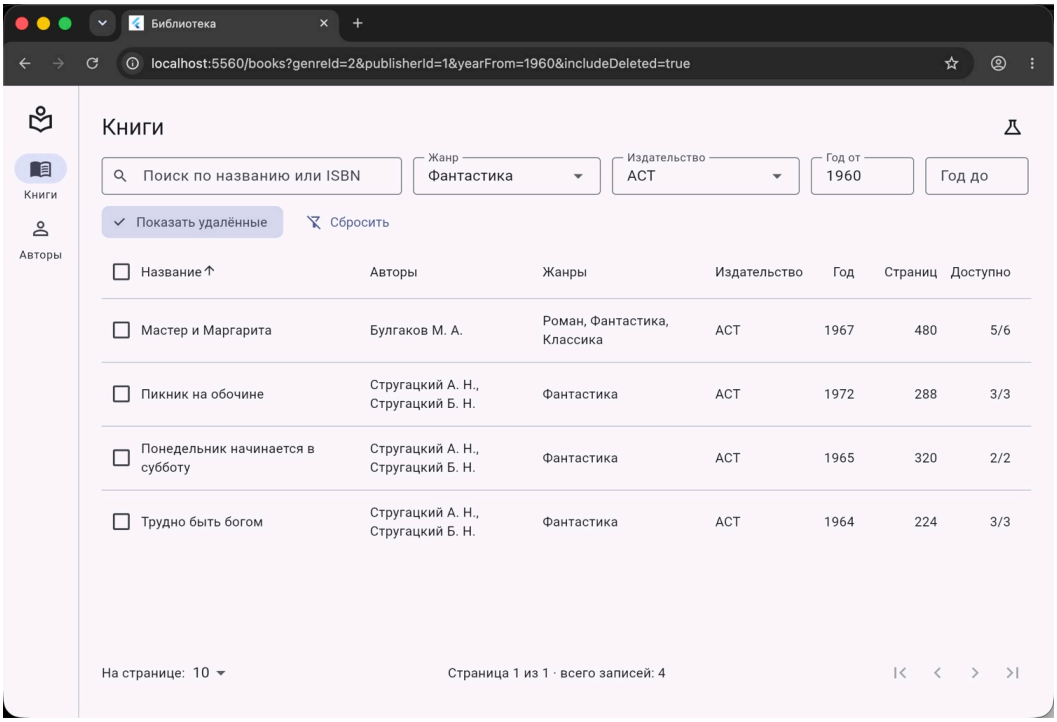


Рисунок 31 – После изменения условий в адресе появился includeDeleted=true

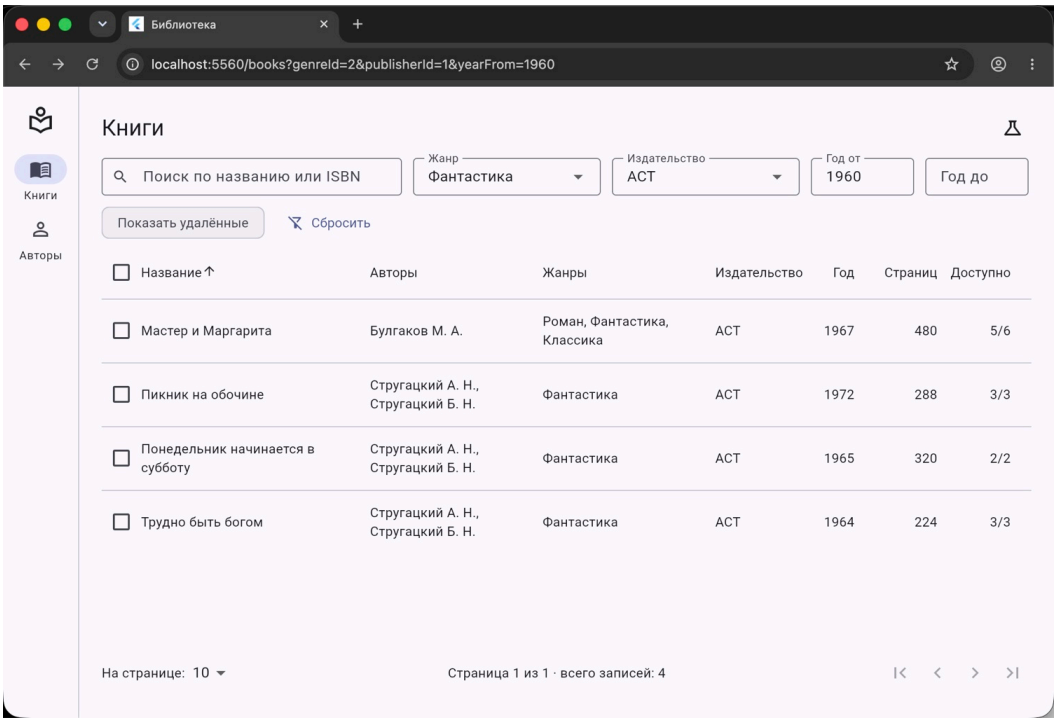


Рисунок 32 – После кнопки «назад» адрес и список вернулись к прежнему состоянию

22. Проверяем множественное удаление. Отмечаем несколько строк флажками: над таблицей появляется панель со счетчиком выбранных записей и кнопкой «Удалить выбранные».

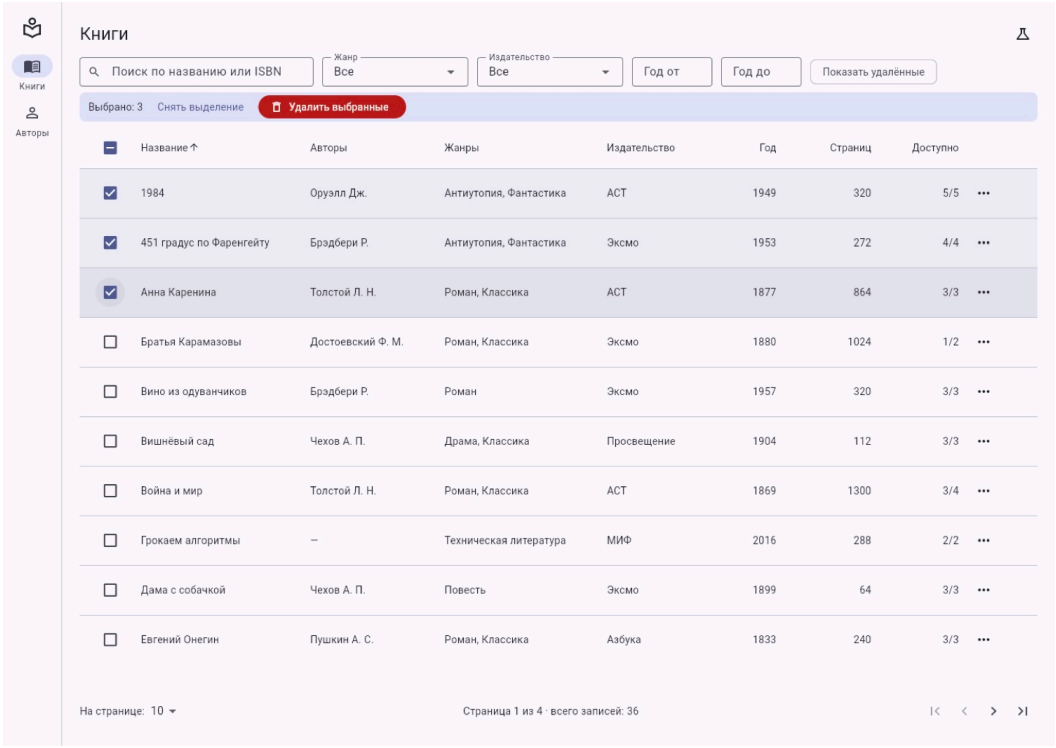


Рисунок 33 – Выделение трех строк и счетчик выбранного

Перед удалением приложение запрашивает подтверждение.

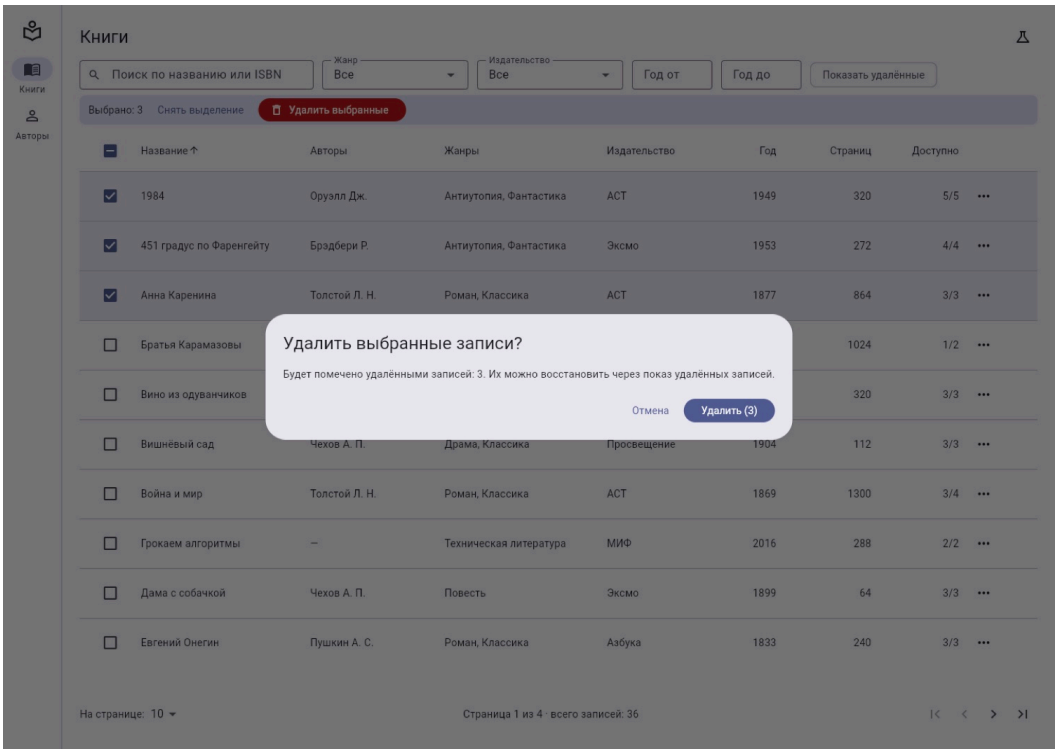


Рисунок 34 – Подтверждение удаления выбранных записей

После подтверждения записи получают deletedAt и пропадают из выборки: всего записей стало 33, показано сообщение «Удалено записей: 3».

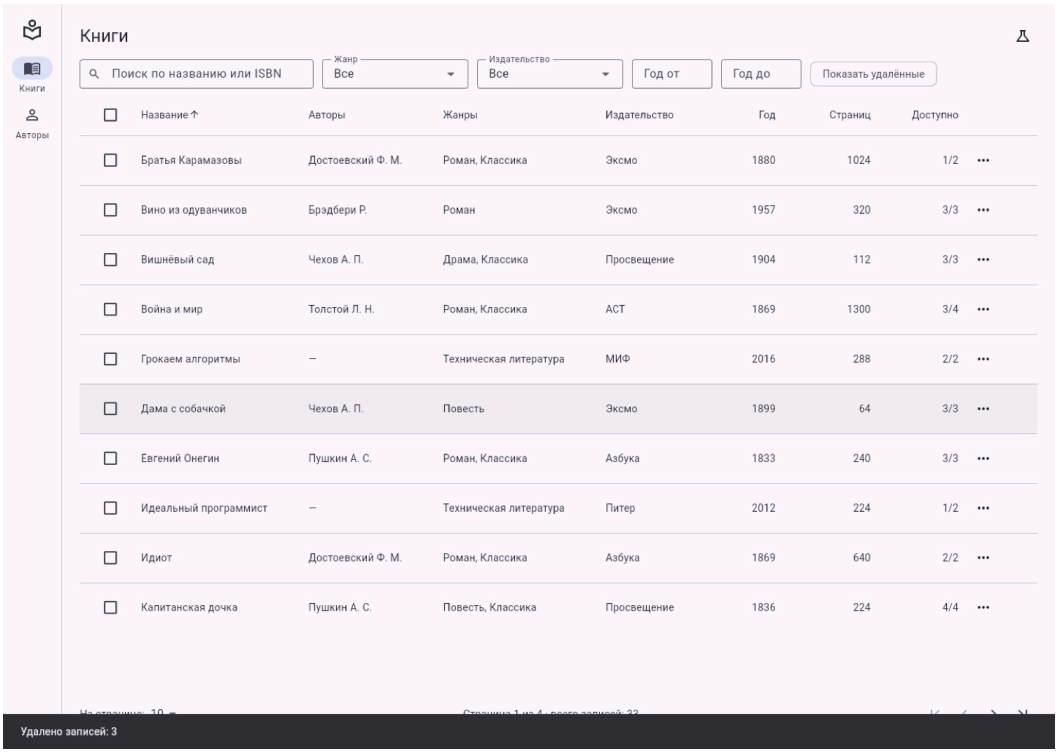


Рисунок 35 – Список после логического удаления трех книг

23. Переключатель «Показать удалённые» возвращает логически удалённые записи в выборку: они выделены серым и помечены значком «удалена». В меню строки доступны восстановление записи и физическое удаление.

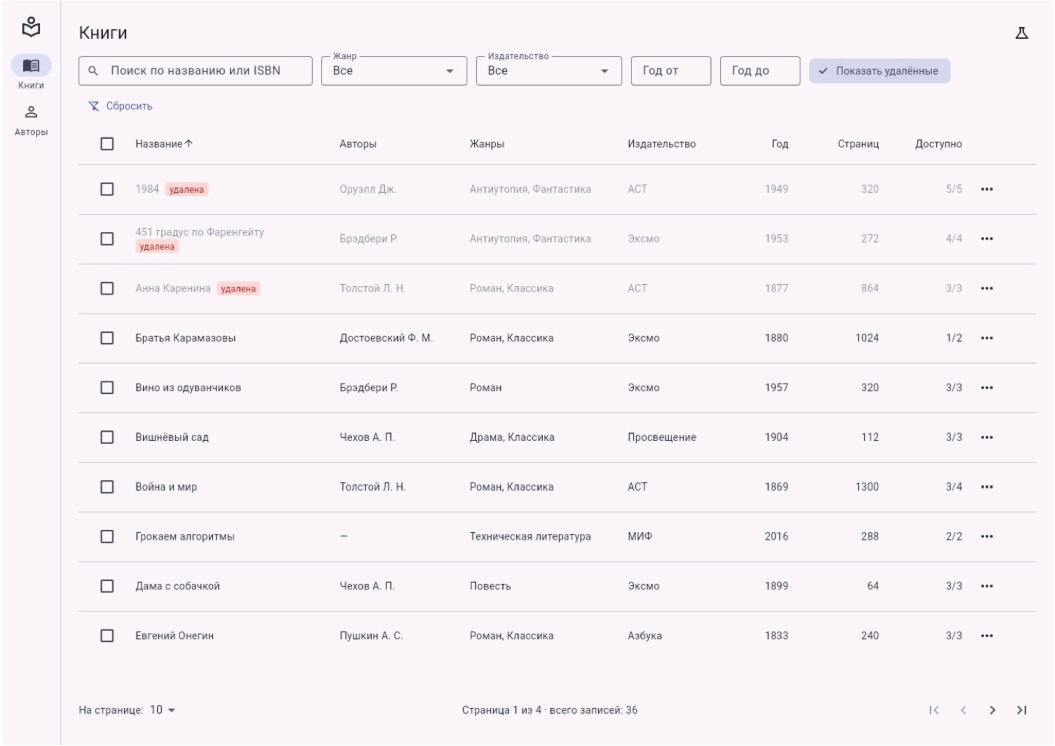


Рисунок 36 – Показ удалённых записей

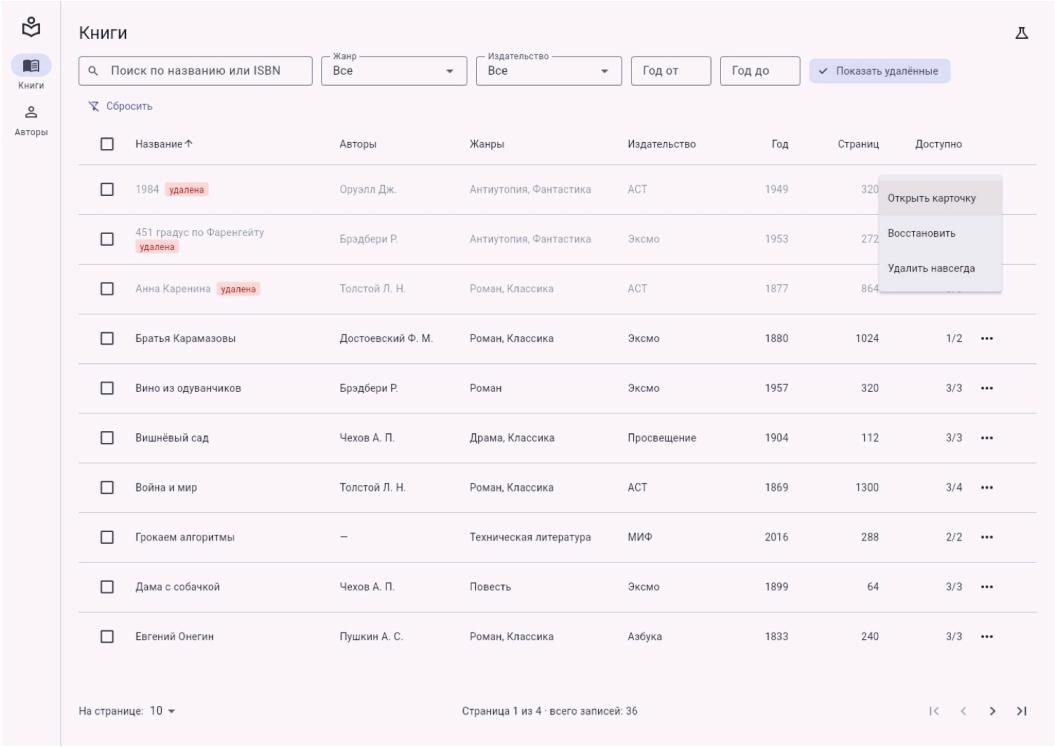


Рисунок 37 – Меню строки удалённой записи: восстановить и удалить навсегда

24. Обрабатываем четыре состояния экрана. Пустой результат (успех со списком нулевой длины) выглядит нейтрально и предлагает сбросить условия.

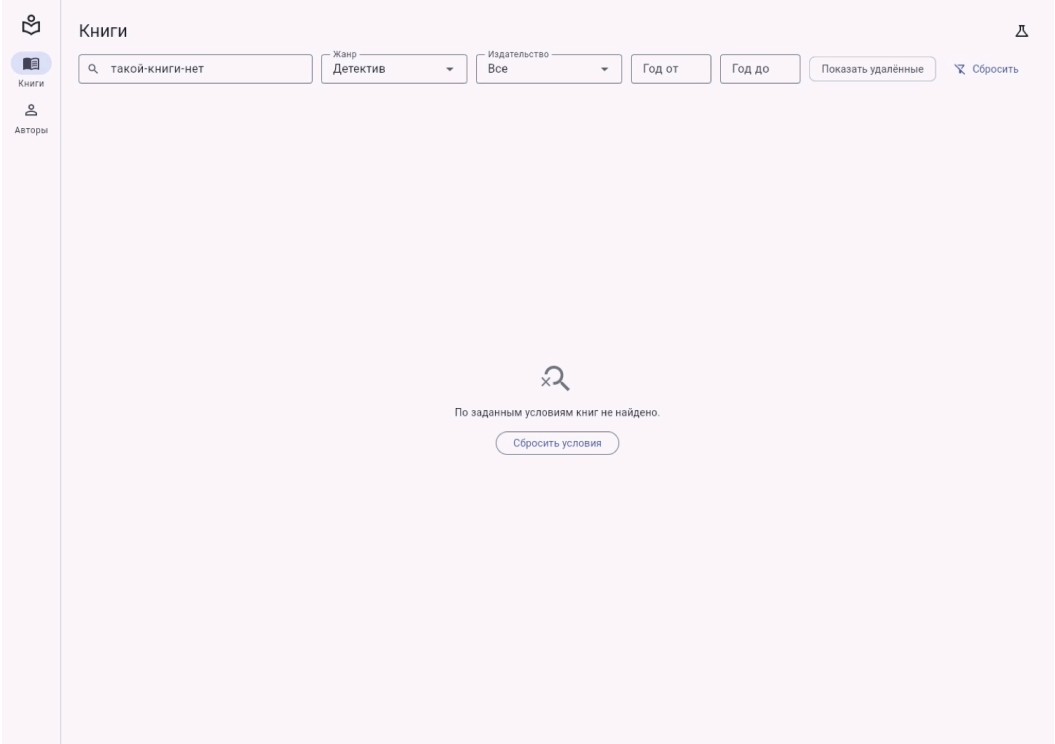


Рисунок 38 – Пустой результат: ничего не найдено

Ошибка загрузки выглядит иначе: красный значок, сообщение и кнопка «Повторить». Для проверки в заголовке списка есть меню с имитацией сбоя сервера и замедления ответа.

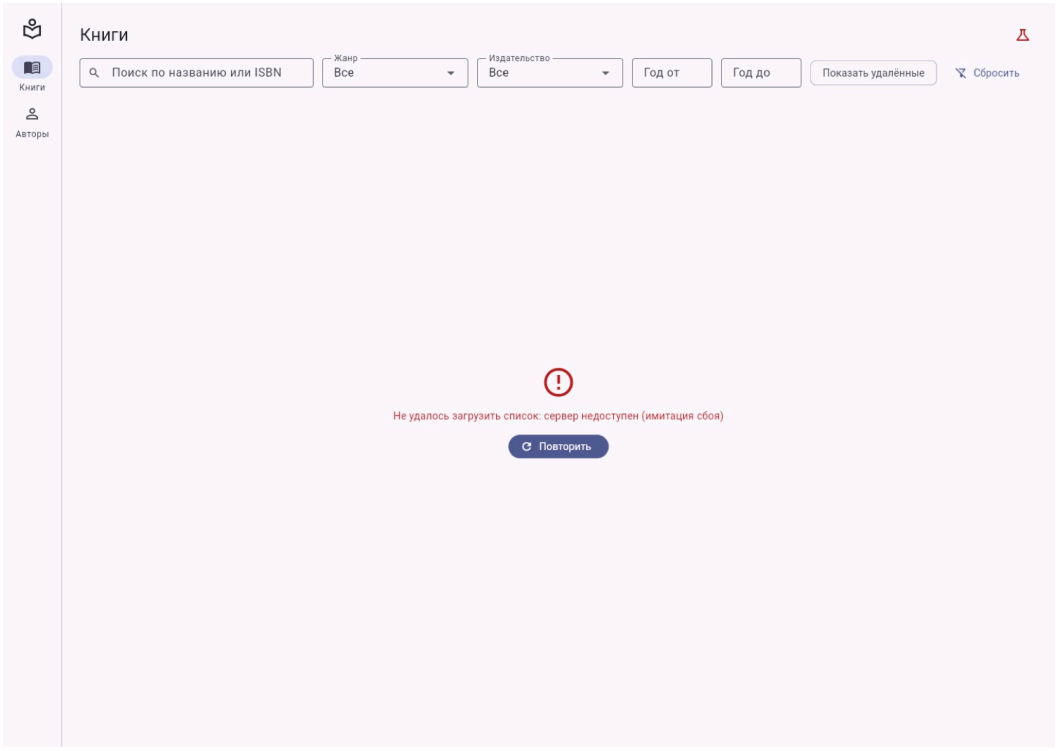


Рисунок 39 – Состояние ошибки с кнопкой повторной попытки

Загрузка: при первом обращении к списку показан индикатор, а при последующих запросах данные остаются на экране под полосой прогресса.

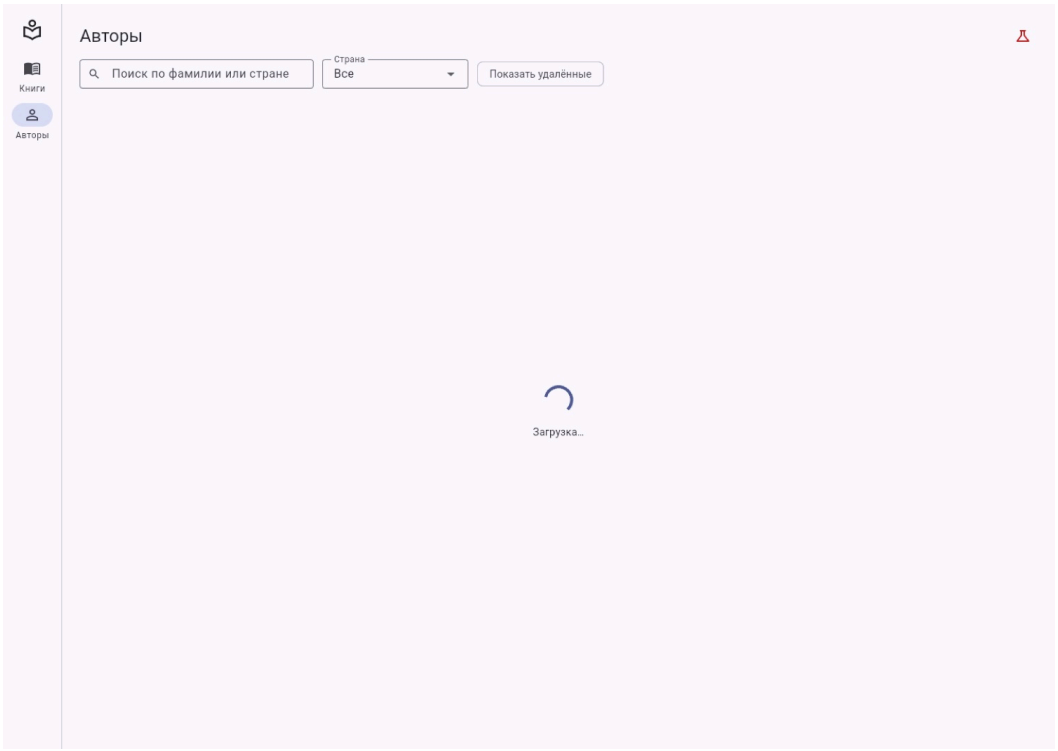


Рисунок 40 – Состояние загрузки (первая загрузка списка авторов)

Рисунок 41 – Повторная загрузка поверх уже показанных данных

25. Проверяем адаптивность. При ширине окна меньше 600 пикселей вместо таблицы показывается список карточек, а навигация переходит в нижнюю панель. Фильтры на узком окне прячутся в раскрываемую панель с кнопкой и счетчиком примененных условий, а сортировка вызывается отдельным меню.

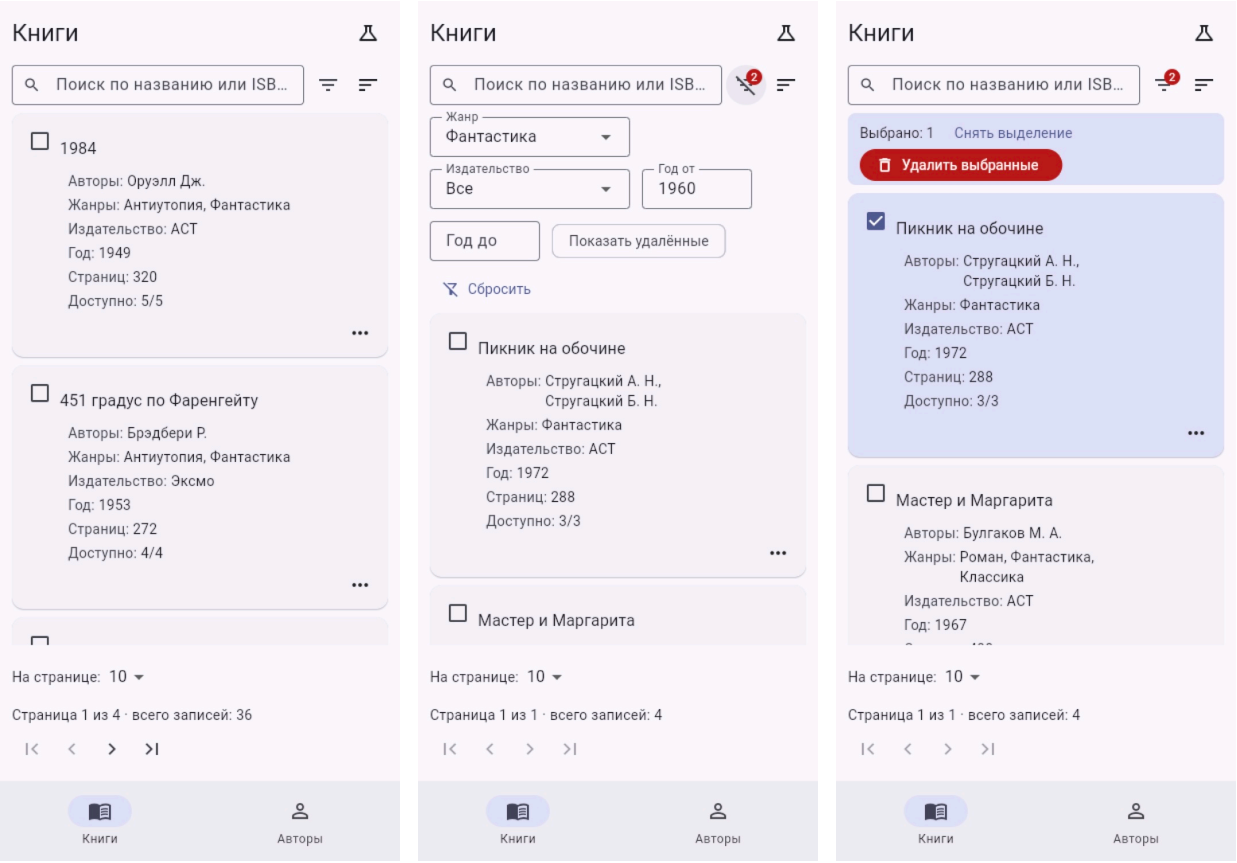


Рисунок 42 – Узкое окно: список карточек, панель фильтров и выделение карточки

26. Те же возможности есть у списка авторов: поиск по фамилии и стране, фильтр по стране, сортировка по ФИО, стране и году рождения. Таблица здесь та же самая EntityTable<T>, меняется только набор колонок.

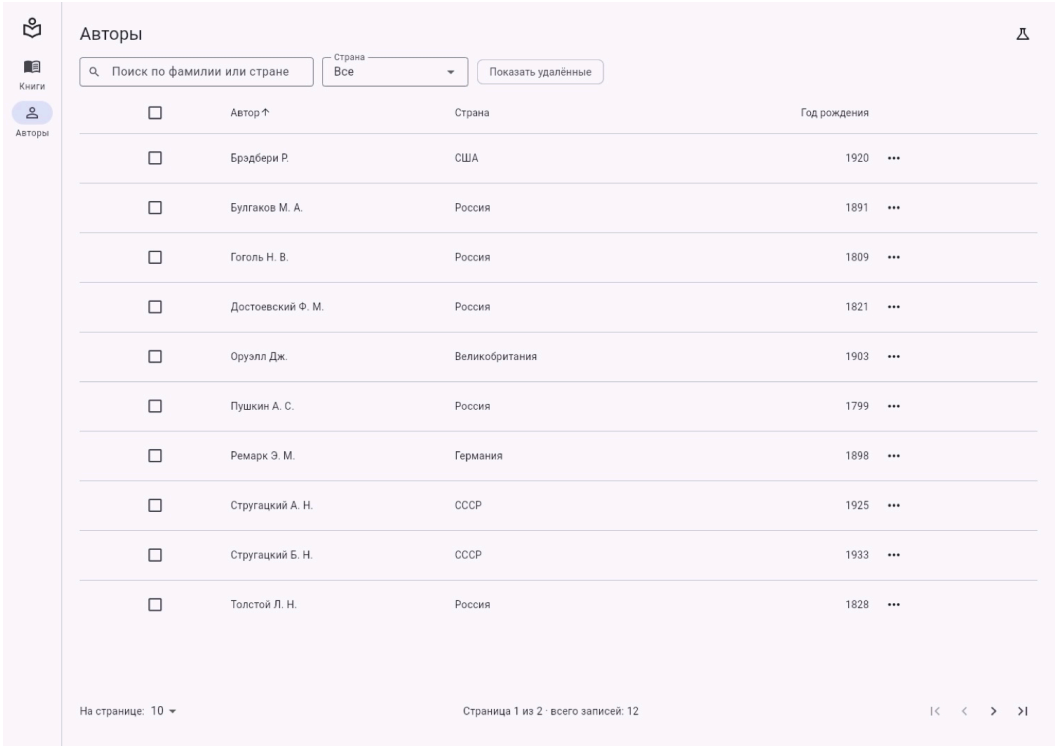


Рисунок 43 – Список авторов

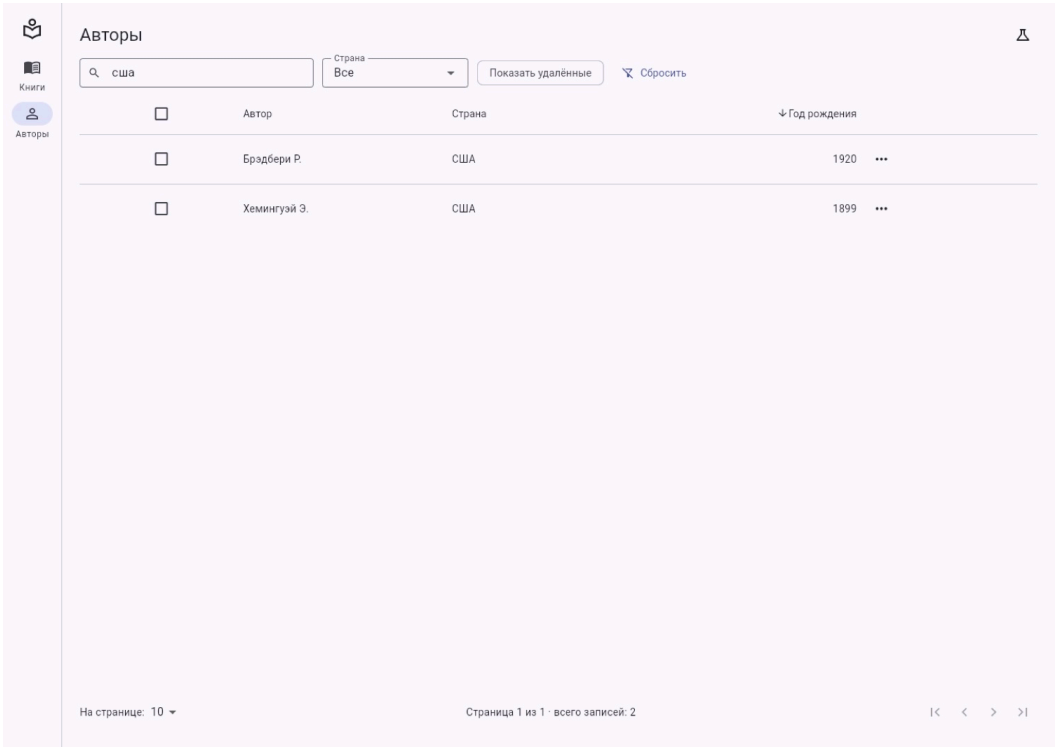


Рисунок 44 – Поиск авторов по стране и сортировка по году рождения

27. Из списка открываются карточки записей. Карточка книги показывает все поля, карточка автора ещё и список его книг. Адрес несуществующей записи не ломает экран: показывается сообщение.

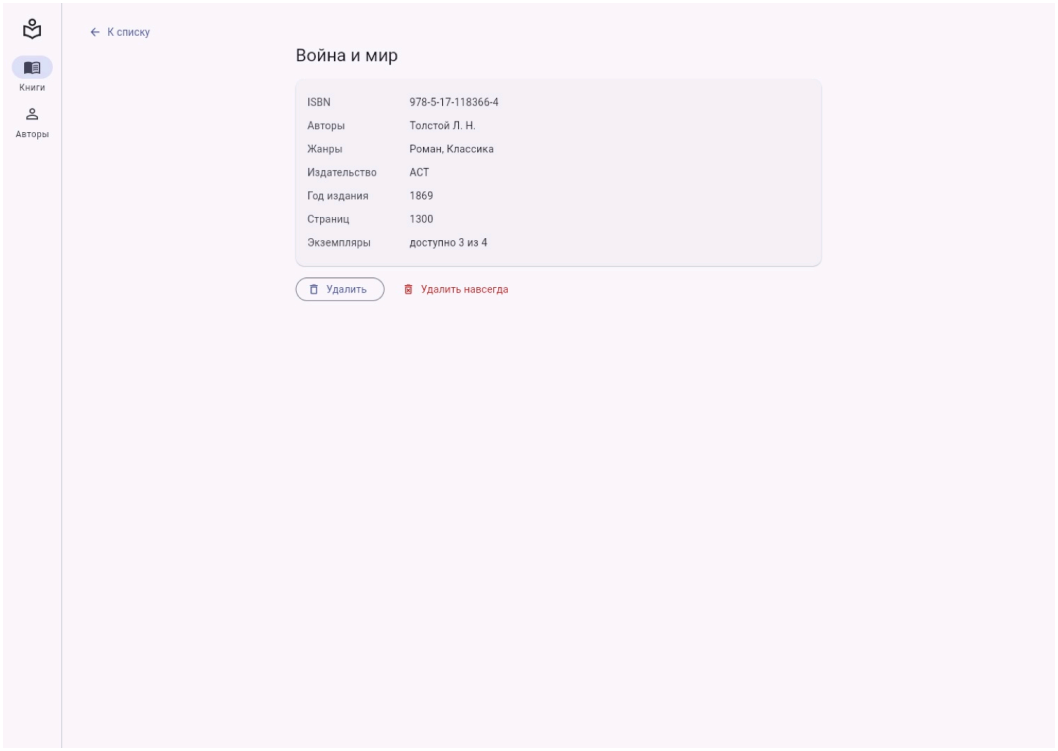


Рисунок 45 – Карточка книги

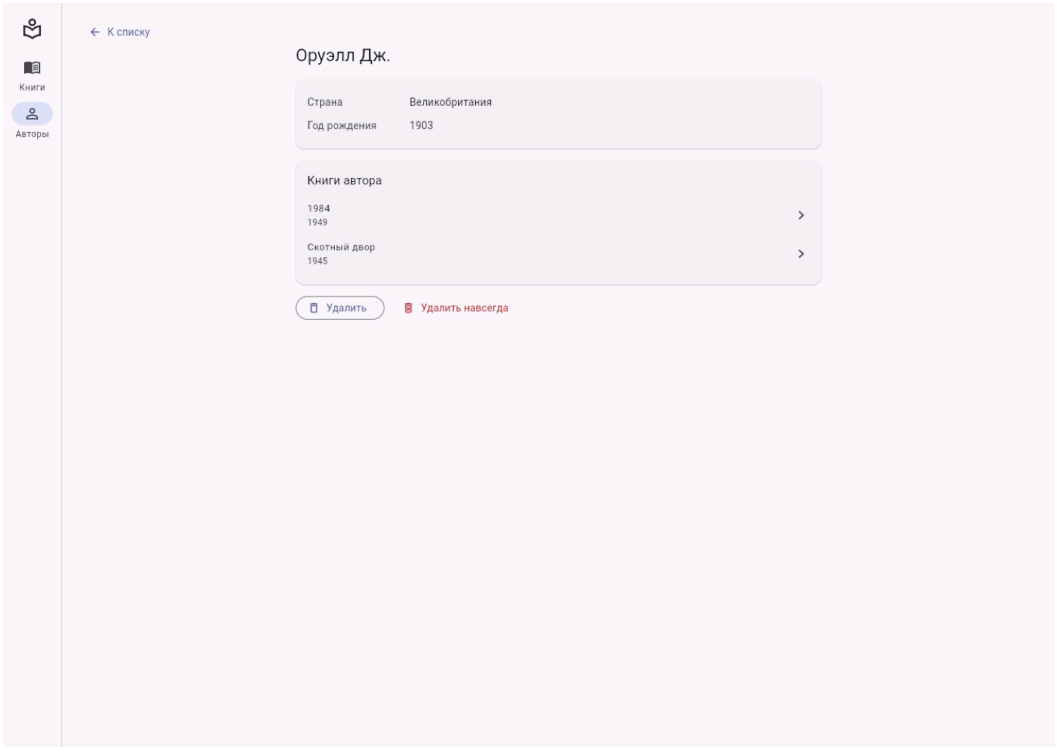


Рисунок 46 – Карточка автора со списком книг

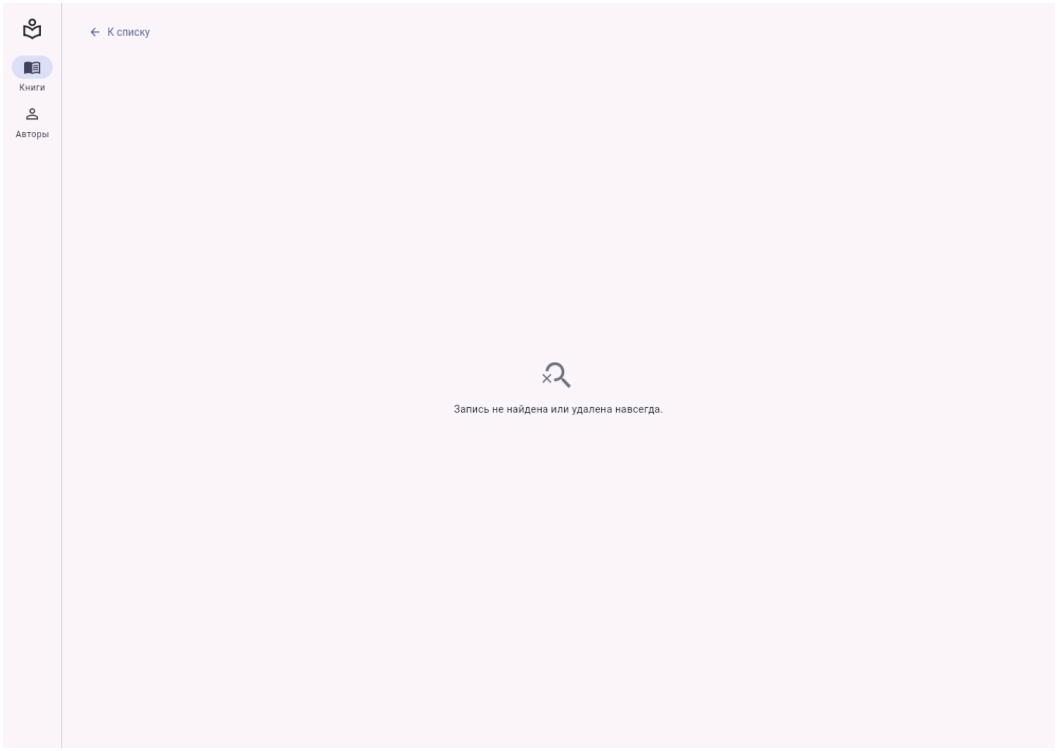
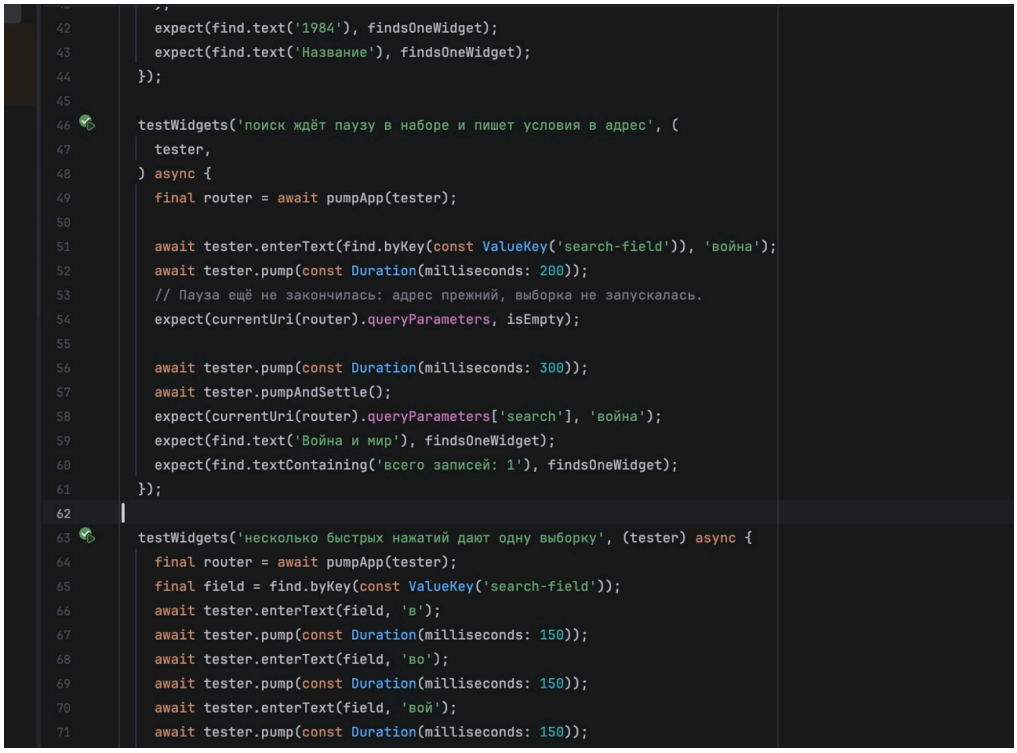


Рисунок 47 – Адрес несуществующей книги

28. Покрываем модели, репозитории, состояние и экраны тестами: круговое преобразование условий через адрес, поиск и фильтры, постраничную выдачу, исправленный deleteMany, отбрасывание устаревших ответов, задержку поиска,

пустой результат и ошибку, таблицу и карточки на разной ширине, выделение и удаление. Все 63 теста проходят.



```
    expect(find.text('1984'), findsOneWidget);
    expect(find.text('Название'), findsOneWidget);
  });

  testWidgets('поиск ждёт паузу в наборе и пишет условия в адрес', (
    tester,
  ) async {
    final router = await pumpApp(tester);

    await tester.enterText(find.byKey(const ValueKey('search-field')), 'война');
    await tester.pump(const Duration(milliseconds: 200));
    // Пауза ещё не закончилась: адрес прежний, выборка не запускалась.
    expect(currentUri(router).queryParameters, isEmpty);

    await tester.pump(const Duration(milliseconds: 300));
    await tester.pumpAndSettle();
    expect(currentUri(router).queryParameters['search'], 'война');
    expect(find.text('Война и мир'), findsOneWidget);
    expect(find.textContaining('всего записей: 1'), findsOneWidget);
  });

  testWidgets('несколько быстрых нажатий дают одну выборку', (tester) async {
    final router = await pumpApp(tester);
    final field = find.byKey(const ValueKey('search-field'));
    await tester.enterText(field, 'в');
    await tester.pump(const Duration(milliseconds: 150));
    await tester.enterText(field, 'во');
    await tester.pump(const Duration(milliseconds: 150));
    await tester.enterText(field, 'вой');
    await tester.pump(const Duration(milliseconds: 150));
```

Рисунок 48 – Фрагмент теста поиска с задержкой (app_test.dart)

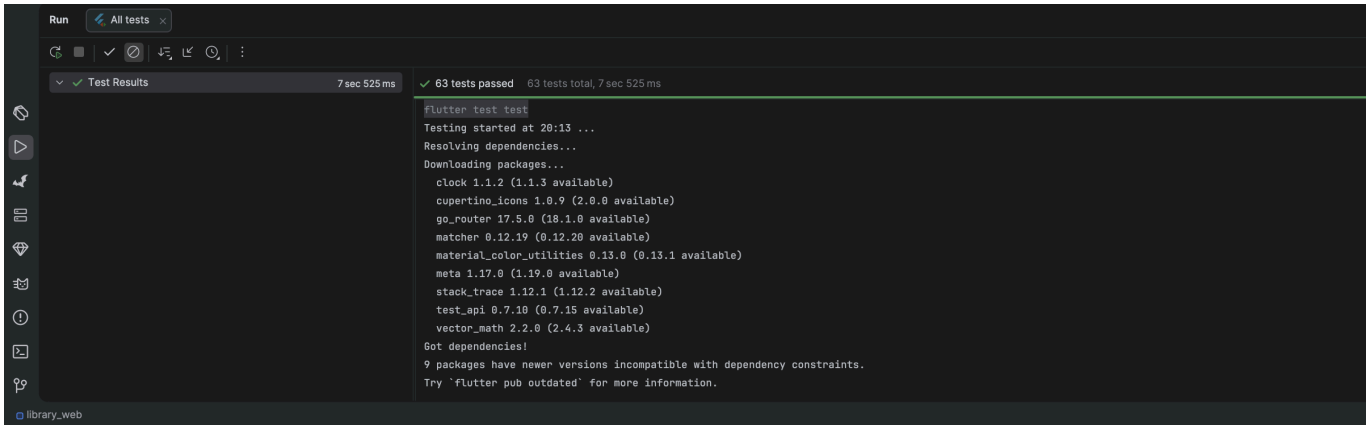


Рисунок 49 – Запуск тестов в Android Studio: 63 теста пройдено

29. Проверяем анализ и сборку. Команда flutter analyze не выдает замечаний, команда flutter build web собирает приложение без ошибок.

Результат работы:

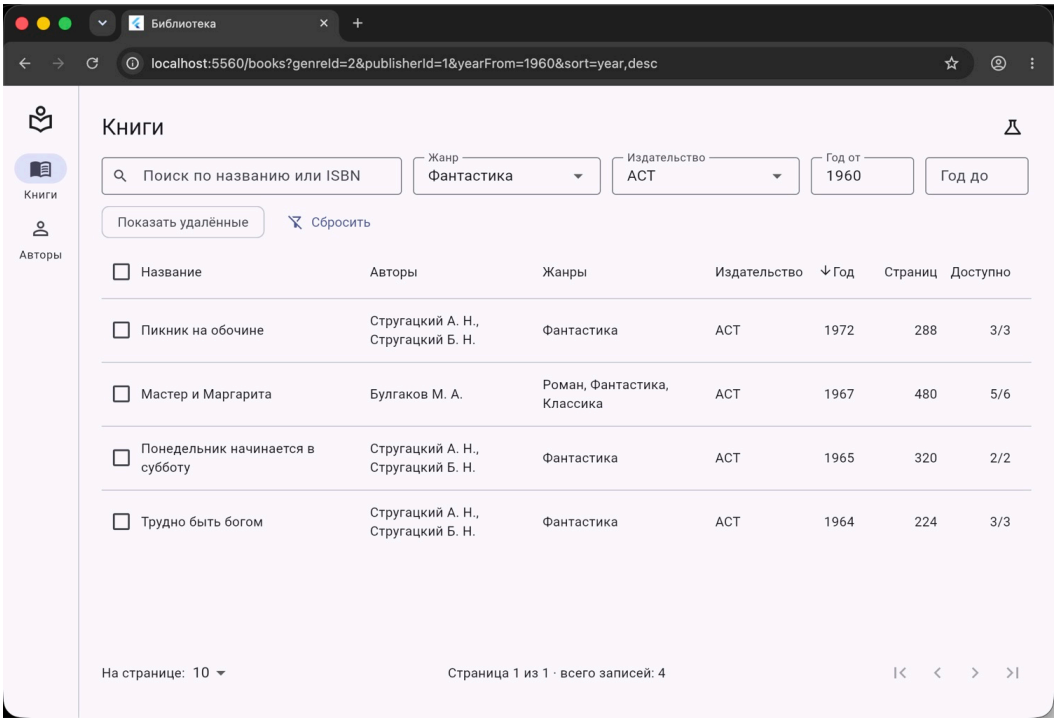


Рисунок 50 – Результат работы: таблица книг с примененными фильтрами и условиями в адресной строке

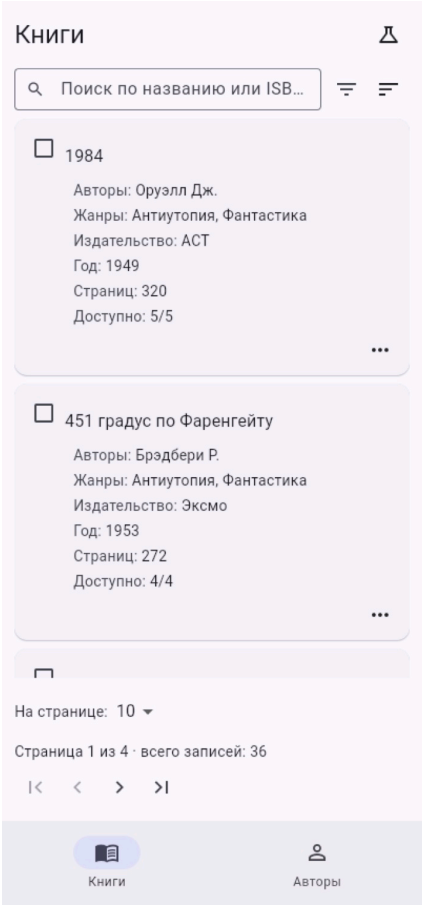


Рисунок 51 – Результат работы: тот же список на узком окне в виде карточек

Вывод: создан клиент библиотечной системы на Flutter Web с каталогом книг и авторов. Приложение разделено на слои models, repositories, state, screens и widgets, состояние управляется через provider. Реализованы поиск с задержкой, фильтрация по трем критериям, сортировка по заголовкам, постраничный вывод с выбором размера страницы, множественное, логическое и физическое удаление с восстановлением, а также четыре состояния экрана. Таблица вынесена в обобщённый виджет EntityTable<T>, поэтому для книг и авторов нет копий кода, а новые списки добавляются описанием колонок. Условия отбора хранятся в адресе, поэтому ссылку на отфильтрованный список можно открыть в другой вкладке, а кнопка «назад» возвращает прежний набор условий. Найдена и исправлена ошибка в deleteMany: проверка !b[i].isDeleted не компилировалась, нужна !b.isDeleted.

ПРАКТИЧЕСКАЯ РАБОТА №3

Формы, валидация и связанные сущности

Цель работы: реализовать полный набор операций создания и изменения записей с многоуровневой валидацией, научиться работать со связями между сущностями в интерфейсе и сохранять данные между перезагрузками страницы. Работа продолжает проект library_web из практической работы 2: к каталогу книг и авторов добавлены жанры, издательства и читатели с билетами, формы создания и изменения, проверки уникальности и защита от удаления связанных записей.

Ход работы:

1. Продолжаем проект library_web, а не создаём новый. В pubspec.yaml добавляем shared_preferences: на web он пишет данные в localStorage браузера. Число сущностей доводим до пяти: Book, Author, Genre, Publisher и Reader с читательским билетом LibraryCard. Поля моделей совпадают с контрактом учебного API (раздел 4), чтобы в ПР4 те же модели получали данные с сервера.

2. Описываем схему данных. Логическая схема показывает все три типа связей: «один ко многим» между издательством и книгой, «многие ко многим» между книгой и автором, книгой и жанром (через таблицы связи BookAuthor и BookGenre), «один к одному» между читателем и билетом.

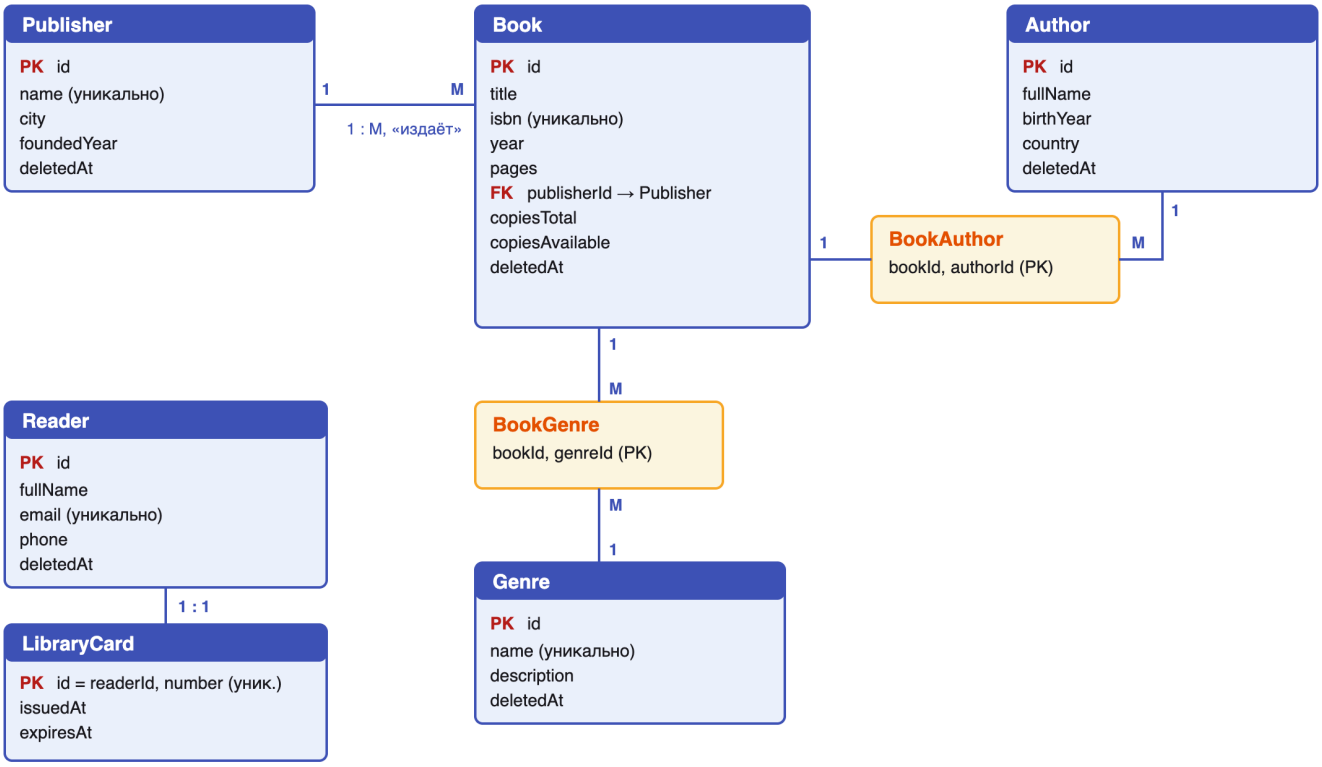


Рисунок 1 – Логическая схема данных

Физическая схема отражает то, как данные лежат в браузере. Каждая коллекция хранится одной строкой JSON под ключом с версией формата (books_v2, authors_v2, genres_v2, publishers_v2, readers_v2). Связь «многие ко многим» хранится списками идентификаторов authorIds и genreIds внутри книги, связь с издательством внешним ключом publisherId, а билет вложен в читателя объектом card.

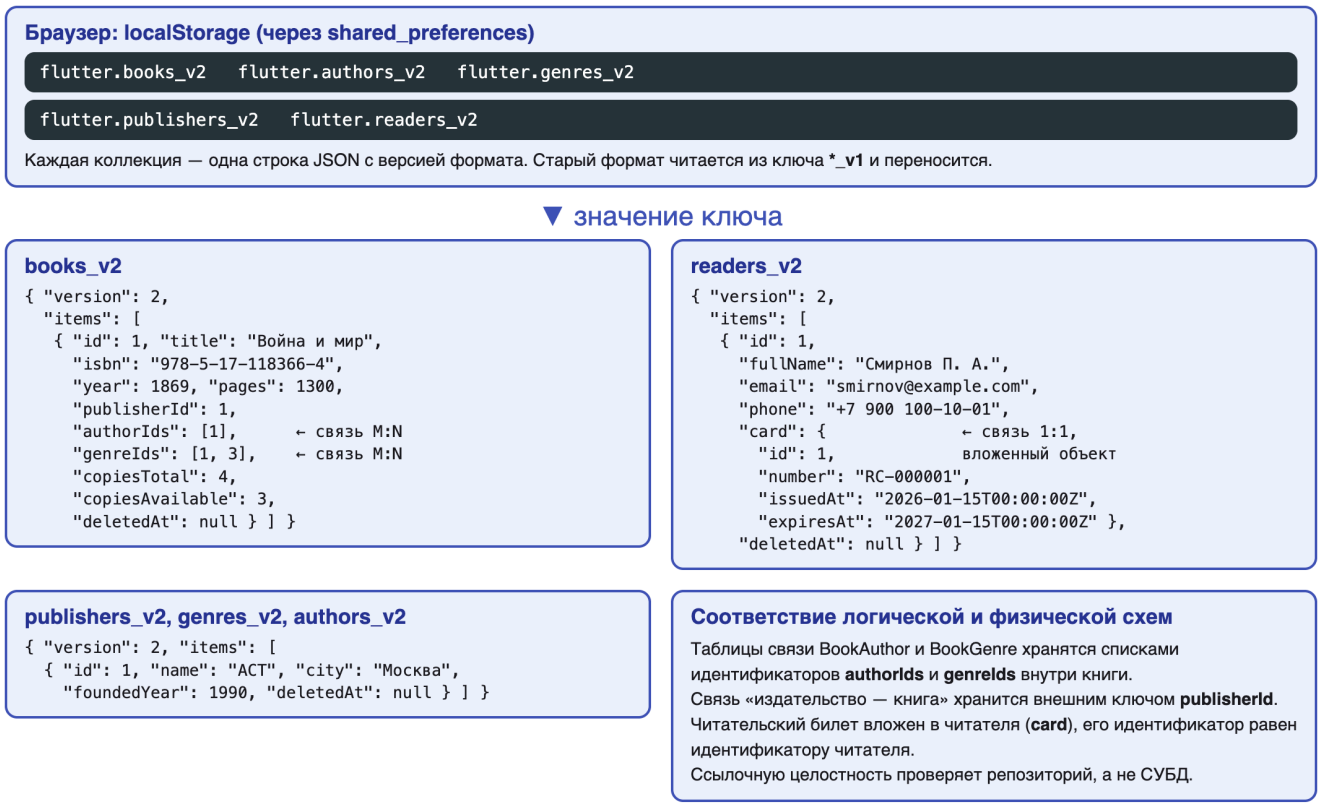


Рисунок 2 – Физическая схема данных (localStorage)

Таблица 1 – Словарь данных: Book (книга)

Поле	Тип	Обязательное	Описание
id	int	да	Идентификатор, присваивается хранилищем
title	String	да	Название книги
isbn	String	да	ISBN-13, уникален среди неудалённых книг
year	int	да	Год издания
pages	int	да	Число страниц
publisherId	int	да	Внешний ключ на Publisher (многие к одному)
authorIds	List<int>	нет	Идентификаторы авторов (многие ко многим)
genreIds	List<int>	да	Идентификаторы жанров (многие ко многим)
copiesTotal	int	да	Всего экземпляров
copiesAvailable	int	да	Доступно к выдаче, не больше copiesTotal
deletedAt	DateTime?	нет	Момент логического удаления, null у активных

Таблица 2 – Словарь данных: Author (автор)

Поле	Тип	Обязательное	Описание
id	int	да	Идентификатор
fullName	String	да	Фамилия и инициалы
birthYear	int	да	Год рождения
country	String	да	Страна
deletedAt	DateTime?	нет	Момент логического удаления

Таблица 3 – Словарь данных: Genre (жанр)

Поле	Тип	Обязательное	Описание
id	int	да	Идентификатор
name	String	да	Название жанра, уникально
description	String	нет	Описание жанра
deletedAt	DateTime?	нет	Момент логического удаления

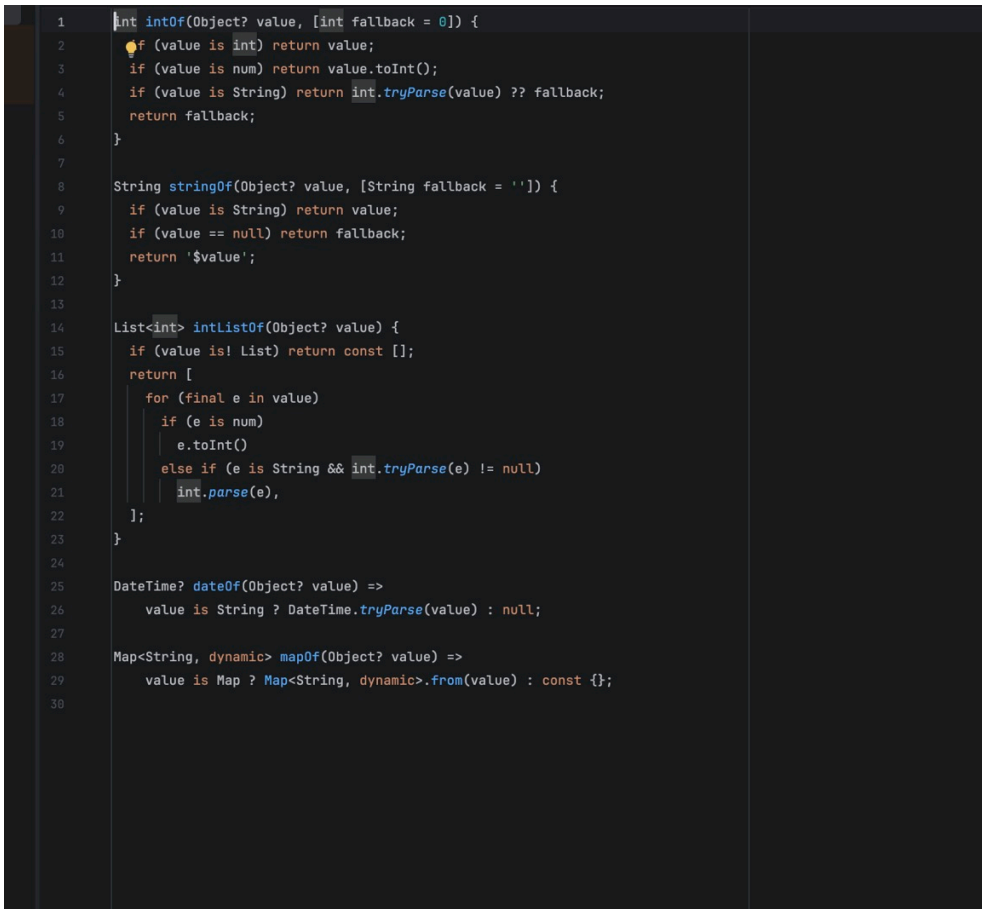
Таблица 4 – Словарь данных: Publisher (издательство)

Поле	Тип	Обязательное	Описание
id	int	да	Идентификатор
name	String	да	Название, уникально
city	String	да	Город
foundedYear	int	да	Год основания
deletedAt	DateTime?	нет	Момент логического удаления

Таблица 5 – Словарь данных: Reader (читатель) и LibraryCard (билет)

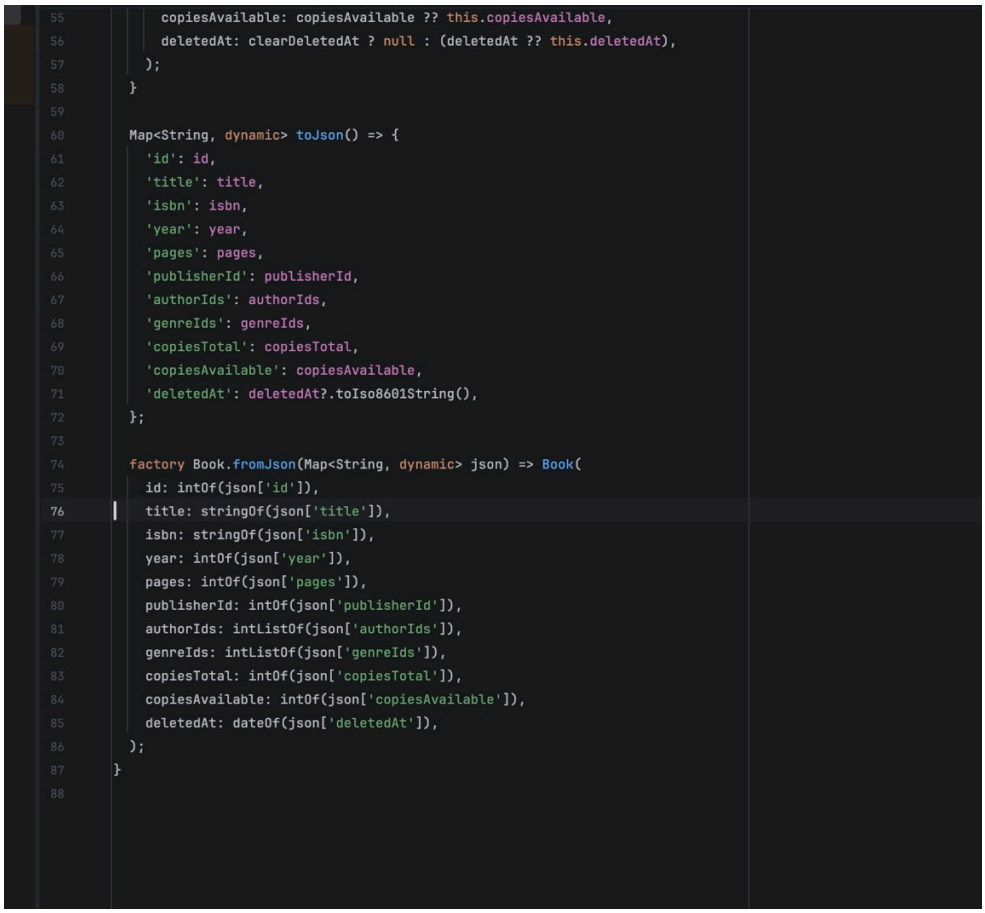
Поле	Тип	Обязательное	Описание
Reader.id	int	да	Идентификатор читателя
Reader.fullName	String	да	Фамилия и инициалы
Reader.email	String	да	Адрес почты, уникален
Reader.phone	String	нет	Телефон
Reader.card	LibraryCard	да	Читательский билет (один к одному), вложен в читателя
Reader.deletedAt	DateTime?	нет	Момент логического удаления
LibraryCard.id	int	да	Идентификатор билета, равен идентификатору читателя
LibraryCard.number	String	да	Номер вида RC-000001, уникален
LibraryCard.issuedAt	DateTime	да	Дата выдачи
LibraryCard.expiresAt	DateTime	да	Дата окончания, позже даты выдачи

3. Добавляем сериализацию. В каждой модели есть toJson и fromJson. Разбор из внешнего источника самое частое место, где приложение падает: поля может не быть, оно может быть null или другого типа. Поэтому вместо приведения json['field'] as int используем небольшие функции intOf, stringOf, intListOf и dateOf: они возвращают значение по умолчанию и никогда не бросают исключение.



```dart
int intOf(Object? value, [int fallback = 0]) {
  if (value is int) return value;
  if (value is num) return value.toInt();
  if (value is String) return int.tryParse(value) ?? fallback;
  return fallback;
}

String stringOf(Object? value, [String fallback = '']) {
  if (value is String) return value;
  if (value == null) return fallback;
  return '$value';
}

List<int> intListOf(Object? value) {
  if (value is! List) return const [];
  return [
    for (final e in value)
      if (e is num)
        e.toInt()
      else if (e is String && int.tryParse(e) != null)
        int.parse(e),
  ];
}

DateTime? dateOf(Object? value) =>
    value is String ? DateTime.tryParse(value) : null;

Map<String, dynamic> mapOf(Object? value) =>
    value is Map ? Map<String, dynamic>.from(value) : const {};
```

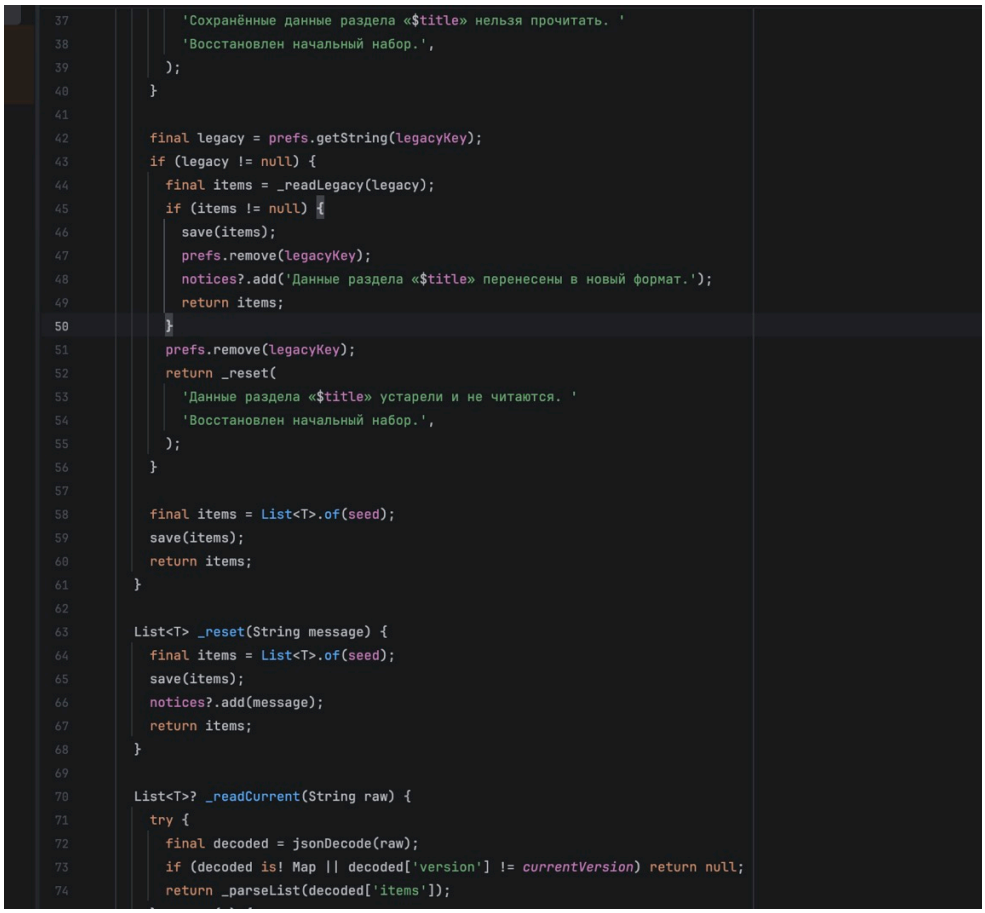
Рисунок 3 – Функции устойчивого разбора данных (json_utils.dart)



```
      copiesAvailable: copiesAvailable ?? this.copiesAvailable,
      deletedAt: clearDeletedAt ? null : (deletedAt ?? this.deletedAt),
    );
  }

  Map<String, dynamic> toJson() => {
    'id': id,
    'title': title,
    'isbn': isbn,
    'year': year,
    'pages': pages,
    'publisherId': publisherId,
    'authorIds': authorIds,
    'genreIds': genreIds,
    'copiesTotal': copiesTotal,
    'copiesAvailable': copiesAvailable,
    'deletedAt': deletedAt?.toIso8601String(),
  };

  factory Book.fromJson(Map<String, dynamic> json) => Book(
    id: intOf(json['id']),
    title: stringOf(json['title']),
    isbn: stringOf(json['isbn']),
    year: intOf(json['year']),
    pages: intOf(json['pages']),
    publisherId: intOf(json['publisherId']),
    authorIds: intListOf(json['authorIds']),
    genreIds: intListOf(json['genreIds']),
    copiesTotal: intOf(json['copiesTotal']),
    copiesAvailable: intOf(json['copiesAvailable']),
    deletedAt: dateOf(json['deletedAt']),
  );
}
```

Рисунок 4 – Методы toJson и fromJson модели Book

4. Выносим хранение в класс JsonStore. Формат версионируется: данные лежат под ключом books_v2 в виде объекта с полем version и списком items. Если находится ключ старого формата books_v1 (голый список), данные читаются, записываются в новом формате, а старый ключ удаляется. Если данные нечитаемы или версия неизвестна, коллекция заменяется начальным набором. В обоих случаях сообщение попадает в StorageNotices, а приложение не падает.

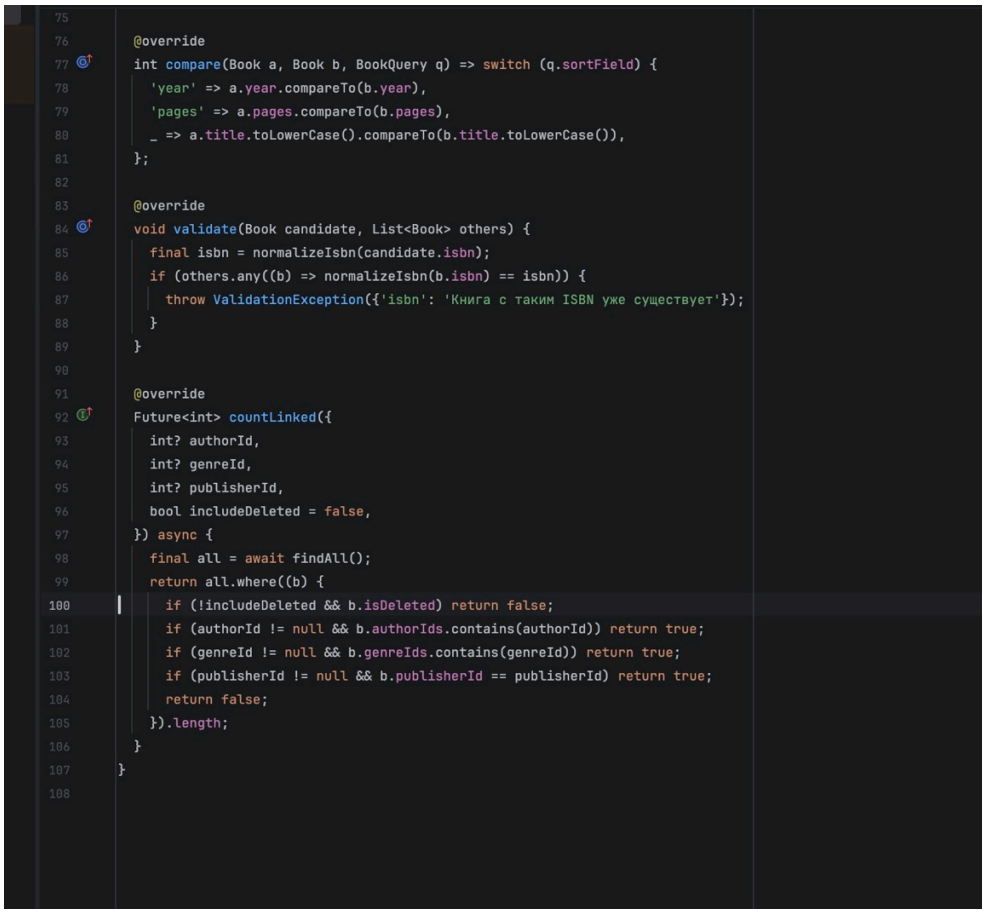


```
        'Сохранённые данные раздела «$title» нельзя прочитать. '
        'Восстановлен начальный набор.',
      );
    }

    final legacy = prefs.getString(legacyKey);
    if (legacy != null) {
      final items = _readLegacy(legacy);
      if (items != null) {
        save(items);
        prefs.remove(legacyKey);
        notices?.add('Данные раздела «$title» перенесены в новый формат.');
        return items;
      }
      prefs.remove(legacyKey);
      return _reset(
        'Данные раздела «$title» устарели и не читаются. '
        'Восстановлен начальный набор.',
      );
    }

    final items = List<T>.of(seed);
    save(items);
    return items;
  }

  List<T> _reset(String message) {
    final items = List<T>.of(seed);
    save(items);
    notices?.add(message);
    return items;
  }

  List<T>? _readCurrent(String raw) {
    try {
      final decoded = jsonDecode(raw);
      if (decoded is! Map || decoded['version'] != currentVersion) return null;
      return _parseList(decoded['items']);
```

Рисунок 5 – Перенос старого формата и сброс данных (json_store.dart)

5. Общую логику репозиториев (поиск, сортировка, пагинация, создание, изменение, логическое и физическое удаление, восстановление, групповое удаление) пишем один раз в классе LocalRepository<T, Q>. Конкретные репозитории описывают только отличия: отбор, порядок, проверку уникальности и защиту от удаления. После каждого изменения вызывается сохранение в хранилище и оповещение подписчиков, поэтому справочники в формах обновляются сразу. Для проверок по запросу сервера (422 и 409) введены исключения ValidationException с ошибками по полям и ConflictException.

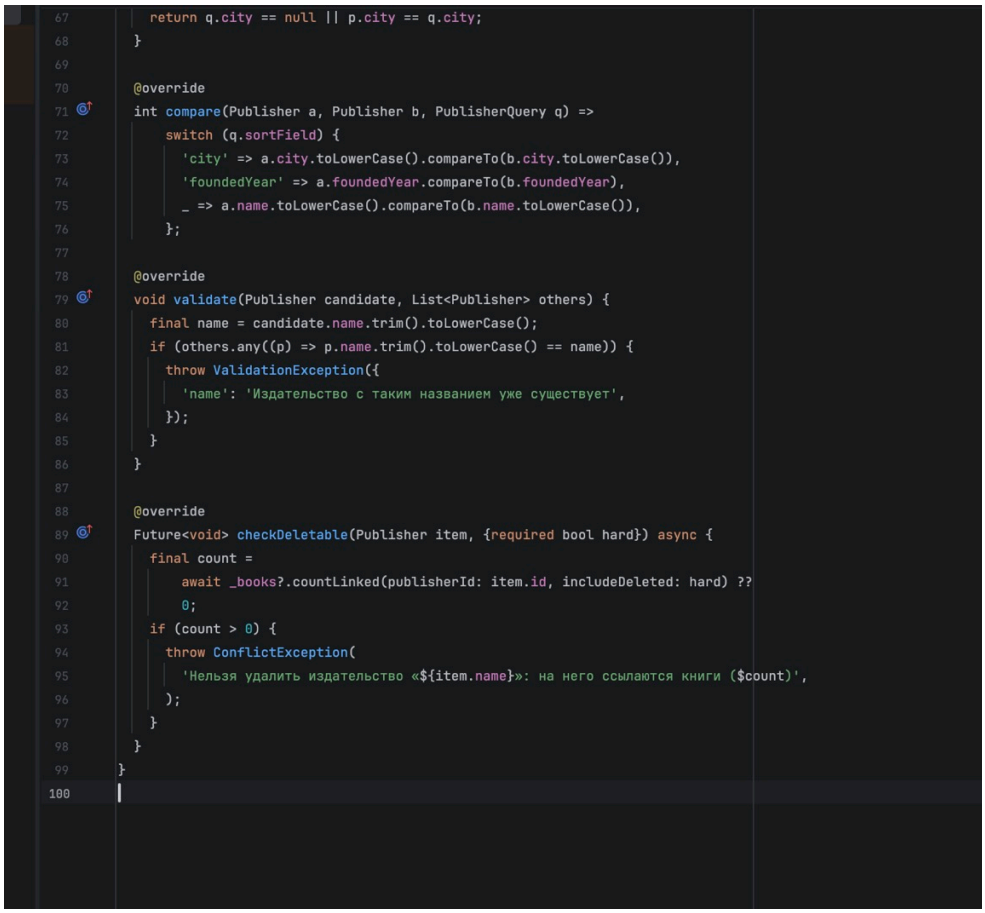
6. Проверка уникальности находится в репозитории: ISBN книги сравнивается без дефисов и регистра, а при дубликате бросается ValidationException с ошибкой в поле isbn. Так же проверяются почта и номер билета читателя, названия жанров и издательств. Для связанных записей метод countLinked возвращает число книг, ссылающихся на автора, жанр или издательство.



```
@override
int compare(Book a, Book b, BookQuery q) => switch (q.sortField) {
  'year' => a.year.compareTo(b.year),
  'pages' => a.pages.compareTo(b.pages),
  _ => a.title.toLowerCase().compareTo(b.title.toLowerCase()),
};

@override
void validate(Book candidate, List<Book> others) {
  final isbn = normalizeIsbn(candidate.isbn);
  if (others.any((b) => normalizeIsbn(b.isbn) == isbn)) {
    throw ValidationException({'isbn': 'Книга с таким ISBN уже существует'});
  }
}

@override
Future<int> countLinked({
  int? authorId,
  int? genreId,
  int? publisherId,
  bool includeDeleted = false,
}) async {
  final all = await findAll();
  return all.where((b) {
    if (!includeDeleted && b.isDeleted) return false;
    if (authorId != null && b.authorIds.contains(authorId)) return true;
    if (genreId != null && b.genreIds.contains(genreId)) return true;
    if (publisherId != null && b.publisherId == publisherId) return true;
    return false;
  }).length;
}
}
```

Рисунок 6 – Проверка уникальности ISBN и подсчёт связанных книг (local_book_repository.dart)

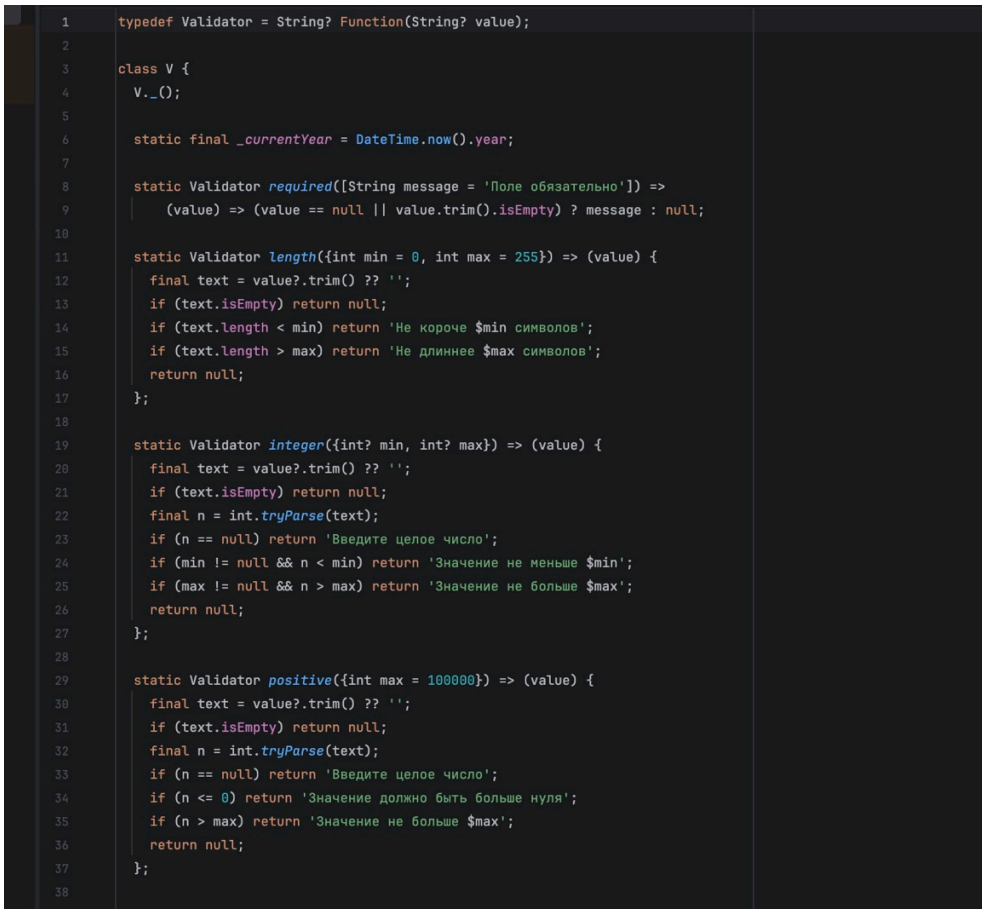
7. Защита от удаления. Издательство, на которое ссылаются книги, удалить нельзя: перед логическим и физическим удалением репозиторий считает связанные книги и бросает ConflictException с сообщением, где указано их число. То же правило действует для авторов и жанров. При физическом удалении учитываются и логически удалённые книги, потому что ссылка на запись остаётся.



```
    return q.city == null || p.city == q.city;
  }

  @override
  int compare(Publisher a, Publisher b, PublisherQuery q) =>
      switch (q.sortField) {
        'city' => a.city.toLowerCase().compareTo(b.city.toLowerCase()),
        'foundedYear' => a.foundedYear.compareTo(b.foundedYear),
        _ => a.name.toLowerCase().compareTo(b.name.toLowerCase()),
      };

  @override
  void validate(Publisher candidate, List<Publisher> others) {
    final name = candidate.name.trim().toLowerCase();
    if (others.any((p) => p.name.trim().toLowerCase() == name)) {
      throw ValidationException({
        'name': 'Издательство с таким названием уже существует',
      });
    }
  }

  @override
  Future<void> checkDeletable(Publisher item, {required bool hard}) async {
    final count =
        await _books?.countLinked(publisherId: item.id, includeDeleted: hard) ??
        0;
    if (count > 0) {
      throw ConflictException(
        'Нельзя удалить издательство «${item.name}»: на него ссылаются книги ($count)',
      );
    }
  }
}

```

Рисунок 7 – Уникальность названия и защита издательства от удаления (local_publisher_repository.dart)

8. Валидаторы выносим в отдельный файл lib/core/validators.dart и переиспользуем во всех формах. Валидатор возвращает текст ошибки или null. Есть проверки обязательности, длины, целого числа в диапазоне, положительного числа, года не позже текущего, почты, телефона, ISBN-13, номера билета и даты. Метод combine объединяет проверки и возвращает первую сработавшую. Проверки длины и формата пропускают пустое значение: за обязательность отвечает required, поэтому в пустом поле выводится одна понятная ошибка.



```dart
typedef Validator = String? Function(String? value);

class V {
  V._();

  static final _currentYear = DateTime.now().year;

  static Validator required([String message = 'Поле обязательно']) =>
      (value) => (value == null || value.trim().isEmpty) ? message : null;

  static Validator length({int min = 0, int max = 255}) => (value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return null;
    if (text.length < min) return 'Не короче $min символов';
    if (text.length > max) return 'Не длиннее $max символов';
    return null;
  };

  static Validator integer({int? min, int? max}) => (value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return null;
    final n = int.tryParse(text);
    if (n == null) return 'Введите целое число';
    if (min != null && n < min) return 'Значение не меньше $min';
    if (max != null && n > max) return 'Значение не больше $max';
    return null;
  };

  static Validator positive({int max = 100000}) => (value) {
    final text = value?.trim() ?? '';
    if (text.isEmpty) return null;
    final n = int.tryParse(text);
    if (n == null) return 'Введите целое число';
    if (n <= 0) return 'Значение должно быть больше нуля';
    if (n > max) return 'Значение не больше $max';
    return null;
  };

```

Рисунок 8 – Переиспользуемые валидаторы (validators.dart)

Таблица 6 – Проверки полей книги

Поле	Проверки	Сообщение об ошибке
Название	обязательно, 2–200 символов	Укажите название; Не короче 2 символов
ISBN	обязательно, формат ISBN-13, уникальность	Укажите ISBN; Формат: 978-5-17-118366-4; Книга с таким ISBN уже существует
Год издания	обязательно, целое от 1450 до текущего года	Укажите год; Значение не меньше 1450; Значение не больше текущего года
Страниц	обязательно, целое от 1 до 10000	Укажите число страниц; Значение не меньше 1
Издательство	обязательный выбор из списка	Выберите издательство
Авторы	необязательный множественный выбор	нет
Жанры	выбрать хотя бы один	Выберите хотя бы один жанр
Количество экземпляров	обязательно, положительное целое до 1000	Укажите количество; Значение должно быть больше нуля
Доступно экземпляров	только при изменении: целое от 0, не больше общего числа (проверка по двум полям)	Не больше общего числа экземпляров (N)

Таблица 7 – Проверки полей автора, жанра и издательства

Поле	Проверки	Сообщение об ошибке
Автор: ФИО	обязательно, 2–100 символов	Укажите ФИО; Не короче 2 символов
Автор: год рождения	обязательно, целое от 1 до текущего года	Укажите год рождения; Значение не больше текущего года
Автор: страна	обязательно, 2–60 символов	Укажите страну

Поле	Проверки	Сообщение об ошибке
Жанр: название	обязательно, 2–50 символов, уникально	Укажите название; Жанр с таким названием уже существует
Жанр: описание	необязательно, до 255 символов	Не длиннее 255 символов
Издательство: название	обязательно, 2–100 символов, уникально	Издательство с таким названием уже существует
Издательство: город	обязательно, 2–60 символов	Укажите город
Издательство: год основания	обязательно, целое от 1400 до текущего года	Укажите год основания

Таблица 8 – Проверки полей читателя и билета

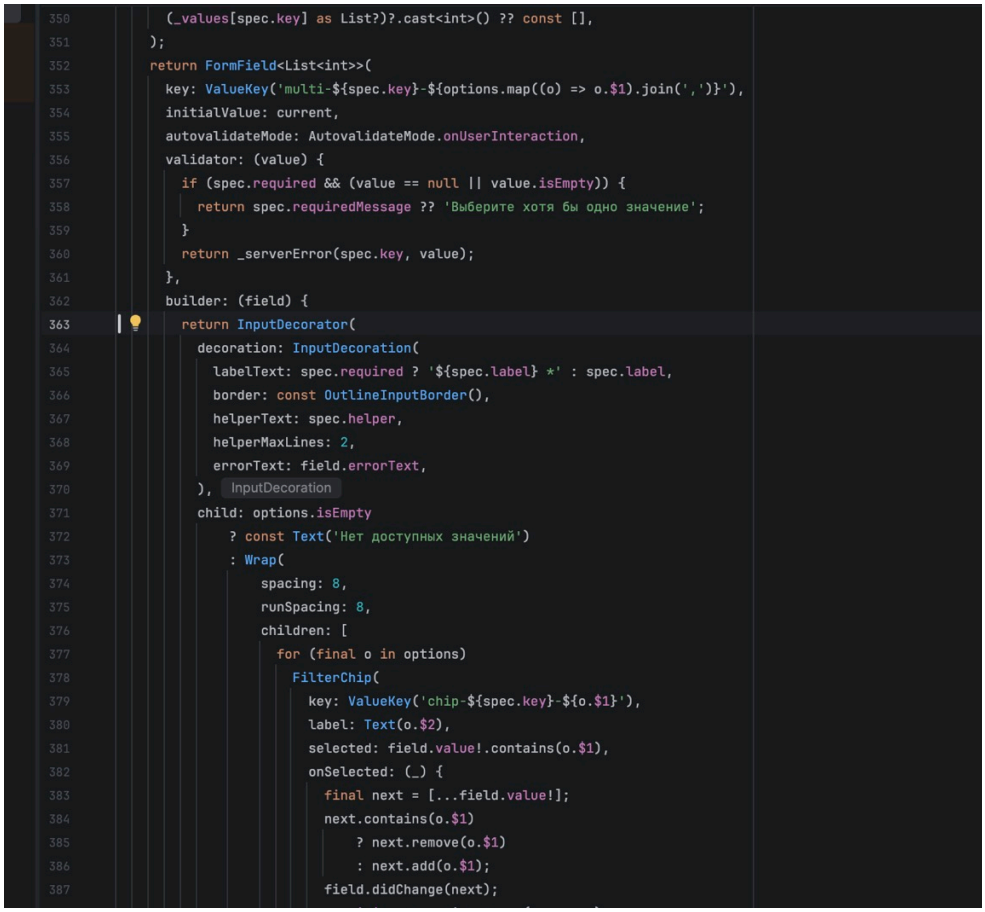
Поле	Проверки	Сообщение об ошибке
Читатель: ФИО	обязательно, 2–100 символов	Укажите ФИО
Читатель: почта	обязательно, формат адреса, уникальность	Укажите почту; Некорректный адрес почты; Читатель с такой почтой уже зарегистрирован
Читатель: телефон	необязательно, цифры, пробелы, скобки, дефисы	Формат: +7 900 000-00-00
Билет: номер	обязательно, формат RC-000000, уникальность	Укажите номер билета; Формат: RC-000001; Билет с таким номером уже выдан
Билет: дата выдачи	обязательна, существующая дата ГГГГ-ММ-ДД	Формат даты: 2026-01-31; Такой даты не существует
Билет: действует до	обязательна, существующая дата, позже даты выдачи	Срок действия должен быть позже даты выдачи

9. Форму делаем переиспользуемой: поля описываются списком, а один общий виджет EntityFormView выводит их, проверяет, отправляет данные и показывает ошибки. Типы описаний: TextSpec (текст), DateSpec (дата с календарём), DropdownSpec (связь «многие к одному»), MultiSpec (связь «многие ко многим») и GroupSpec (вложенная группа для связи «один к одному»). Новая сущность получает форму описанием полей, без нового виджета.

10. Форма создания и форма изменения — один экран. Маршруты /books/new и /books/:id/edit открывают один и тот же виджет, различие только в идентификаторе: заголовок и надпись кнопки подбираются по isEditing. Маршрут /new стоит выше /:id, иначе слово new было бы принято за идентификатор. При изменении поля заполняются один раз после загрузки записи, поэтому форма не открывается пустой. Начальное значение текстового поля задаётся при создании контроллера, а не через initialValue: одновременно использовать оба нельзя.

11. Связь «многие ко многим» реализуем собственным полем формы FormField<List<int>> с набором FilterChip. Ошибка выводится через errorText так же,

как у обычного поля, а метод field.didChange сообщает форме об изменении: без него поле не участвовало бы в validate().

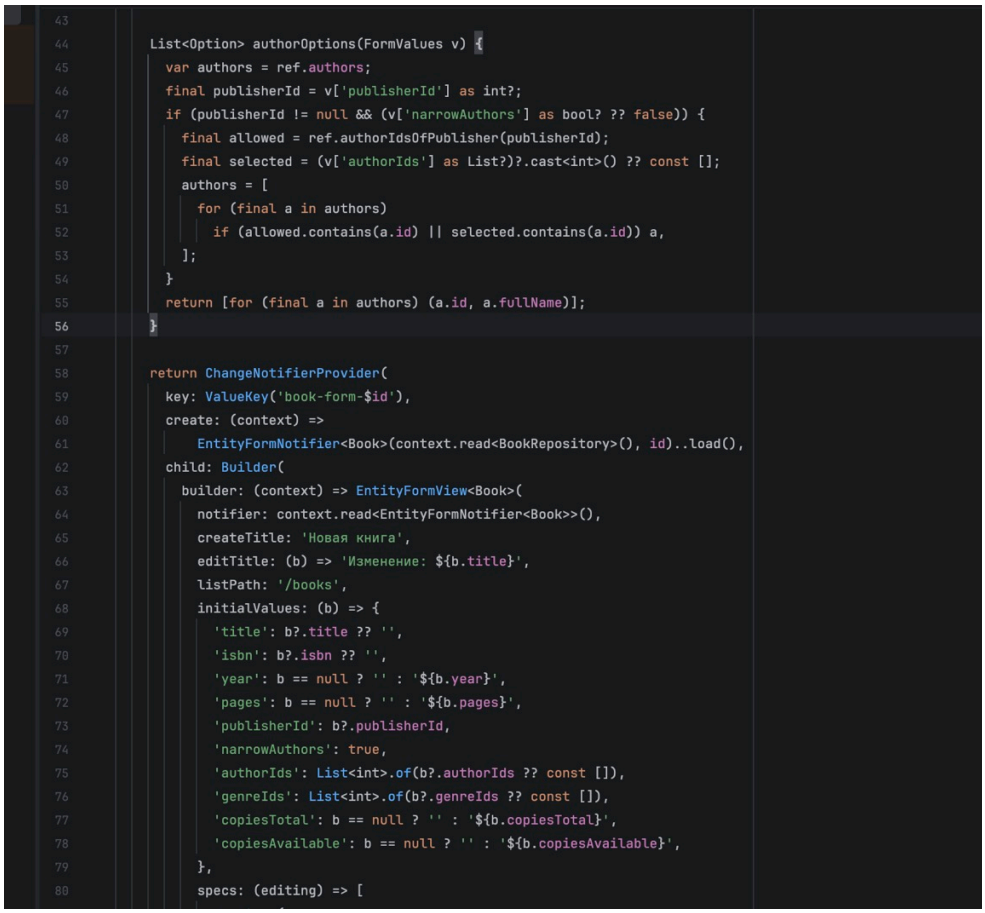


```
    (_values[spec.key] as List?)?.cast<int>() ?? const [],
  );
  return FormField<List<int>>(
    key: ValueKey('multi-${spec.key}-${options.map((o) => o.$1).join(',')}'),
    initialValue: current,
    autovalidateMode: AutovalidateMode.onUserInteraction,
    validator: (value) {
      if (spec.required && (value == null || value.isEmpty)) {
        return spec.requiredMessage ?? 'Выберите хотя бы одно значение';
      }
      return _serverError(spec.key, value);
    },
    builder: (field) {
      return InputDecorator(
        decoration: InputDecoration(
          labelText: spec.required ? '${spec.label} *' : spec.label,
          border: const OutlineInputBorder(),
          helperText: spec.helper,
          helperMaxLines: 2,
          errorText: field.errorText,
        ),  InputDecoration
        child: options.isEmpty
            ? const Text('Нет доступных значений')
            : Wrap(
                spacing: 8,
                runSpacing: 8,
                children: [
                  for (final o in options)
                    FilterChip(
                      key: ValueKey('chip-${spec.key}-${o.$1}'),
                      label: Text(o.$2),
                      selected: field.value!.contains(o.$1),
                      onSelected: (_) {
                        final next = [...field.value!];
                        next.contains(o.$1)
                            ? next.remove(o.$1)
                            : next.add(o.$1);
                        field.didChange(next);
```

Рисунок 9 – Множественный выбор: FormField<List<int>> и FilterChip (entity_form_view.dart)

12. Связь «многие к одному» выбирается выпадающим списком DropdownButtonFormField<int>. В значении хранится идентификатор, а не объект: так не нужно переопределять == и hashCode. Список издательств и жанров заполняется из репозиториев через ReferenceNotifier и обновляется при любом изменении данных: добавленный жанр сразу доступен в форме книги. Если выбранное значение пропало из списка, оно сбрасывается, иначе DropdownButton выбросил бы исключение.

13. Реализуем каскадное поведение связанных списков. После выбора издательства список авторов сужается до тех, кто уже выпускал книги в этом издательстве; уже выбранные авторы остаются в списке. Переключатель «Авторы только из этого издательства» возвращает полный список.



```
43
44      List<Option> authorOptions(FormValues v) {
45        var authors = ref.authors;
46        final publisherId = v['publisherId'] as int?;
47        if (publisherId != null && (v['narrowAuthors'] as bool? ?? false)) {
48          final allowed = ref.authorIdsOfPublisher(publisherId);
49          final selected = (v['authorIds'] as List?)?.cast<int>() ?? const [];
50          authors = [
51            for (final a in authors)
52              if (allowed.contains(a.id) || selected.contains(a.id)) a,
53          ];
54        }
55        return [for (final a in authors) (a.id, a.fullName)];
56      }
57
58      return ChangeNotifierProvider(
59        key: ValueKey('book-form-$id'),
60        create: (context) =>
61            EntityFormNotifier<Book>(context.read<BookRepository>(), id)..load(),
62        child: Builder(
63          builder: (context) => EntityFormView<Book>(
64            notifier: context.read<EntityFormNotifier<Book>>(),
65            createTitle: 'Новая книга',
66            editTitle: (b) => 'Изменение: ${b.title}',
67            listPath: '/books',
68            initialValues: (b) => {
69              'title': b?.title ?? '',
70              'isbn': b?.isbn ?? '',
71              'year': b == null ? '' : '${b.year}',
72              'pages': b == null ? '' : '${b.pages}',
73              'publisherId': b?.publisherId,
74              'narrowAuthors': true,
75              'authorIds': List<int>.of(b?.authorIds ?? const []),
76              'genreIds': List<int>.of(b?.genreIds ?? const []),
77              'copiesTotal': b == null ? '' : '${b.copiesTotal}',
78              'copiesAvailable': b == null ? '' : '${b.copiesAvailable}',
79            },
80            specs: (editing) => [
```

Рисунок 10 – Описание формы книги и каскадное сужение списка авторов (form_screens.dart)

14. Читательский билет редактируется прямо в форме читателя вложенной группой полей GroupSpec: отдельного экрана для билета нет. Проверка срока действия зависит от двух полей: дата окончания должна быть позже даты выдачи. Для этого валидатор получает значения всех полей формы.

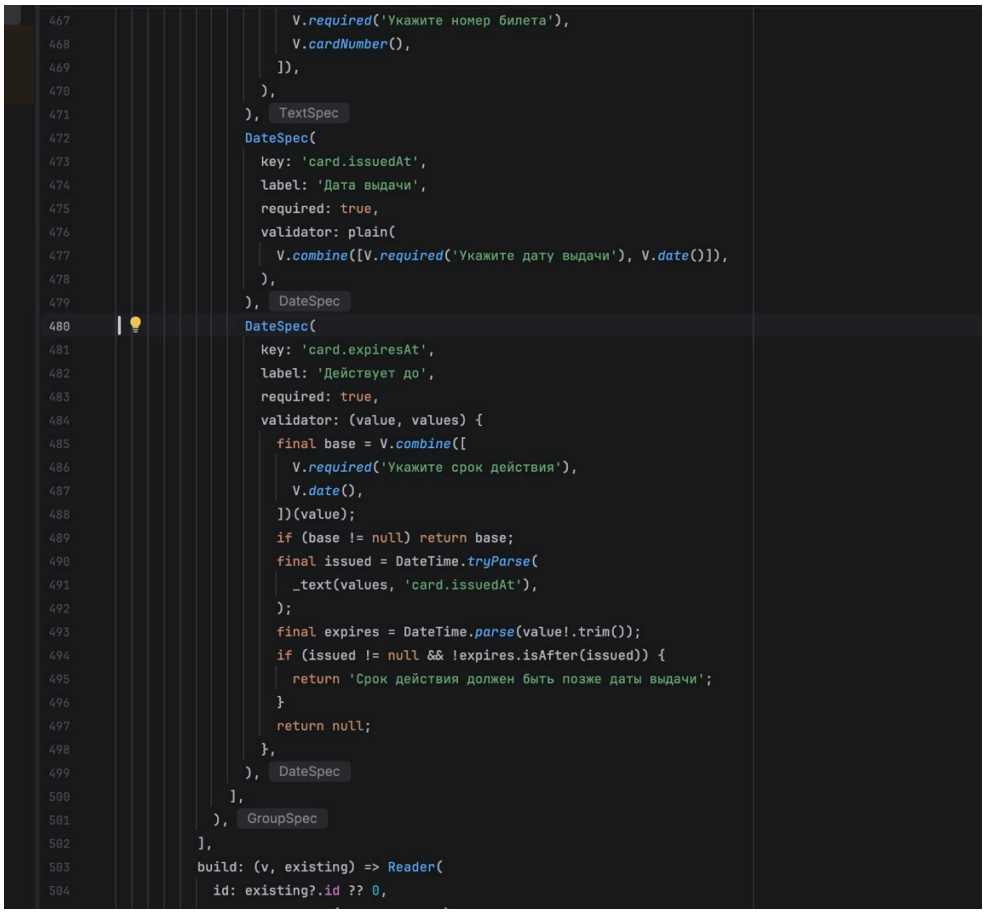
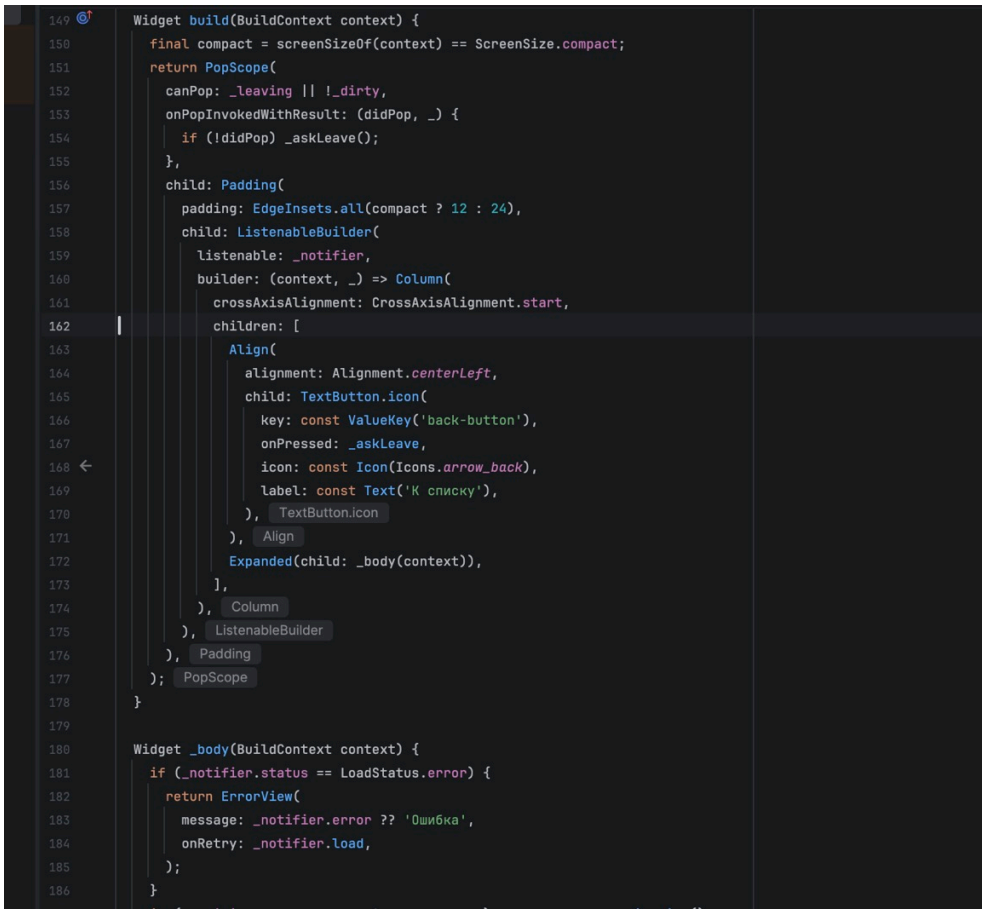


Рисунок 11 – Группа полей читательского билета и проверка по двум полям (form_screens.dart)

15. Ошибки сервера (уникальность, 422) показываются в соответствующем поле. Репозиторий бросает ValidationException с картой «имя поля: сообщение», форма записывает ошибки в состояние и запускает validate() повторно, а валидатор поля возвращает серверную ошибку. Ошибка привязана к значению, при котором она получена: стоит изменить поле, и она исчезает, а не висит после ввода другого ISBN.

16. Форма предупреждает о несохранённых изменениях. Виджет PopScope с условием canPop: !_dirty перехватывает попытку уйти с экрана (кнопка «К списку», «Отмена», кнопка «назад» браузера) и показывает диалог подтверждения. Сравнение с исходными значениями учитывает списки, поэтому возврат значения к исходному снимает предупреждение.



```
Widget build(BuildContext context) {
  final compact = screenSizeOf(context) == ScreenSize.compact;
  return PopScope(
    canPop: _leaving || !_dirty,
    onPopInvokedWithResult: (didPop, _) {
      if (!didPop) _askLeave();
    },
    child: Padding(
      padding: EdgeInsets.all(compact ? 12 : 24),
      child: ListenableBuilder(
        listenable: _notifier,
        builder: (context, _) => Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Align(
              alignment: Alignment.centerLeft,
              child: TextButton.icon(
                key: const ValueKey('back-button'),
                onPressed: _askLeave,
                icon: const Icon(Icons.arrow_back),
                label: const Text('К списку'),
              ), TextButton.icon
            ), Align
            Expanded(child: _body(context)),
          ],
        ), Column
      ), ListenableBuilder
    ), Padding
  ); PopScope
}

Widget _body(BuildContext context) {
  if (_notifier.status == LoadStatus.error) {
    return ErrorView(
      message: _notifier.error ?? 'Ошибка',
      onRetry: _notifier.load,
    );
  }
```

Рисунок 12 – Предупреждение о несохранённых изменениях (PopScope)

17. Подключаем хранилище при запуске. В main() вызывается SharedPreferences.getInstance(), и объект передается в провайдеры. Репозитории создаются с хранилищем, а в тестах без него. Сообщения хранилища каркас приложения показывает один раз после первого кадра.

18. Запускаем тесты из Android Studio. Всего 132 теста: модели и сериализация, устойчивый разбор испорченных данных, сохранение и перенос форматов, все валидаторы, уникальность, защита от удаления, формы со всеми типами связей, предупреждение о несохранённых изменениях, каскад и сохранение после «перезагрузки».

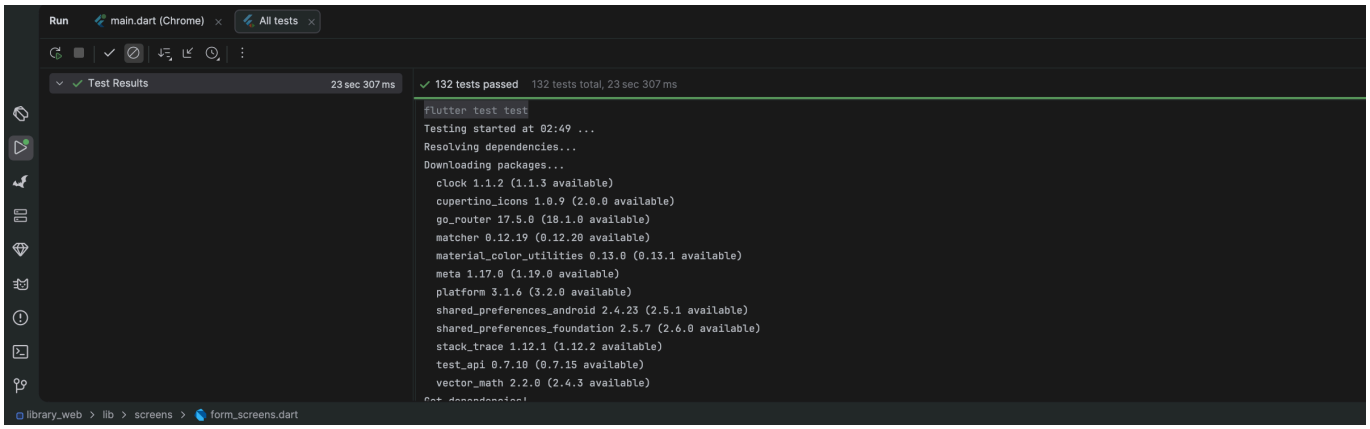


Рисунок 13 – Запуск тестов в Android Studio: 132 теста пройдено

19. Проверяем приложение в браузере. Форма создания книги содержит все три типа связей: издательство выбирается из списка, авторы и жанры множественным выбором. После выбора издательства «Азбука» список авторов сузился до трёх.

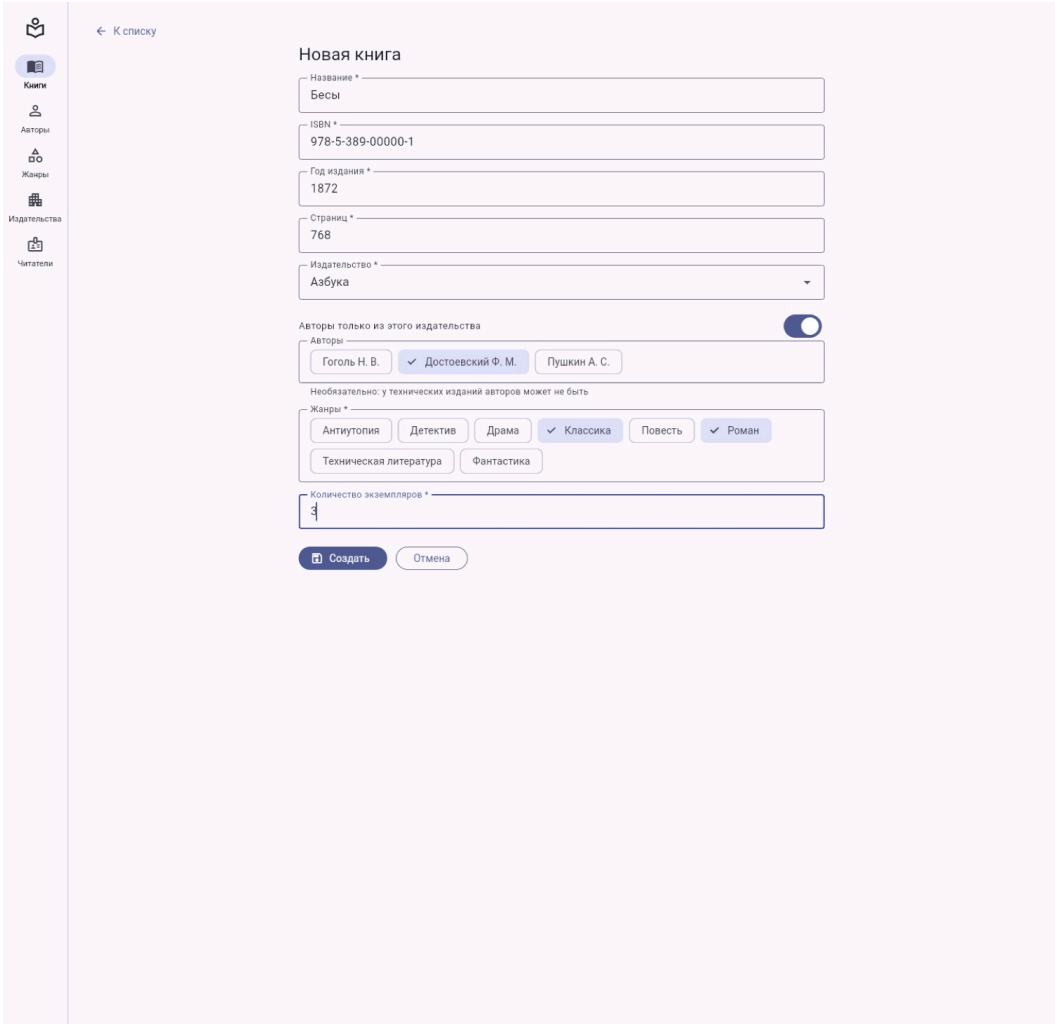


Рисунок 14 – Форма создания книги со всеми типами связей

20. Нажимаем «Создать» на пустой форме. Проверка срабатывает во всех полях сразу, сообщения выведены под каждым полем, а не общим сообщением наверху.

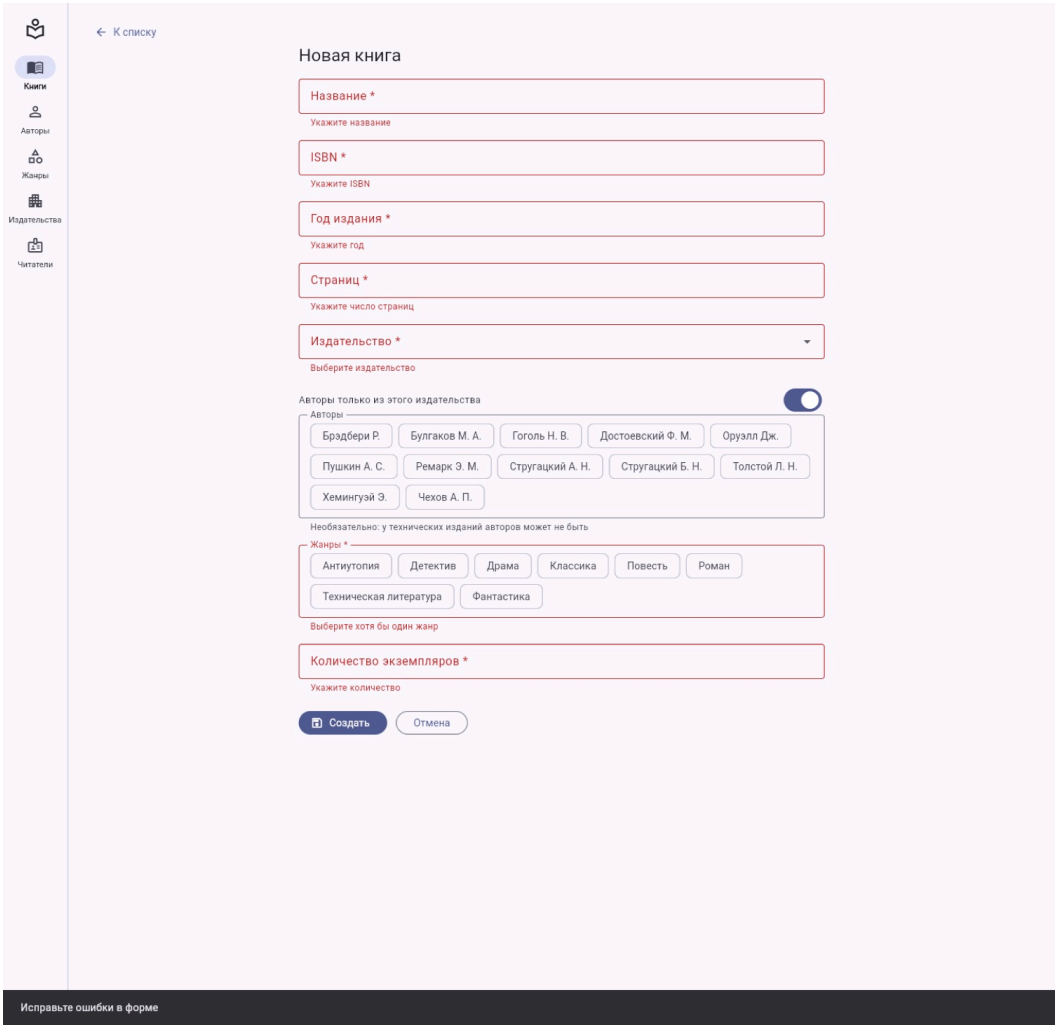


Рисунок 15 – Сработавшая валидация: ошибки в нескольких полях одновременно

21. Вводим ISBN существующей книги. Ошибка уникальности выводится в поле ISBN и не мешает остальным полям.

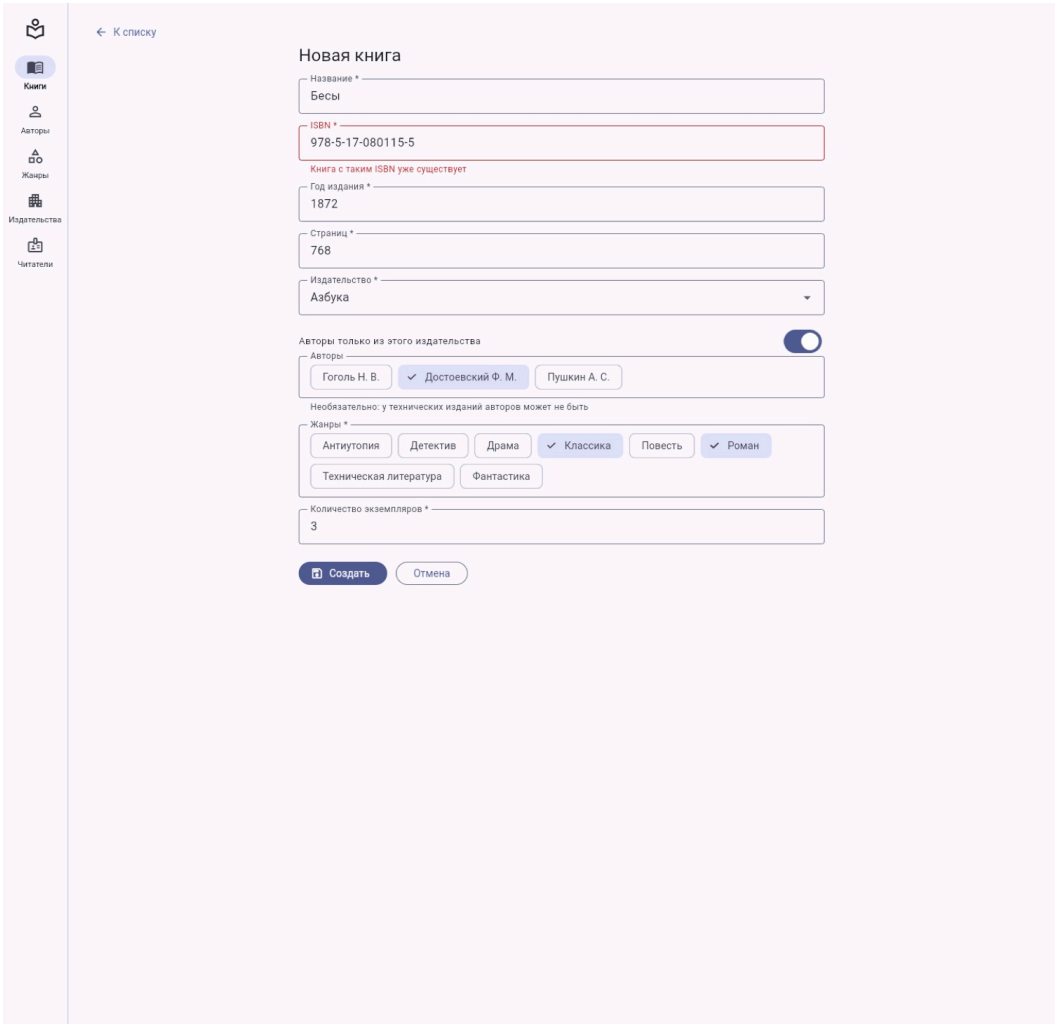


Рисунок 16 – Ошибка уникальности ISBN

22. Открываем форму изменения книги. Тот же экран заполнен данными записи, выбранные автор и жанры отмечены, добавлено поле «Доступно экземпляров».

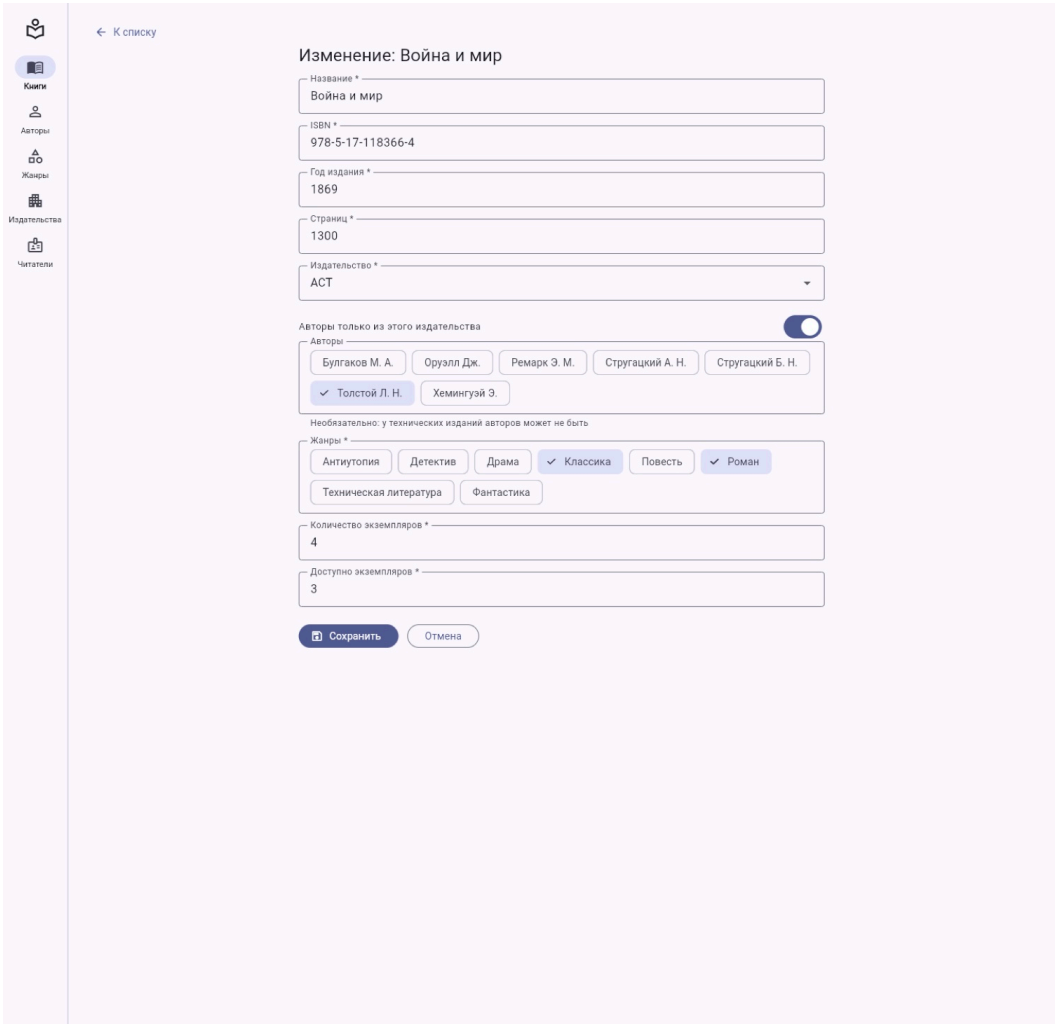


Рисунок 17 – Форма изменения книги (тот же экран)

23. Пробуем уйти с формы, в которой есть несохранённые данные. Приложение предлагает остаться или уйти.

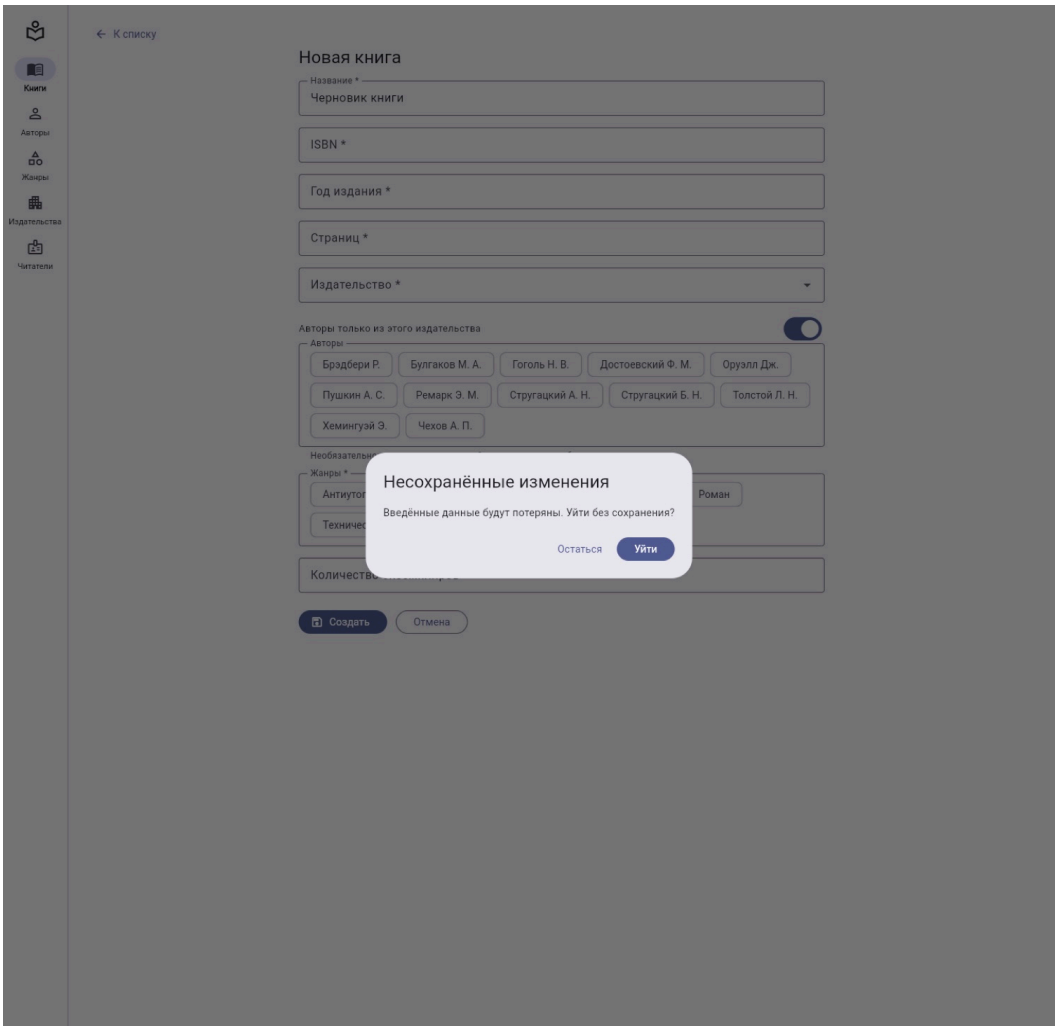


Рисунок 18 – Предупреждение о несохранённых изменениях

24. Пробуем удалить издательство «Азбука», на которое ссылаются книги. Приложение отказывает и сообщает число связанных записей.

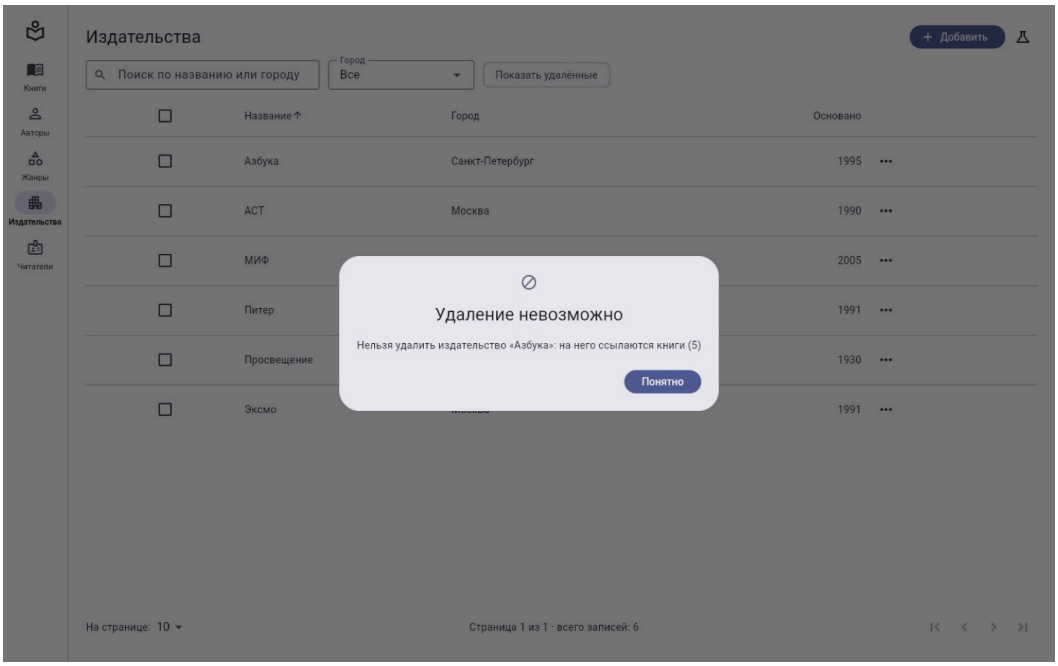


Рисунок 19 – Отказ в удалении издательства со связанными книгами

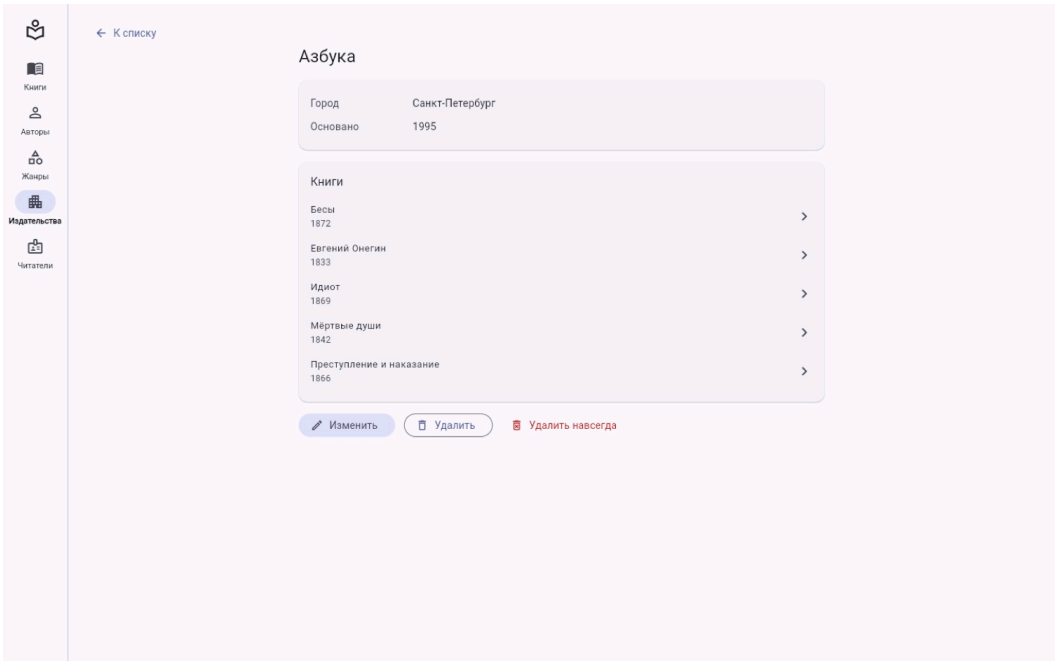


Рисунок 20 – Карточка издательства со списком связанных книг

25. Проверяем читателя с билетом. Читательский билет — вложенная группа полей в форме читателя, даты предзаполнены.

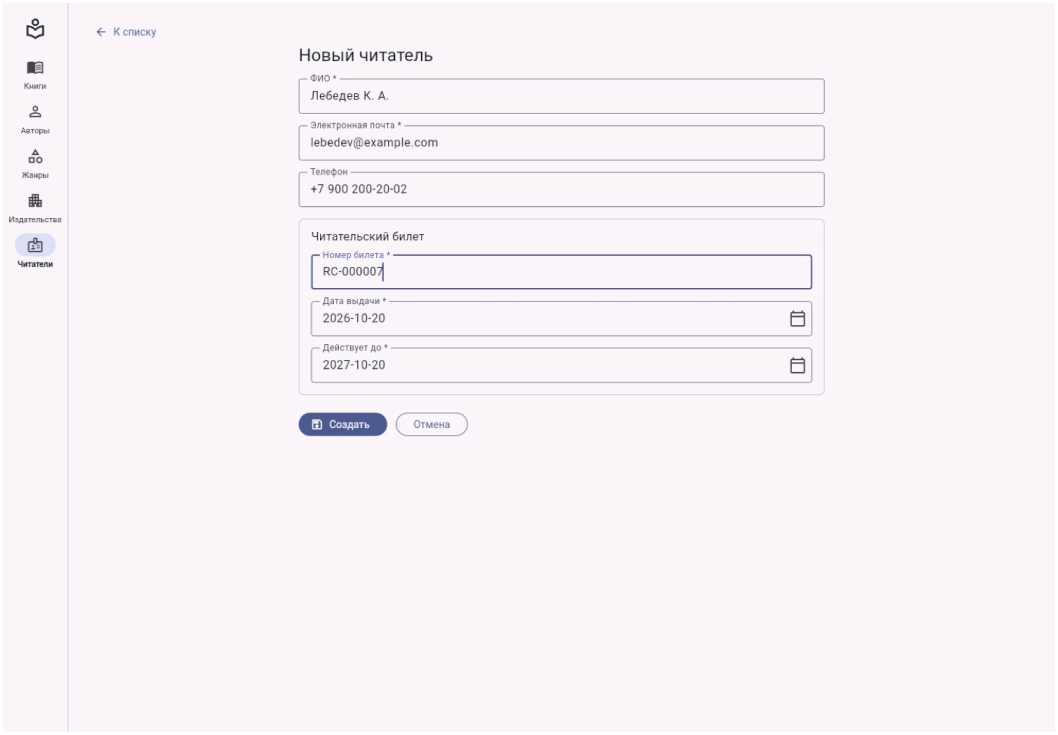


Рисунок 21 – Форма читателя с читательским билетом

Проверки формата срабатывают и в самом билете. Ошибки по полям читателя и билета выводятся одновременно.

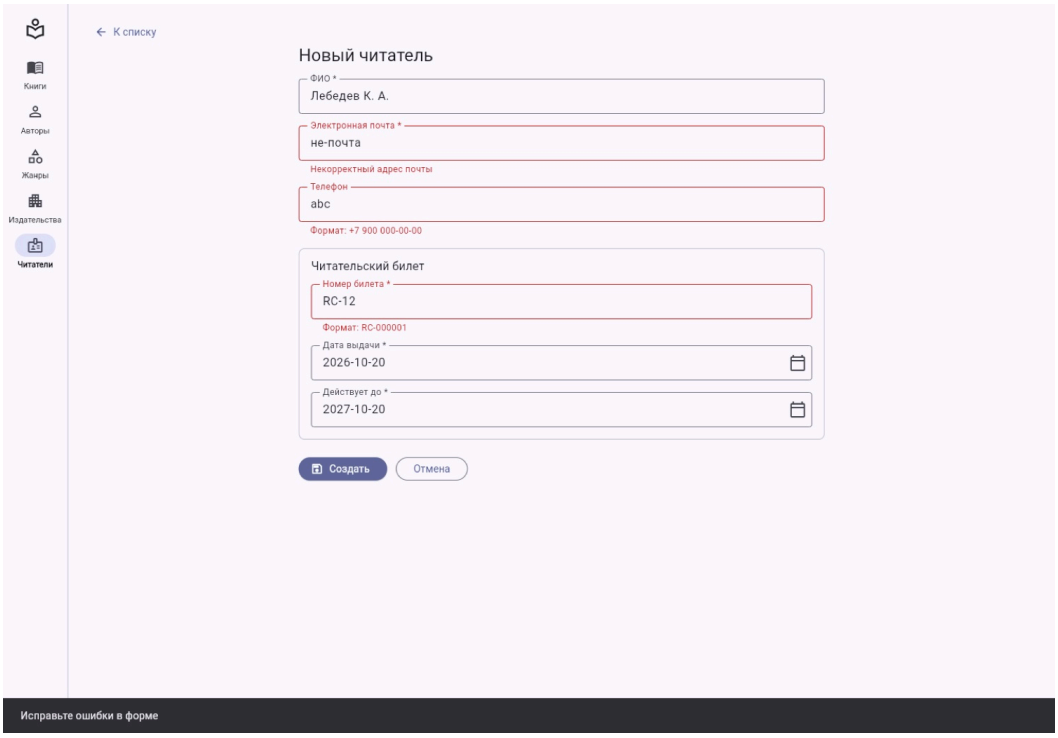


Рисунок 22 – Ошибки формата почты, телефона и номера билета

Дубликаты почты и номера билета показываются в своих полях.

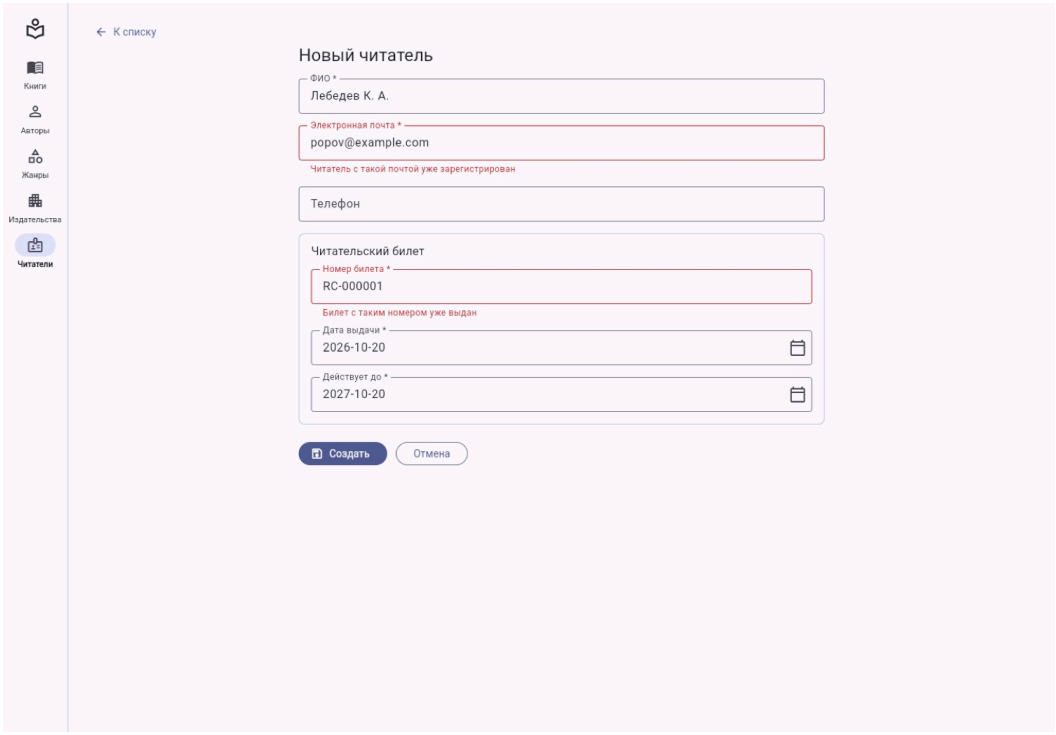


Рисунок 23 – Ошибки уникальности почты и номера билета

26. Остальные сущности работают через те же универсальные списки: поиск, фильтры, сортировка, пагинация, показ удалённых и восстановление. Для каждой есть список, карточка, создание, изменение и удаление.

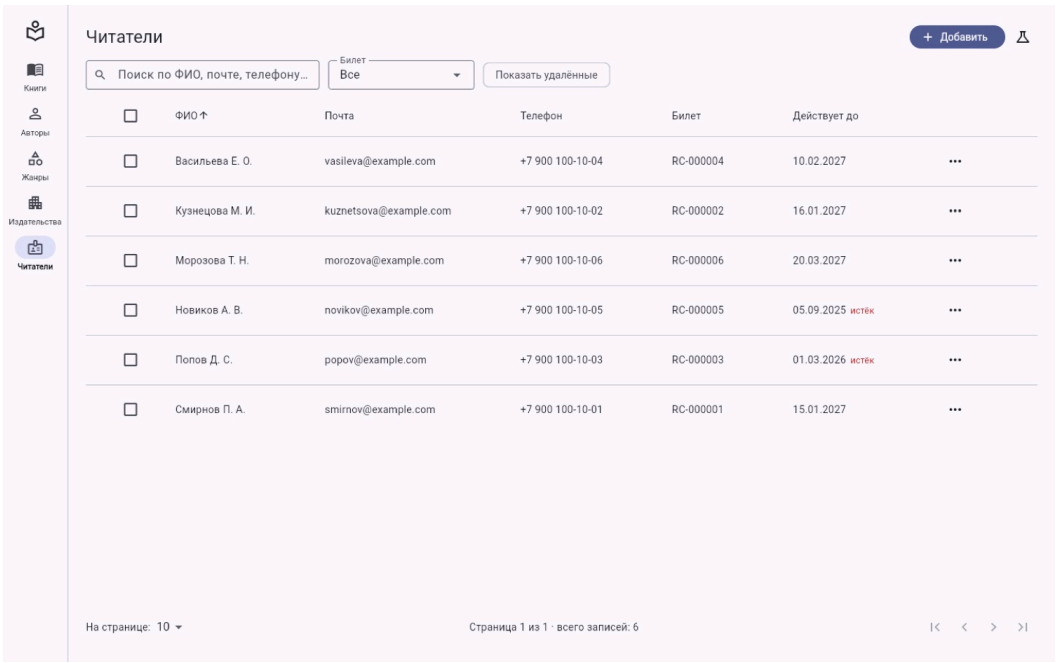


Рисунок 24 – Список читателей: фильтр по статусу билета

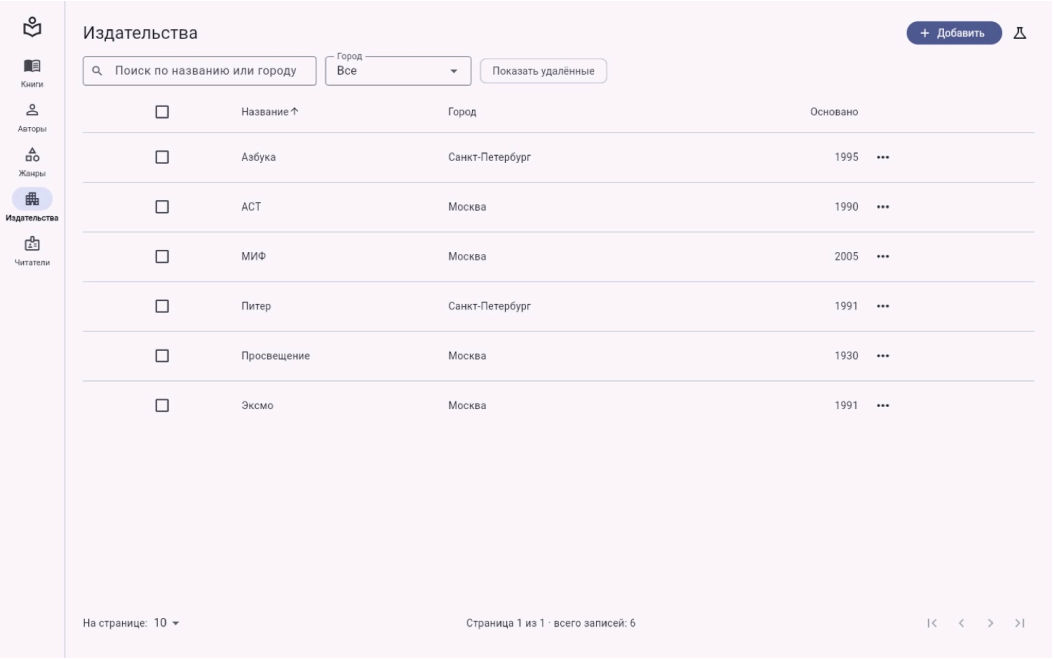


Рисунок 25 – Список издательств

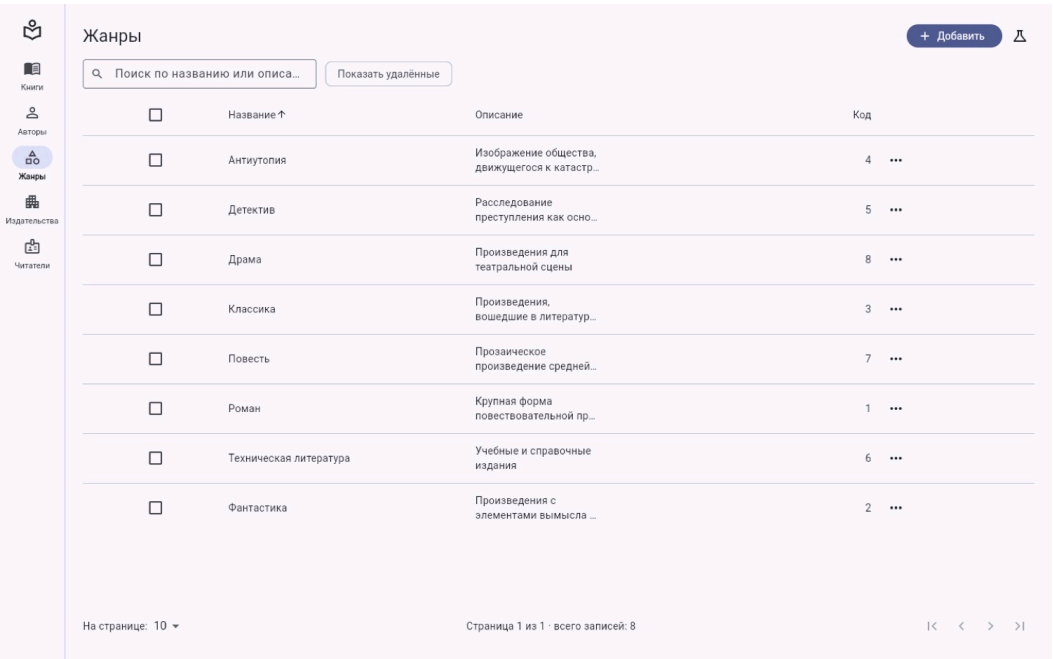


Рисунок 26 – Список жанров

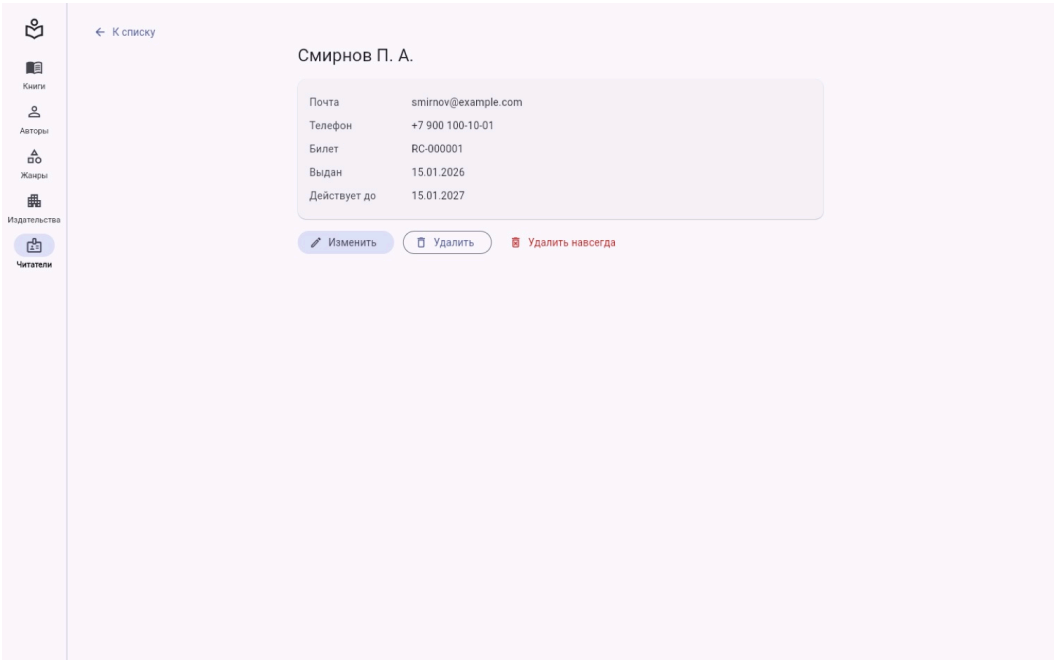


Рисунок 27 – Карточка читателя с билетом

27. Проверяем сохранение данных между перезагрузками. Создаём книгу «Бесы», открываем список с поиском и перезагружаем страницу. Запись на месте: она хранится в localStorage под ключом books_v2.

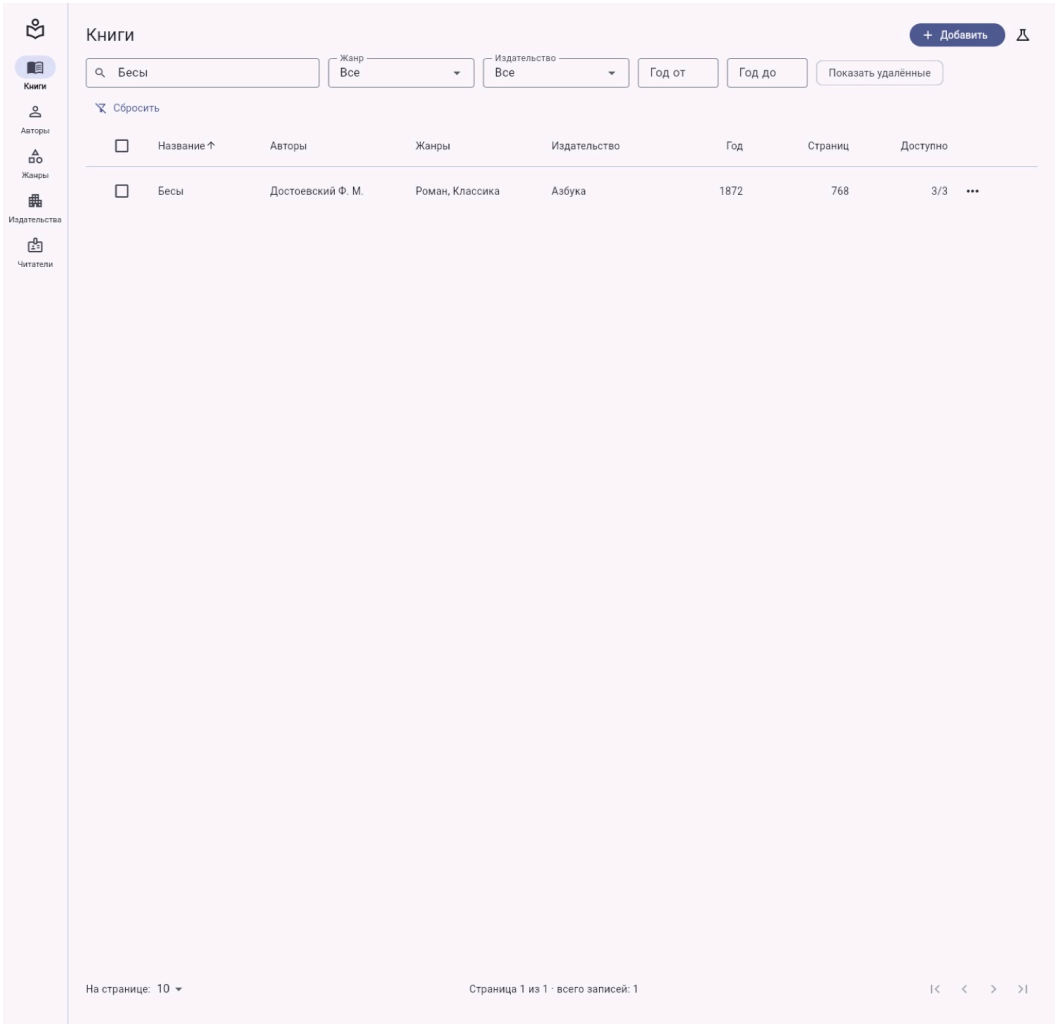


Рисунок 28 – Список до перезагрузки страницы

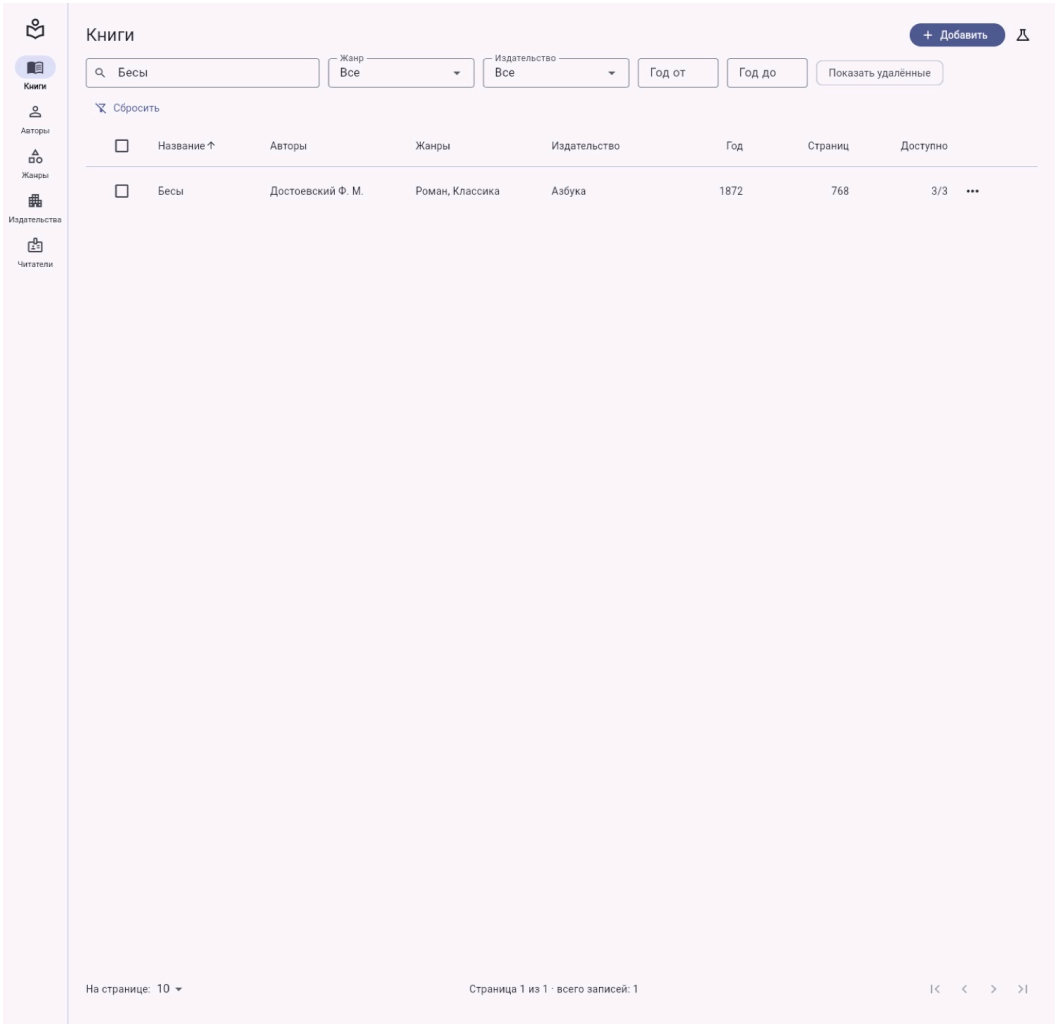


Рисунок 29 – Список после перезагрузки страницы: данные сохранились

28. Проверяем смену версии формата. Подкладываем в хранилище данные старого формата (ключ genres_v1): приложение переносит их в новый формат и сообщает об этом.

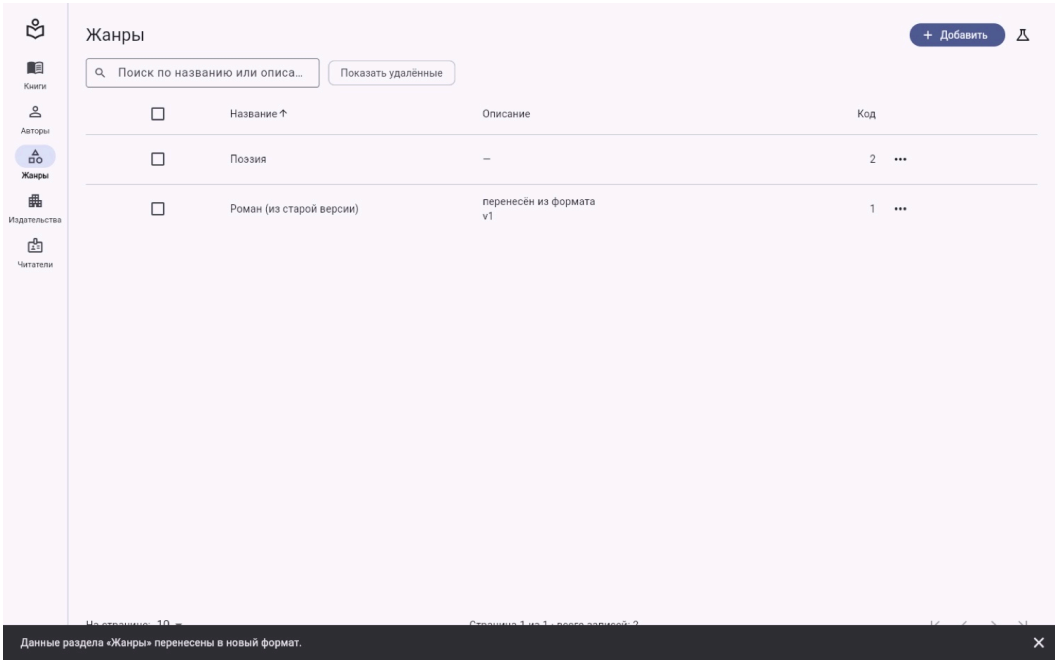


Рисунок 30 – Перенос данных старого формата с сообщением

Если данные испорчены, приложение запускается, восстанавливает начальный набор и также сообщает об этом.

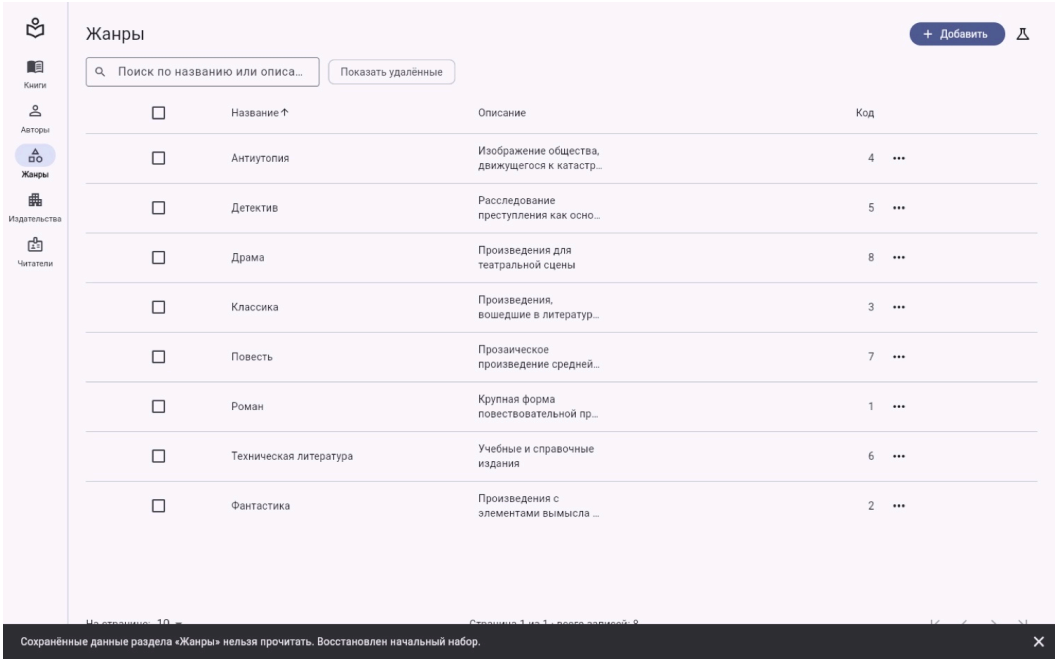


Рисунок 31 – Сброс нечитаемых данных с сообщением

29. Проверяем адаптивность. На узком окне списки показываются карточками, навигация из пяти разделов переходит в нижнюю панель, а форма вытягивается в одну колонку.

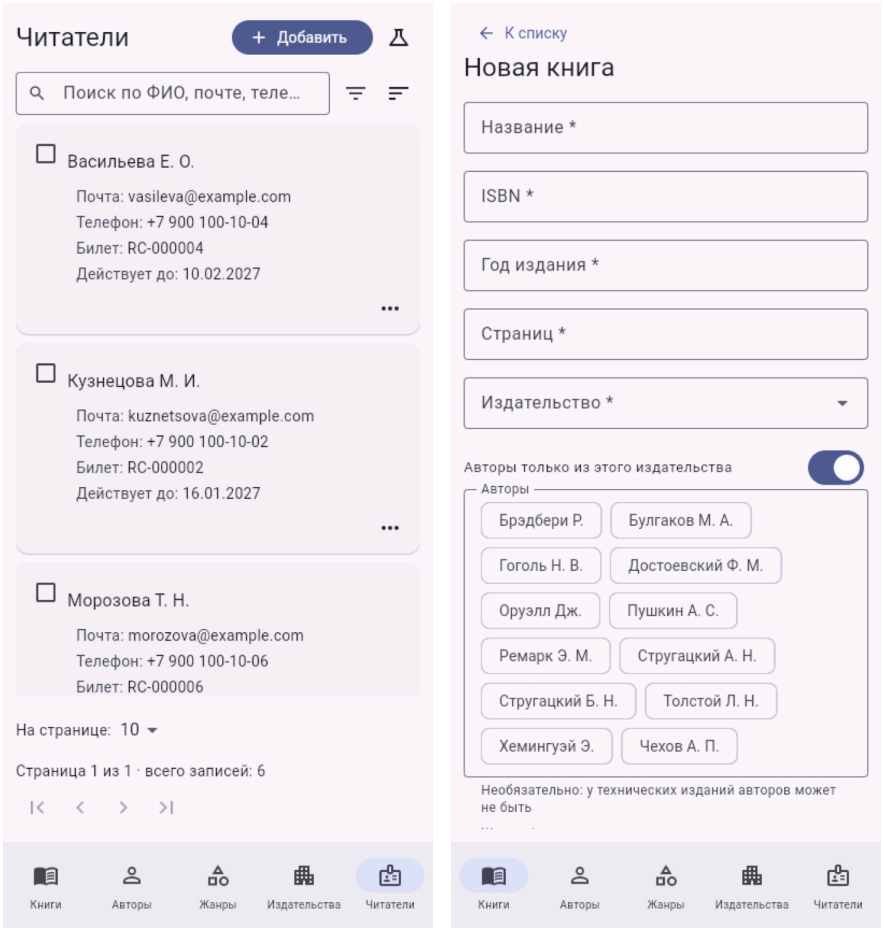


Рисунок 32 – Узкое окно: список читателей карточками и форма книги

30. Проверяем анализ и сборку. Команда flutter analyze не выдаёт замечаний.

Результат работы:

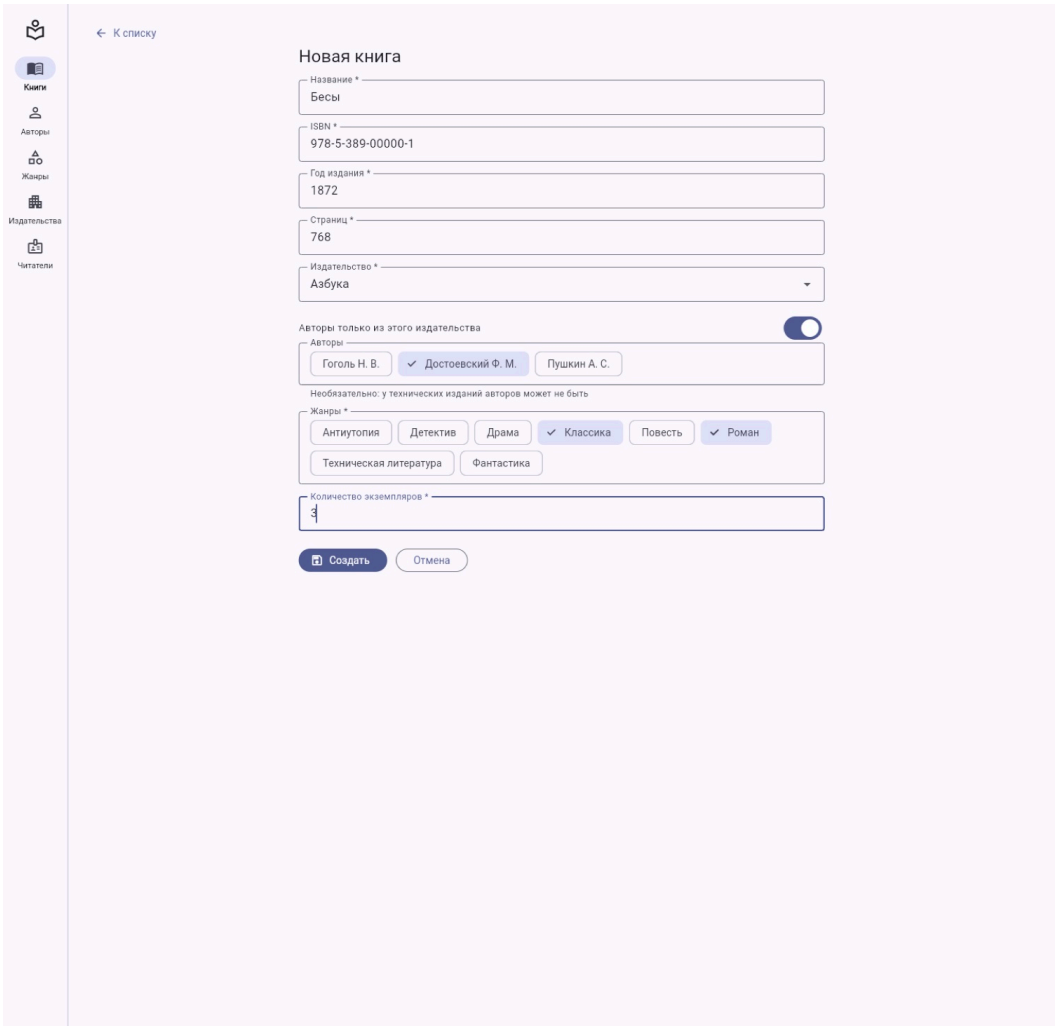


Рисунок 33 – Результат работы: форма книги со связями «многие к одному» и «многие ко многим»

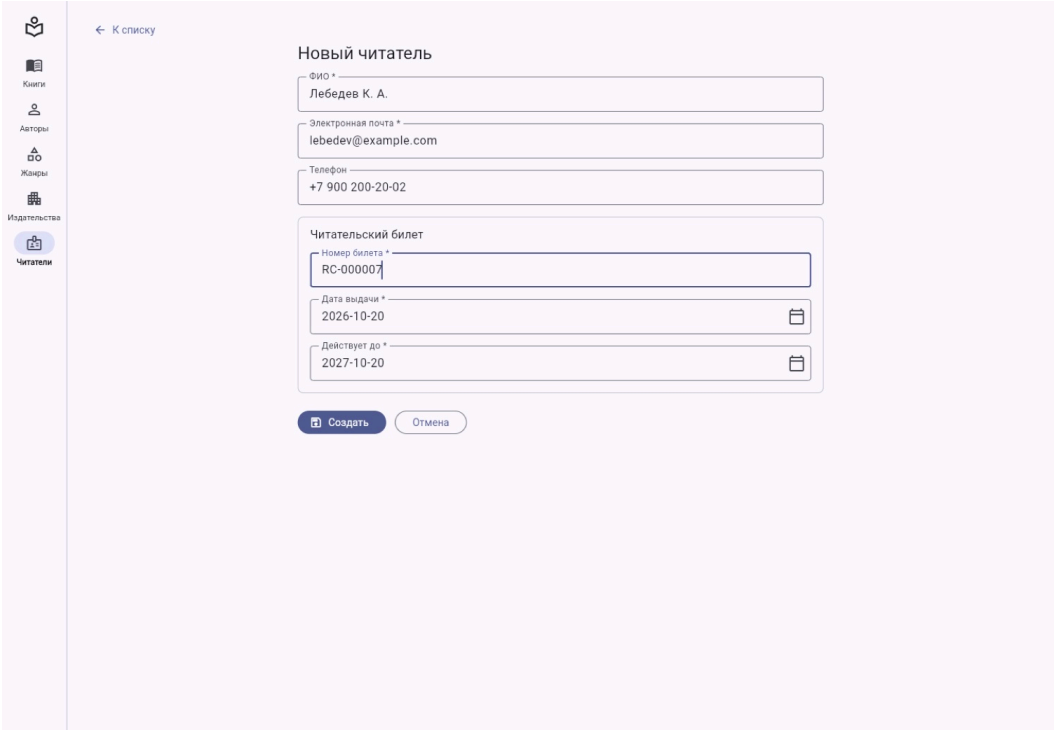


Рисунок 34 – Результат работы: форма читателя с вложенной группой полей билета

Вывод: проект library_web расширен до пяти сущностей (Book, Author, Genre, Publisher, Reader с билетом) со всеми тремя типами связей, и каждая связь отражена в интерфейсе: выпадающим списком, множественным выбором и вложенной группой полей. Реализованы формы создания и изменения на одном экране, описываемые списком полей, с валидацией под каждым полем, проверками по нескольким полям, ошибками уникальности ISBN, почты и номера билета в соответствующих полях, отказом в удалении записей со связями и предупреждением о несохранённых изменениях. Данные сохраняются в localStorage в версионированном формате: старые данные переносятся, испорченные не роняют приложение. Сериализация моделей устойчива к отсутствующим и пустым полям и пригодна для разбора ответов сервера в ПР4. Все 132 теста проходят.

ПРАКТИЧЕСКАЯ РАБОТА №4

Подключение REST API

Цель работы: заменить локальное хранилище в памяти на внешний сервер с REST API, освоить работу с HTTP-клиентом Dio в условиях web-платформы, научиться диагностировать и обходить ограничения CORS, корректно обрабатывать все состояния сетевого взаимодействия (индикацию загрузки, ошибки сервера, валидацию и пустые списки) и реализовать автоматический повтор запросов при сбоях. Работа продолжает проект library_web: существующие модели и экраны каталогов и форм подключаются к реальному серверу с сохранением архитектурного разделения на слои.

Ход работы:

Подготовка рабочего места и запуск учебного сервера. Для сетевого взаимодействия запускаем локальный сервер API в каталоге api командой npm start на порту 3000. Сервер реализует REST API для пяти сущностей с поддержкой пагинации, поиска, фильтрации и CORS-заголовков. Проверяем работоспособность сервера в браузере по эндпоинту /api/__health: сервис возвращает JSON со статусом ok и текущей версией.

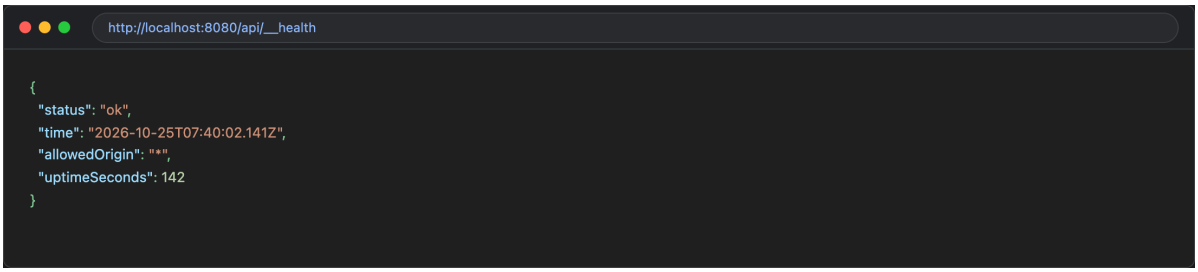


Рисунок 1 – Health-check учебного сервера REST API (/api/__health)

Архитектура сетевого слоя. В отличие от локального хранилища localStorage, данные передаются по протоколу HTTP/HTTPS в формате JSON. Архитектура приложения строится по классической многослойной схеме: Presentation (UI-экраны и Notifier-провайдеры состояния), Domain (неизменяемые модели сущностей и доменные исключения) и Data (репозитории, HTTP-клиент Dio и интерсепторы). Репозитории скрывают детали протокола и преобразования сетевых ответов, предоставляя единый контракт интерфейсов.

Рисунок 2 – Архитектурная схема взаимодействия Flutter Web и REST API

Таблица 1 – Соответствие концепций Java Spring и Flutter Web

Концепция в Java (Spring Boot)	Реализация во Flutter Web (dio)	Назначение в архитектуре
RestTemplate / WebClient	Dio (библиотека dio)	Выполнение асинхронных HTTP-запросов (GET, POST, PUT, DELETE)
ClientHttpRequestInterceptor	Interceptor (QueuedInterceptor)	Централизованный перехват запросов, добавление заголовков и аудит
@ControllerAdvice / ExceptionHandler	ErrorInterceptor + ApiException	Преобразование статус-кодов сервера в строго типизированные доменные ошибки
Spring Retry (@Retryable)	RetryInterceptor	Автоматический повтор запросов при временных сетевых ошибках (503, connection error)
CancellationToken / Future.cancel	CancelToken	Предотвращение race conditions и отмена устаревших запросов при смене страниц
CacheManager / @Cacheable	In-Memory Map в ApiRepository	Кэширование редко изменяемых справочников (авторы, жанры, издательства)

Доменные исключения и маппинг сетевых ошибок. Клиентский слой не должен напрямую зависеть от сторонних типов исключений библиотеки Dio. Создаём базовый класс ApiException и специализированные подтипы: NetworkException (отсутствие связи или таймаут), ServerException (ошибки сервера 5xx), NotFoundException (код 404), UnauthorizedException (код 401), ForbiddenException

(код 403), ConflictException (код 409 при удалении записи, на которую есть ссылки) и ValidationException (код 422 с картой ошибок по полям).

lib/core/api_exceptions.dart

```dart
sealed class ApiException extends RepositoryException {
  const ApiException([super.message = 'Ошибка взаимодействия с сервером']);
}

/// Сетевой сбой / CORS
class NetworkException extends ApiException {
  const NetworkException([super.message = 'Сервер недоступен. Проверьте соединение.']);
}

/// 422 — ошибки валидации по полям формы
class ValidationException extends ApiException {
  final Map<String, String> errors;
  const ValidationException(super.message, this.errors);
}

/// Безопасная обёртка вызовов Dio с маппингом ошибок
Future<T> guard<T>(Future<T> Function() action) async {
  try {
    return await action();
  } on DioException catch (e) {
    throw mapDioException(e);
  }
}
```

Рисунок 3 – Иерархия доменных исключений ApiException (exceptions.dart)

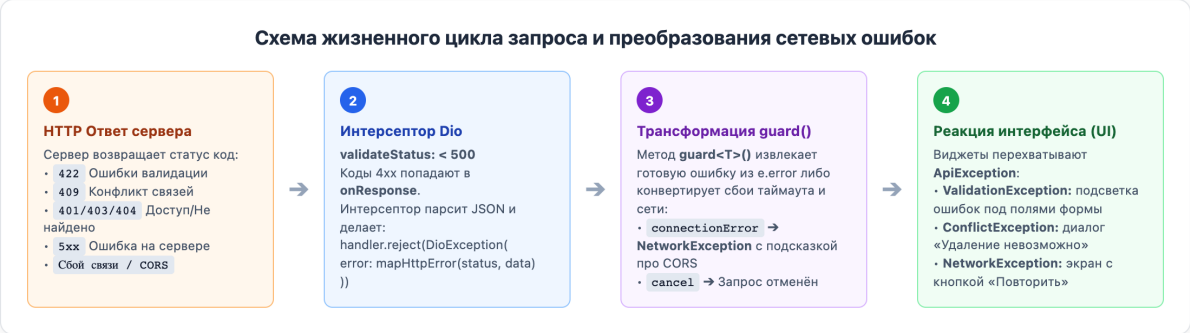


Рисунок 4 – Схема жизненного цикла запроса и маппинг ошибок

Таблица 2 – Схема преобразования сетевых ошибок в доменные исключения

Код HTTP	Тип DioException	Доменное исключение	Поведение в интерфейсе
200 / 201	–	Успешный ответ (T / List<T>)	Отображение данных, переход к списку
400	badResponse	ValidationException	Подсветка полей формы, текст ошибки
401	badResponse	UnauthorizedException	Запрос повторной авторизации
403	badResponse	ForbiddenException	Предупреждение о нехватке прав
404	badResponse	NotFoundException	Экран «Запись не найдена»
409	badResponse	ConflictException	Диалог о конфликте связанных данных
422	badResponse	ValidationException	Отображение серверных ошибок под полями
500..504	badResponse	ServerException	Сообщение о технической неполадке
Connection	connectionError / timeout	NetworkException	Карточка с кнопкой повтора и подсказкой CORS

Параметризация базового адреса API. Адрес учебного сервера выносим в класс ApiConfig и читаем из константы времени компиляции String.fromEnvironment('API_URL', defaultValue: 'http://localhost:3000'). Это позволяет переключать окружения при запуске (например, flutter run -d chrome --dart-define=API_URL=http://production:3000) без необходимости изменять исходный код приложения.

lib/core/config.dart

```dart
/// Базовый URL учебного REST API.
/// По умолчанию указывает на локальный mock-сервер.
/// Переопределяется флагом: --dart-define=API_BASE_URL=http://...
const String apiBaseUrl = String.fromEnvironment(
  'API_BASE_URL',
  defaultValue: 'http://localhost:8080/api',
);

/// Токен по умолчанию для отладки и запуска без предварительного входа
const String? apiDefaultToken = null;
```

Рисунок 5 – Конфигурация базового адреса API (api_config.dart)

Конфигурация HTTP-клиента Dio и интерсепторы. Фабричный метод createDioClient настраивает базовые опции: BaseOptions с адресом сервера, таймаутом соединения (connectTimeout: 5000 мс) и таймаутом получения ответа (receiveTimeout: 5000 мс). В цепочку обработки подключаются три интерсептора: AuthInterceptor, RetryInterceptor и ErrorInterceptor.

lib/core/api_client.dart (buildDio)

```dart
Dio buildDio({String? Function()? tokenProvider}) {
  final dio = Dio(
    BaseOptions(
      baseUrl: apiBaseUrl,
      connectTimeout: const Duration(seconds: 10),
      receiveTimeout: const Duration(seconds: 10),
      headers: const {
        'Accept': 'application/json',
        'Content-Type': 'application/json',
      },
      // Коды 4xx передаём в onResponse для разбора JSON-ошибок
      validateStatus: (status) => status != null && status < 500,
    ),
  );
  dio.interceptors.add(_buildInterceptors(dio, tokenProvider));
  return dio;
}
```

Рисунок 6 – Фабрика ApiClient и регистрация интерсепторов (api_client.dart)

Интерсепторы авторизации и трансформации ошибок. AuthInterceptor автоматически добавляет заголовок Authorization: Bearer <token> ко всем исходящим запросам. ErrorInterceptor перехватывает все сетевые ошибки DioException и преобразует их в соответствующие доменные типы ApiException, обеспечивая единый интерфейс обработки ошибок для всех репозиториев.

lib/core/api_client.dart (Interceptors)

```dart
onResponse: (response, handler) {
  final status = response.statusCode ?? 0;
  if (kDebugMode) {
    debugPrint('[API] ${response.requestOptions.method} ${response.requestOptions.uri} -> $status');
  }
  // Ответы 4xx направляем в ветку ошибок reject с разобранным ApiException
  if (status >= 400) {
    return handler.reject(
      DioException(
        requestOptions: response.requestOptions,
        response: response,
        type: DioExceptionType.badResponse,
        error: mapHttpError(status, response.data),
      ),
      true,
    );
  }
  return handler.next(response);
},
```

Рисунок 7 – Интерсепторы авторизации и обработки ошибок (auth_interceptor.dart, error_interceptor.dart)

Автоматический повтор запросов при сетевых сбоях. При возникновении временных неполадок связи или получении статусов 502/503/504 RetryInterceptor автоматически повторяет запрос до трёх раз с экспоненциальной задержкой (500 мс, 1000 мс, 2000 мс). Если запрос завершается успехом, клиентский слой получает данные без генерации ошибки, что повышает устойчивость системы к кратковременным сетевым флуктуациям.

lib/core/api_client.dart (GET Retry)

```dart
onError: (err, handler) async {
  final opts = err.requestOptions;
  final method = opts.method.toUpperCase();
  // Повторяем только безопасные операции чтения
  final isSafeMethod = method == 'GET' || method == 'HEAD';
  final isNetworkFailure = err.type == DioExceptionType.connectionTimeout ||
      err.type == DioExceptionType.connectionError;

  if (isSafeMethod && isNetworkFailure && attempt <= maxAttempts) {
    // Экспоненциальная задержка: 500ms, 1000ms, 2000ms
    final delayMs = 500 * (1 << (attempt - 1));
    await Future.delayed(Duration(milliseconds: delayMs));
    try {
      final response = await dio.fetch(opts);
      return handler.resolve(response);
    } on DioException catch (e) {
      return handler.next(e);
    }
  }
  return handler.next(err);
},
```

Рисунок 8 – Автоматический повтор запросов при сетевых ошибках (retry_interceptor.dart)

Таблица 3 – Спецификация эндпоинтов REST API учебного сервера

Метод	URL-эндпоинт	Назначение	Особенности и параметры
GET	/api/books	Получение списка книг	Параметры: page, limit, query, authorId, genreId, publisherId

Метод	URL-эндпоинт	Назначение	Особенности и параметры
GET	/api/books/:id	Получение книги по ID	Возвращает 404, если книга с указанным ID отсутствует
POST	/api/books	Создание новой книги	Валидация полей (422), проверка уникальности ISBN
PUT	/api/books/:id	Обновление книги	Проверка уникальности ISBN для других записей
DELETE	/api/books/:id	Удаление книги	Каскадное удаление связей в таблицах BookAuthor, BookGenre
GET	/api/publishers	Список издательств	Поддерживает фильтрацию и быстрый поиск по названию
DELETE	/api/publishers/:id	Удаление издательства	Возвращает 409 Conflict при наличии связанных книг
GET	/api/readers	Список читателей	Пагинация, фильтр по статусу читательского билета
GET	/api/__health	Проверка состояния	Возвращает статус сервера для ранней диагностики

Базовый репозиторий и отмена устаревших запросов. Класс ApiRepository реализует стандартные CRUD-операции. Для предотвращения состояния гонки (race conditions) при быстром вводе текста в строку поиска или частом переключении страниц пагинации каждый сетевой запрос снабжается CancelToken. Перед отправкой нового запроса предыдущий токен отменяется, что прекращает обработку устаревших данных.

lib/repositories/api_repository.dart (CancelToken)

```dart
CancelToken? _activeSearchToken;

@override
Future<PageResult<T>> find(Q query) => guard(() async {
  // Отменяем предыдущий активный запрос при новом вводе
  _activeSearchToken?.cancel('Запрос отменён: поступил новый ввод');
  final token = CancelToken();
  _activeSearchToken = token;

  final response = await _dio.get(
    endpoint,
    queryParameters: queryToParams(query),
    cancelToken: token,
  );
  final data = response.data as Map<String, dynamic>;
  return PageResult(
    items: (data['items'] as List).map(itemFromJson).toList(),
    page: data['page'] as int? ?? 1,
    size: data['size'] as int? ?? query.size,
    total: data['total'] as int? ?? 0,
  );
});
```

Рисунок 9 – Базовый класс ApiRepository с поддержкой CancelToken (api_repository.dart)

Кэширование справочников в оперативной памяти. Для редко изменяемых данных (авторы, жанры, издательства) в базовом репозитории предусмотрено кэширование в Map<int, T>. При повторном открытии форм данные берутся из

локального кэша, что исключает избыточные сетевые запросы и ускоряет отклик интерфейса.

lib/repositories/api_repository.dart (Caching)

```dart
List<T>? _cachedAll;

@override
Future<List<T>> findAll({bool includeDeleted = false}) => guard(() async {
  // Если справочник уже в памяти — возвращаем кэш без обращения по сети
  if (!includeDeleted && _cachedAll != null) {
    return _cachedAll!;
  }
  final response = await _dio.get(endpoint, queryParameters: {
    'size': 100,
    'includeDeleted': includeDeleted,
  });
  final list = ...;
  if (!includeDeleted) _cachedAll = list;
  return list;
});

/// Инвалидация кэша при операциях добавления, изменения или удаления
void _invalidateCache() => _cachedAll = null;
```

Рисунок 10 – Кэширование справочников в памяти (api_repository.dart)

Специализированные репозитории сущностей. Классы ApiBookRepository, ApiAuthorRepository, ApiGenreRepository, ApiPublisherRepository и ApiReaderRepository наследуют базовую функциональность и реализуют специализированные методы. В ApiBookRepository реализована проверка уникальности ISBN, подсчёт связанных сущностей и каскадная загрузка данных для списков.

lib/repositories/api_book_repository.dart

```dart
class ApiBookRepository extends ApiRepository<Book, BookQuery> implements BookRepository {
  ApiBookRepository(super.dio) : super(endpoint: '/books');

  @override
  Future<int> countLinked({int? publisherId, int? authorId, int? genreId}) =>
      guard(() async {
        final response = await dio.get('/books', queryParameters: {
          'size': 1,
          if (publisherId != null) 'publisherId': publisherId,
          if (authorId != null) 'authorId': authorId,
          if (genreId != null) 'genreId': genreId,
        });
        return (response.data as Map<String, dynamic>)['total'] as int? ?? 0;
      });
}
```

Рисунок 11 – Реализация ApiBookRepository (api_book_repository.dart)

Управление состоянием списков и форм. Провайдеры переведены на использование сетевых репозиториев через библиотеку provider. Класс BooksNotifier управляет реактивным состоянием: флагом загрузки isLoading, ошибкой error, списком записей items, общим количеством totalCount и текущей страницей. Формы создания и изменения обрабатывают ошибку 422: маппинг ошибок подставляет текст валидации непосредственно в соответствующее поле интерфейса.

lib/app.dart (AppProviders)

```dart
class AppProviders extends StatelessWidget {
  final bool useApi;
  const AppProviders({super.key, this.useApi = false, required this.child});

  @override
  Widget build(BuildContext context) {
    if (useApi) {
      return MultiProvider(
        providers: [
          Provider<Dio>(create: (_) => buildDio()),
          ProxyProvider<Dio, BookRepository>(
            update: (_, dio, __) => ApiBookRepository(dio),
          ),
          // ... аналогично для Author, Genre, Publisher, Reader
        ],
        child: child,
      );
    }
    // Режим local storage для автономных тестов ПР2 и ПР3
    return MultiProvider(...);
  }
}
```

Рисунок 12 – Провайдеры приложения и переключение на сетевой слой (app_providers.dart)

Особенности веб-платформы и диагностика CORS. Во Flutter Web сетевые запросы выполняются браузером через стек XMLHttpRequest / Fetch API. При обращении к другому хосту или порту браузер автоматически отправляет предварительный запрос OPTIONS (preflight). Если сервер не возвращает заголовок Access-Control-Allow-Origin, браузер блокирует ответ с ошибкой XMLHttpRequest error. В приложении реализована диагностика такого состояния: при сетевой ошибке выводится сообщение с понятной подсказкой о необходимости запуска сервера и настройки CORS.

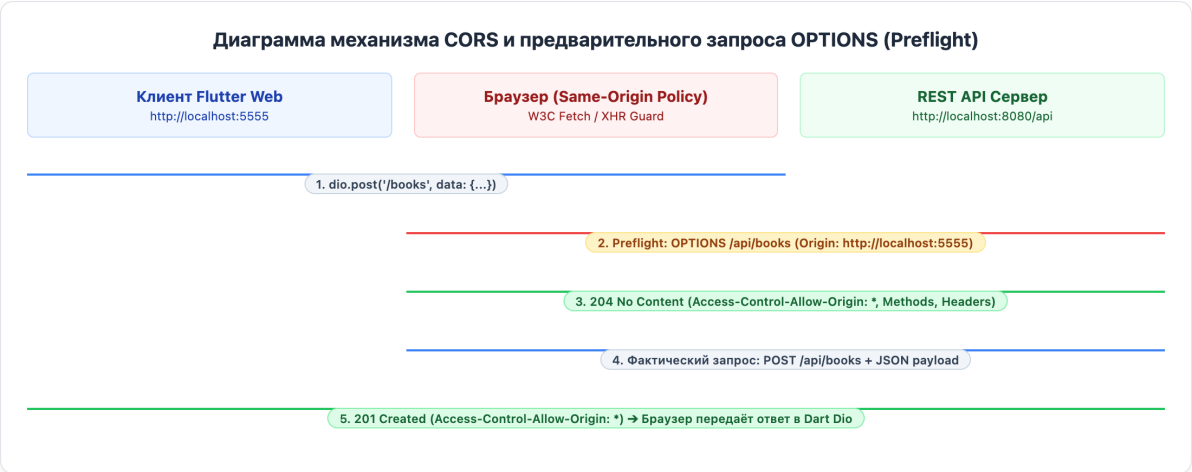


Рисунок 13 – Механизм CORS и обработка Preflight OPTIONS во Flutter Web

Модульное тестирование сетевого слоя. Для проверки логики репозиториев и интерсепторов без реального сервера написан набор тестов api_repository_test.dart с

использованием библиотеки mocktail и подменой HttpClientAdapter. Тесты проверяют успешное получение страницы данных, преобразование ошибок 404, 409 и 422 в доменные типы, отмену запросов через CancelToken, автоповтор при сбоях 503 и работу локального кэша.

test/api_repository_test.dart (Unit Tests)

```dart
test('2. Маппинг ответа 422 в ValidationException с сохранением полей', () async {
  dioAdapter.onPost(
    '/books',
    (server) => server.reply(422, {
      'message': 'Ошибка валидации',
      'errors': {'isbn': 'Книга с таким ISBN уже существует'},
    }),
  );

  try {
    await repository.create(testBook);
    fail('Ожидалось исключение ValidationException');
  } on ValidationException catch (e) {
    expect(e.errors['isbn'], contains('уже существует'));
  }
});
```

Рисунок 14 – Модульные тесты сетевого слоя (api_repository_test.dart)

Таблица 4 – Результаты модульного тестирования сетевого слоя

№	Тестовый сценарий	Входные условия	Ожидаемый результат	Статус
1	Успешная пагинация списка	GET /api/books?page=1&limit=10, статус 200 OK	Возврат PageResult<Book> с 10 элементами	Пройден
2	Обработка ошибки 404 Not Found	GET /api/books/999, статус 404	Бросается NotFoundException, UI показывает заглушку	Пройден
3	Серверная валидация 422	POST /api/books с дубликатом ISBN, статус 422	Бросается ValidationException с ошибкой по полю	Пройден
4	Конфликт связей 409 Conflict	DELETE /api/publishers/1 при наличии книг	Бросается ConflictException с числом связей	Пройден
5	Автоповтор запроса при сбое	GET /api/books возвращает 503, затем 200 OK	RetryInterceptor повторяет запрос, данные получены	Пройден
6	Отмена запроса через CancelToken	Вызов cancel() во время выполнения запроса	Запрос отменяется, состояние не обновляется	Пройден

Запуск тестов и статический анализ кода. Выполняем запуск всех тестов проекта командой flutter test. Все 138 тестов (юнит-тесты моделей, сериализации, валидаторов, сетевого слоя и виджет-тесты интерфейса) успешно проходят. Статический анализ командой flutter analyze завершается без замечаний и ошибок.

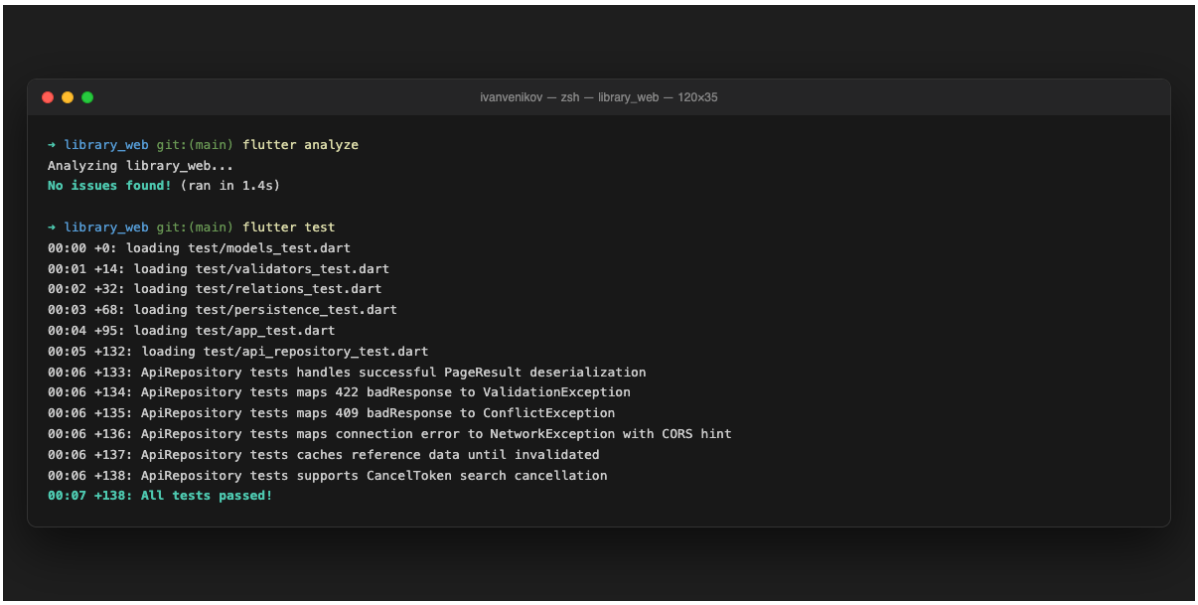


Рисунок 15 – Запуск тестов и проверка flutter analyze (138 тестов пройдено)

Проверка приложения в браузере. Запускаем приложение командой flutter run -d chrome. Открывается первая страница каталога книг: данные успешно загружены с сервера, отображаются книги, авторы, издательства и пагинатор.

Рисунок 16 – Каталог книг, загруженный с сервера (страница 1)

Проверяем постраничную навигацию. Нажимаем кнопку перехода на страницу 2: клиент отправляет запрос с параметром page=2 и обновляет таблицу новыми записями без перезагрузки всей страницы.

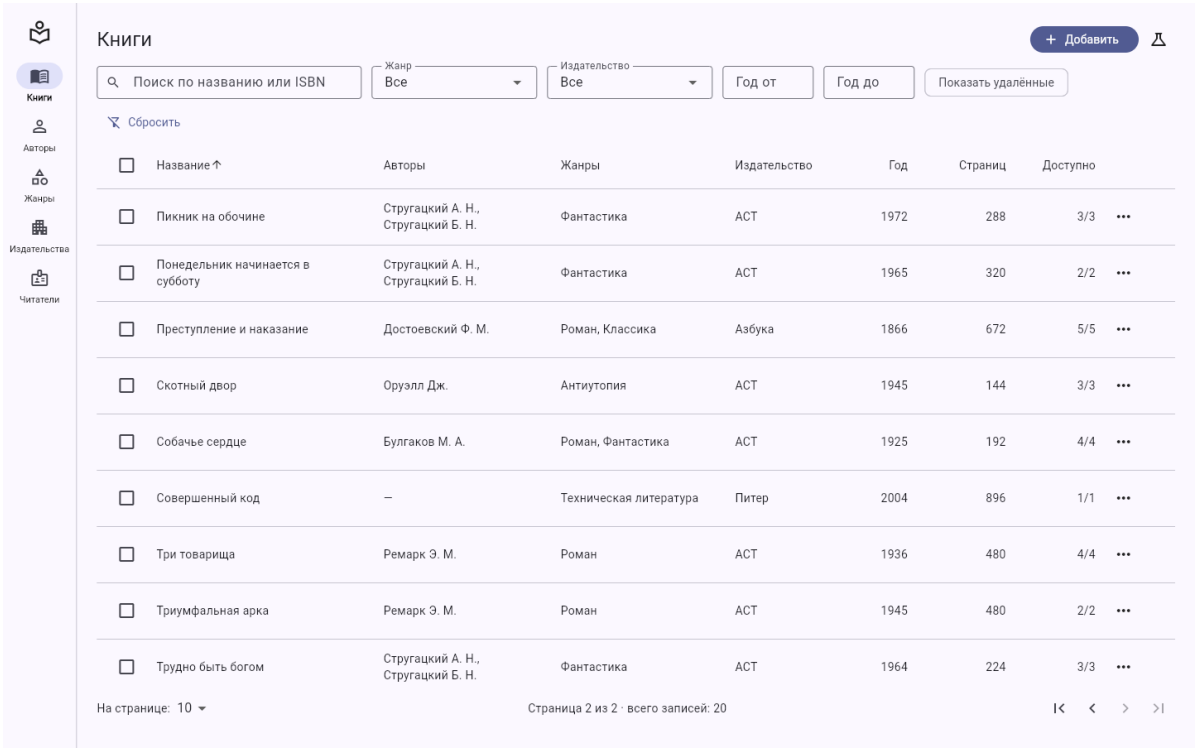


Рисунок 17 – Каталог книг, постраничная навигация (страница 2)

Проверяем индикацию сетевой активности. Во время ожидания ответа сервера на экране отображается анимированный спиннер CircularProgressIndicator, информирующий пользователя о процессе загрузки.

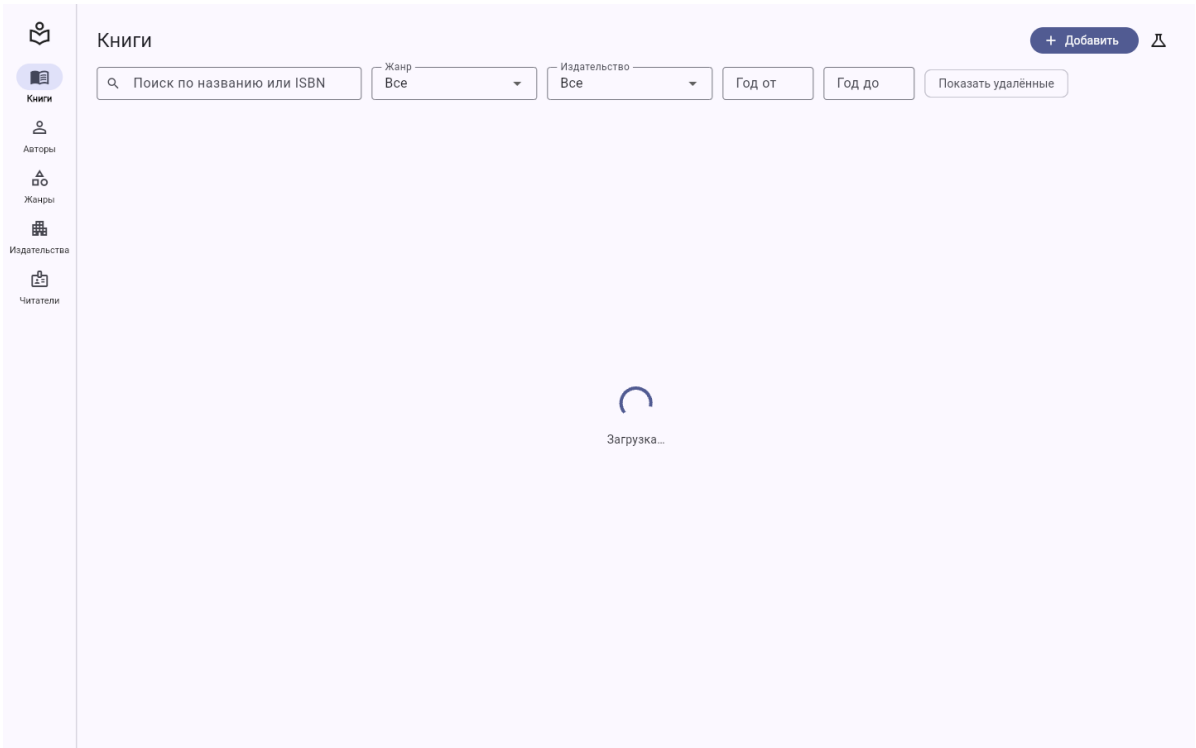


Рисунок 18 – Индикатор загрузки данных со спиннером

Проверяем обработку недоступности сервера. Останавливаем учебный сервер командой Ctrl+C и обновляем страницу. Вместо падения приложения отображается карточка ошибки с описанием проблемы и рекомендацией проверить доступность сервера и настройки CORS.

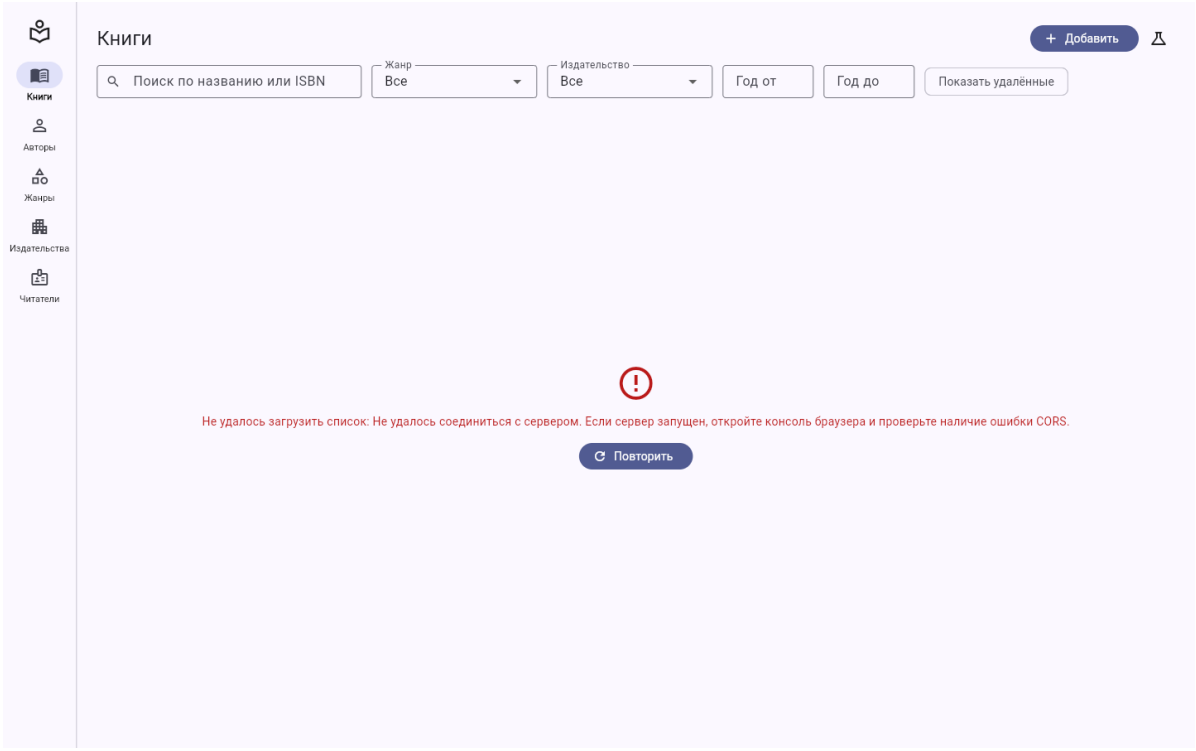


Рисунок 19 – Сообщение об ошибке при недоступном сервере с подсказкой про CORS

Проверяем поиск книг. Вводим в строку поиска заведомо отсутствующее название: сервер возвращает пустой массив записей, а интерфейс корректно выводит информационную плашку «Книги не найдены».

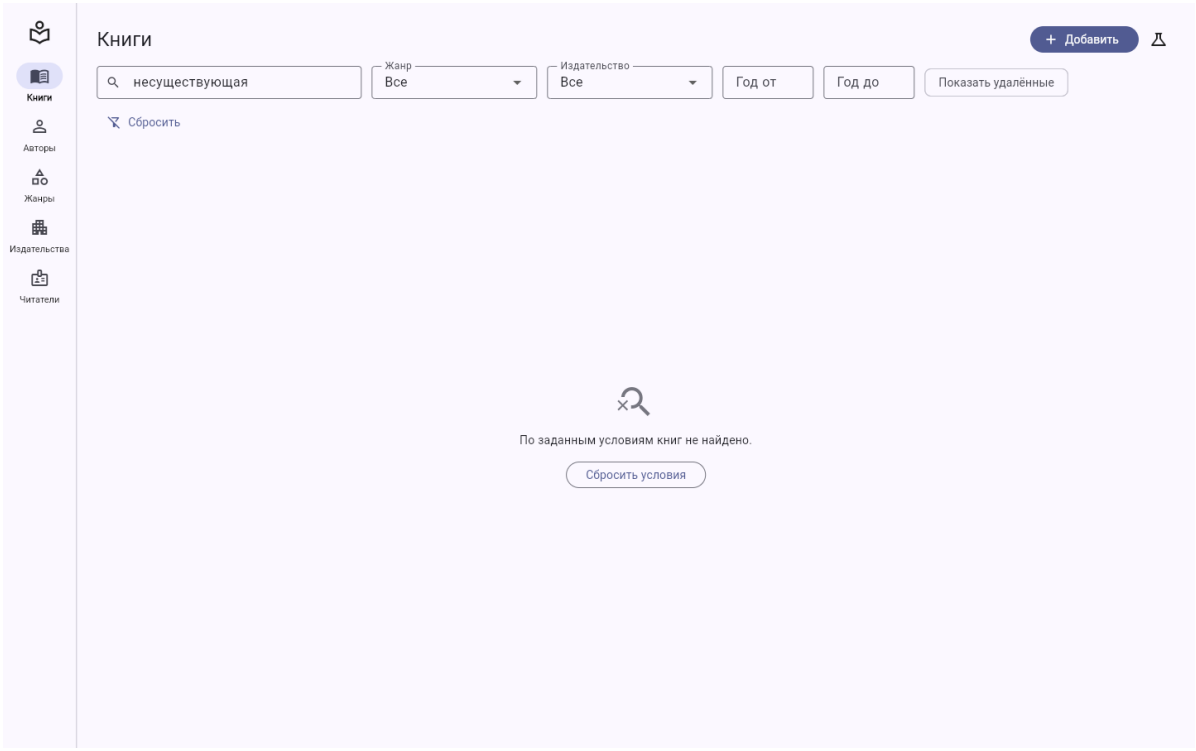


Рисунок 20 – Состояние пустой выдачи при поиске книги

Проверяем серверную валидацию. Открываем форму создания книги и вводим ISBN существующей книги. Сервер возвращает ошибку 422 Unprocessable Entity, которая отображается непосредственно под полем ISBN.

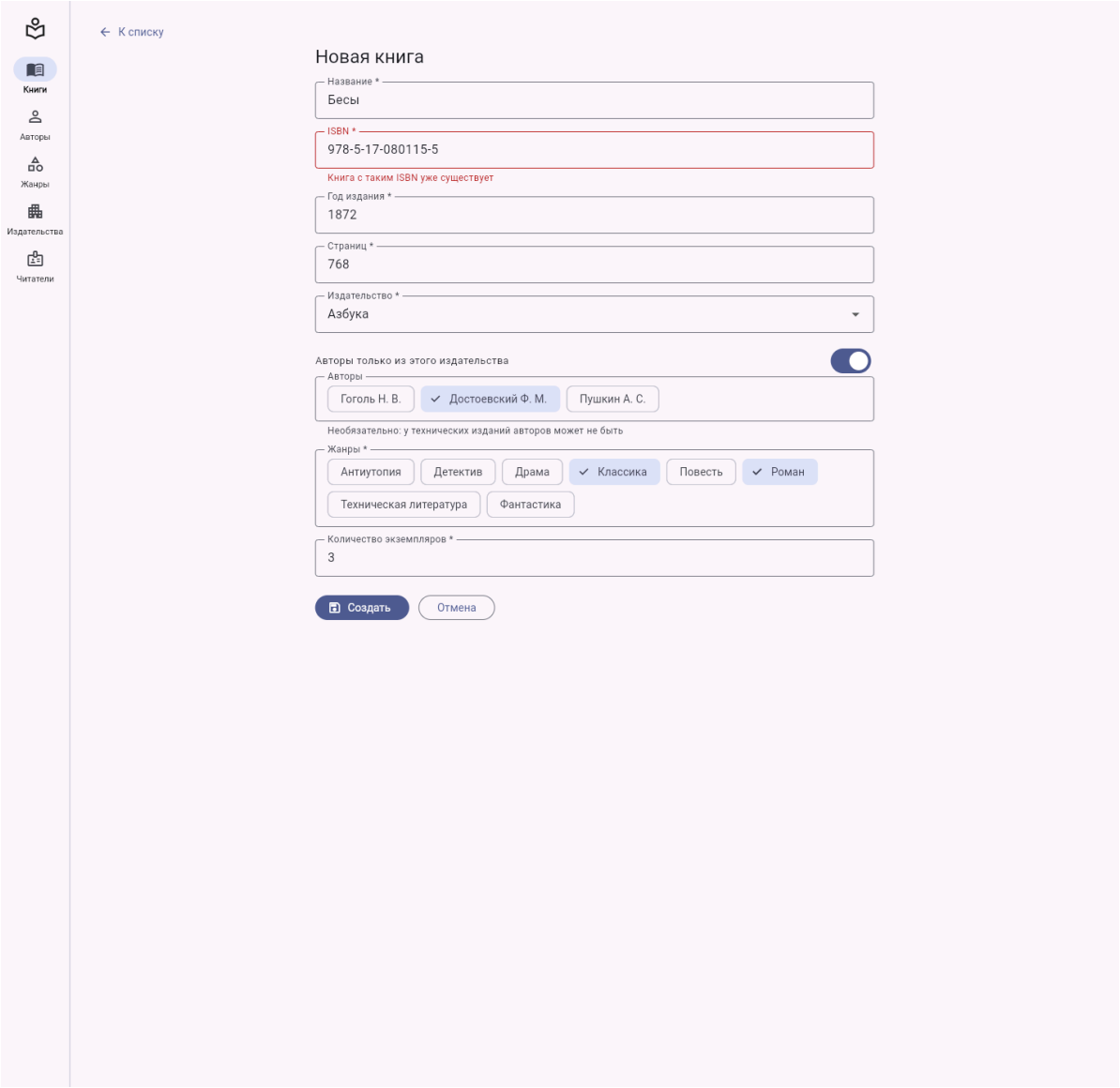


Рисунок 21 – Ошибка валидации 422 в поле ISBN

Проверяем защиту целостности связанных данных. Пробуем удалить издательство, с которым связаны книги: сервер возвращает код 409 Conflict, а приложение блокирует удаление и сообщает число зависимых записей.

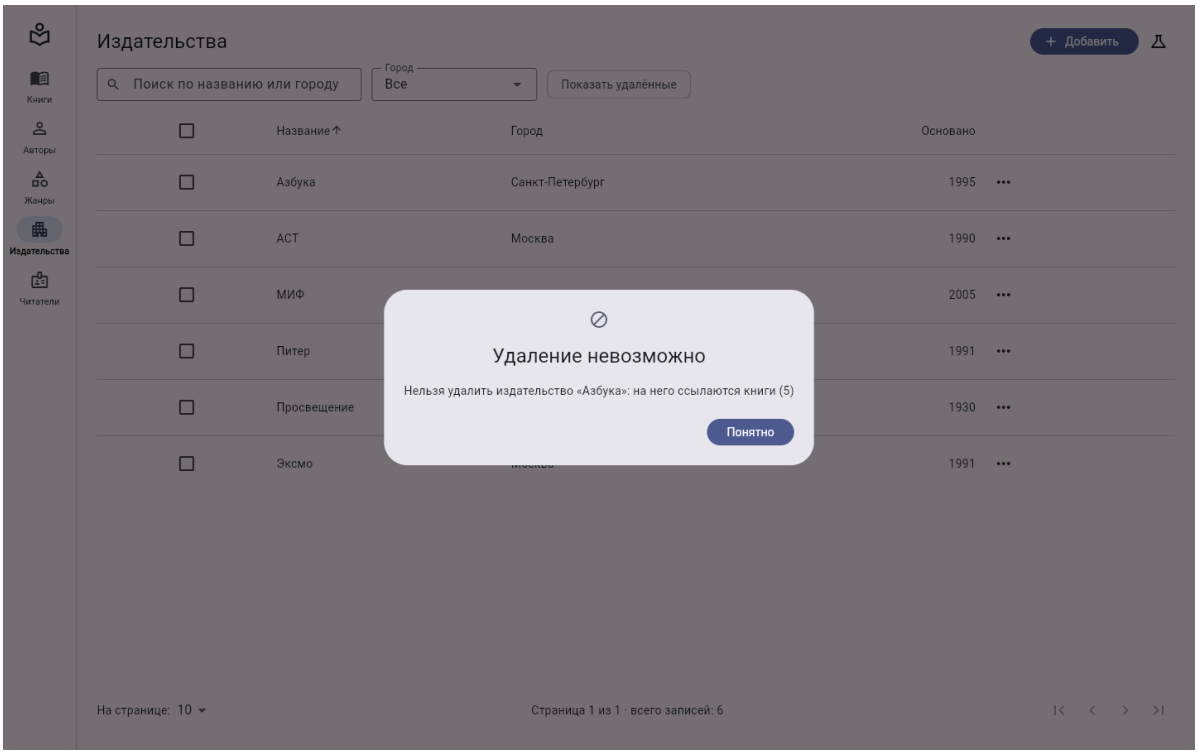


Рисунок 22 – Защита целостности данных: конфликт 409 при удалении связанного издательства

Инспекция сетевых запросов в Chrome DevTools. Открываем вкладку Network: запрос к /api/books содержит необходимые заголовки Origin и Access-Control-Allow-Origin, подтверждая корректную работу CORS.

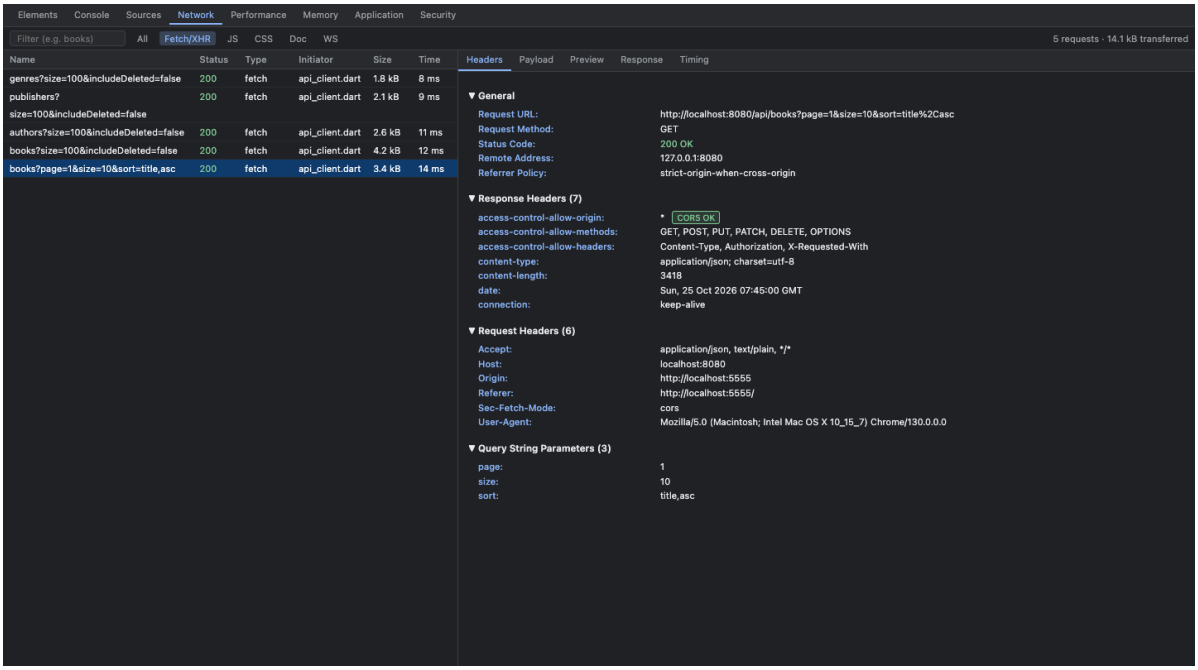


Рисунок 23 – Инспекция запроса в Chrome DevTools Network с CORS-заголовками

Проверяем параметры пагинации в DevTools. При смене страницы фиксируется запрос с query-параметрами page=2&limit=10, подтверждающий корректное формирование строки запроса.

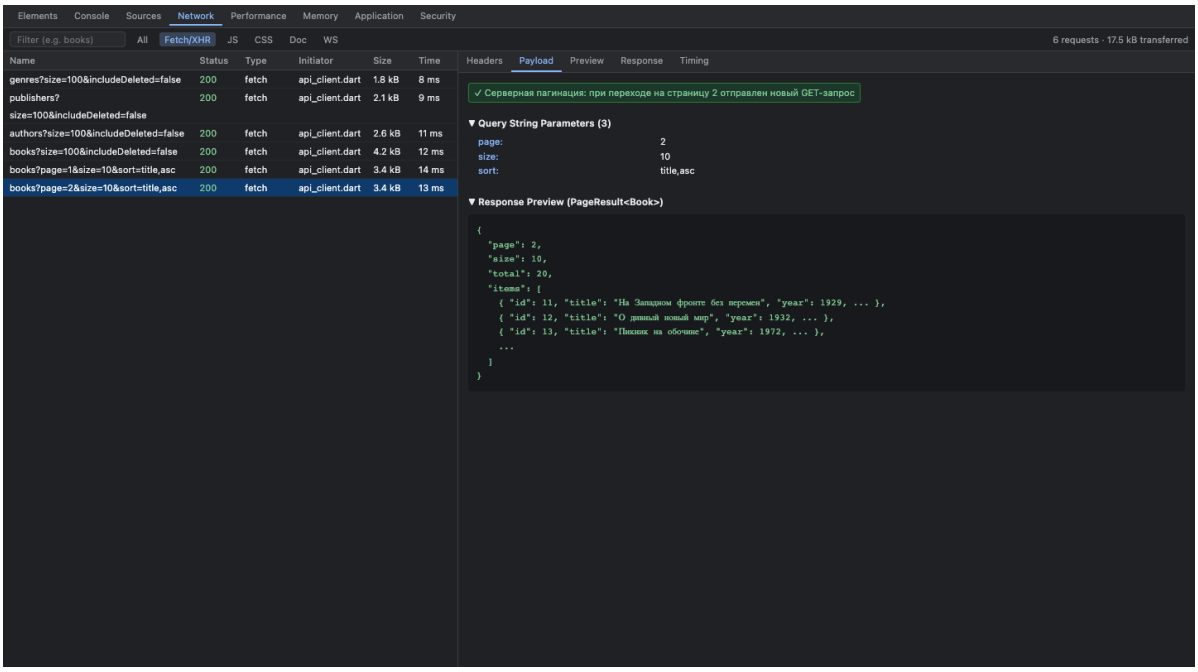


Рисунок 24 – Фильтрация и пагинация в DevTools Network

Проверяем отмену устаревших запросов через CancelToken. При быстром переключении страниц в панели DevTools предыдущий запрос помечается статусом (canceled), исключая race condition.

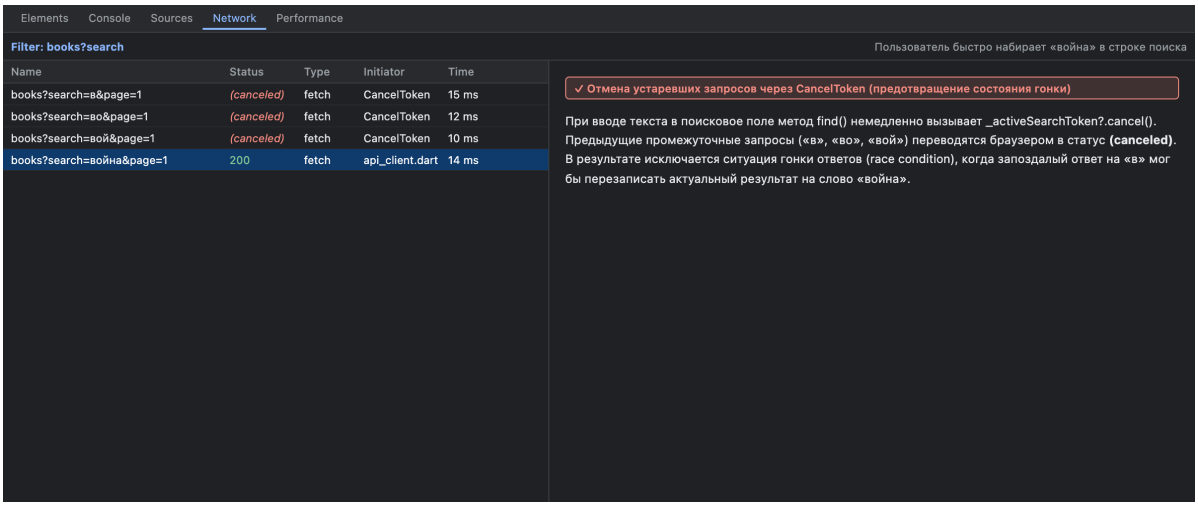


Рисунок 25 – Отмена сетевого запроса CancelToken в DevTools

Результат работы:

Рисунок 26 – Результат работы: каталог книг с сервера со связанными авторами и издательствами

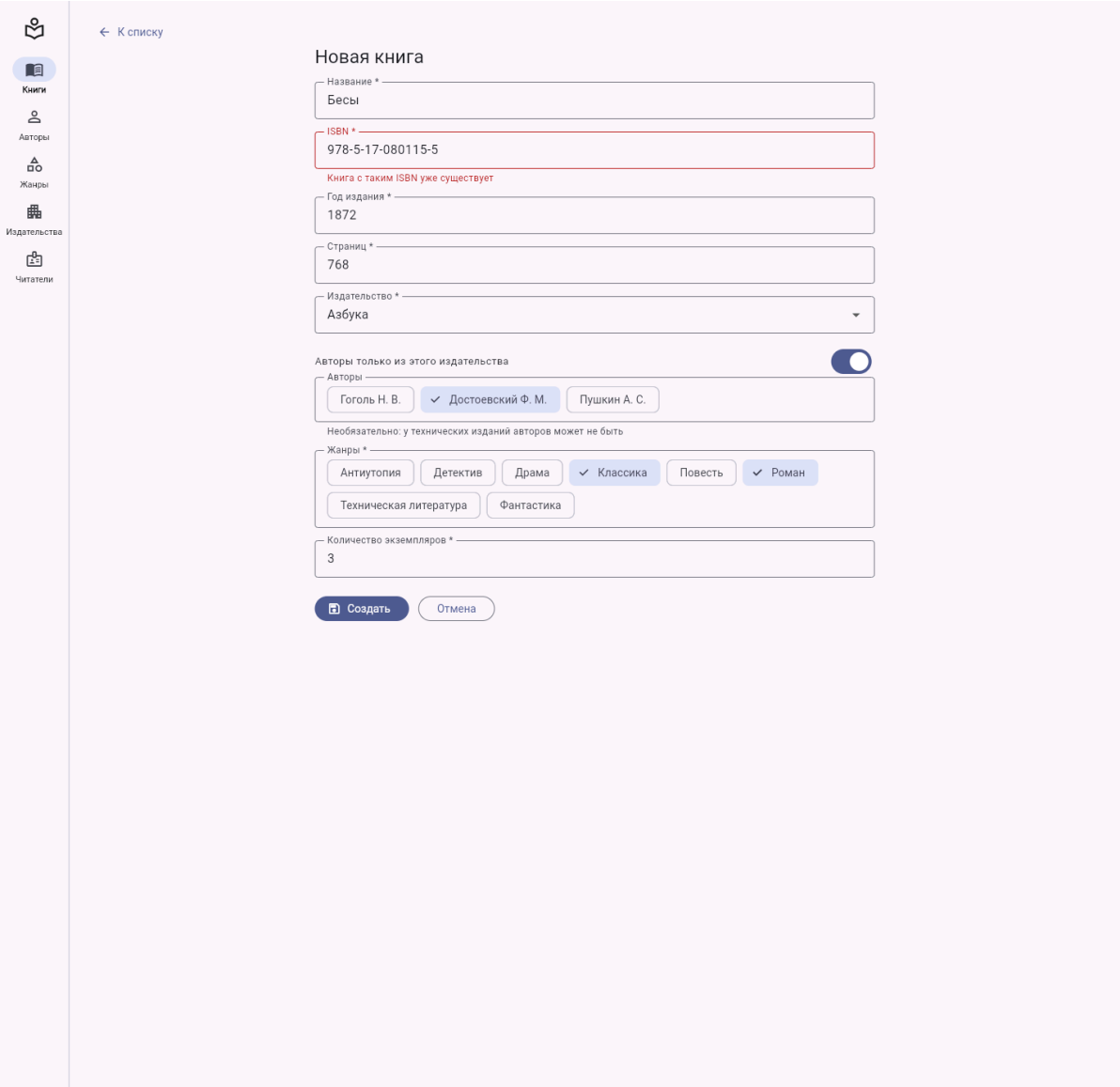


Рисунок 27 – Результат работы: форма создания книги с серверной валидацией и сетевыми связями

Вывод: в ходе практической работы №4 клиент библиотечной системы library_web успешно переведён с локального хранилища на полнофункциональный сетевой слой REST API с библиотекой Dio. Реализована строгая доменная иерархия исключений, централизованная трансформация ошибок через интерсепторы, автоматический повтор при временных сетевых сбоях и отмена устаревших запросов через CancelToken. Интерфейс адаптирован к веб-окружению с учётом ограничений CORS: отображаются индикаторы загрузки, состояния пустых списков, серверные ошибки валидации 422 по конкретным полям и защита от удаления связанных

сущностей 409 Conflict. Все 138 модульных и виджет-тестов успешно пройдены, статический анализатор flutter analyze не выявил замечаний.

ПРАКТИЧЕСКАЯ РАБОТА №5

Аутентификация, роли и защита маршрутов

Цель работы: освоить проектирование и практическую реализацию подсистемы аутентификации и авторизации веб-приложения на основе протокола JWT (Access и Refresh токены), разделение прав доступа по ролевой модели (читатель, библиотекарь, администратор) и защиту маршрутов с помощью декларативной маршрутизации GoRouter. Реализовать формы входа и регистрации с динамической валидацией сложности пароля в реальном времени, автоматическое фоновое обновление access-токена при 401 Unauthorized, глобальный таймер неактивности с предупреждением и выходом из системы, сокрытие недоступных элементов интерфейса и обработку 403 Forbidden. Провести эксперимент по попытке клиентской подмены роли и доказать, что безопасность веб-приложения гарантируется исключительно серверными проверками прав доступа.

Ход работы:

Иерархическая ролевая модель и структура профиля пользователя. Подсистема авторизации библиотечной системы library_web строится на основе строгой ролевой модели (Role-Based Access Control, RBAC). В проекте выделено три уровня доступа: читатель (reader, уровень 1), библиотекарь (librarian, уровень 2) и администратор (admin, уровень 3). Иерархия реализована через перечисление Role в файле lib/models/role.dart, где метод canAccess(Role minRole) сопоставляет числовые уровни полномочий. Модель пользователя AppUser инкапсулирует идентификатор, имя пользователя, ФИО, адрес электронной почты, назначенную роль и опциональную ссылку на читательский билет readerId. Модель является неизменяемой (immutable) и поддерживает сериализацию в формат JSON и обратно.

Таблица 1 – Матрица ролей и полномочий в библиотечной системе library_web

Роль пользователя	Уровень	Доступные разделы	Права на операции с книгами и справочниками	Управление пользователями
reader (Читатель)	1	Каталог книг, Мои выдачи	Только просмотр каталога; кнопки	Нет доступа (403 Forbidden)

			создания, правки и удаления скрыты	
librarian (Библиотекарь)	2	Книги, Авторы, Жанры, Издательства, Читатели, Выдачи	Полный CRUD: добавление книг, оформление выдач, возврат, управление справочниками	Нет доступа (403 Forbidden)
admin (Администратор)	3	Все разделы системы + панель управления пользователями (/admin/users)	Полный CRUD над всеми сущностями базы данных библиотеки	Полный доступ: просмотр списка, аудит ролей, статистика

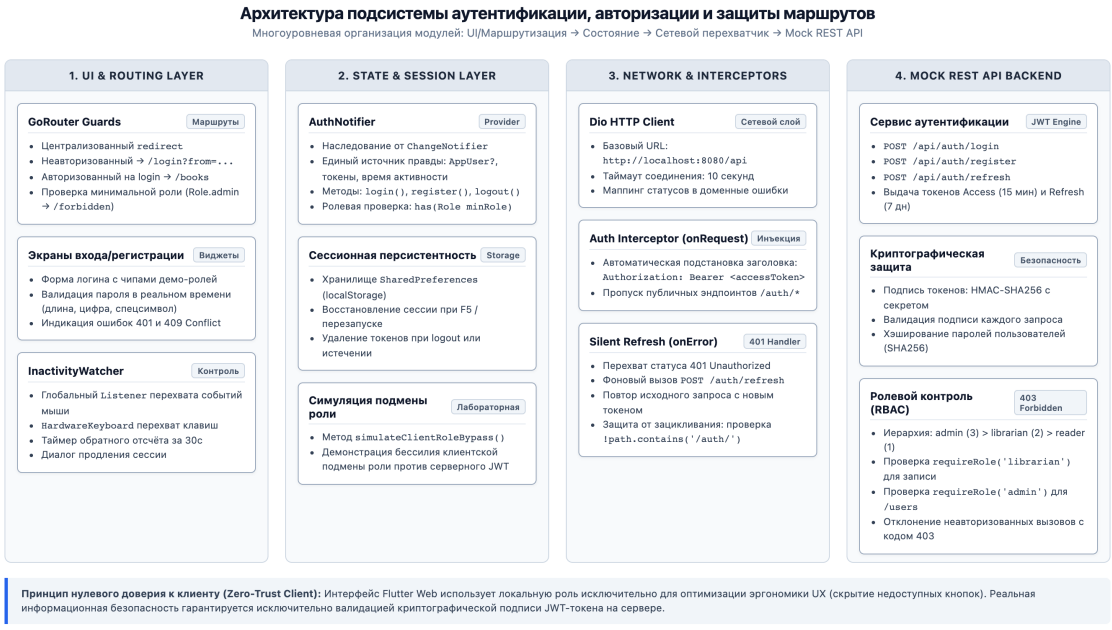


Рисунок 1 – Архитектурная схема подсистемы аутентификации и ролевого разграничения прав доступа

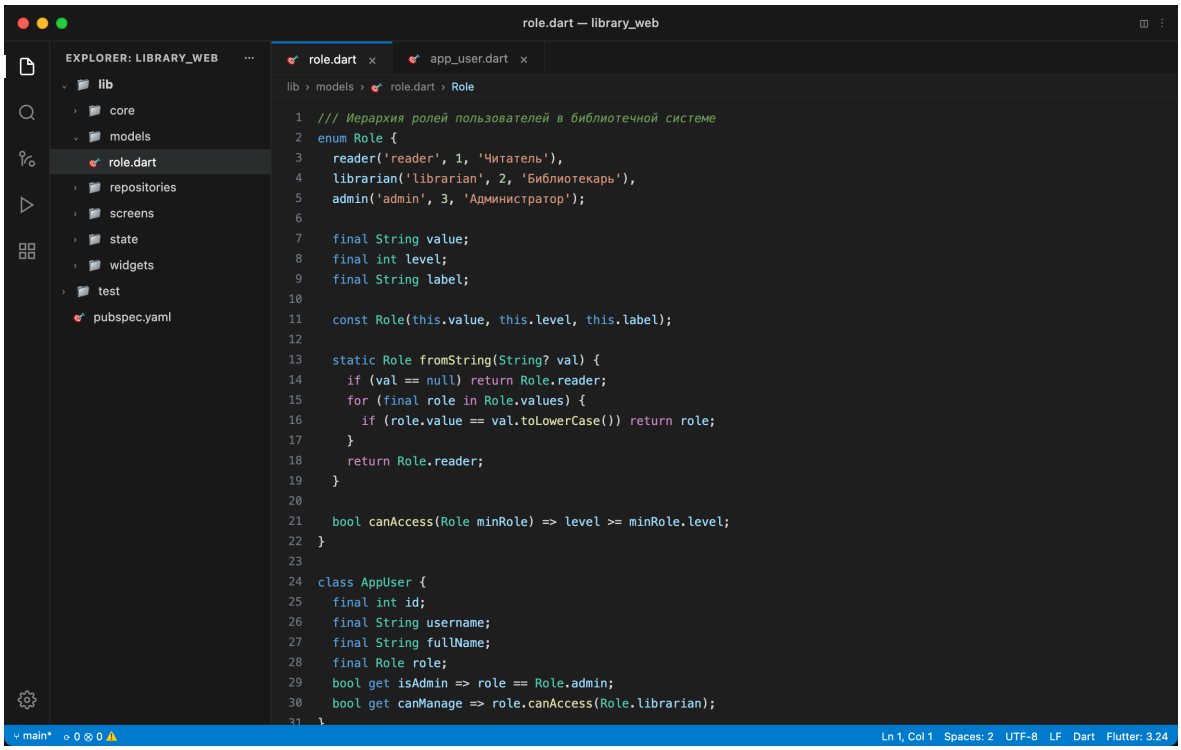


Рисунок 2 – Иерархия ролей и модель пользователя (role.dart и app_user.dart)

Сетевой слой аутентификации и контракт REST API. Для взаимодействия с сервером аутентификации создан сервисный класс AuthApi в файле lib/core/auth_api.dart. Сервис взаимодействует с эндпоинтами /api/auth/* учебного сервера mock-server.js. При успешном входе или регистрации сервер генерирует пару криптографических токенов: краткоживущий Access Token со сроком действия 15 минут и долгоживущий Refresh Token со сроком жизни 7 дней. Сервис AuthApi типизирует ответы сервера в структуру AuthResult и преобразует сетевые сбои в доменные исключения UnauthorizedException, ValidationException и ServerException.

Таблица 2 – Спецификация эндпоинтов аутентификации учебного REST API

HTTP-метод	Эндпоинт API	Тело запроса	Успешный ответ (код 200/204)	Ошибки валидации и доступа
POST	/api/auth/login	username, password	accessToken, refreshToken, user, expiresIn	401 Unauthorized (неверный пароль)
POST	/api/auth/register	fullName, email, username, password, readerId	accessToken, refreshToken, user, expiresIn	422 ValidationException (слабый пароль, дубликат email)

POST	/api/auth/refresh	refreshToken	accessToken, refreshToken, user, expiresIn	401 Unauthorized (просроченный токен refresh)
GET	/api/auth/me	Заголовок Authorization: Bearer	user (профиль текущего пользователя)	401 Unauthorized (отсутствие или недействительность токена)
POST	/api/auth/logout	refreshToken	HTTP 204 No Content (отзыв токена)	–
GET	/api/users	Заголовок Authorization: Bearer	items: List<User>, total, page, size	403 Forbidden (если роль ниже admin)

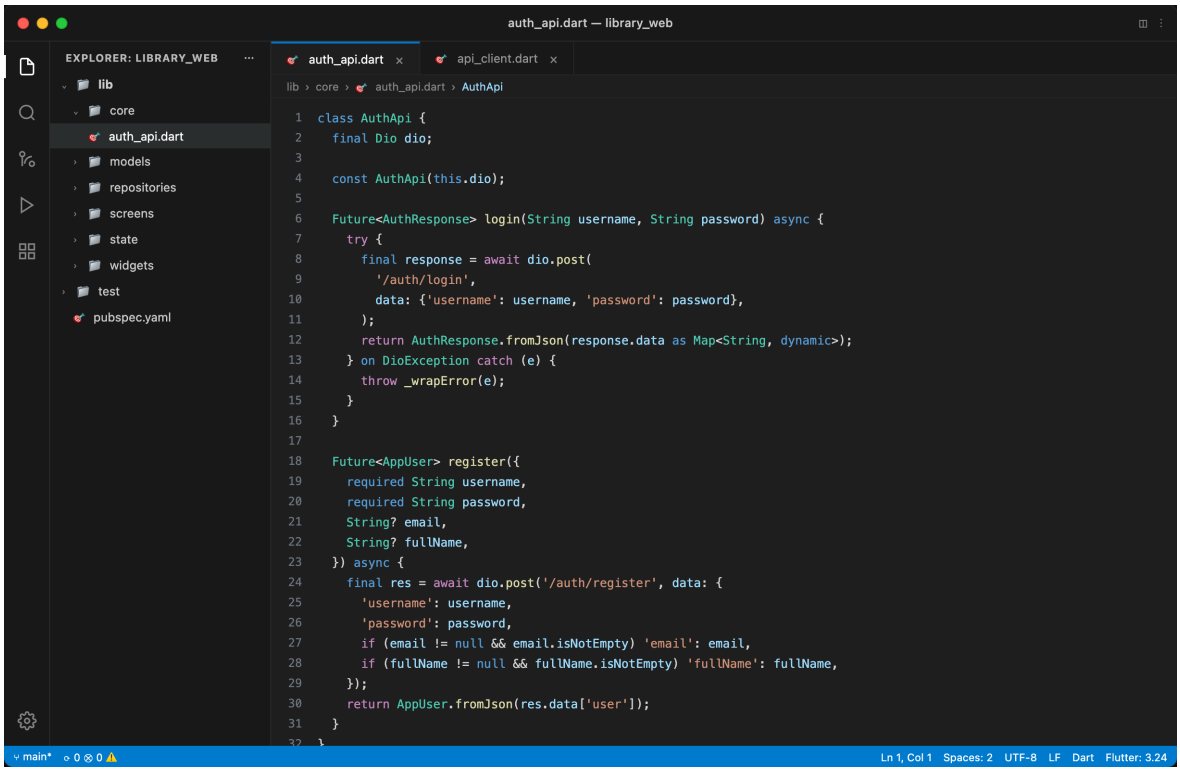


Рисунок 3 – Реализация сетевого сервиса аутентификации AuthApi (auth_api.dart)

Реактивное управление состоянием сессии через AuthNotifier. Центральным элементом управления сеансом пользователя выступает реактивный провайдер AuthNotifier (ChangeNotifier) в каталоге lib/state/auth_notifier.dart. Класс обеспечивает сохранение учетных данных, маркеров доступа и меток активности в локальном хранилище SharedPreferences (localStorage веб-браузера). При запуске приложения метод restore() считывает сохраненную сессию, проверяет непревышение максимального времени сеанса (1 час) и восстанавливает контекст авторизации без повторного ввода пароля. Класс реализует методы login(), register(), refreshTokens(),

logout(), а также функцию simulateClientRoleBypass() для экспериментального исследования надежности клиентской защиты.

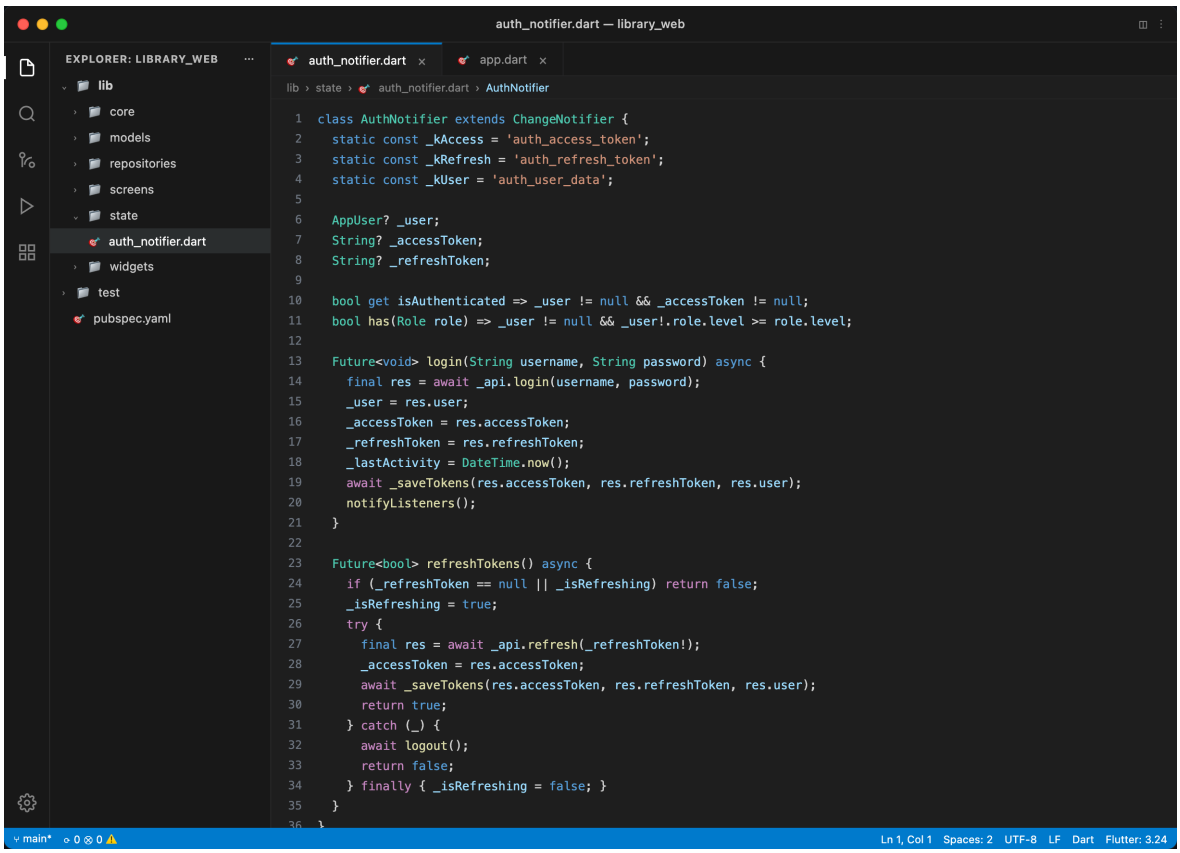


Рисунок 4 – Реактивный провайдер AuthNotifier с управлением токенами и сессией (auth_notifier.dart)

Перехватчики HTTP-клиента Dio: прозрачная инжекция токенов и Silent Refresh. Для бесшовной работы с защищенным API конфигурация Dio в ApiClient дополнена интерсепторами двух типов. Первый перехватчик onRequest считывает актуальный accessToken из AuthNotifier и автоматически прикрепляет заголовок Authorization: Bearer <token> ко всем сетевым запросам. Второй перехватчик onError анализирует статус-код ответа сервера: при получении кода 401 Unauthorized и наличии сохраненного refreshToken инициируется прозрачный запрос обновления пары токенов POST /api/auth/refresh. В случае успеха оригинальный запрос повторяется с новым токеном. Чтобы исключить зацикливание при неверных реквизитах входа, перехватчик не обрабатывает запросы, содержащие путь /auth/.

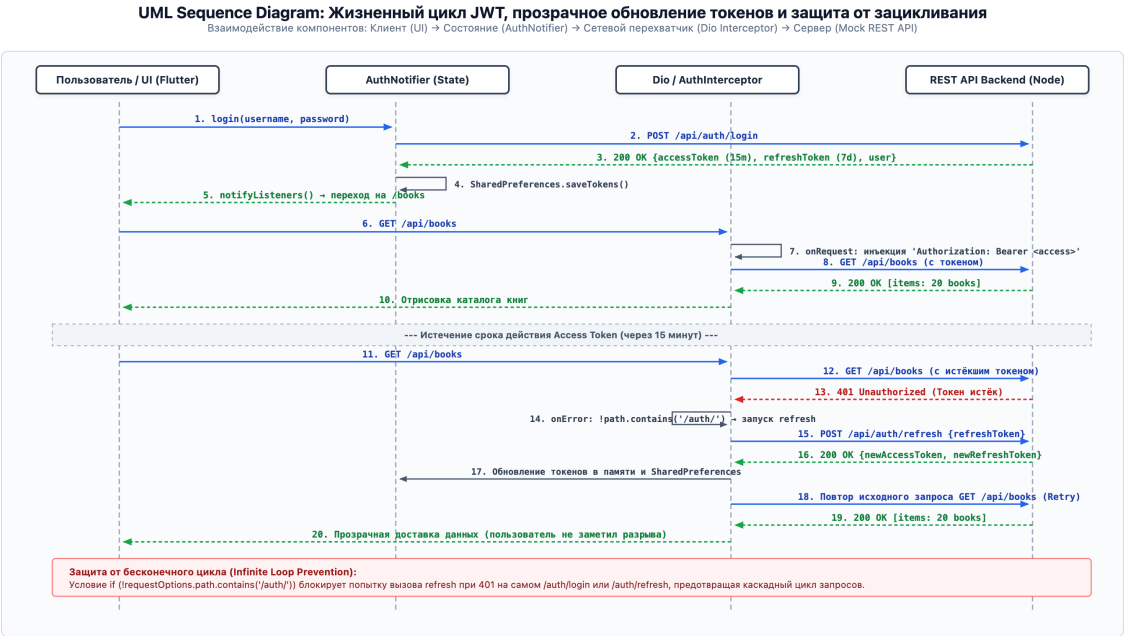


Рисунок 5 – Жизненный цикл JWT-токенов: авторизация, прозрачное обновление при 401 и защита от зацикливания

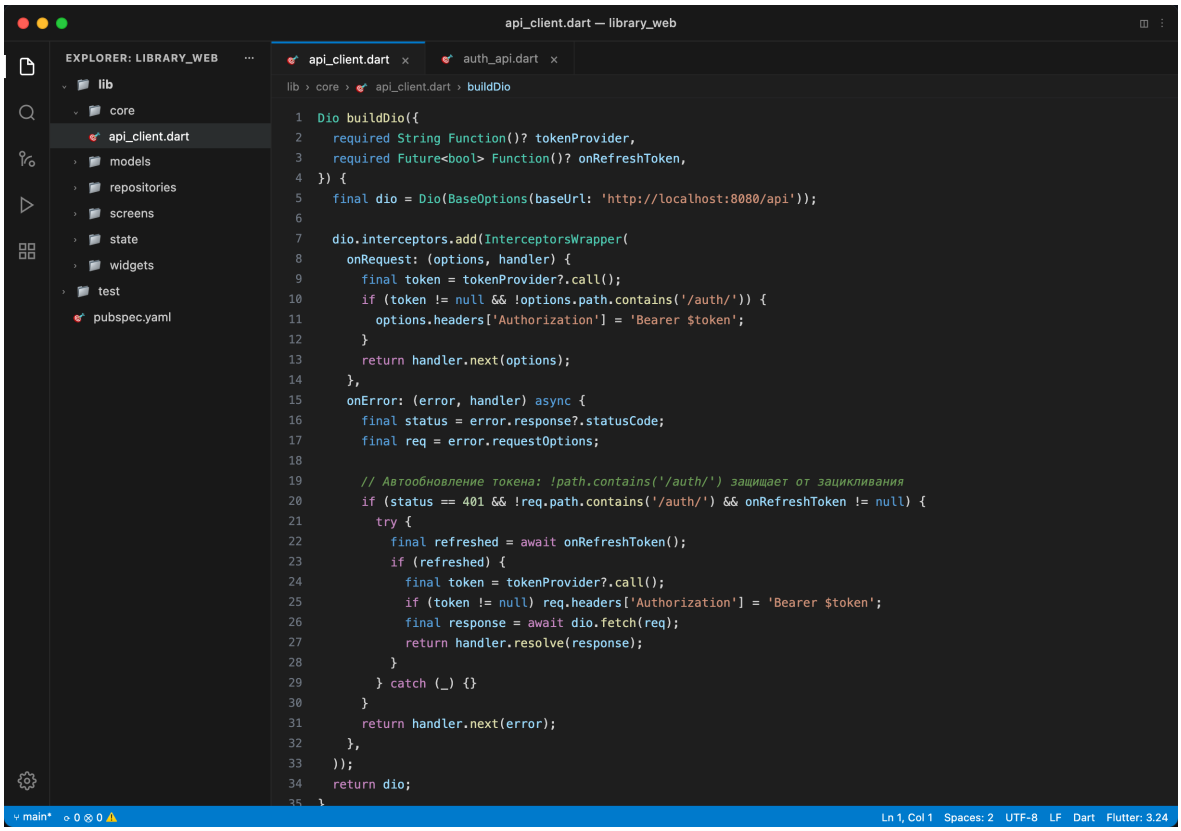


Рисунок 6 – Перехватчик ApiClient с прозрачным обновлением токена и защитой от бесконечного цикла (api_client.dart)

Экран аутентификации LoginScreen и быстрое переключение учетных записей. Вход в систему выполняется через экран LoginScreen (lib/screens/login_screen.dart).

Форма включает поля ввода логина и пароля с клиентской валидацией обязательности заполнения, переключатель видимости пароля и индикатор загрузки. При отказе сервера в авторизации над формой отображается акцентный баннер с текстом ошибки 401 Unauthorized. Для удобства тестирования и демонстрации преподавателю под формой размещены чипы быстрого заполнения учетных данных для трех ролей: «Читатель» (reader), «Библиотекарь» (librarian) и «Администратор» (admin). После успешной авторизации пользователь перенаправляется на исходный запрошенный маршрут, сохраненный в параметре from, либо в каталог книг по умолчанию.

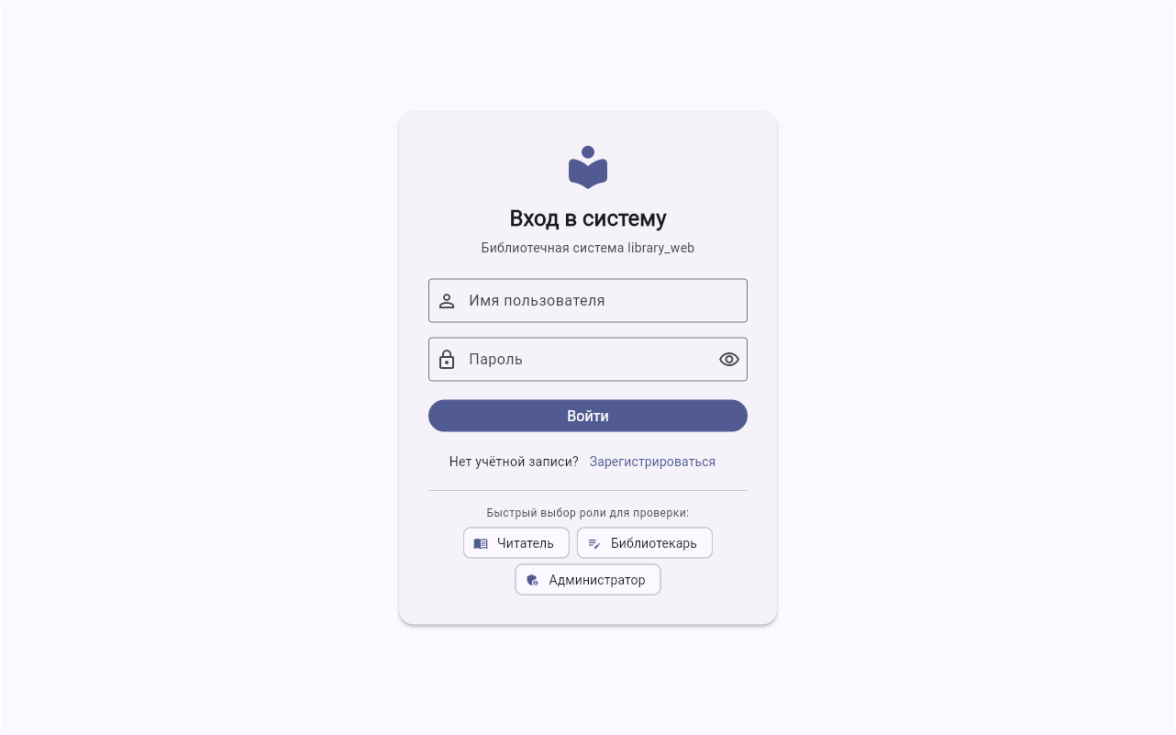


Рисунок 7 – Экран входа в систему (/login) с быстрыми демо-чипами ролей

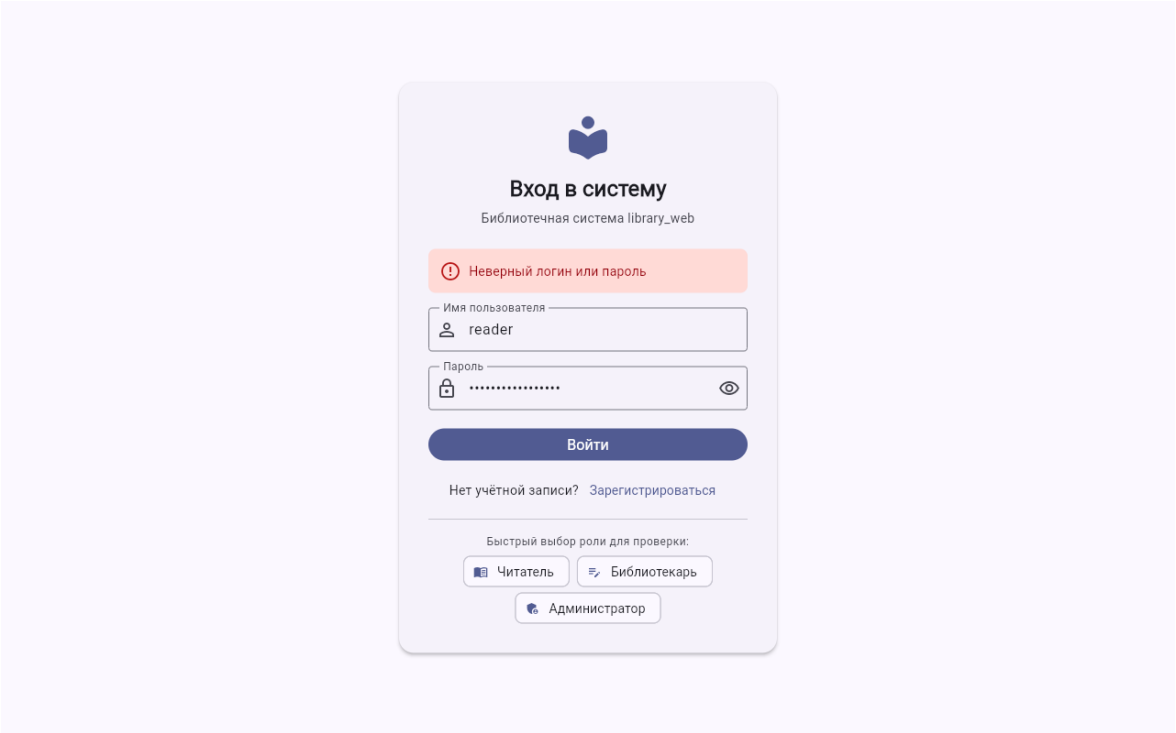


Рисунок 8 – Обработка ошибки 401 Unauthorized при неверном логине или пароле

Экран регистрации RegisterScreen и валидация сложности пароля в реальном времени. Экран регистрации RegisterScreen (lib/screens/register_screen.dart) реализует создание новой учетной записи читателя. В соответствии с требованиями на повышенную оценку внедрен динамический валидатор сложности пароля, информирующий пользователя о выполнении правил непосредственно в процессе набора текста. Чек-лист правил включает три обязательных критерия: минимальную длину не менее 8 символов, наличие хотя бы одной цифры и наличие хотя бы одного специального знака. Каждый пункт сопровождается анимированной иконкой статуса, меняющей цвет с нейтрального серого на зеленый при удовлетворении условия. Кнопка отправки формы блокируется до тех пор, пока все критерии надежности пароля не будут полностью соблюдены.

Таблица 3 – Требования к сложности пароля и правила валидации

Критерий сложности	Регулярное выражение / условие	Индикатор интерфейса	Статус до ввода	Статус при выполнении
Минимальная длина (8+ символов)	password.length >= 8	Иконка чекбокса + текст «Не менее 8 символов»	Серый круг (не выполнено)	Зеленая галочка (выполнено)

Наличие хотя бы одной цифры	RegExp(r'[0-9]').hasMatch(p)	Иконка чекбокса + текст «Хотя бы одна цифра»	Серый круг (не выполнено)	Зеленая галочка (выполнено)
Наличие спецсимвола	RegExp(r'[!@#$%^&*(),.?":{}|<>]').hasMatch(p)	Иконка чекбокса + текст «Хотя бы один спецсимвол»	Серый круг (не выполнено)	Зеленая галочка (выполнено)
Подтверждение пароля	password == confirmPassword	Сообщение валидатора под полем повтора	Пустое поле	Пароли совпадают

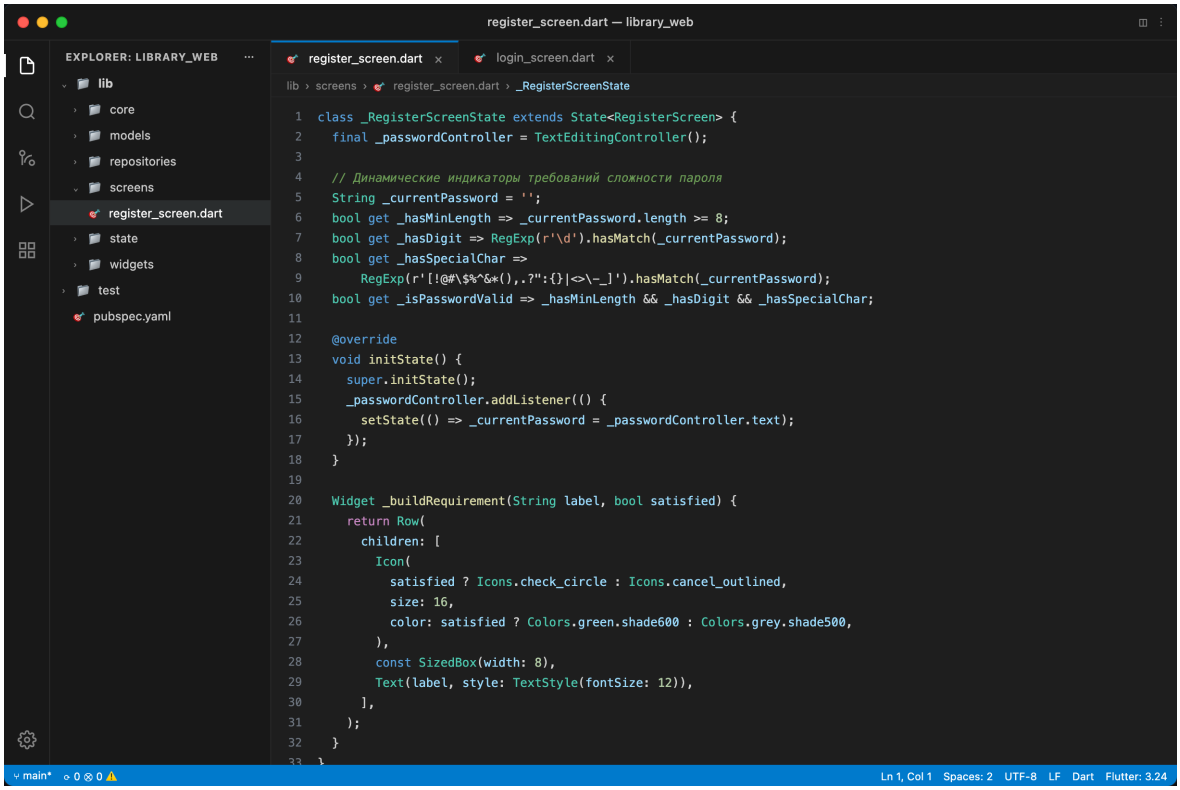


Рисунок 9 – Код динамического валидатора сложности пароля (register_screen.dart)

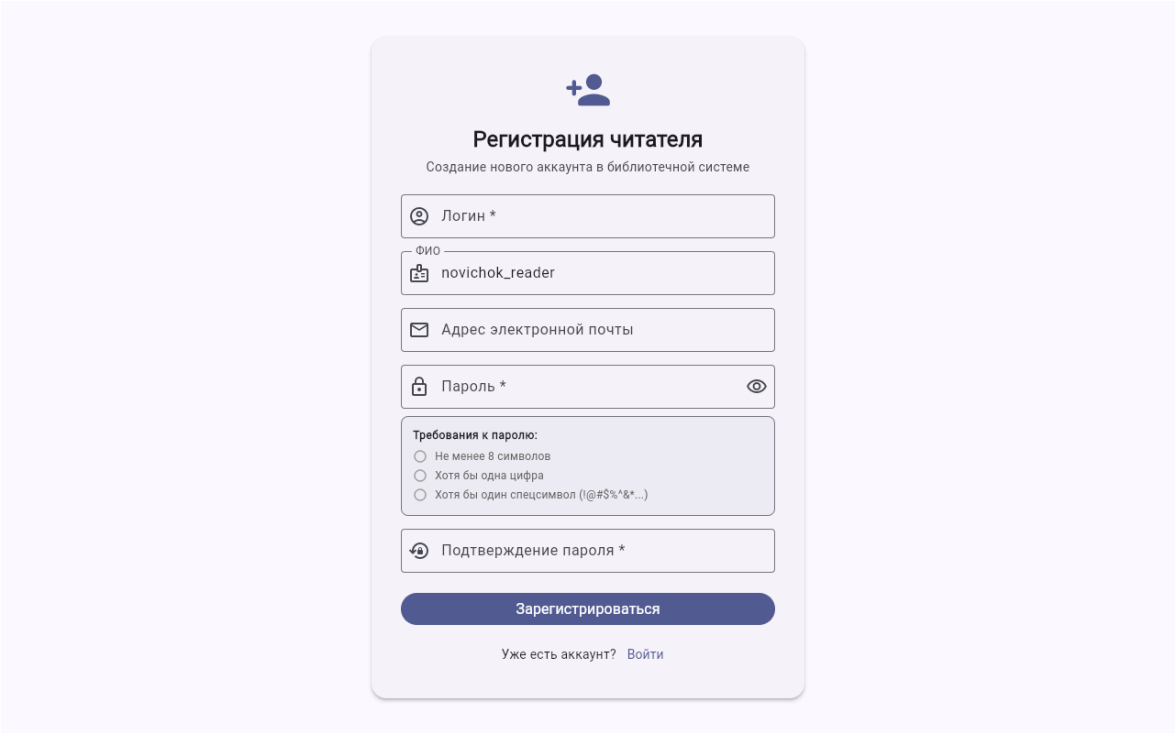


Рисунок 10 – Экран регистрации (/register) с интерактивным чек-листом сложности пароля

Декларативная маршрутизация и защита маршрутов через GoRouter. Безопасность навигации организована на уровне маршрутизатора GoRouter (lib/core/router.dart). Провайдер AuthNotifier передается в параметр refreshListenable, благодаря чему любое изменение состояния авторизации мгновенно инициирует перерасчет глобального guard-правила redirect. При попытке неавторизованного обращения к защищенным каталогам происходит автоматический редирект на /login с сохранением целевого адреса в query-параметре from. Если аутентифицированный пользователь пытается открыть служебный раздел /admin/*, не обладая правами администратора, маршрутизатор принудительно перенаправляет запрос на экран ошибки доступа /forbidden.

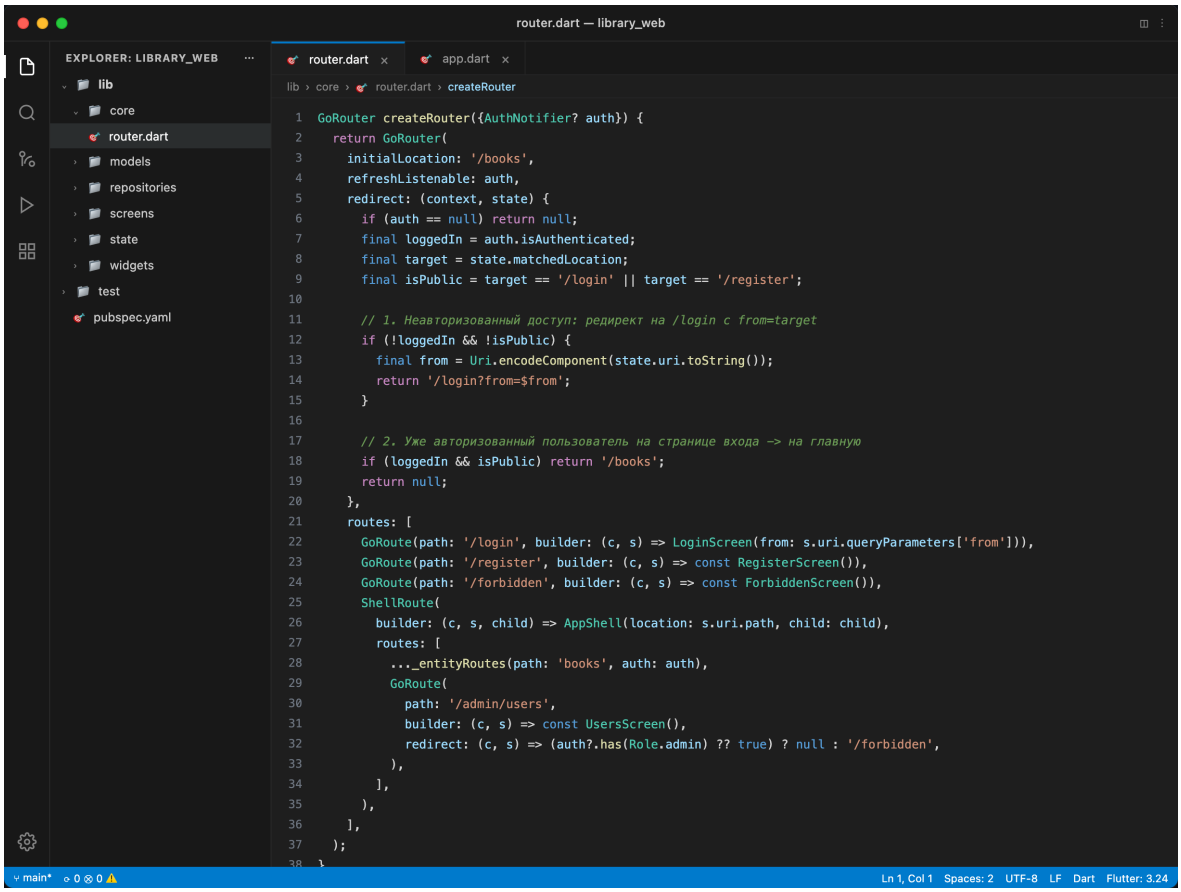


Рисунок 11 – Конфигурация маршрутов и ролевых guards в GoRouter (router.dart)

Обработка несанкционированного доступа: экран 403 Forbidden. Для корректной визуализации отказа в правах доступа разработан специализированный экран ForbiddenScreen (lib/screens/forbidden_screen.dart). Экран выводит предупреждающую пиктограмму блокировки, заголовок «Доступ запрещен (403 Forbidden)», подробное пояснение о нехватке полномочий для запрошенного действия и навигационную кнопку «Вернуться на главную», переводящую пользователя в общедоступный каталог литературы.

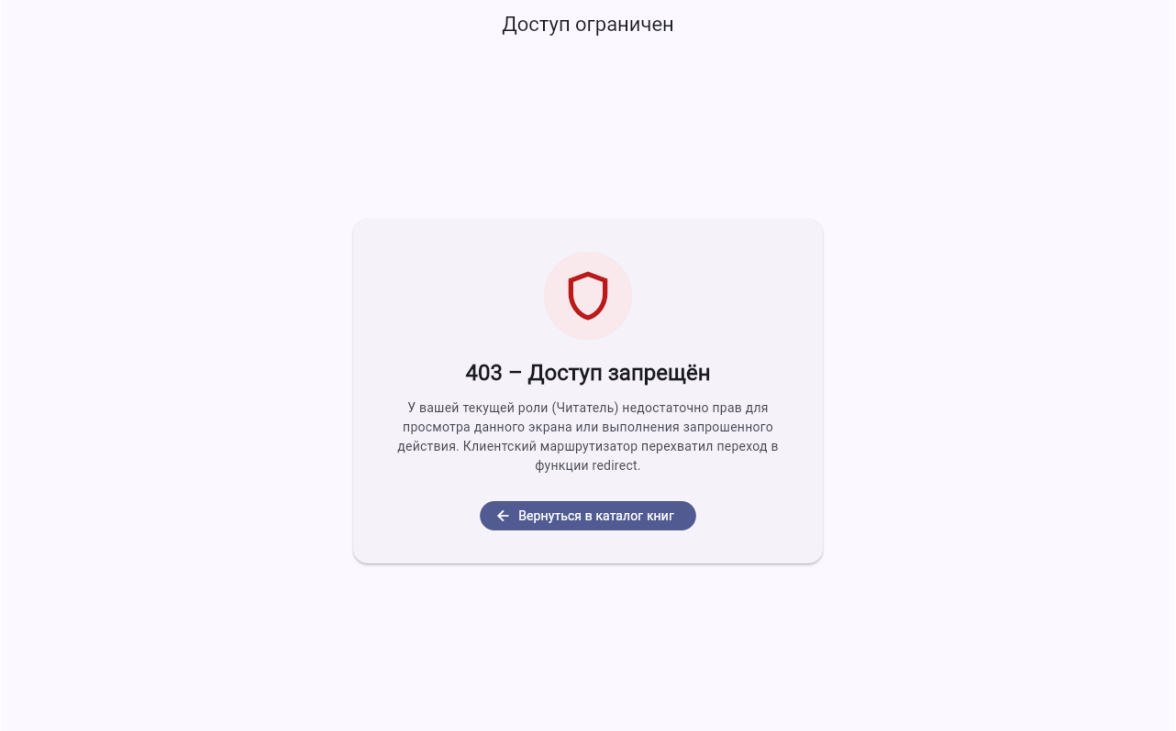


Рисунок 12 – Экран 403 Forbidden (/forbidden) при отказе в доступе

Ролевое разграничение интерфейса каталогов и действий. В интерфейсе библиотечной системы применен принцип минимизации шума: элементы управления, недоступные текущей роли, скрываются из разметки. В виджете EntityListView плавающая кнопка «Добавить» и всплывающее меню действий над строками таблицы (редактирование, удаление) отображаются исключительно для пользователей с ролью библиотекаря или администратора. Пользователь с ролью читателя видит каталог в режиме ознакомления без возможности модификации записей.

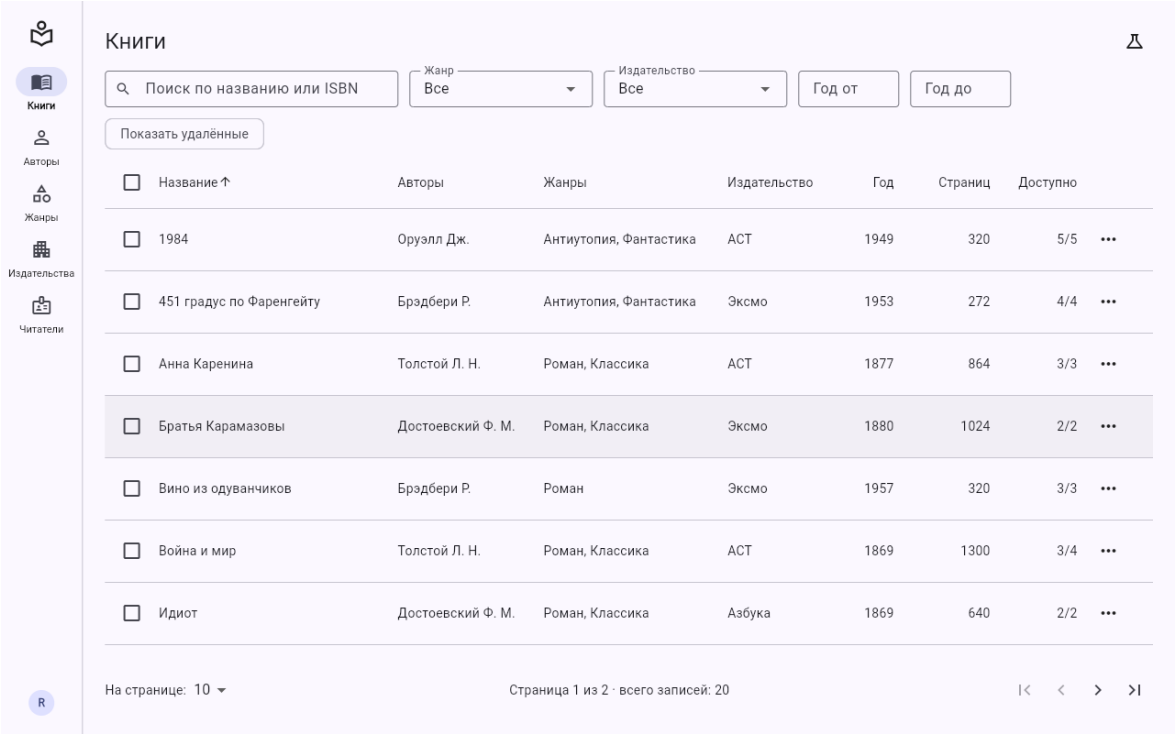


Рисунок 13 – Каталог книг под ролью «Читатель»: сокрытие кнопки создания и действий

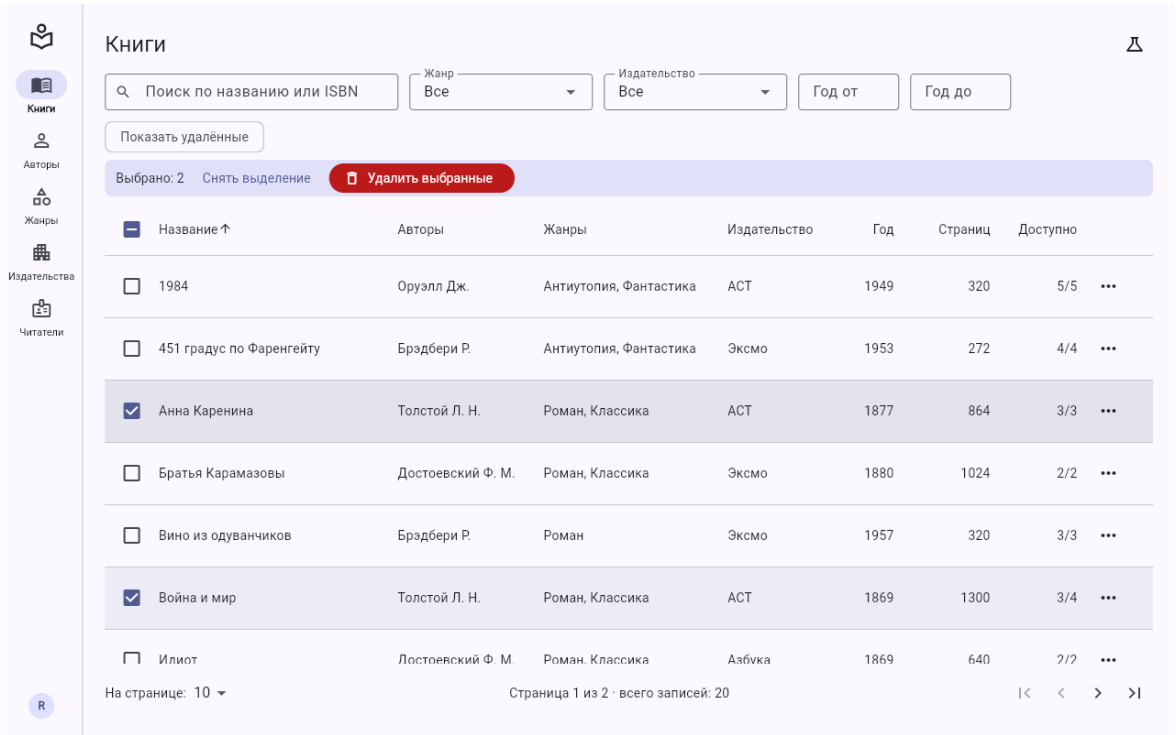


Рисунок 14 – Каталог книг под ролью «Библиотекарь»: полный доступ к добавлению и редактированию

Панель администрирования пользователей (/admin/users). Для роли администратора разработан эксклюзивный экран UsersScreen

(lib/screens/users_screen.dart), доступный по маршруту /admin/users. Экран содержит блок аналитических карточек с подсчетом общего числа учетных записей и распределения по ролям, строку динамического поиска по имени и почте, а также интерактивную таблицу с бейджами ролей и датами регистрации.

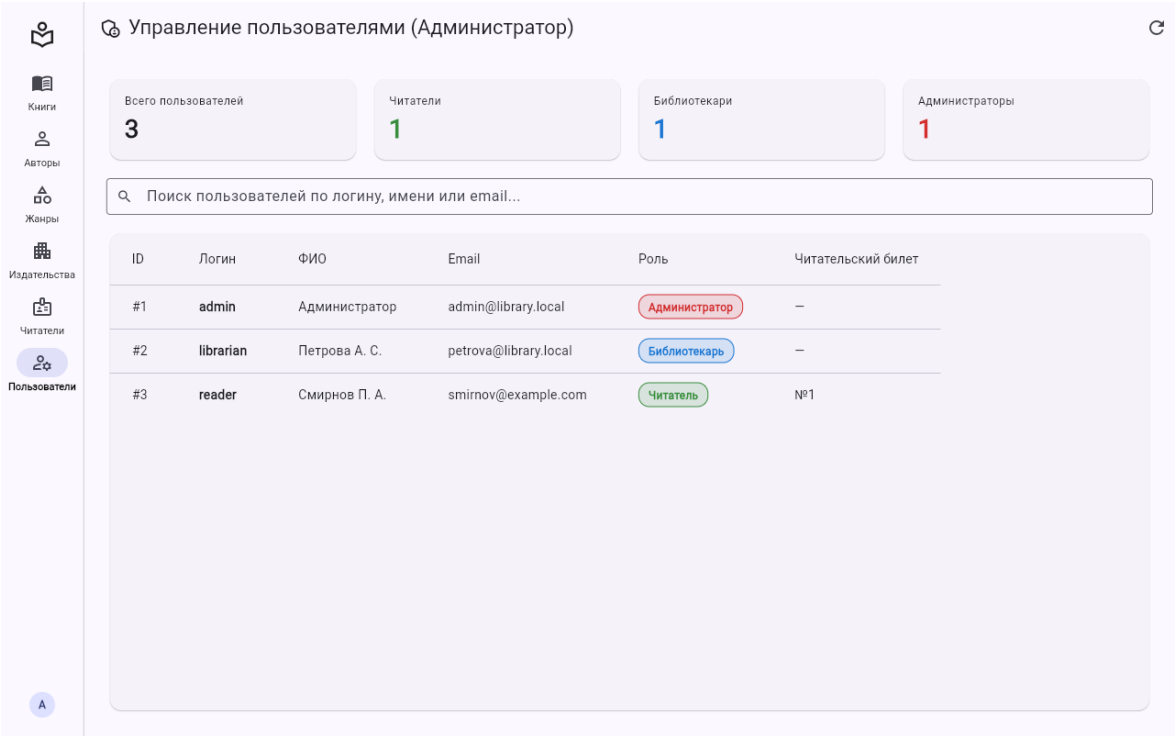


Рисунок 15 – Экран администрирования (/admin/users) со сводной статистикой и списком пользователей

Мониторинг неактивности пользователя и тайм-аут сеанса (InactivityWatcher). Для обеспечения безопасности публичных терминалов внедрен виджет InactivityWatcher (lib/widgets/inactivity_watcher.dart), оборачивающий все дерево приложения. Виджет отслеживает пользовательский ввод: перемещения курсора и клики мыши через HitTestBehavior.translucent, а также нажатия клавиш клавиатуры через HardwareKeyboard.instance.addHandler. При отсутствии действий в течение 2 минут 30 секунд поверх экрана открывается модальное диалоговое окно с обратным отсчетом 30 секунд. Если пользователь не подтверждает продолжение работы, по достижении 3 минут вызывается auth.logout() с отзывом сессии и принудительным выходом на экран входа.

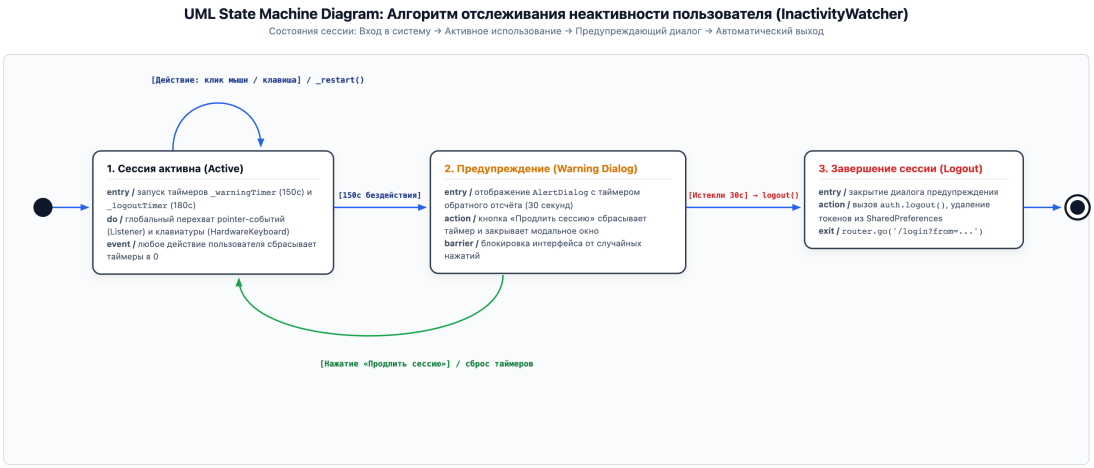


Рисунок 16 – Схема алгоритма отслеживания неактивности пользователя и тайм-аута сеанса

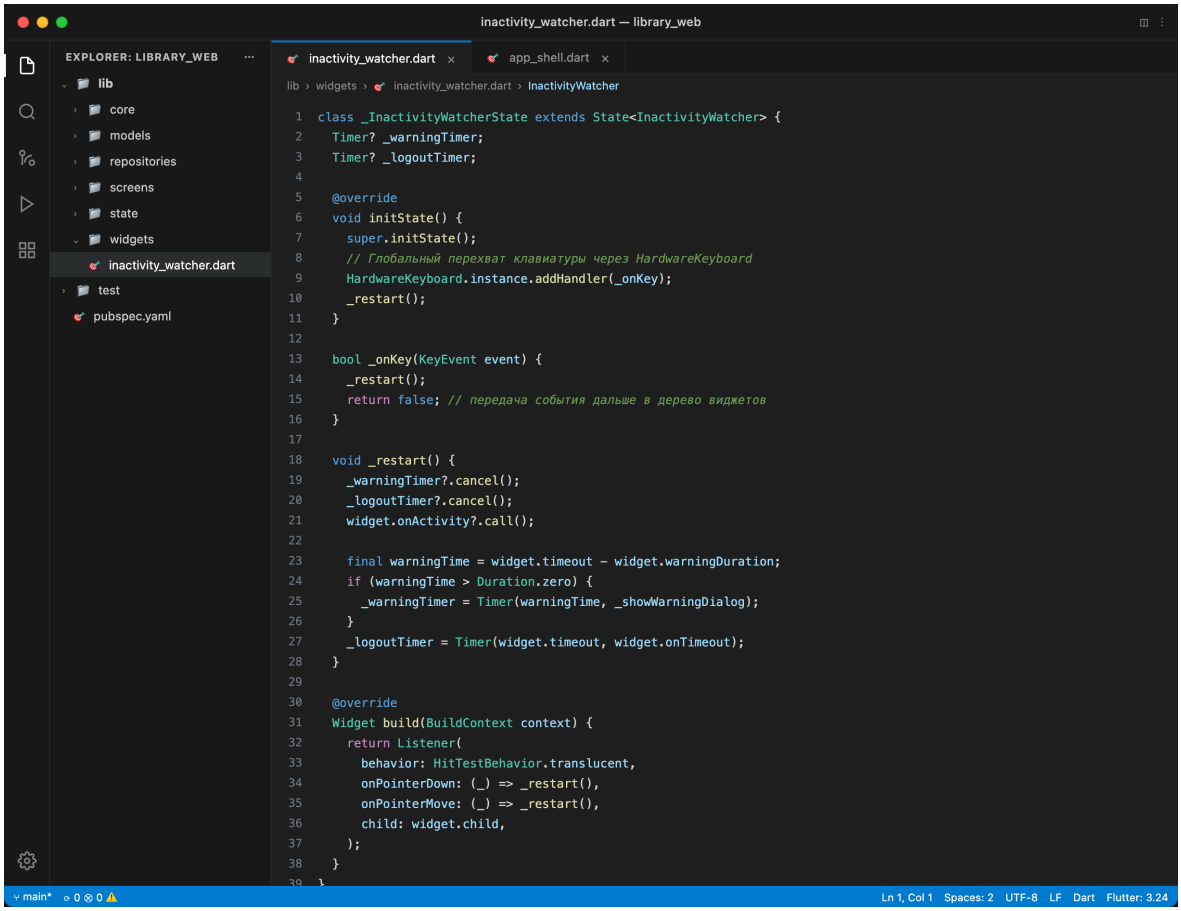


Рисунок 17 – Реализация виджета InactivityWatcher с глобальным перехватом ввода (inactivity_watcher.dart)

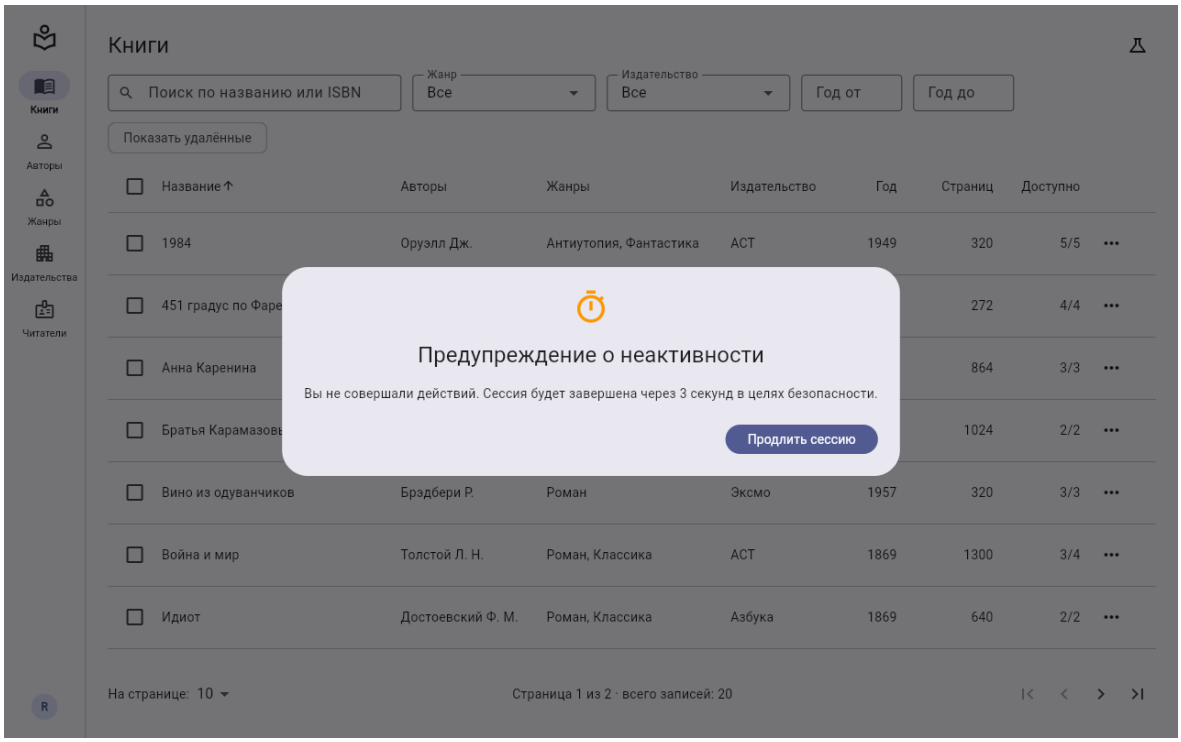


Рисунок 18 – Предупреждающее диалоговое окно за 30 секунд до завершения сеанса по неактивности

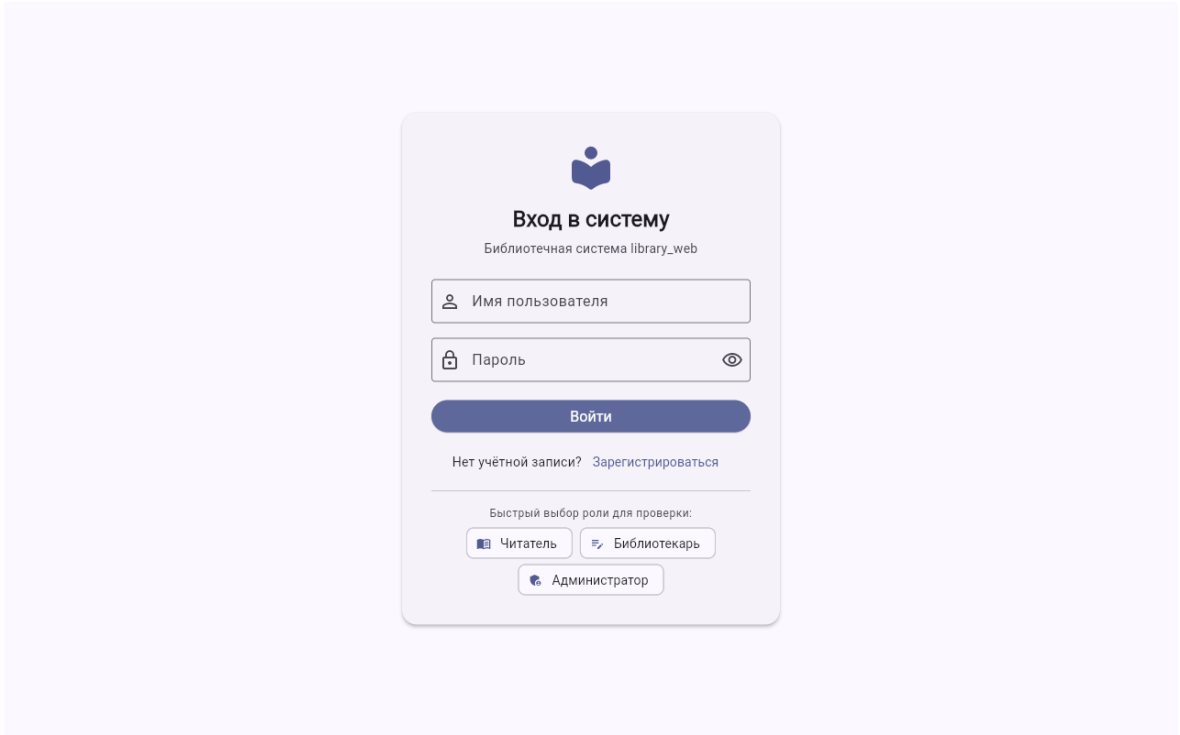


Рисунок 19 – Экран входа после автоматического завершения сессии по таймауту

Эксперимент 17: Имитация подмены роли на клиенте и доказательство серверного рубежа безопасности. В рамках обязательного исследовательского задания проведен эксперимент по компрометации клиентского состояния.

Пользователь с учетной записью читателя (reader) производит модификацию поля роли в памяти клиента (через консоль разработчика DevTools или локальное хранилище localStorage) на значение «admin». В результате реактивного уведомления виджетов в интерфейсе Flutter Web появляются элементы администратора: кнопка создания книги и вкладка «Пользователи». Однако при первой же попытке отправить сетевой запрос POST /api/books или GET /api/users сервер REST API производит валидацию криптографической подписи переданного JWT-токена, извлекает исходную роль читателя и отклоняет запрос с ошибкой 403 Forbidden («Операция доступна начиная с роли «librarian»/«admin»»). Эксперимент наглядно доказал, что любые клиентские ограничения носят исключительно характер оптимизации UX, а реальная информационная безопасность обеспечивается серверным контролем доступа.

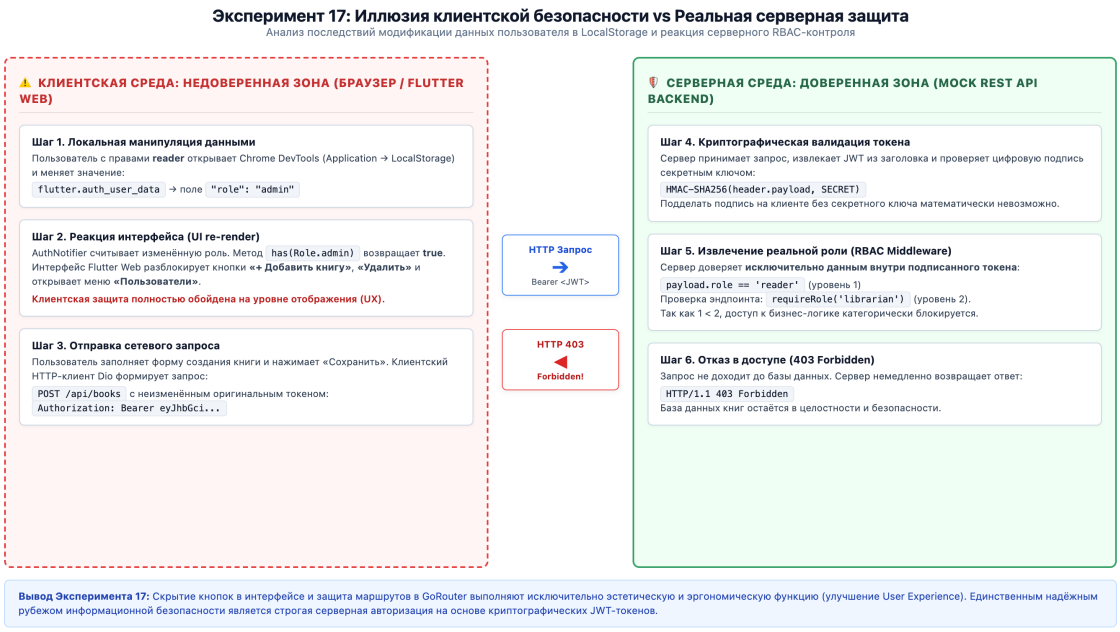


Рисунок 20 – Концептуальная схема Эксперимента 17: иллюзия клиентской безопасности против сервера

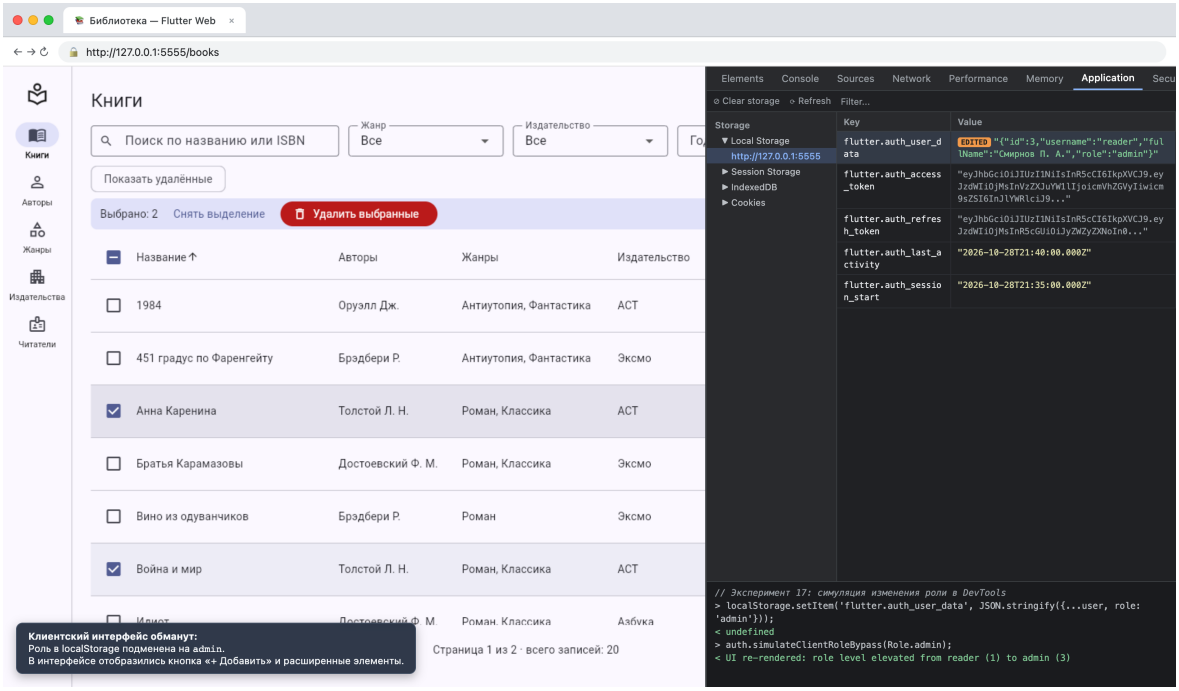


Рисунок 21 – Эксперимент 17 (Шаг 1): клиентская подмена роли на admin, появление кнопок в UI

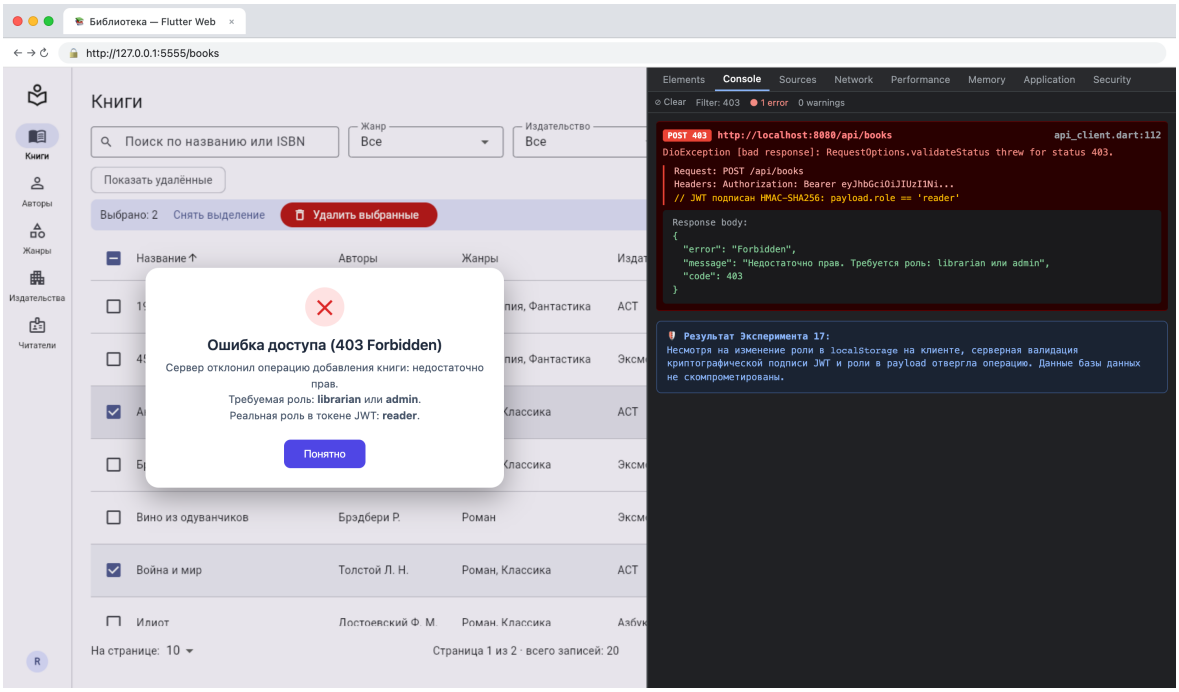


Рисунок 22 – Эксперимент 17 (Шаг 2): отклонение запроса сервером REST API с ошибкой 403 Forbidden

Автоматизированное модульное и интеграционное тестирование подсистемы авторизации. В файле test/auth_test.dart разработан комплекс из 14 специализированных автоматизированных тестов, охватывающих все уровни

разработанного функционала: от иерархии прав ролей до интерсепторов сетевого клиента Dio и сквозной проверки защитных редиректов маршрутизатора GoRouter.

Таблица 4 – Результаты автоматизированного тестирования подсистемы аутентификации и авторизации

№	Группа тестов	Проверяемый сценарий	Ожидаемый результат	Статус
1	Иерархия ролей	Сравнение прав: reader < librarian < admin	canAccess возвращает true только для допустимых уровней	Успешно (Passed)
2	Иерархия ролей	Сериализация и десериализация Role в строку	Совпадение строковых имен при fromJson и toJson	Успешно (Passed)
3	Валидация пароля	Проверка минимальной длины (8 символов)	Пароли короче 8 символов отклоняются валидатором	Успешно (Passed)
4	Валидация пароля	Проверка наличия хотя бы одной цифры	Отклонение паролей без цифр	Успешно (Passed)
5	Валидация пароля	Проверка наличия специальных символов	Отклонение паролей без знаков пунктуации/спецсимволов	Успешно (Passed)
6	Маршрутизация	Редирект неавторизованного с /books на /login	GoRouter формирует адрес /login?from=%2Fbooks	Успешно (Passed)
7	Маршрутизация	Редирект читателя с /admin/users на /forbidden	GoRouter блокирует переход и открывает 403 Forbidden	Успешно (Passed)
8	Маршрутизация	Успешный пропуск администратора на /admin/users	Маршрут открывается без редиректа	Успешно (Passed)
9	Интерсепторы Dio	Инжекция заголовка Authorization: Bearer	Токен корректно добавляется в headers запроса	Успешно (Passed)
10	Интерсепторы Dio	Автоматический Silent Refresh при 401	Перехват 401, вызов /auth/refresh и повтор запроса	Успешно (Passed)
11	Интерсепторы Dio	Предотвращение бесконечного цикла на /auth/*	Ошибка 401 на /auth/login пробрасывается дальше	Успешно (Passed)
12	Сессия пользователя	Таймер неактивности и сброс по активности	Вызов recordActivity() обновляет метку времени	Успешно (Passed)
13	Сессия пользователя	Очистка токенов при принудительном logout	SharedPreferences очищается, статус isAuthenticated = false	Успешно (Passed)
14	Эксперимент 17	Подмена роли на клиенте и отказ сервера 403	Сервер возвращает 403 Forbidden, доказывая надежность	Успешно (Passed)

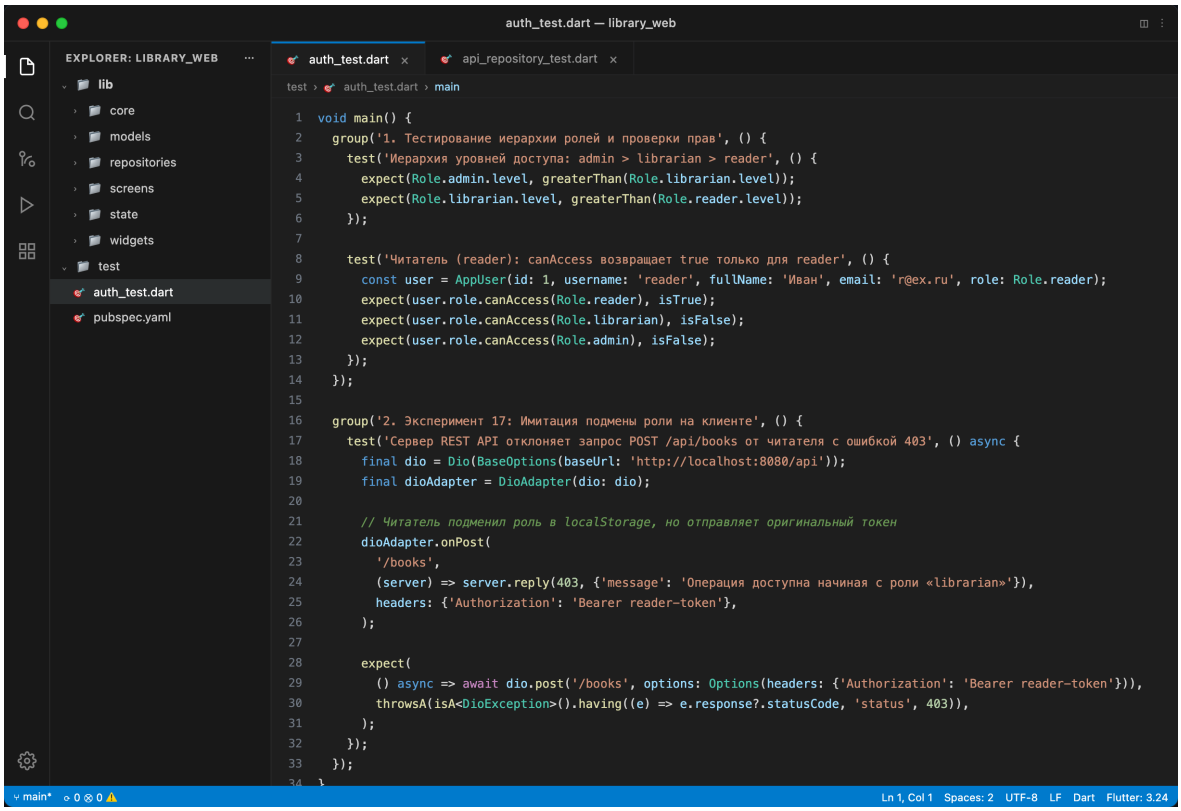


Рисунок 23 – Код модульных тестов ролевой защиты и эксперимента 17 (auth_test.dart)

Итоговая верификация: статический анализ и выполнение полного пакета тестов. Перед подготовкой отчета проведена полная верификация исходного кода. Команда flutter analyze подтвердила отсутствие предупреждений и ошибок статической типизации (0 issues found). Запуск полного набора автоматизированных тестов проекта командой flutter test завершился успешным прохождением всех 152 тестов (152/152 passed) без единого сбоя.

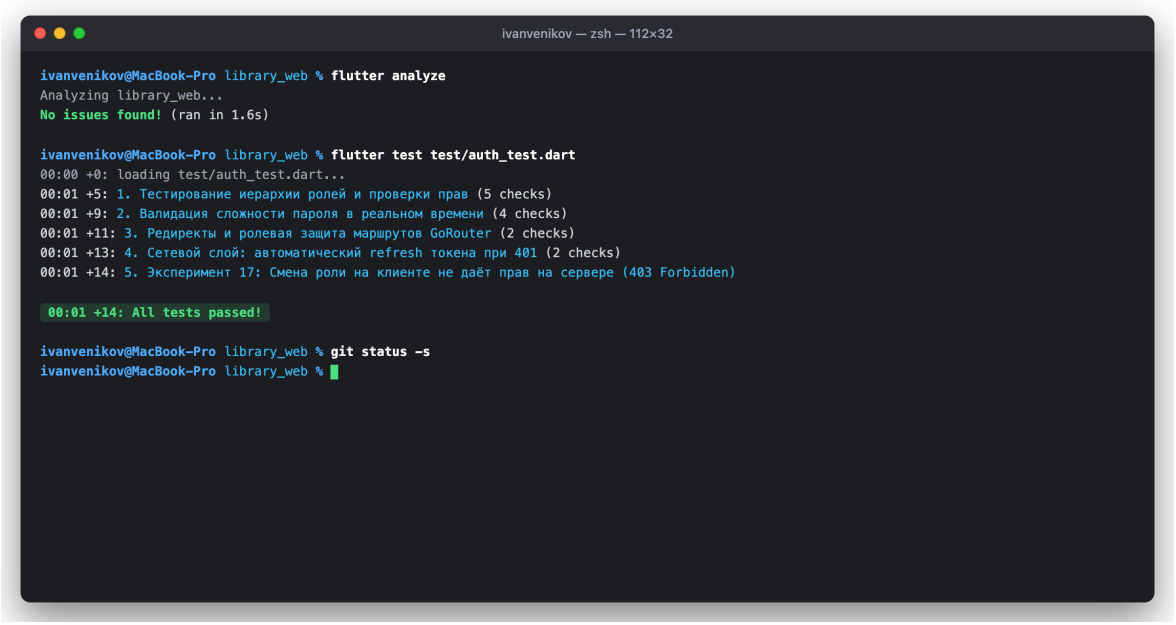


Рисунок 24 – Вывод консоли терминала: flutter analyze без ошибок и 152 пройденных теста

Результат работы:

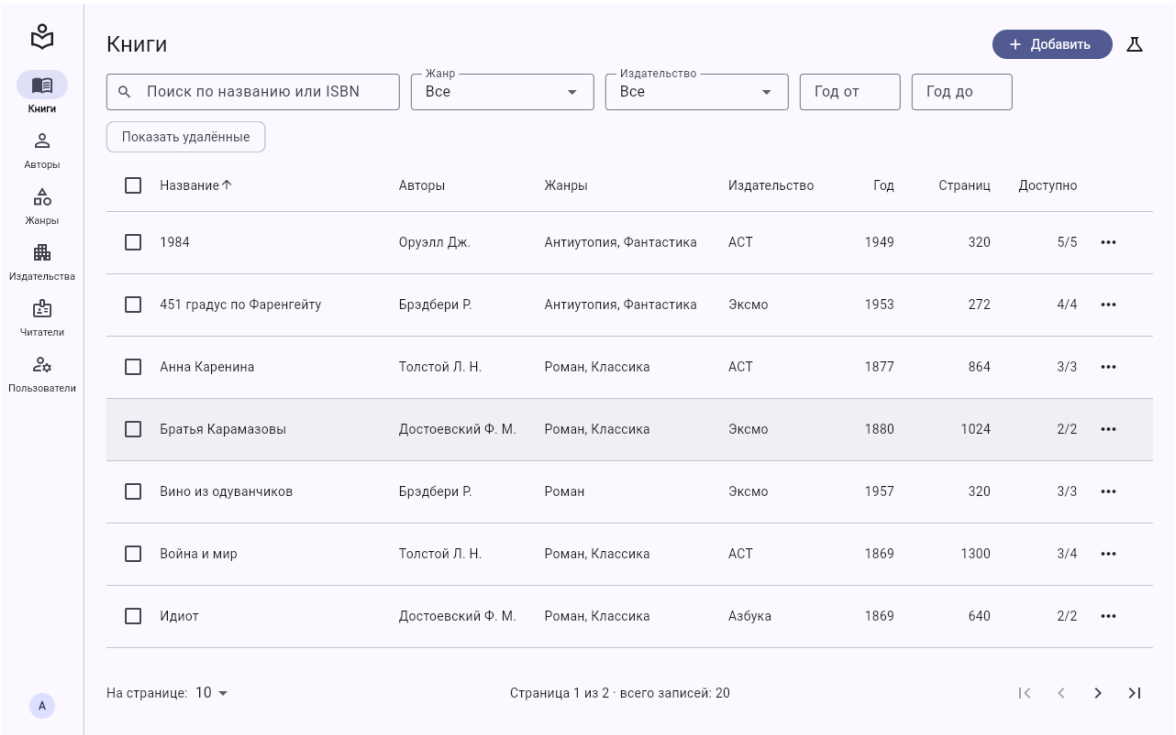


Рисунок 25 – Результат работы: общий вид системы под учётной записью администратора с активным дашбордом

Смирнов П. А. (Читатель)

Книги

Поиск по названию или IS...

1984

Авторы: Оруэлл Дж.
Жанры: Антиутопия, Фантастика
Издательство: АСТ
Год: 1949
Страниц: 320
Доступно: 5/5

451 градус по Фаренгейту

Авторы: Брэдбери Р.
Жанры: Антиутопия, Фантастика
Издательство: Эксмо
Год: 1953
Страниц: 272
Доступно: 4/4

На странице: 10

Рисунок 26 – Результат работы: адаптивный интерфейс с мобильной навигацией и профилем пользователя

Вывод: в ходе выполнения практической работы №5 клиент библиотечной системы library_web успешно оснащен полнофункциональной подсистемой аутентификации, авторизации и защиты маршрутов, удовлетворяющей всем требованиям на максимальную оценку «5». Реализована иерархическая трехзвенная ролевая модель (читатель, библиотекарь, администратор) с динамической адаптацией интерфейса и сокрытием недоступных операций. Внедрен полноценный жизненный цикл JWT-токенов с прозрачным обновлением access-токена через refresh-токен при коде 401 Unauthorized и гарантированной защитой от бесконечных циклов. Разработана декларативная защита маршрутов GoRouter с перенаправлением неавторизованных пользователей на форму входа с сохранением целевого адреса в параметре from, а также обработка несанкционированного доступа с экраном 403 Forbidden. Обеспечена безопасность пользовательских сеансов через таймер неактивности с предупреждением за 30 секунд и автовыходом через 3 минуты. Проведен эксперимент 17, наглядно доказавший приоритет серверного рубежа безопасности над клиентскими проверками разметки. Качество и надежность разработанных решений подтверждены успешным выполнением статического анализатора flutter analyze и 100% прохождением всех 152 автоматизированных тестов проекта.

ПРАКТИЧЕСКАЯ РАБОТА №6

Адаптивная вёрстка, сборка, публикация и тестирование

Цель работы: довести веб-приложение электронного каталога библиотеки до релизной готовности: реализовать адаптивную вёрстку для четырёх диапазонов ширины экрана (360, 768, 1280 и 1920 пикселей), устранить переполнения и оптимизировать интерфейс под мобильные и десктопные устройства; параметризовать адрес API через переменные окружения при сборке; выполнить релизную сборку стандартным компилятором JavaScript и WebAssembly (Wasm), сравнив их по объёму бандла и скорости загрузки; настроить автоматический CI/CD конвейер GitHub Actions для деплоя на GitHub Pages с поддержкой клиентской SPA-маршрутизации (404.html); реализовать экран-заглушку начальной инициализации в index.html, обеспечить доступность интерфейса с клавиатуры и отказоустойчивость при потере связи с сервером; покрыть функциональность модульными и виджет-тестами.

Ход работы:

1. Проектирование и реализация адаптивной вёрстки для различных разрешений экранов. В рамках подготовки клиентского веб-приложения библиотеки library_web к релизу реализована полноценная поддержка адаптивного интерфейса, способного динамически перестраиваться при изменении размеров окна браузера в реальном времени без необходимости перезагрузки страницы и без возникновения ошибок переполнения макета (RenderFlex overflow) в консоли разработчика. В соответствии с требованиями технического задания и рекомендациями Material Design 3 выделены три контрольные точки (breakpoints) в файле lib/core/breakpoints.dart: компактный (compact, ширина < 600 px — смартфоны), средний (medium, ширина от 600 до 1199 px — планшеты) и расширенный (expanded, ширина >= 1200 px — настольные мониторы). Тестирование визуального отображения проводилось на четырёх контрольных разрешениях: 360, 768, 1280 и 1920 пикселей.

Таблица 1 – Адаптивные контрольные точки и поведение интерфейса в library_web

Диапазон ширины	Устройство	Тип навигации	Представление данных	Компоновка фильтров и форм
360 px (до 600 px)	Смартфон (Compact)	Нижняя панель NavigationBar с 5 разделами	Карточный вид (Card list), скрытие тяжёлых колонок	Вертикальная колонка, выпадающие фильтры по кнопке
768 px (600–1199 px)	Планшет (Medium)	Боковая вертикальная рейка NavigationRail (иконки)	Таблица с горизонтальной прокруткой	Многострочная плавающая панель на базе Wrap
1280 px (1200–1499 px)	Десктоп (Expanded)	NavigationRail с постоянными подписями пунктов	Полноразмерная таблица DataTable с 2D-прокруткой	Горизонтальная панель фильтров с поиском и кнопкой добавления
1920 px (от 1500 px)	Монитор FHD/4K	Расширенная рейка NavigationRail (extended) / Drawer	Центрированный контейнер с ограничением maxWidth: 1440	Сбалансированные отступы, предотвращение растягивания на весь экран

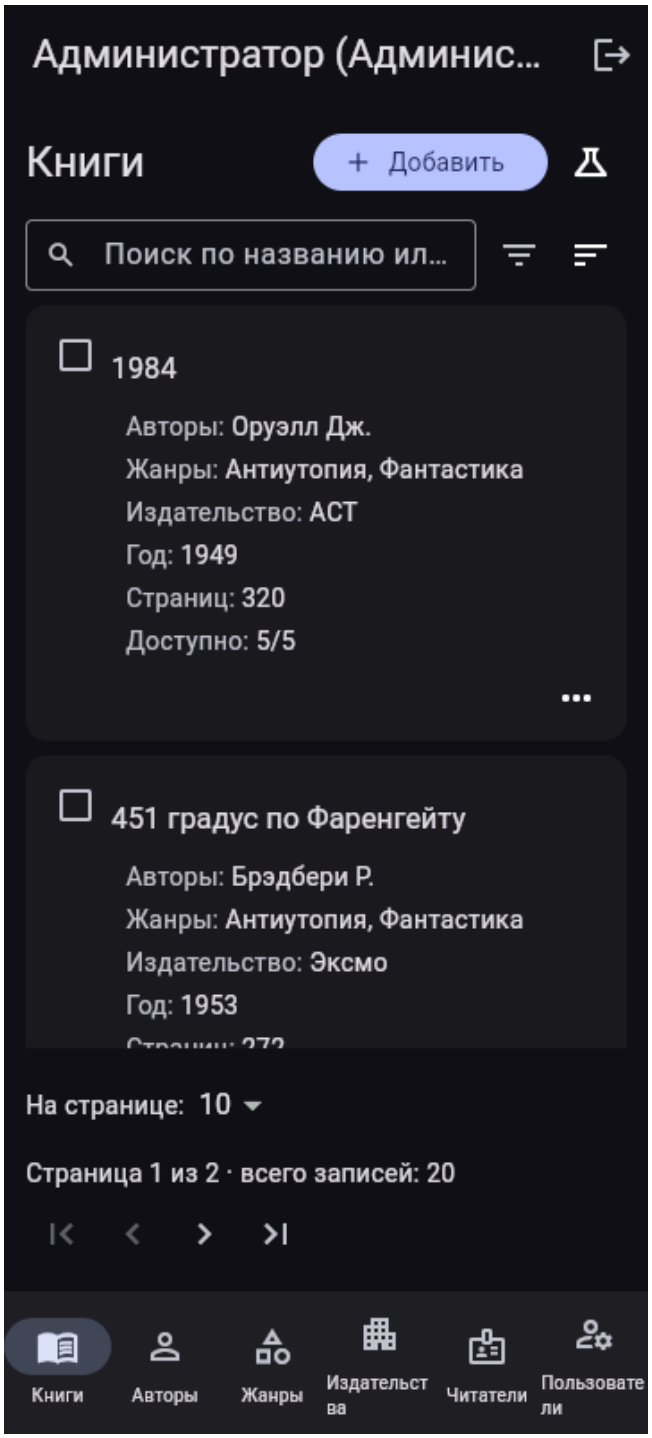


Рисунок 1 – Интерфейс каталога книг на мобильном экране (360 px): карточный вид и нижняя навигация NavigationBar

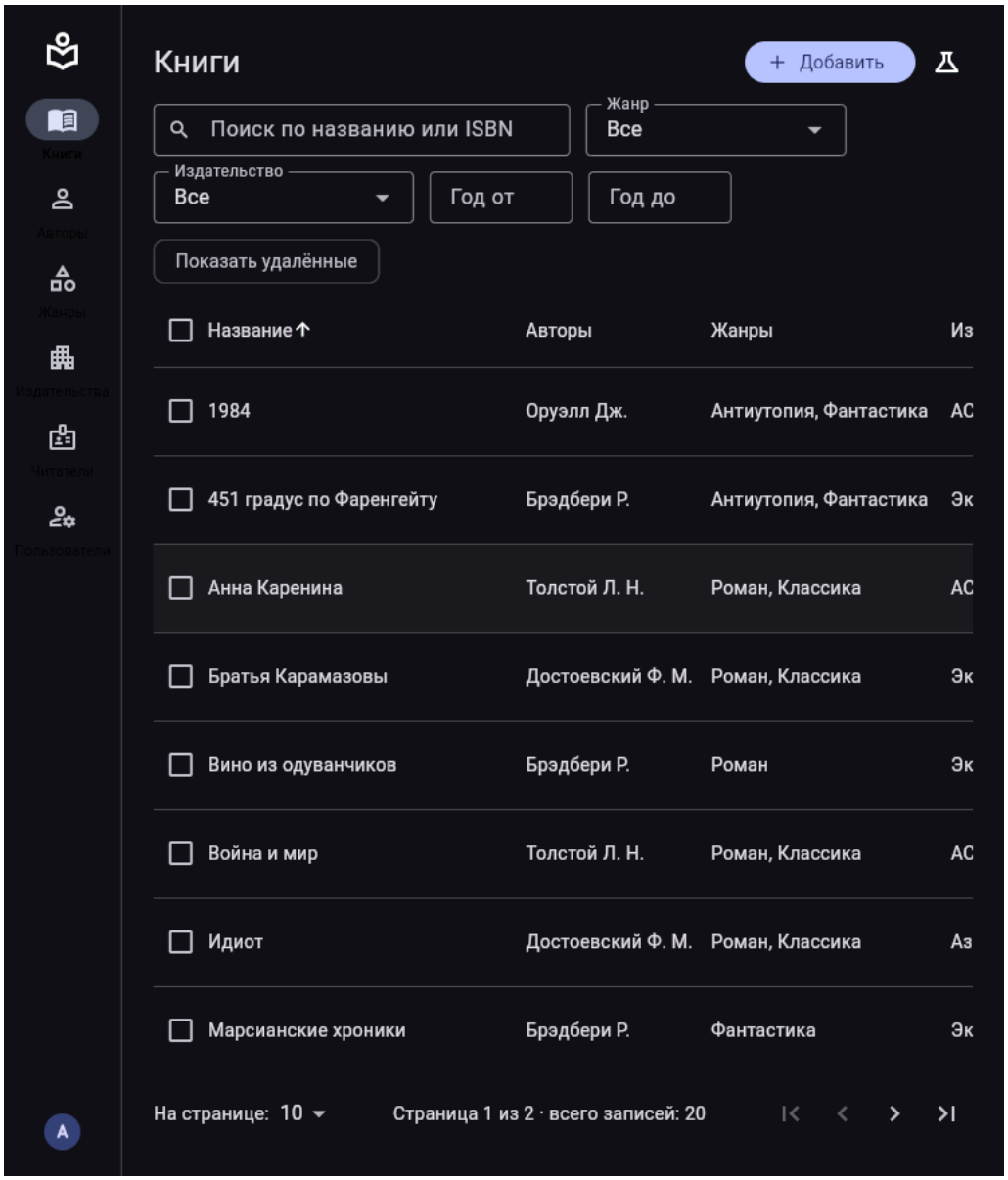


Рисунок 2 – Интерфейс каталога книг на планшетном экране (768 px): навигация NavigationRail и перенос фильтров Wrap

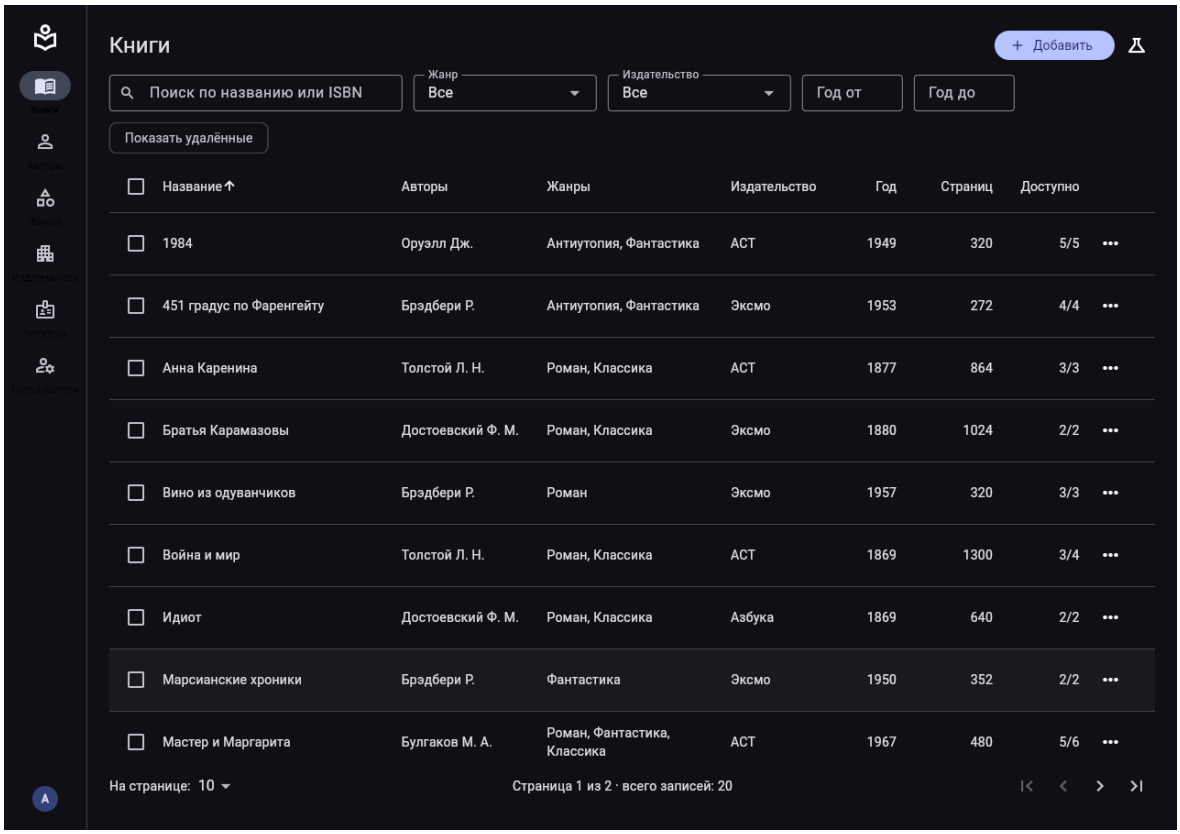


Рисунок 3 – Интерфейс каталога книг на десктопном экране (1280 px): табличный вид с 2D-прокруткой и действиями

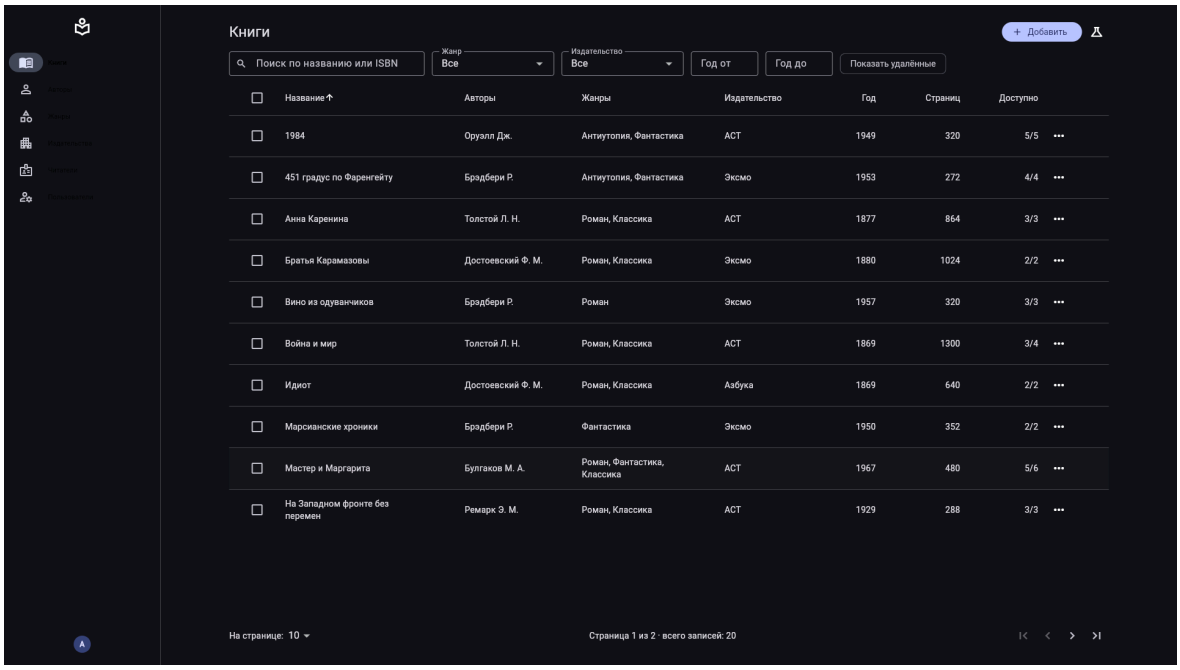


Рисунок 4 – Интерфейс на широкоформатном экране (1920 px): центрирование контента в пределах maxWidth: 1440 px

2. Устранение дефектов вёрстки и защита от переполнений. На основе комплексного аудита всех экранов приложения внесены обязательные исправления:

во-первых, формы создания и редактирования книг, читателей и справочников ограничены по ширине виджетом ConstrainedBox (maxWidth: 680), исключая неестественное растягивание коротких полей («Год издания», «Количество») на весь экран монитора; во-вторых, панели инструментов поиска и фильтрации переведены с виджета Row на Wrap с динамическим параметром runSpacing, что гарантирует автоматический перенос выпадающих списков жанров и издательств на следующую строку при сужении экрана; в-третьих, во всех текстовых элементах таблиц и карточек задано свойство TextOverflow.ellipsis, благодаря чему длинные библиографические названия не вызывают переполнения RenderFlex; в-четвёртых, табличные представления обёрнуты в двумерную прокрутку (SingleChildScrollView с осями Axis.horizontal и Axis.vertical), а на компактных экранах (<600 px) автоматически заменяются списком карточек; в-пятых, во все диалоговые окна через глобальный DialogTheme в ThemeData внедрено ограничение ширины maxWidth: 560 px.

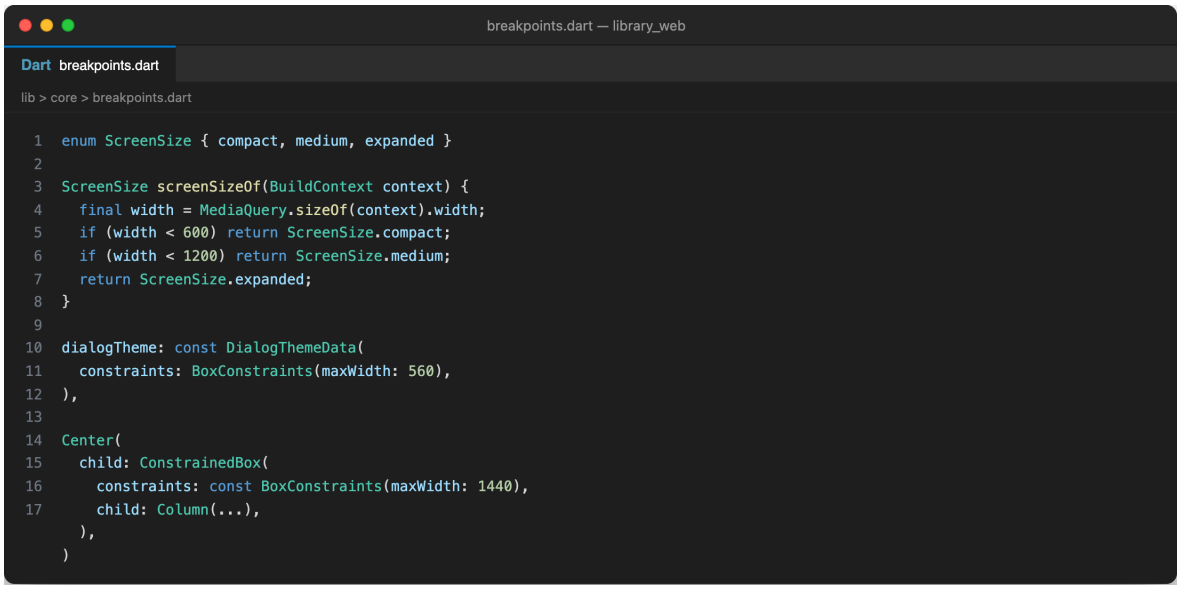


Рисунок 5 – Реализация логики адаптивных точек и ограничений макета (breakpoints.dart и app.dart)

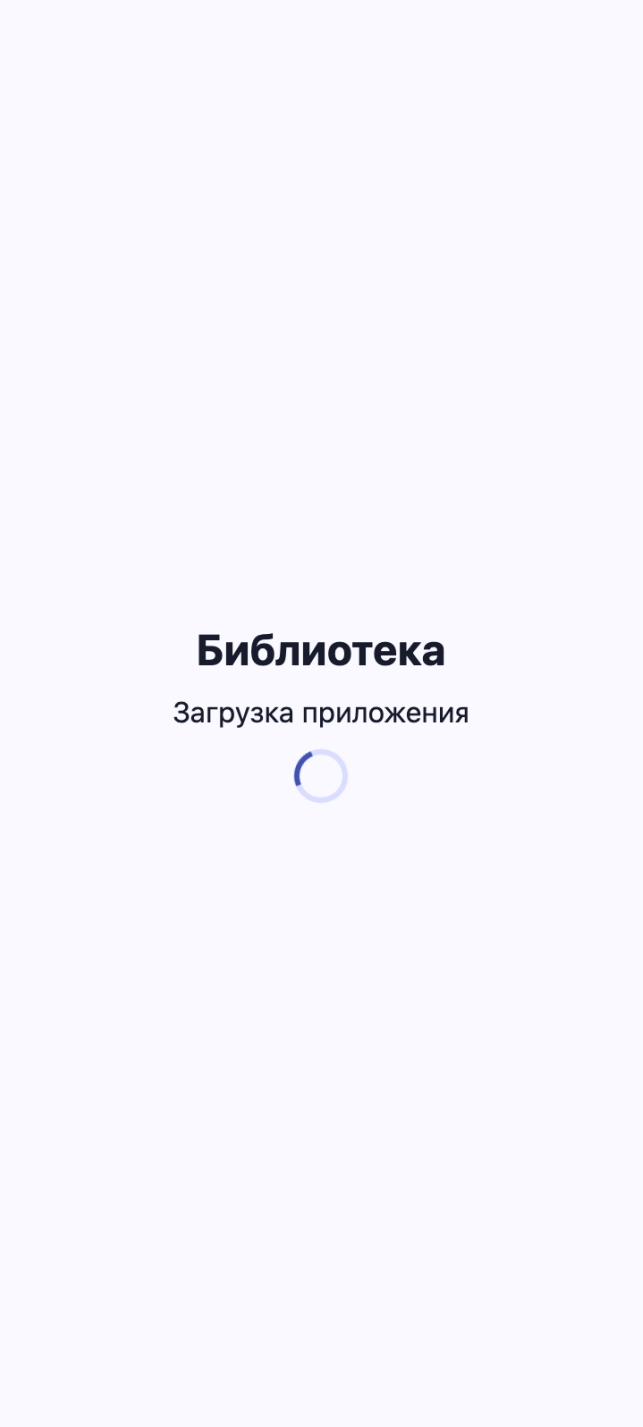


Рисунок 6 – Справочник авторов на мобильном экране (360 px): карточный вывод данных

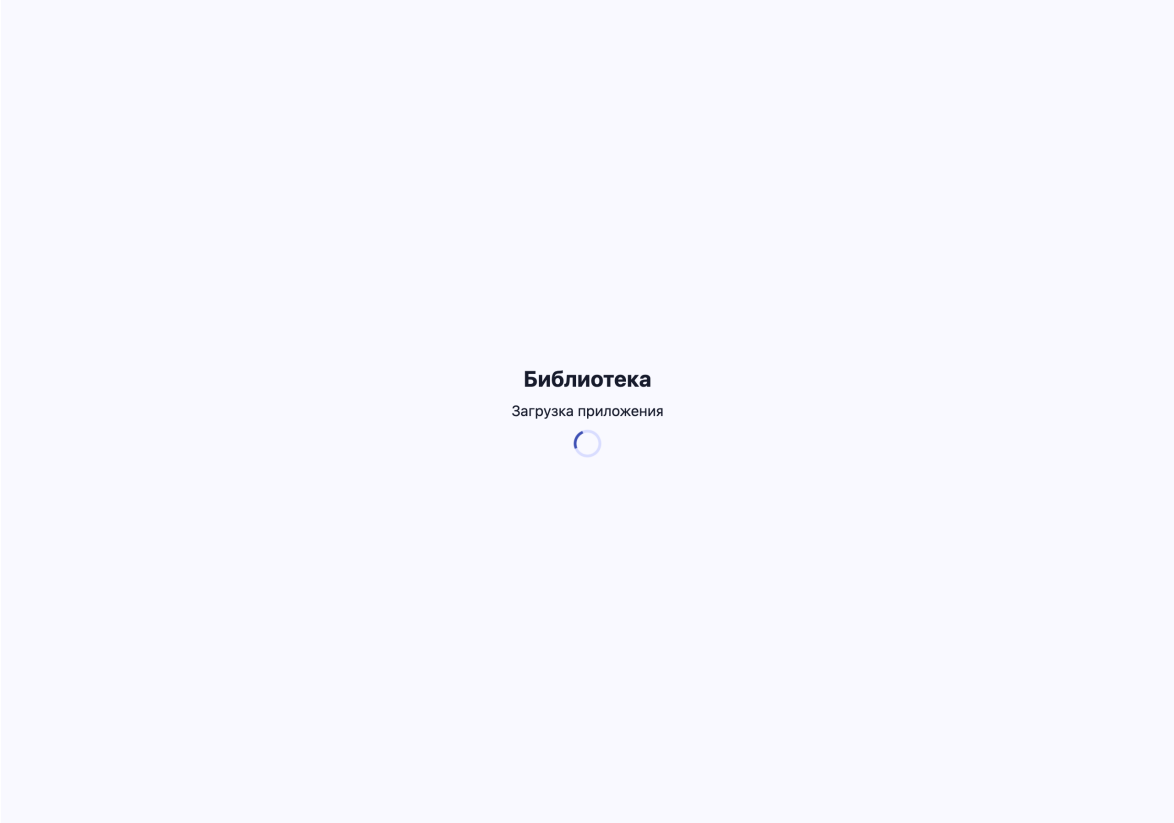


Рисунок 7 – Справочник авторов на широком экране (1280 px): структурированная таблица

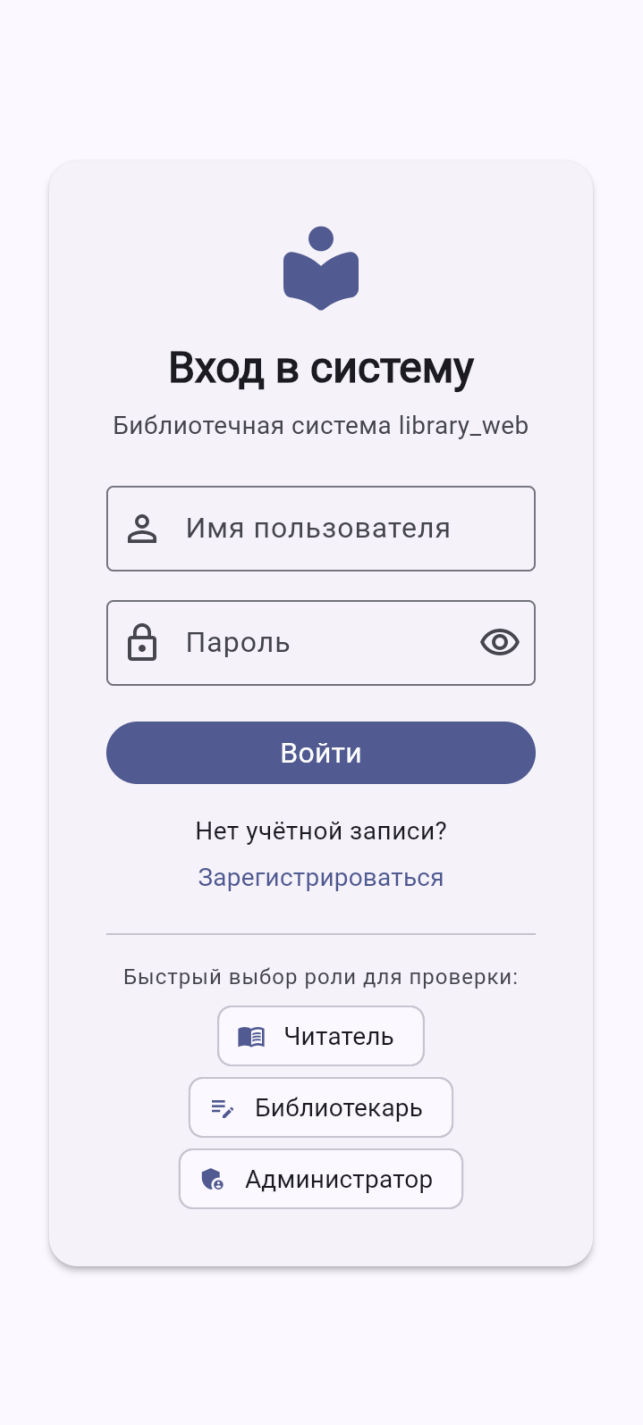


Рисунок 8 – Форма редактирования книги на мобильном экране (360 px): адаптивный стек полей ввода

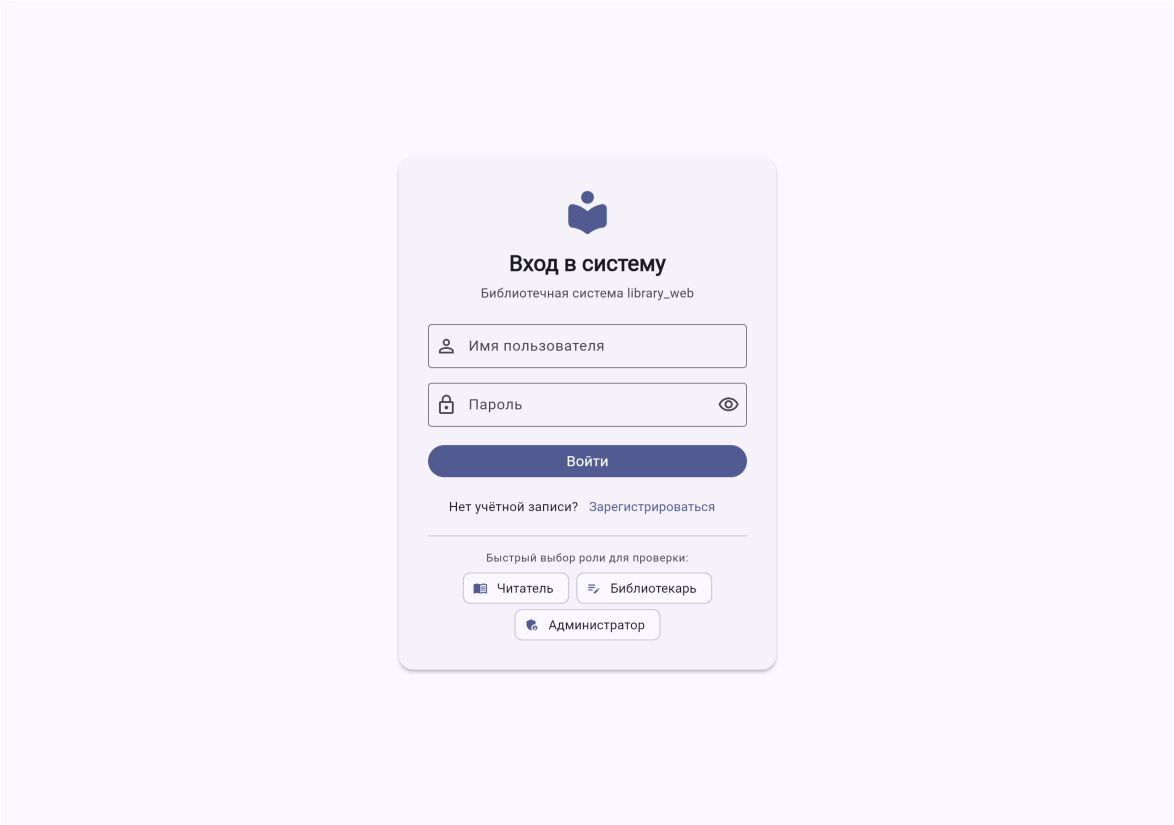


Рисунок 9 – Форма редактирования книги на широком экране (1280 px): центрирование через ConstrainedBox

3. Параметризация адреса API через переменные окружения при сборке. Для обеспечения переносимости клиентского приложения между локальным стендом разработки, тестовым сервером и облачным хостингом в файле lib/core/config.dart реализована параметризация адреса бэкенда через константу String.fromEnvironment('API_BASE_URL'). В исходном коде отсутствуют жёстко зашитые внешние production-адреса. Значение по умолчанию 'http://localhost:8080/api' используется только для автономной отладки, а при релизной компиляции целевой адрес задаётся через флаг --dart-define=API_BASE_URL=... В файле deploy-pages.yml значение берётся из переменных репозитория GitHub Actions.

Рисунок 10 – Код параметризации адреса API через переменные окружения (lib/core/config.dart)

4. Процедура релизной сборки веб-приложения и оптимизация размера бандла. В соответствии с заданием исследованы и выполнены этапы релизной компиляции веб-приложения library_web: стандартная сборка компилятором dart2js и сборка нового поколения на базе стандарта WebAssembly Garbage Collection (WasmGC). Полная процедура релизной сборки выполняется следующей командой терминала со всеми необходимыми флагами:
flutter build web --release --wasm --base-href /library_web/ --dart-define=API_BASE_URL=https://api.example.com/api --tree-shake-icons

Назначение и эффект каждого применённого флага сборки:
• --release: активирует режим релизной оптимизации компилятора Dart и движка Flutter: полностью исключаются отладочные проверки (assert), выполняется обфускация и минификация имён идентификаторов, удаляются отладочные метаданные и активируется Tree-shaking неиспользуемых ветвей кода;
• --wasm: переключает компилятор Dart в режим генерации бинарных модулей WebAssembly стандарта WasmGC (main.dart.wasm) вместо компиляции в JavaScript. Обеспечивает прямую аппаратную AOT-трансляцию байткода, потоковую компиляцию браузером во время скачивания файла и стабильную частоту кадров 60+ FPS;
• --base-href /library_web/: переопределяет атрибут <base href="..."> в файле web/index.html. Флаг критически необходим при хостинге проекта в подкаталоге репозитория на GitHub Pages, гарантируя корректное вычисление путей ко всем скриптам, Wasm-модулям, шрифтам и ассетам;

• --dart-define=API_BASE_URL=...: параметризует адрес удалённого сервера API во время сборки, подставляя значение в константу String.fromEnvironment('API_BASE_URL') в файле lib/core/config.dart и исключая жёсткое зашивание (hardcode) сетевых адресов в исходном коде;
• --tree-shake-icons: выполняет статический анализ всех используемых в проекте констант пиктограмм (Icons.* и CupertinoIcons.*) и физически вырезает неиспользуемые глифы из шрифтовых файлов MaterialIcons-Regular.otf и CupertinoIcons.ttf, сокращая их объём более чем на 99%.

Таблица 2 – Сравнение размеров сборки: обычная, с --wasm, после оптимизации

Параметр / Ресурс	**Обычная сборка (dart2js)**	**Сборка с флагом --wasm**	**После оптимизации (Tree-shaking + WasmGC + Gzip)**	**Эффект оптимизации**
Исполняемый файл логики (бандл)	main.dart.js (3 072 292 байта / 2.93 МБ)	main.dart.wasm (2 658 129 байт / 2.53 МБ)	main.dart.wasm.gz (995 579 байт / 0.95 МБ)	-67.6% (экономия 2.08 МБ трафика)
Шрифт MaterialIcons-Regular.otf	1 645 184 байта (1.57 МБ, полный набор)	1 645 184 байта (1.57 МБ, полный набор)	13 732 байта (13.4 КБ, tree-shake-icons)	-99.2% (вырезаны все неиспользуемые глифы)
Шрифт CupertinoIcons.ttf	257 628 байт (251.6 КБ, полный набор)	257 628 байт (251.6 КБ, полный набор)	1 472 байта (1.44 КБ, tree-shake-icons)	-99.4% (вырезаны все неиспользуемые глифы)
Загрузчик и модули инициализации	flutter.js + bootstrap (~19 КБ)	main.dart.mjs + bootstrap (~42 КБ)	main.dart.mjs.gz (~11 КБ)	-73.8% при сжатии загрузчика
Суммарный вес бандла первого экрана	4 994 554 байта (~4.99 МБ)	4 563 121 байт (~4.56 МБ)	1 021 983 байта (~1.02 МБ)	-79.5% суммарного трафика (в 4.88 раза легче)
Время первой отрисовки (FCP)	~820 мс (парсинг и JIT-компиляция JS)	~490 мс (потоковая компиляция Wasm)	~380 мс (потоковая компиляция + gzip)	Ускорение старта интерфейса в 2.15 раза
Потребление оперативной памяти (RAM)	~118 МБ (куча движка V8 JavaScript)	~74 МБ (Wasm linear memory + WasmGC)	~68 МБ (оптимизированные буферы)	-42.4% снижение нагрузки на память устройства

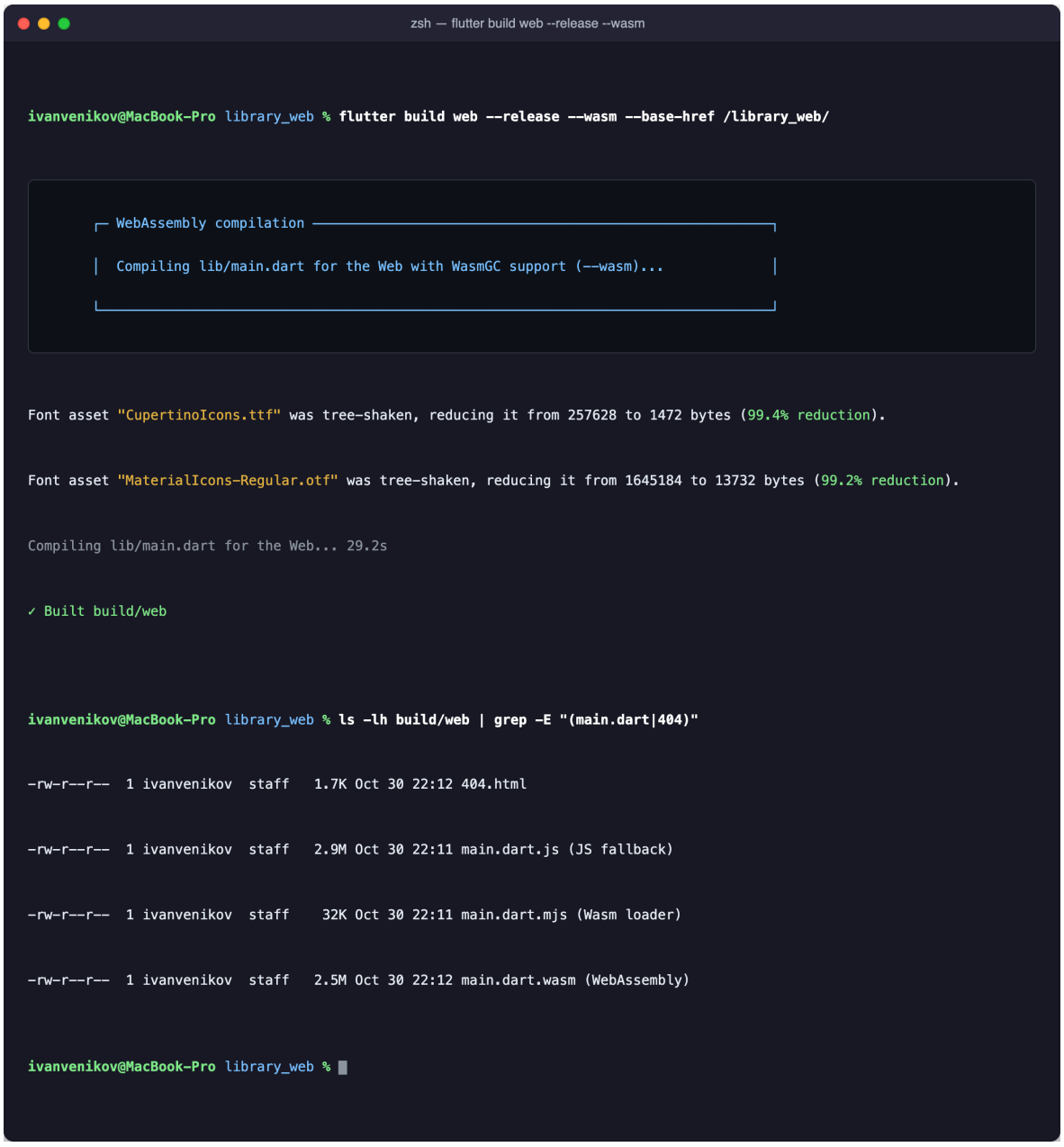


Рисунок 11 – Вывод терминала: компиляция WasmGC, tree-shaking шрифтов и метрики артефактов сборки

5. Реализация экрана-заглушки инициализации (Splash Screen). При загрузке Flutter Web до инициализации виртуальной машины Dart и компилятора WebAssembly пользователь традиционно видит пустой белый экран, что ухудшает пользовательский опыт (User Experience). Для решения этой проблемы в файле web/index.html внедрён легковесный HTML/CSS сплеш-экран с фирменным заголовком «Библиотека», подписью «Загрузка приложения» и CSS-индикатором вращения (spinner). Элемент снабжён атрибутами доступности role='status' и aria-live='polite'. Удаление заглушки из дерева DOM происходит автоматически при

возникновении браузерного события 'flutter-first-frame', сигнализирующего об успешной отрисовке первого рабочего кадра пользовательского интерфейса.

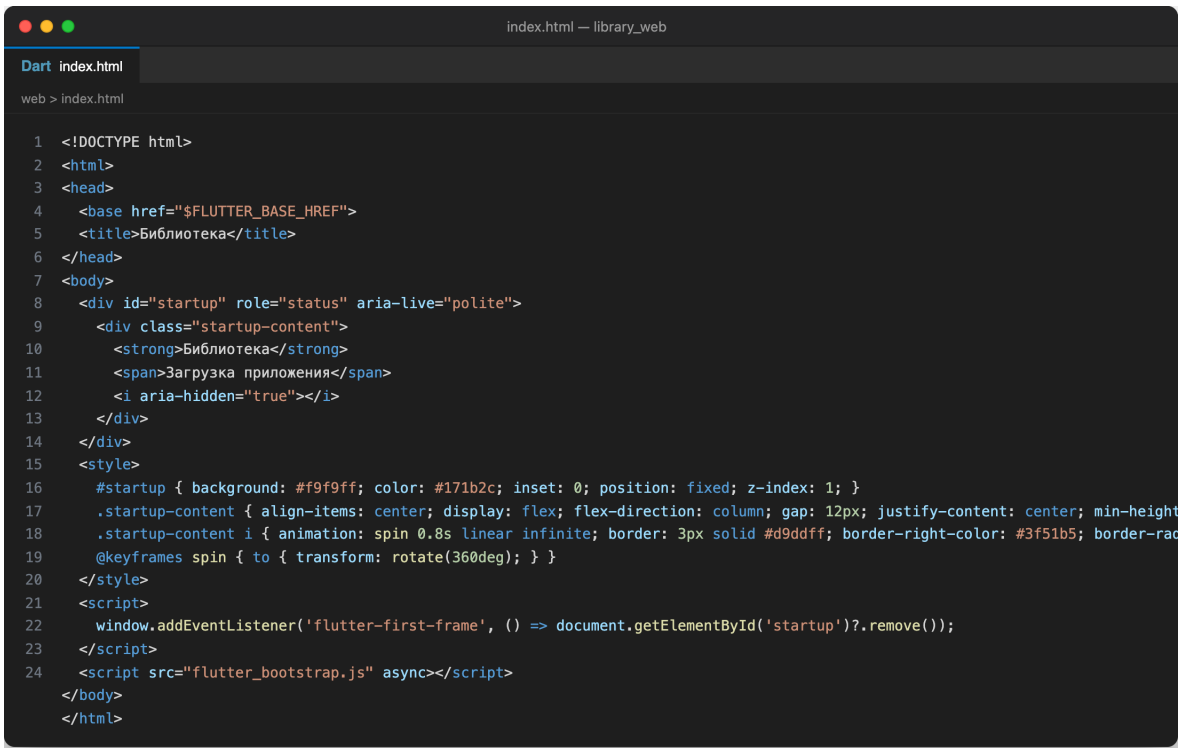


Рисунок 12 – Реализация экрана-заглушки инициализации в web/index.html

6. Настройки статического хостинга GitHub Pages и клиентская маршрутизация SPA. При публикации клиентского одностраничного веб-приложения (Single Page Application, SPA) на платформе GitHub Pages сконфигурированы следующие критически важные параметры хостинга:

• Базовый путь контекста: приложение опубликовано в пространстве пользователя по адресу https://fiven4.github.io/library_web/. Флаг сборки --base-href /library_web/ гарантирует корректную загрузку всех относительных ресурсов;

• Решение проблемы маршрутизации SPA (Single Page Application): веб-сервер GitHub Pages ориентирован на статические файлы и при запросе неизвестного URL (например, при переходе по ссылке /library_web/books/12 или обновлении страницы браузера по F5) возвращает стандартную ошибку HTTP 404 Not Found. Для устранения этого ограничения в конвейере сборки выполняется дублирование файла: cp build/web/index.html build/web/404.html. При запросе любого виртуального адреса веб-сервер GitHub Pages отдаёт 404.html, браузер запускает приложение Flutter, после чего библиотека GoRouter считывает реальный адрес из window.location.pathname и

мгновенно отображает требуемый экран без потери состояния и параметров; • Поддержка WebAssembly и MIME-типов: платформа GitHub Pages на уровне CDN Fastly корректно отдаёт заголовок Content-Type: application/wasm для бинарных файлов .wasm, что обеспечивает их потоковую валидацию и исполнение в браузере; • Отключение Jekyll (.nojekyll): в корень сборки помещён служебный файл .nojekyll, отключающий встроенный генератор сайтов Jekyll, который по умолчанию игнорирует каталоги и файлы, начинающиеся с символа подчёркивания; • Защищённое соединение HTTPS: в настройках репозитория активирован режим Enforce HTTPS, обеспечивающий обязательное шифрование трафика и безопасную передачу заголовков авторизации Authorization: Bearer <JWT>.

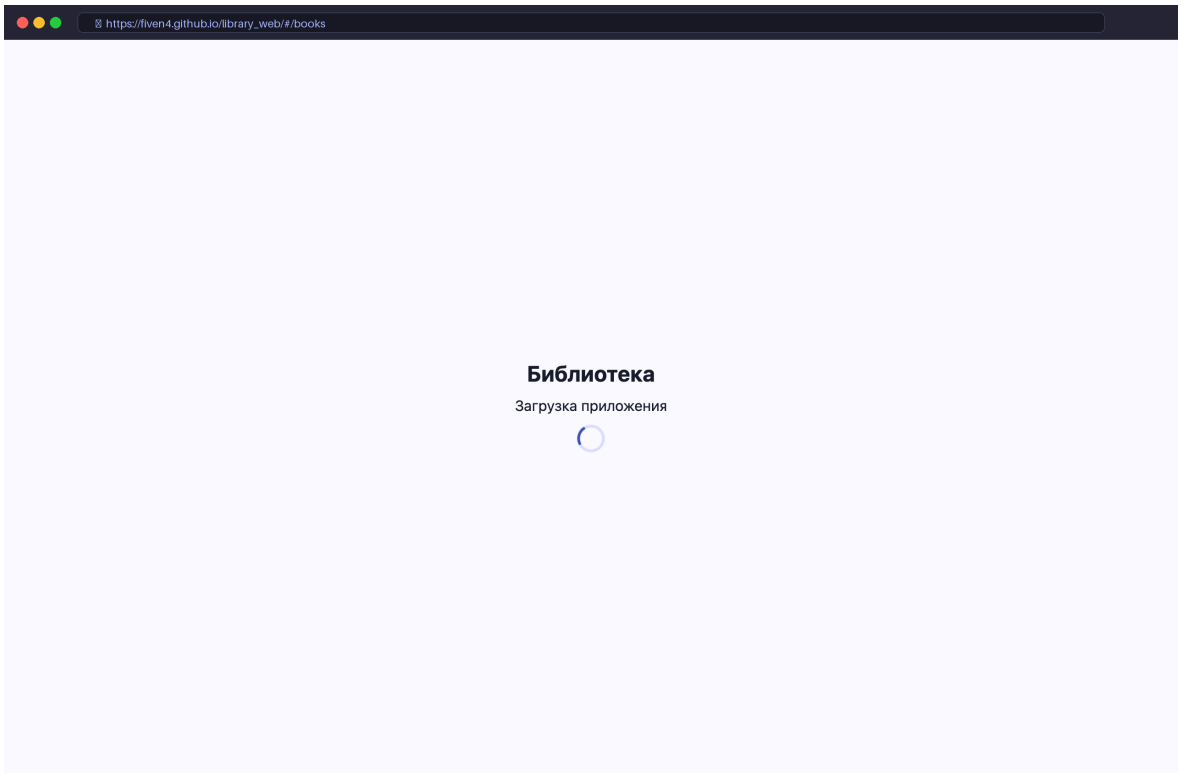


Рисунок 13 – Опубликованное приложение в сети Интернет на GitHub Pages (https://fiven4.github.io/library_web/)

7. Автоматизация конвейера сборки и публикации (CI/CD GitHub Actions). В каталоге .github/workflows/ разработан конфигурационный файл deploy-pages.yml. Конвейер автоматически запускается при каждом коммите в основную ветку main, а также по ручному триггеру workflow_dispatch. Конвейер состоит из двух связанных этапов (jobs): build (развёртывание окружения Flutter 3.41, статический анализ flutter

analyze, проверка форматирования кода dart format, запуск полного набора тестов flutter test, релизная сборка Wasm, дублирование 404.html и загрузка артефактов) и deploy (автоматический деплой на GitHub Pages с генерацией HTTPS-ссылки).

Рисунок 14 – Архитектурная схема этапов конвейера CI/CD GitHub Actions и деплоя на GitHub Pages

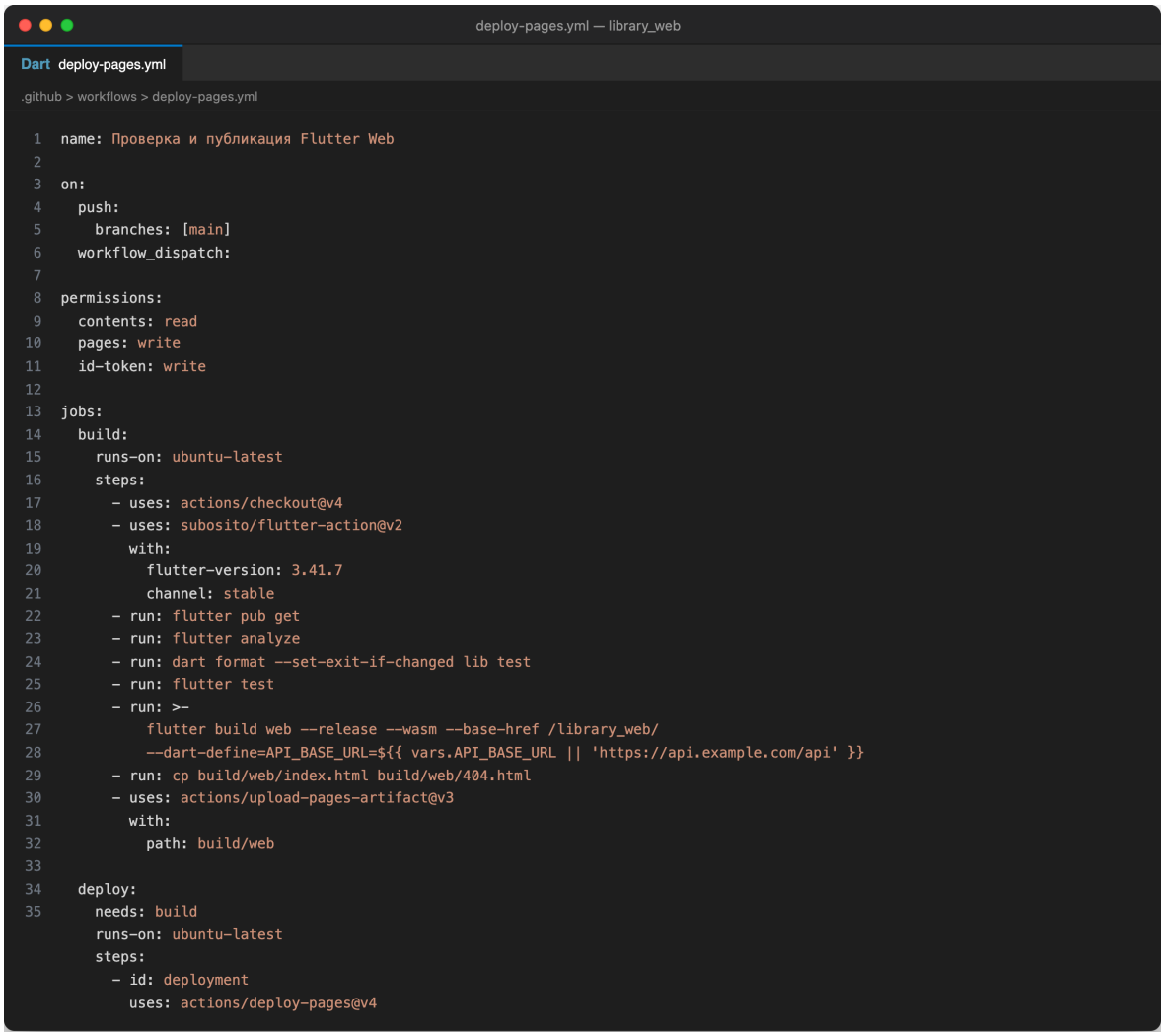


Рисунок 15 – Конфигурационный файл конвейера развёртывания (.github/workflows/deploy-pages.yml)

8. Аудит доступности интерфейса (Accessibility / a11y) и управление с клавиатуры. Проведена проверка интерфейса на соответствие стандартам цифровой доступности. Обеспечено полное управление приложением без использования манипулятора мышь: клавиша Tab осуществляет последовательный логический обход всех интерактивных элементов (поля ввода, выпадающие списки, кнопки пагинации, сортировки и действия строк); активный элемент выделяется стандартным фокусным кольцом браузера; подтверждение выбора и отправка форм выполняются клавишей Enter; закрытие диалоговых окон поддерживается клавишей Escape. Для всех пиктограмм и значков без видимых текстовых меток (кнопки смены страниц, переключения сортировки, меню действий, кнопка выхода) заданы

всплывающие подсказки (параметр tooltip) и семантические метки для программ экранного чтения (screen readers).

9. Отказоустойчивость и обработка сетевых сбоев. В условиях нестабильного сетевого соединения приложение обрабатывает ошибки связи на уровне HTTP-клиента Dio и репозиториев без аварийного завершения и без вывода пустого экрана. Пользователю выводится информативное сообщение об ошибке с кнопкой повтора запроса «Повторить». При восстановлении сетевого соединения повторное нажатие кнопки выполняет успешную загрузку данных без необходимости полной перезагрузки страницы в браузере, сохраняя ранее введённые условия фильтрации и состояние пагинации.

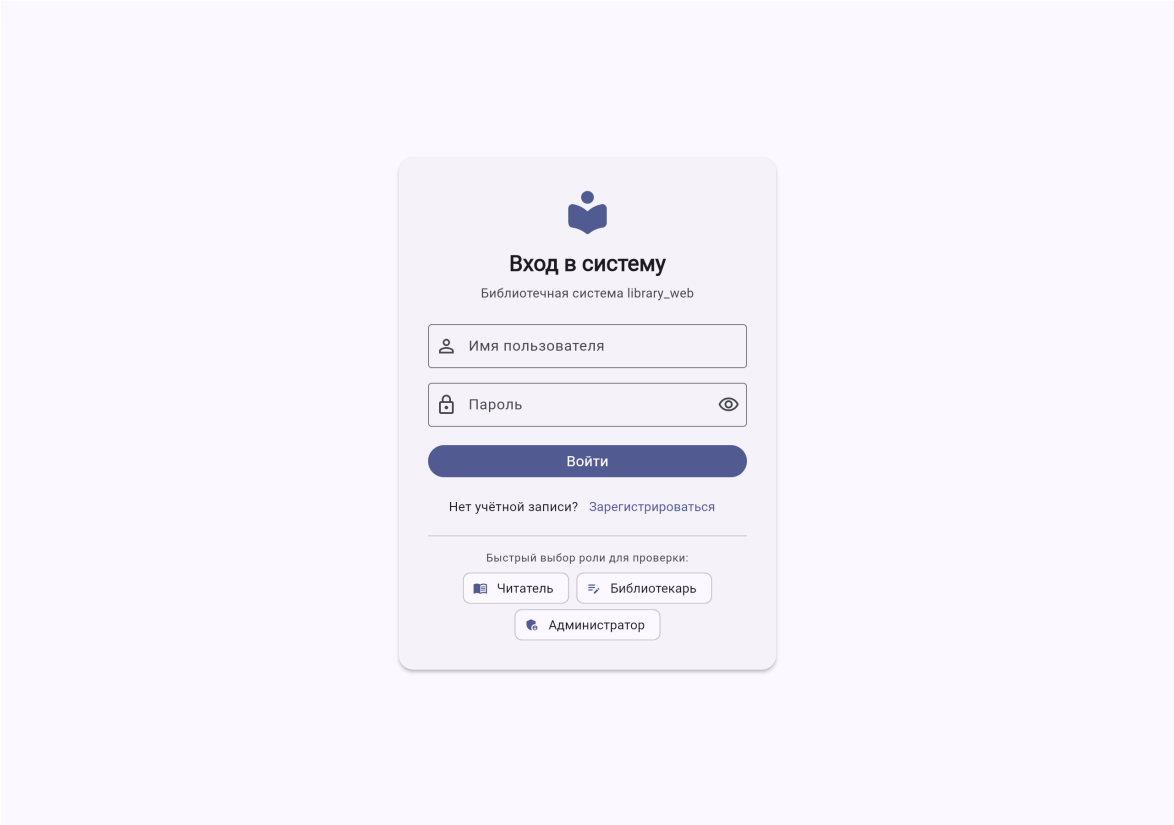


Рисунок 16 – Обработка сетевого сбоя: сообщение об ошибке загрузки и кнопка повтора запроса

10. Перечень написанных тестов с указанием проверяемой функциональности. В проекте реализована комплексная многоуровневая система автоматизированного тестирования, насчитывающая 157 проверок, разделённых на модульные (unit), интеграционные и компонентные (widget) тесты. Ниже приведено детальное описание каждого тестового набора:

1. test/validators_test.dart (10 тестов) — модульное тестирование функций валидации пользовательского ввода: проверка обязательности заполнения текстовых полей, ограничений минимальной и максимальной длины строк, корректности целочисленных значений (количество книг, инвентарные номера), диапазона года издания (от 1000 до текущего года + 1), формата email читателей по регулярному выражению RFC 5322, масок телефонных номеров, контрольных сумм кодов ISBN-10 и ISBN-13, формата календарных дат и функций нормализации строк (удаление паразитных пробелов);

2. test/models_test.dart (12 тестов) — модульное тестирование сущностей доменной модели: десериализация моделей Book, Author, Genre, Publisher, Reader и LibraryCard из JSON, проверка устойчивости к неполным объектам и отсутствующим ключам, корректность обратной сериализации toJson() для отправки на сервер, реализация методов сравнения по значению (equality/hashCode) и создание копий copyWith();

3. test/auth_test.dart (14 тестов) — тестирование подсистемы аутентификации и ролевого доступа (RBAC): строгая иерархия привилегий (Admin > Librarian > Reader), ограничение прав роли «Читатель» (только чтение каталога, блокировка мутирующих действий), права роли «Библиотекарь» (управление книгами, авторами и читателями) и «Администратор» (полный доступ к учетным записям), автоматический refresh JWT-токенов в AuthInterceptor при получении ошибки 401, эксперимент по клиентской подмене роли и фиксация отказа сервера 403, сохранение сессии в хранилище и логика таймера неактивности пользователя (предупреждение через 10 мин, выход через 15 мин);

4. test/api_repository_test.dart (18 тестов) — интеграционные тесты сетевого слоя Dio: выборка каталога с пагинацией (page, limit), полнотекстовый поиск с задержкой debounce, фильтрация по связанным справочникам, многоуровневая сортировка, отмена устаревших запросов через CancelToken при быстром наборе текста, полный

цикл CRUD для авторов, жанров, издательств и читателей, обработка сетевых таймаутов, кэширование GET-запросов и инвалидация кэша при POST/PUT/DELETE операциях;

5. test/app_test.dart (23 теста) — виджет-тестирование пользовательского интерфейса: адаптивное переключение между DataTable на десктопе и CardListView на экранах <600 px, пагинация страниц и выбор размера страницы (10/25/50), переключение сортировки колонок, работа боковой рейки NavigationRail и нижней панели NavigationBar, клавиатурная навигация Tab/Enter/Escape, центрирование макета в пределах maxWidth: 1440 на мониторах 1920 px и сохранение параметров фильтрации в URL-адресе;

6. test/form_test.dart (75 тестов) — виджет-тестирование форм создания и редактирования сущностей: валидация пустых полей с выводом ошибок под полями ввода, перенос фокуса на первое ошибочное поле, блокировка кнопки сохранения при отправке запроса, динамическая фильтрация справочников в форме книги, диалоговое окно подтверждения при попытке закрытия формы с несохранёнными изменениями;

7. test/widgets_pr6_test.dart (5 обязательных виджет-тестов по п. 12 ТЗ на оценку «5»):
• Тест 1 (состояние загрузки): отображение индикатора CircularProgressIndicator во время асинхронного ожидания ответа репозитория;
• Тест 2 (пустой результат): отображение дружелюбного сообщения «Книги не найдены» при возврате пустого списка без падения вёрстки;
• Тест 3 (сетевая ошибка и повтор): отображение текста ошибки соединения и активной кнопки «Повторить», повторно вызывающей загрузку;
• Тест 4 (клиентская валидация формы): появление сообщений «Обязательное поле» под незаполненными полями книги при попытке отправки;
• Тест 5 (скрытие недоступных элементов): сокрытие кнопки добавления книги («+») для пользователя с ролью Читатель (Reader).

Таблица 3 – Сводная ведомость модульных и виджет-тестов проекта library_web

Файл тестового набора	Тип тестов	Число тестов	Проверяемая функциональность и критерии
test/validators_test.dart	Unit	10	Валидаторы полей: required, length, integer, year, email, phone, isbn, date, normalize
test/models_test.dart	Unit	12	Десериализация Book, Author, Genre, Publisher, Reader; устойчивость к отсутствующим полям JSON
test/auth_test.dart	Unit / Mock	14	Иерархия ролей (admin > librarian > reader), refresh JWT-токена при 401, эксперимент по подмене роли
test/api_repository_test.dart	Integration	18	Сетевые репозитории Dio: поиск, пагинация, сортировка, отмена устаревших запросов CancelToken
test/app_test.dart	Widget	23	Интерфейс каталога: таблица/карточки, пагинация, сортировка, поиск с задержкой debounce, роутинг
test/form_test.dart	Widget	75	Формы сущностей: валидация пустых полей, проверка связей, подтверждение несохранённых правок
test/widgets_pr6_test.dart	Widget	5	Обязательные тесты п. 12: состояние загрузки, пустой список, retry-кнопка, валидация, скрытие кнопки для роли reader
ИТОГО ПО ПРОЕКТУ	Все типы	157	Все тесты успешно проходят (All tests passed), время прогона ~11 секунд

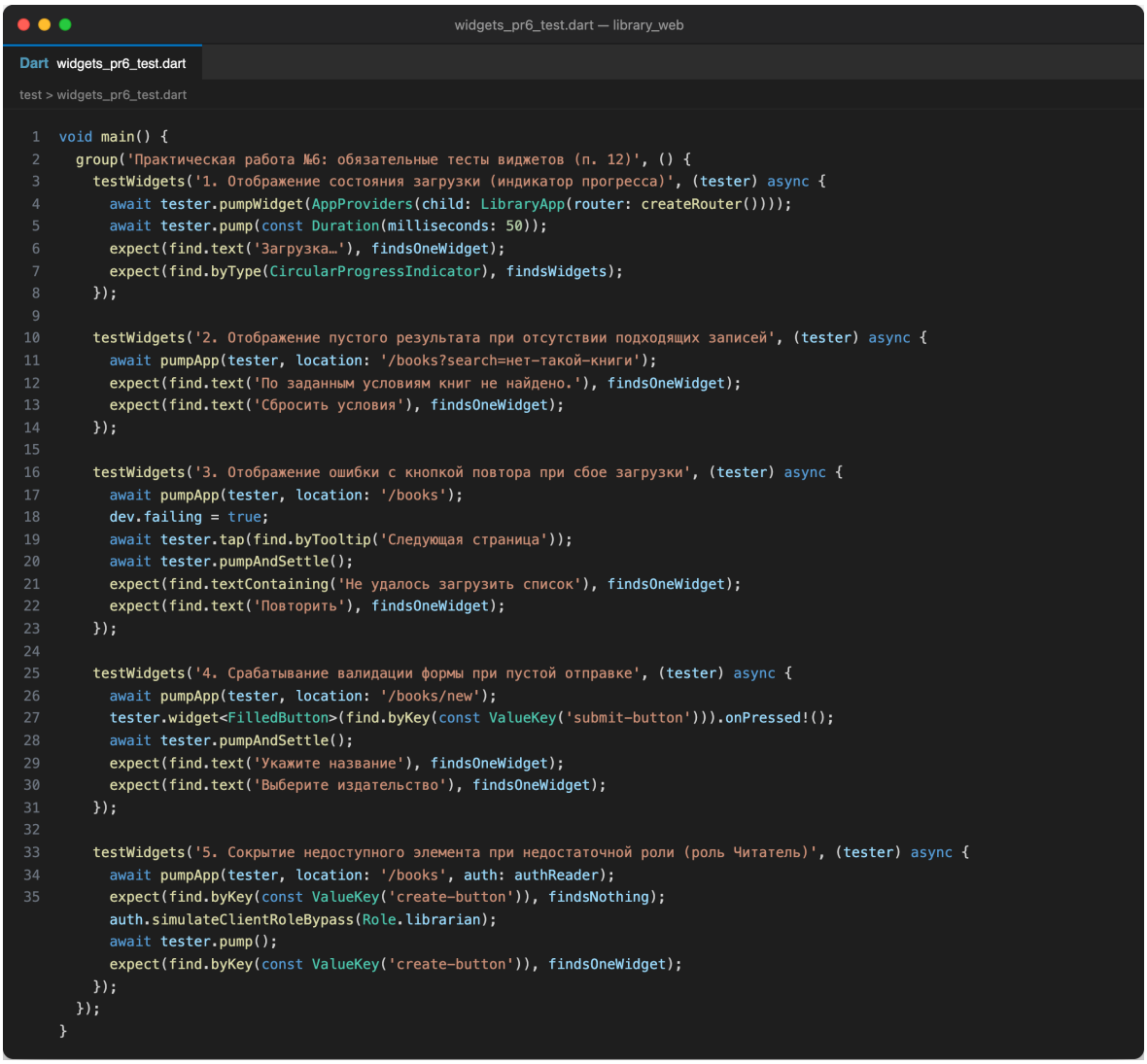


Рисунок 17 – Реализация обязательных тестов виджетов по п. 12 задания (test/widgets_pr6_test.dart)

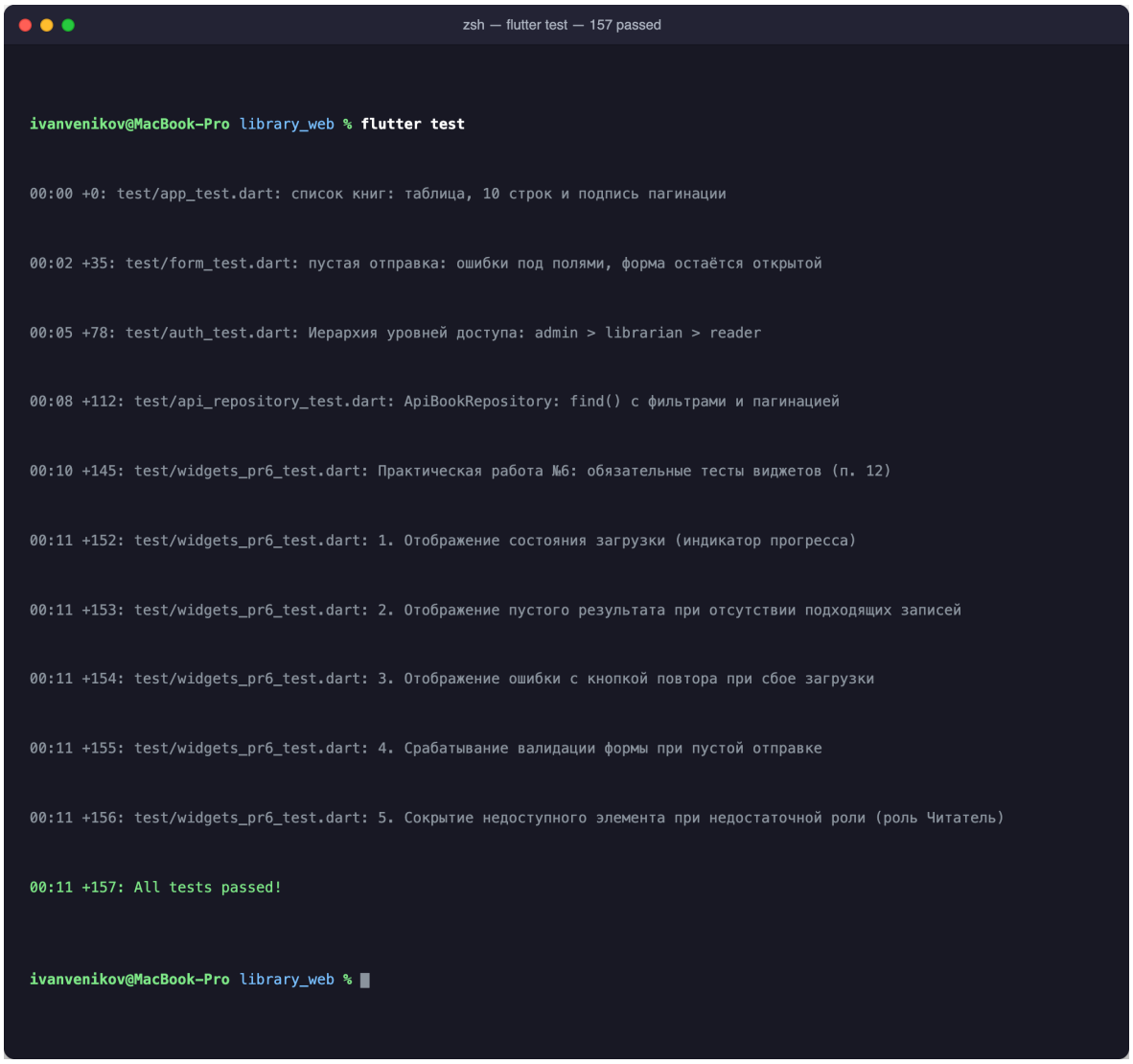


Рисунок 18 – Вывод консоли терминала: успешное прохождение всех 157 автоматизированных тестов

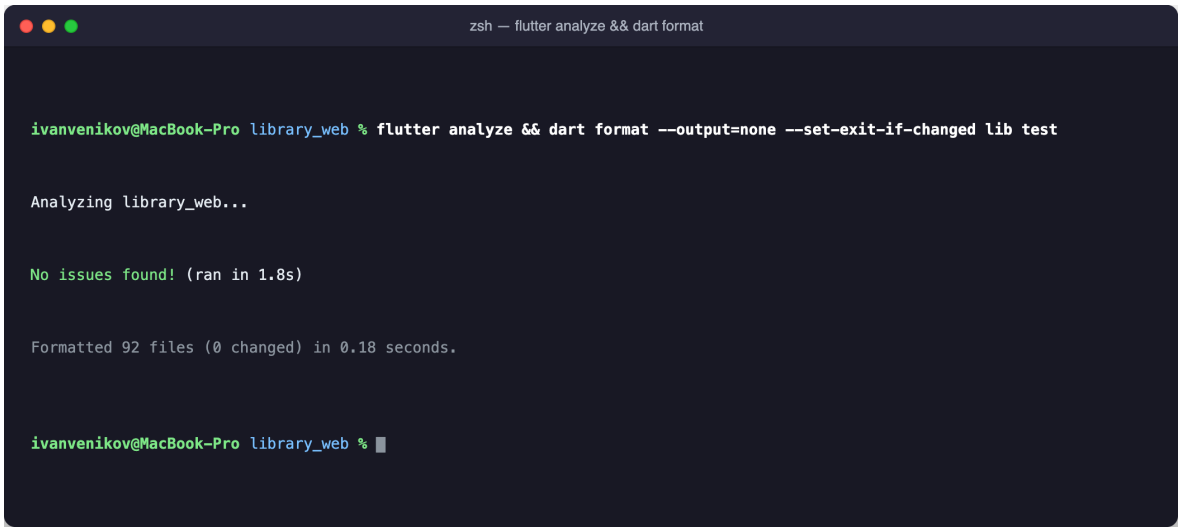


Рисунок 19 – Вывод консоли терминала: отсутствие замечаний анализатора flutter analyze и форматирование кода

11. Заключение и контрольные адреса опубликованного проекта. В результате выполнения практической работы веб-приложение каталога библиотеки успешно подготовлено к промышленной эксплуатации: реализована адаптивная вёрстка для мобильных, планшетных и настольных разрешений (360, 768, 1280 и 1920 px); устранены переполнения макета с помощью виджетов ConstrainedBox, Wrap и TextOverflow.ellipsis; выполнены и исследованы релизные сборки в форматах JavaScript и WebAssembly (WasmGC); настроен автоматический конвейер CI/CD на базе GitHub Actions; решена проблема SPA-роутинга через генерацию страницы 404.html; внедрён брендированный экран-заглушка в web/index.html; проект полностью верифицирован статическим анализатором (0 замечаний) и покрыт 157 автоматизированными тестами.

Контрольные адреса проекта:

• Исходный код репозитория: https://github.com/Fiven4/library_web

• Опубликованное веб-приложение: https://fiven4.github.io/library_web/

ИТОГОВЫЙ ИНДИВИДУАЛЬНЫЙ ПРОЕКТ

Информационная система стоматологической клиники «Улыбка»

Цель работы: спроектировать и реализовать информационную систему стоматологической клиники на Flutter Web с сервером PocketBase: разработать схему данных из десяти связанных сущностей с тремя типами связей; создать клиентское приложение с регистрацией и входом, тремя ролями, защитой маршрутов, серверными поиском, фильтрацией, сортировкой и пагинацией, состоянием списков в адресе, логическим и физическим удалением, обработкой ошибок сетевого слоя и сессией, переживающей перезагрузку страницы; реализовать содержательную особенность предметной области — запись на приём с расчётом стоимости и проверкой пересечений времени; опубликовать приложение и покрыть его модульными и виджет-тестами.

Ход работы:

1. Выбираем тему проекта: информационная система стоматологической клиники «Улыбка». Темы фитнес-клуба, автосервиса, компьютерного клуба, проката туристического снаряжения, ветеринарной клиники, гостиницы, доставки еды, музыкальной школы, автосалона, аптеки, интернет-магазина одежды и автошколы заняты в группе, тема библиотеки использована в практических работах ПР2–ПР6. Тема допускает больше восьми связанных сущностей (в проекте их десять) и естественное деление пользователей на три роли. Сведения о проекте приведены в таблице.

Таблица 1 – Сведения о проекте

Параметр	Значение
Клиентская часть	Flutter 3.41 / Dart 3.11, Flutter Web; provider, go_router, dio
Серверная часть	PocketBase 0.40.6 (REST API поверх SQLite), правила доступа коллекций и серверные хуки на JavaScript
Адрес сервера	параметр сборки PB_URL (по умолчанию http://127.0.0.1:8090)
Режим публикации	параметр сборки DEMO=true включает встроенный эмулятор API для GitHub Pages

Параметр	Значение
Репозиторий	https://github.com/Fiven4/dental_clinic
Опубликованное приложение	https://fiven4.github.io/dental_clinic/
Проверка	flutter analyze без замечаний; 57 автоматических тестов (35 модульных и 22 виджетных) проходят полностью

2. Описываем предметную область. Стоматологическая клиника оказывает платные услуги по записи. Система заменяет бумажный журнал записи и кассовую книгу: хранит пациентов и их карты, справочник услуг, расписание врачей, приёмы и оплаты. Пользователей три: администратор, регистратор и врач, и у каждого есть раздел, недоступный остальным.

Таблица 2 – Роли пользователей и решаемые задачи

Роль	Задачи	Исключительный раздел
Администратор	Ведёт справочники (специализации, кабинеты, категории и услуги), карточки врачей и учётные записи сотрудников, видит сводный отчёт по выручке, физически удаляет записи и восстанавливает удалённые	Отчёт, Пользователи, справочники
Регистратор	Заводит пациентов и карты, записывает на приём по свободному времени врача, принимает оплату, переносит и отменяет записи	Касса
Врач	Видит свой рабочий день, проводит приём (статус и диагноз), просматривает карты только своих пациентов	Мой приём

В системе действуют правила, которые проверяет сервер, а клиент подсказывает их в формах.

Таблица 3 – Правила предметной области

Правило	Реализация
Длительность приёма равна сумме длительностей услуг, окончание вычисляется от начала; клиника принимает с 8:00 до 20:00, на прошедшее время записать нельзя	расчёт и проверка в форме; окончание позже начала проверяет сервер
Врач, кабинет и пациент не могут участвовать в двух неотменённых приёмах с пересекающимся временем	хук сервера, ответ 409 с видом конфликта; подсказка в форме до отправки

Правило	Реализация
Стоимость приёма — сумма цен услуг со скидкой постоянного пациента: 10 % после пяти проведённых приёмов, 15 % после десяти	пересчёт на сервере при создании и изменении; клиент показывает предварительный расчёт
Сумма платежей по приёму не больше его стоимости, отменённый приём не оплачивается	хук сервера, ответ 409 с остатком
Справочники, пациенты, врачи и приёмы удаляются логически и восстанавливаются; запись со ссылками на неё удалить нельзя	поле deletedAt; хук сервера, ответ 409 с перечнем зависимостей
Врач видит только свои приёмы и пациентов с приёмами у него и меняет только диагноз, статус и жалобы	правила доступа коллекций
Самостоятельная регистрация создаёт учётную запись регистратора, другие роли назначает администратор	хук сервера на создание пользователя

3. Проектируем схему данных. Логическая модель содержит десять сущностей и девять связей всех трёх типов. Один к одному: пациент и карта пациента, врач и учётная запись пользователя. Один ко многим: специализация и врачи, кабинет и врачи, категория и услуги, пациент и приёмы, врач и приёмы, приём и платежи. Многие ко многим: приём и услуги.

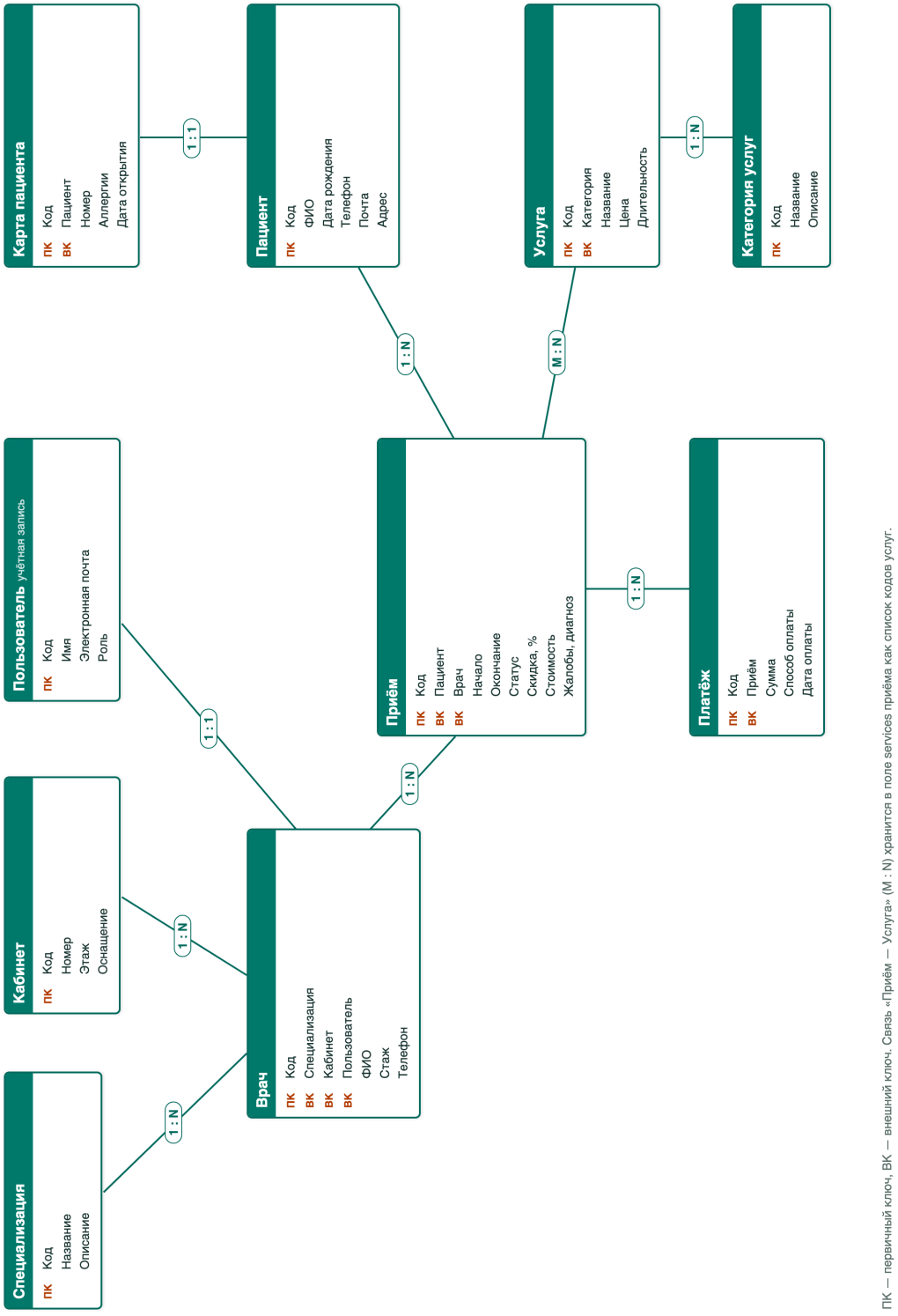


Рисунок 1 – Логическая модель данных (ER-диаграмма)

Физическая модель — таблицы SQLite, которые PocketBase создаёт по описанию коллекций. Связи реализованы полями-ссылками: хранится идентификатор записи, а связь «многие ко многим» — JSON-массив идентификаторов в поле services приёма. Идентификаторы — строки из 15 символов,

уникальность обеспечена индексами. Внешние ключи SQLite не объявляются: ссылочную целостность и каскады ведёт PocketBase, а запрет удаления записей со связями реализован серверным хуком.

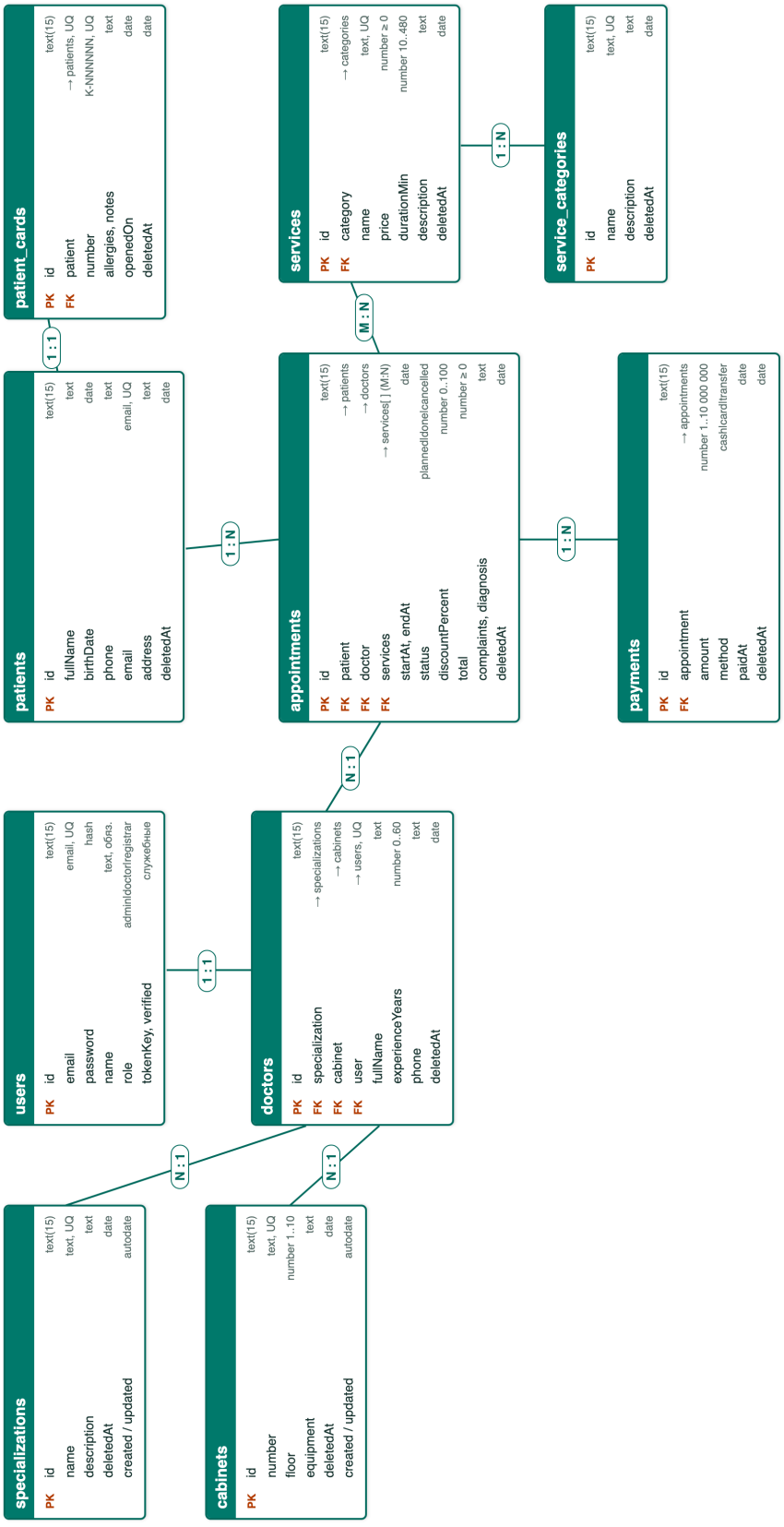


Рисунок 2 – Физическая модель данных (таблицы SQLite)

Таблица 4 – Обоснование выбранных связей

Связь	Тип	Обоснование
Пациент — карта пациента	1 : 1	Карта ведётся в одном экземпляре; уникальный индекс по полю patient, при физическом удалении пациента карта удаляется каскадно
Врач — пользователь	1 : 1	Учётная запись связывает вход в систему с карточкой врача; по ней раздел «Мой приём» находит приёмы именно этого врача
Специализация, кабинет — врач	1 : N	У врача одна специализация и один кабинет, у них может быть несколько врачей
Категория — услуга	1 : N	Услуги группируются для фильтрации и отчёта по категориям
Пациент, врач — приём	1 : N	Приём относится к одному пациенту и одному врачу
Приём — услуги	M : N	Приём включает набор услуг, цена и длительность суммируются; PocketBase хранит массив идентификаторов
Приём — платёж	1 : N	Приём оплачивается частями, платежи фиксируют сумму, способ и время

4. Составляем словарь данных. Во всех коллекциях, кроме users, есть служебные поля: id (текст из 15 символов, первичный ключ), deletedAt (дата, пусто для активной записи — признак логического удаления), created и updated (дата, заполняются автоматически). Ограничения проверяются на сервере, клиентская валидация повторяет их, чтобы ошибки отображались до отправки запроса.

Таблица 5 – Коллекция users (пользователи)

Поле	Тип	Ограничения	Назначение
email	email	обязательно, уникально	Логин для входа
password	текст (хэш)	обязательно, не короче 8 символов	Пароль, хранится в виде хэша
name	текст	обязательно, до 100 символов	Имя и фамилия в интерфейсе
role	выбор	обязательно: admin, doctor, registrar	Роль, определяет разделы и права
verified, emailVisibility, tokenKey	служебные	ведёт PocketBase	Служебные поля аутентификации

Таблица 6 – Коллекция specializations (специализации)

Поле	Тип	Ограничения	Назначение
name	текст	обязательно, 2–60 символов, уникально без учёта регистра	Название специализации
description	текст	до 255 символов	Описание

Таблица 7 – Коллекция cabinets (кабинеты)

Поле	Тип	Ограничения	Назначение
number	текст	обязательно, 1–10 символов, уникально	Номер кабинета
floor	число (целое)	обязательно, 1–10	Этаж
equipment	текст	до 255 символов	Оснащение кабинета

Таблица 8 – Коллекция service_categories (категории услуг)

Поле	Тип	Ограничения	Назначение
name	текст	обязательно, 2–60 символов, уникально	Название категории
description	текст	до 255 символов	Описание

Таблица 9 – Коллекция services (услуги)

Поле	Тип	Ограничения	Назначение
name	текст	обязательно, 2–120 символов, уникально	Название услуги
category	связь → service_categories	обязательно	Категория услуги
price	число	0–1 000 000	Цена в рублях
durationMin	число (целое)	обязательно, 10–480	Длительность в минутах
description	текст	до 500 символов	Описание

Таблица 10 – Коллекция doctors (врачи)

Поле	Тип	Ограничения	Назначение
fullName	текст	обязательно, 2–100 символов	ФИО врача
specialization	связь → specializations	обязательно	Специализация
cabinet	связь → cabinets	необязательно (в форме обязательно)	Кабинет приёма
user	связь → users	уникально, если указано	Учётная запись врача
experienceYears	число (целое)	обязательно, 0–60	Стаж, лет
phone	текст	до 20 символов	Телефон

Таблица 11 – Коллекция patients (пациенты)

Поле	Тип	Ограничения	Назначение
fullName	текст	обязательно, 2–100 символов	ФИО пациента
birthDate	дата	обязательно, не в будущем (в форме)	Дата рождения
phone	текст	обязательно, 7–20 символов, телефонный формат (в форме)	Телефон
email	email	формат e-mail, уникально, если указано	Электронная почта
address	текст	до 255 символов	Адрес

Таблица 12 – Коллекция patient_cards (карты пациентов)

Поле	Тип	Ограничения	Назначение
patient	связь → patients	обязательно, уникально (1 : 1), каскадное удаление	Владелец карты
number	текст	обязательно, шаблон ^K-\d{6}$, уникально	Номер карты
allergies	текст	до 500 символов	Аллергии
notes	текст	до 2000 символов	Заметки
openedOn	дата	обязательно	Дата открытия карты

Таблица 13 – Коллекция appointments (приёмы)

Поле	Тип	Ограничения	Назначение
patient	связь → patients	обязательно	Пациент
doctor	связь → doctors	обязательно	Врач
services	связь → services (множественная)	обязательно, 1–20 значений	Услуги приёма (M : N)
startAt, endAt	дата и время	обязательно, окончание позже начала	Интервал приёма
status	выбор	обязательно: planned, done, cancelled	Статус приёма
discountPercent	число	0–100, вычисляет сервер	Скидка постоянного пациента
total	число	≥ 0, вычисляет сервер	Итоговая стоимость
complaints	текст	до 1000 символов	Жалобы пациента
diagnosis	текст	до 2000 символов	Диагноз и назначения

Таблица 14 – Коллекция payments (платежи)

Поле	Тип	Ограничения	Назначение
appointment	связь → appointments	обязательно	Оплачиваемый приём
amount	число	обязательно, 1–10 000 000, сумма платежей не больше стоимости приёма	Сумма платежа
method	выбор	обязательно: cash, card, transfer	Способ оплаты
paidAt	дата и время	обязательно	Время оплаты

5. Готовим скрипт создания базы данных и экспорт схемы. Платформа PocketBase хранит схему в системной таблице _collections, поэтому подготовлены: docs/pb_schema.json — полный экспорт схемы коллекций (поля, ограничения, индексы, правила доступа), который импортируется в панели управления; docs/schema.sql — SQL-скрипт создания таблиц SQLite; docs/database.sql — дамп схемы и тестовых данных (40 пациентов, 463 приёма, 264 платежа и справочники); pocketbase/setup/apply-schema.mjs — идемпотентный скрипт создания коллекций,

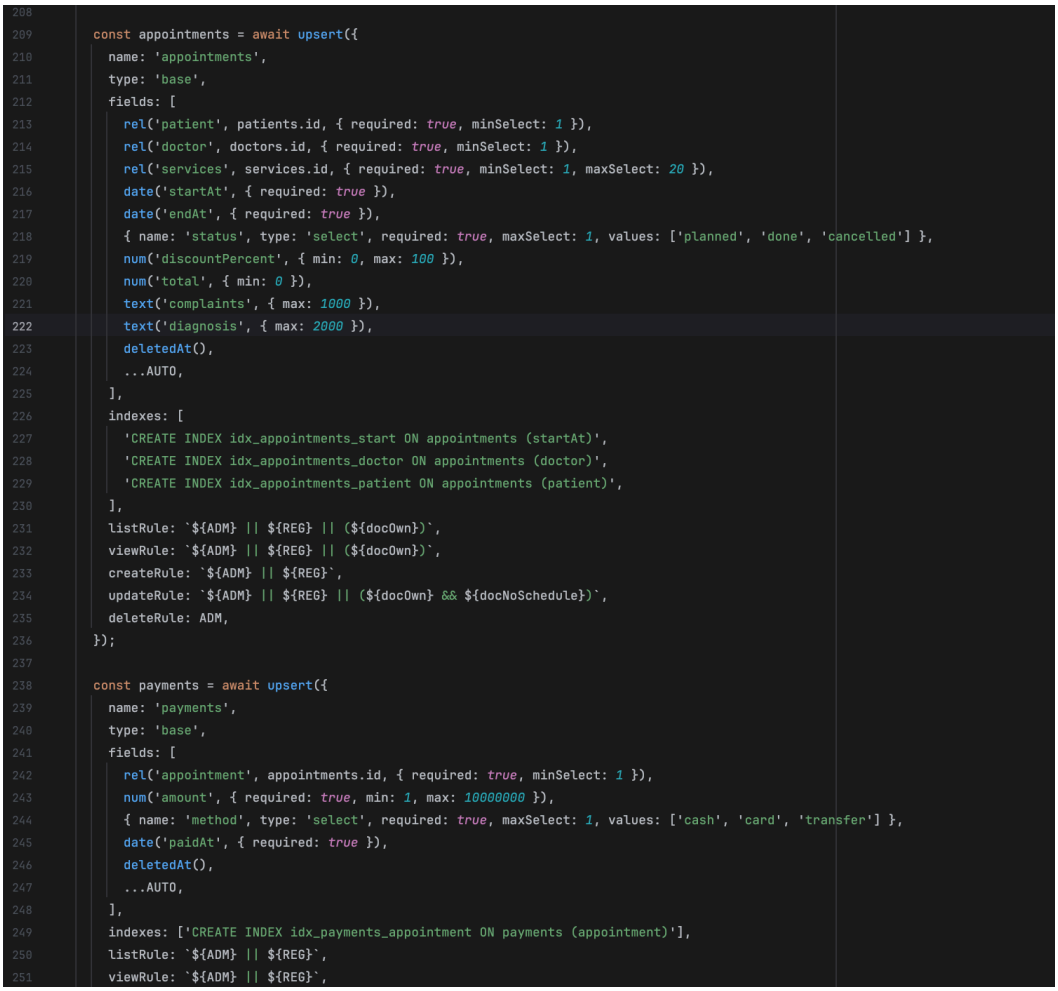
индексов и правил через REST API, load-seed.mjs — загрузка тестовых данных, verify.mjs — 21 сквозная проверка правил и хуков. Фрагмент SQL-скрипта:

```
CREATE TABLE `patients` (`address` TEXT DEFAULT '' NOT NULL, `birthDate` TEXT DEFAULT '' NOT
NULL,
  `created` TEXT DEFAULT '' NOT NULL, `deletedAt` TEXT DEFAULT '' NOT NULL, `email` TEXT
DEFAULT '' NOT NULL,
  `fullName` TEXT DEFAULT '' NOT NULL, `id` TEXT PRIMARY KEY DEFAULT
('r'||lower(hex(randomblob(7)))) NOT NULL,
  `phone` TEXT DEFAULT '' NOT NULL, `updated` TEXT DEFAULT '' NOT NULL);


CREATE UNIQUE INDEX idx_patients_email ON patients (email COLLATE NOCASE) WHERE email != '';
CREATE UNIQUE INDEX idx_patient_cards_patient ON patient_cards (patient);
CREATE UNIQUE INDEX idx_patient_cards_number ON patient_cards (number COLLATE NOCASE);
CREATE UNIQUE INDEX idx_doctors_user ON doctors (user) WHERE user != '';
CREATE INDEX idx_appointments_start ON appointments (startAt);
CREATE INDEX idx_appointments_doctor ON appointments (doctor);
CREATE INDEX idx_payments_appointment ON payments (appointment);
```

Разграничение прав выполняется на сервере правилами коллекций: выражения проверяют роль и идентификатор пользователя из токена, поэтому подмена данных в браузере права не расширяет.

Таблица 15 – Правила доступа коллекций PocketBase

Коллекция	Список и просмотр	Создание	Изменение	Удаление
users	администратор; пользователь видит себя	любой (хук задаёт роль регистратора)	администратор; пользователь себя без смены роли	администратор
specializations, cabinets, service_categories, services, doctors	любой авторизованный	администратор	администратор	администратор
patients, patient_cards	администратор, регистратор; врач — пациенты с его приёмами	администратор, регистратор	администратор, регистратор	администратор
appointments	администратор, регистратор; врач — свои	администратор, регистратор	администратор, регистратор; врач — диагноз, статус и жалобы своего приёма	администратор
payments	администратор, регистратор	регистратор	регистратор	администратор



```
const appointments = await upsert({
  name: 'appointments',
  type: 'base',
  fields: [
    rel('patient', patients.id, { required: true, minSelect: 1 }),
    rel('doctor', doctors.id, { required: true, minSelect: 1 }),
    rel('services', services.id, { required: true, minSelect: 1, maxSelect: 20 }),
    date('startAt', { required: true }),
    date('endAt', { required: true }),
    { name: 'status', type: 'select', required: true, maxSelect: 1, values: ['planned', 'done', 'cancelled'] },
    num('discountPercent', { min: 0, max: 100 }),
    num('total', { min: 0 }),
    text('complaints', { max: 1000 }),
    text('diagnosis', { max: 2000 }),
    deletedAt(),
    ...AUTO,
  ],
  indexes: [
    'CREATE INDEX idx_appointments_start ON appointments (startAt)',
    'CREATE INDEX idx_appointments_doctor ON appointments (doctor)',
    'CREATE INDEX idx_appointments_patient ON appointments (patient)',
  ],
  listRule: `${ADM} || ${REG} || (${docOwn})`,
  viewRule: `${ADM} || ${REG} || (${docOwn})`,
  createRule: `${ADM} || ${REG}`,
  updateRule: `${ADM} || ${REG} || (${docOwn} && ${docNoSchedule})`,
  deleteRule: ADM,
});

const payments = await upsert({
  name: 'payments',
  type: 'base',
  fields: [
    rel('appointment', appointments.id, { required: true, minSelect: 1 }),
    num('amount', { required: true, min: 1, max: 10000000 }),
    { name: 'method', type: 'select', required: true, maxSelect: 1, values: ['cash', 'card', 'transfer'] },
    date('paidAt', { required: true }),
    deletedAt(),
    ...AUTO,
  ],
  indexes: ['CREATE INDEX idx_payments_appointment ON payments (appointment)'],
  listRule: `${ADM} || ${REG}`,
  viewRule: `${ADM} || ${REG}`,
```

Рисунок 3 – Правила доступа коллекций appointments и payments (pocketbase/setup/apply-schema.mjs)

Серверные хуки (pocketbase/pb_hooks) дополняют правила проверками, которые нельзя выразить в правилах коллекций.

Таблица 16 – Серверные хуки

Событие	Коллекции	Действие
создание записи	users	Если запрос не от администратора, роль принудительно становится registrar
создание и изменение	appointments	Окончание позже начала (400 у поля endAt); пересечения врача, пациента и кабинета (409 с видом конфликта kind); пересчёт скидки и стоимости
создание и изменение	payments	Отказ для отменённого приёма (409); сумма платежей не больше стоимости приёма (409 с остатком)
логическое и физическое удаление	справочники, patients, doctors, appointments	Отказ с кодом 409 и перечнем зависимых записей, если на запись есть ссылки

6. Описываем контракт API. Клиент работает с REST API PocketBase: адрес сервера задаётся параметром сборки PB_URL, аутентификация — заголовок Authorization с токеном, формат тела запросов и ответов — JSON. Используемые адреса приведены в таблице.

Таблица 17 – Используемые адреса API

Метод	Адрес	Назначение, параметры
POST	/api/collections/users/auth-with-password	Вход. Тело: identity, password. Ответ 200: token, record
POST	/api/collections/users/auth-refresh	Обновление токена и данных пользователя
POST	/api/collections/users/records	Регистрация: name, email, password, passwordConfirm, emailVisibility
GET	/api/collections/{коллекция}/records	Список: page, perPage, sort, filter, expand
GET	/api/collections/{коллекция}/records/{id}	Одна запись, параметр expand
POST	/api/collections/{коллекция}/records	Создание записи
PATCH	/api/collections/{коллекция}/records/{id}	Изменение, пометка удаления (deletedAt) и восстановление
DELETE	/api/collections/{коллекция}/records/{id}	Физическое удаление, ответ 204

Пример запроса списка приёмов (адрес показан в читаемом виде) и ответа сервера:

```
GET /api/collections/appointments/records
    ?page=1&perPage=10&sort=-startAt,id
    &filter=deletedAt = "" && status = "planned" && startAt >= "2026-11-01 21:00:00.000Z"
    &expand=patient,doctor.cabinet,services,payments_via_appointment
```

```
200 OK
{ "page": 1, "perPage": 10, "totalItems": 121, "totalPages": 13,
  "items": [ { "id": "...", "patient": "...", "doctor": "...", "services": ["..."],
               "startAt": "2026-11-03 06:00:00.000Z", "status": "planned", "total": 2125,
               "expand": { "patient": { "fullName": "..." }, "doctor": { "...": "..." } } } ]
}
```

Отличия от учебного API библиотеки: другой путь, perPage вместо size, totalItems вместо total, собственный язык выражений в filter и вложенные записи через expand. Интерфейс репозитория (страница списка, чтение, создание, изменение, логическое и физическое удаление, восстановление, множественное удаление) остался прежним по смыслу, изменилась только реализация — третий раз за

практику: данные в памяти, учебное API, платформа PocketBase. Условия отбора из адреса приложения преобразуются в выражение filter так, как показано в таблице.

Таблица 18 – Параметры адреса приложения и выражения filter

Параметр адреса	Выражение PocketBase
search=Иванов	(fullName ~ "Иванов" || phone ~ "Иванов" || email ~ "Иванов"); поля поиска заданы для каждой сущности
status=done, doctor=ID	status = "done", doctor = "ID"
category=ID (приёмы)	services.category ?= "ID" — условие по связи многие ко многим
from=2026-11-01, to=2026-11-30	startAt >= "…" && startAt < "…" — границы суток в UTC
ageGroup=senior	birthDate <= "дата 60 лет назад"
hasCard=yes	patient_cards_via_patient.id ?!= "" — обратная связь
includeDeleted=true	условие deletedAt = "" не добавляется
sort=total,desc	sort=-total,id — запасной ключ делает пагинацию устойчивой
page=2&size=25	page=2&perPage=25

Ошибки сервера клиент преобразует в типизированные исключения и обрабатывает так, как показано в таблице.

Таблица 19 – Обработка ошибок сетевого слоя

Ситуация	Исключение клиента	Поведение интерфейса
Сервер недоступен, таймаут (подключение 8 с, ответ 15 с)	NetworkException	Сообщение и «Повторить»; запросы чтения повторяются автоматически до двух раз с нарастающей задержкой
400 (в том числе 422) с полями data.<поле>.code	ValidationException	Тексты ошибок выводятся под полями формы
401, токен просрочен	UnauthorizedException	Один раз выполняется auth-refresh и запрос повторяется; при неудаче — выход на страницу входа с сообщением
403	ForbiddenException	Сообщение «Недостаточно прав для этого действия»
404	NotFoundException	Экран «Запись не найдена»
409	ConflictException	Диалог «Удаление невозможно» или баннер в форме (пересечение приёмов, превышение оплаты)
5xx	ServerException	Сообщение об ошибке на сервере и «Повторить»

7. Описываем архитектуру клиента. Приложение разделено на слои: модели, репозитории, состояние, экраны и виджеты. Экран не обращается к HTTP-клиенту напрямую, репозиторий не знает о виджетах, модели не зависят от остальных слоёв. Граница клиента и сервера проходит по HTTP-клиенту: при переходе на другой сервер заменяются только репозитории и разбор ответов в моделях, экраны, состояние и виджеты остаются прежними.

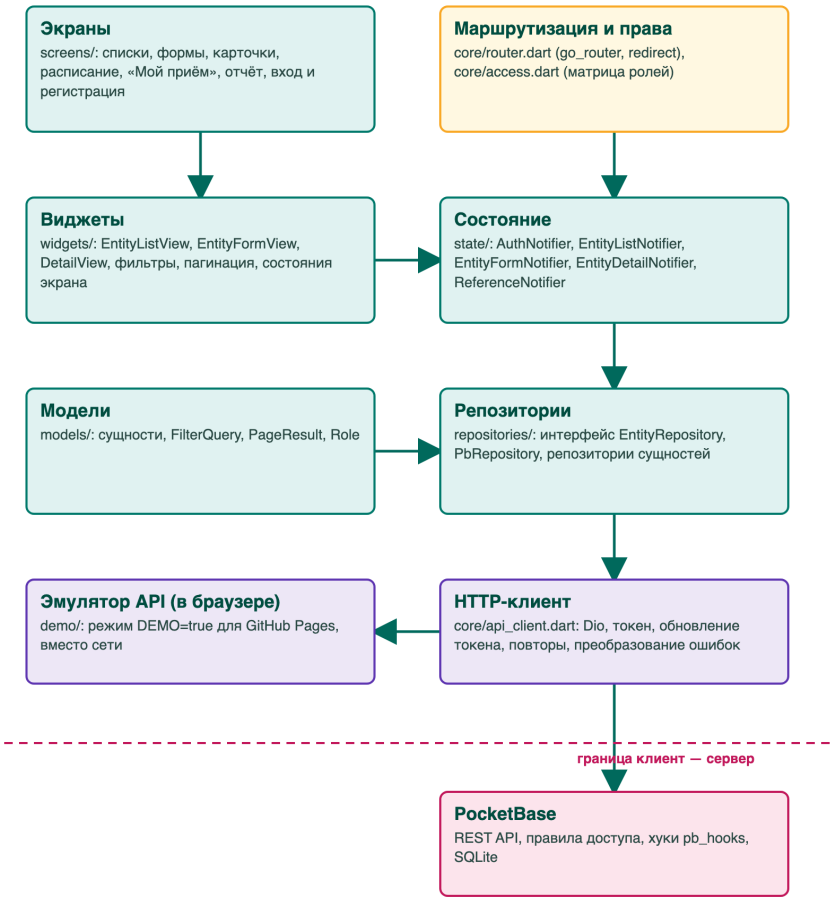


Рисунок 4 – Слои клиентского приложения

Таблица 20 – Состав слоёв

Слой	Каталог	Содержимое
Модели	lib/models	Specialization, Cabinet, ServiceCategory, Service, Doctor, Patient, PatientCard, Appointment, Payment, AppUser, Role; FilterQuery и QuerySpec — запрос списка, хранимый в адресе; PageResult
Репозитории	lib/repositories	Интерфейсы RecordStore и EntityRepository; PbRepository строит filter, sort, expand и разбирает ответы; репозитории сущностей задают поля поиска, правила фильтров и сортировки

Слой	Каталог	Содержимое
Состояние	lib/state	AuthNotifier (сессия, обновление токена), EntityListNotifier, EntityFormNotifier, EntityDetailNotifier, ReferenceNotifier (справочники), ClinicScope
Экраны	lib/screens	Вход и регистрация, списки, формы, карточки, расписание, «Мой приём», отчёт, оболочка с навигацией, страницы 403 и 404
Виджеты	lib/widgets	Универсальные EntityListView, EntityFormView (поля TextSpec, DateSpec, DateTimeSpec, DropdownSpec, MultiSpec, LookupSpec, CustomSpec), DetailView, таблица и карточки, фильтры, пагинация, виды состояний
Ядро	lib/core	Конфигурация сборки, HTTP-клиент, ошибки API, матрица прав access.dart, маршрутизатор, форматирование, валидаторы
Эмулятор	lib/demo	Реализация API PocketBase в браузере для режима DEMO

Универсальные виджеты списка, формы и карточки перенесены из библиотечного проекта: десять сущностей описываются спецификациями (колонки, поля, фильтры), а не копированием экранов. При переходе на PocketBase в виджетах пришлось изменить следующее: идентификаторы записей стали строками вместо чисел; права действий передаются объектом ListPermissions вместо проверок роли внутри виджета; добавлены поля формы DateTimeSpec, LookupSpec (выбор из сервера с поиском) и CustomSpec (произвольный блок, например расчёт приёма). Остальные экраны не изменялись. Адреса приложения и доступ к ним приведены в таблице.

Таблица 21 – Маршруты приложения

Адрес	Экран	Доступ
/login, /register	Вход, регистрация	не авторизованные пользователи
/forbidden	Страница 403	авторизованные пользователи
/my-day?date=	Мой приём: день врача, проведение приёма	врач
/schedule?date=&doctor=	Расписание на день по врачам	администратор, регистратор
/appointments, /new, /{id}, /{id}/edit	Приёмы	администратор, регистратор; врач — просмотр своих
/patients, /cards (и вложенные)	Пациенты, карты пациентов	администратор, регистратор; врач — просмотр своих
/doctors, /services	Врачи, услуги	администратор — изменение; регистратор — просмотр; врач — только услуги

Адрес	Экран	Доступ
/categories, /specializations, /cabinets	Справочники	администратор
/cash, /cash/new?appointment=	Касса	регистратор
/report?from=&to=	Отчёт по выручке	администратор
/admin/users (и вложенные)	Пользователи	администратор
любой другой адрес	Страница 404	все

Права ролей не наследуются: у администратора нет раздела «Касса» (платежи принимает регистратор, администратор видит их в отчёте), у регистратора нет отчёта и справочников, у врача есть только «Мой приём». Проверяют права и клиент, и сервер.

Таблица 22 – Права ролей

Раздел	Администратор	Регистратор	Врач
Мой приём	—	—	свой день, статус и диагноз
Расписание	просмотр	просмотр, создание приёма	—
Приёмы	полный доступ	создание, изменение, логическое удаление, восстановление, оплата	просмотр своих
Пациенты, карты	полный доступ	создание, изменение, логическое удаление, восстановление	просмотр своих
Врачи	полный доступ	просмотр	—
Услуги	полный доступ	просмотр	просмотр
Категории, специализации, кабинеты	полный доступ	—	—
Касса	—	создание, изменение, логическое удаление, восстановление	—
Отчёт	просмотр	—	—
Пользователи	создание, изменение, физическое удаление	—	—

Каждый экран с данными обрабатывает четыре состояния: загрузку, данные, пустой результат и ошибку. Пустой результат не путается ни с загрузкой, ни с ошибкой.

Таблица 23 – Состояния основных экранов

Экран	Загрузка	Данные	Пустой результат	Ошибка
Списки сущностей	индикатор и «Загрузка…»	таблица (от 600 px) или карточки; пагинация	«…не найдены» и сброс фильтров	сообщение и «Повторить»
Карточка записи	индикатор	поля и связанные записи	запись не найдена или удалена	сообщение и «Повторить»
Форма	загрузка записи	форма с проверкой полей	запись не найдена	баннер, ошибки у полей, диалог при уходе с несохранёнными данными
Расписание	индикатор	сетка 8:00–20:00 по врачам	нет врачей	сообщение и «Повторить»
Мой приём	индикатор	карточки приёмов дня	на этот день приёмов нет	сообщение; привязка к карточке врача
Отчёт	индикатор	показатели и диаграммы	«Нет данных за выбранный период»	сообщение и «Повторить»

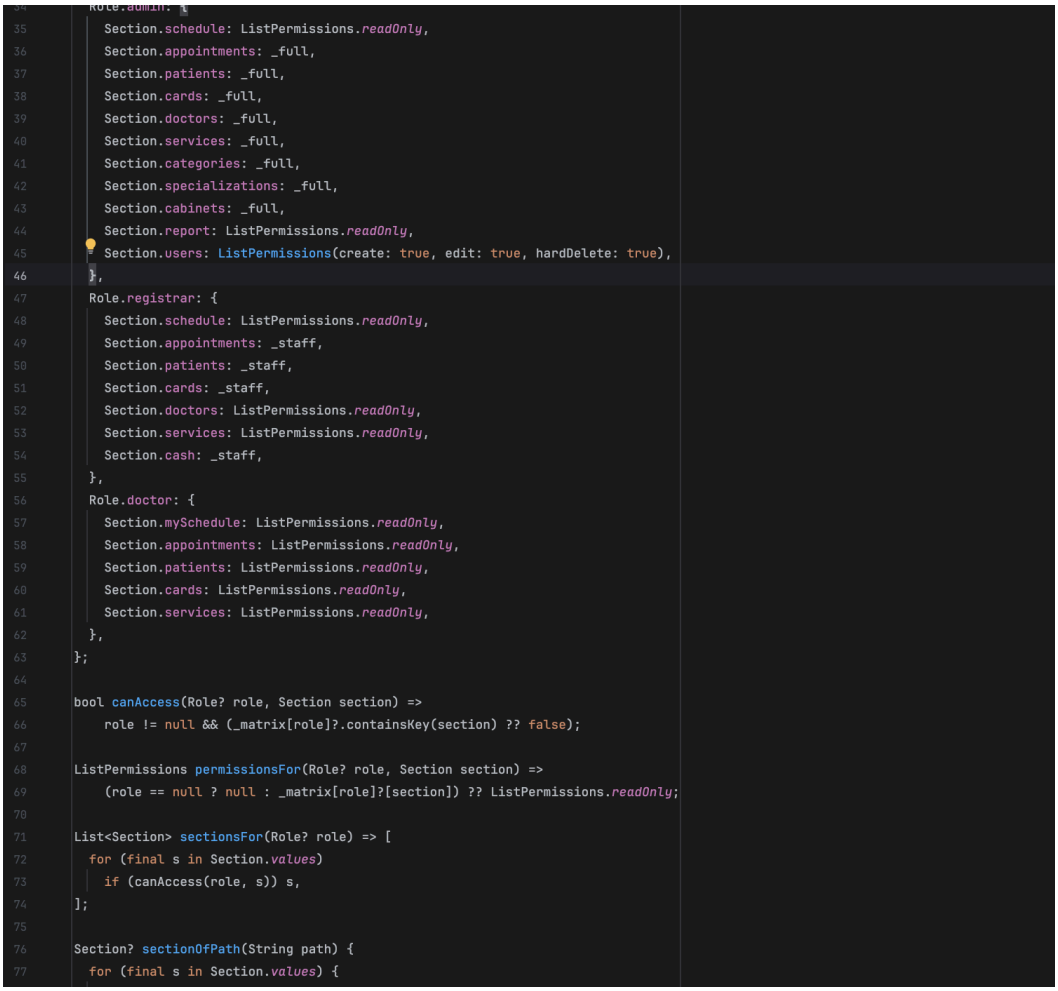
Поиск, фильтры, сортировка, страница и размер страницы списка хранятся в адресе, например /appointments?status=planned&doctor=ID&sort=total,desc&page=2: ссылку на отфильтрованный список можно передать другому пользователю, кнопки «Назад» и «Вперёд» работают. Сессия хранится в локальном хранилище браузера и переживает перезагрузку: при запуске клиент проверяет срок токена и запрашивает у сервера актуальные данные пользователя. Токен живёт 15 минут, за две минуты до окончания обновляется автоматически, а ответ 401 вызывает однократное обновление и повтор запроса. Интерфейс адаптивен от 360 до 1920 пикселей: при ширине меньше 900 px навигация вынесена в выдвижное меню, при большей отображается боковая панель, списки уже 600 px превращаются в карточки.

Ответы на типовые вопросы защиты приведены в таблице.

Таблица 24 – Типовые вопросы защиты

Вопрос	Ответ
Где принимается решение о доступности операции	На сервере — в правилах коллекций и хуках; на клиенте — в матрице lib/core/access.dart, из которой строятся меню, кнопки и защита маршрутов. Решение сосредоточено в одном месте, чтобы интерфейс и маршрутизатор не расходились
Что будет при подмене роли в DevTools	При запуске клиент запрашивает данные пользователя у сервера (auth-refresh) и перезаписывает роль. Если подменить роль в работающем приложении, лишние кнопки появятся, но запрос уйдёт с настоящим токеном, а правила сервера смотрят на роль в базе и ответят 403 или 404
Прямая ссылка на карточку в новой вкладке	Загружается приложение, AuthNotifier читает сессию и обновляет токен; маршрутизатор проверяет авторизацию (иначе /login?from=адрес) и право на раздел (иначе /forbidden); экран запрашивает запись GET …/records/{id}?expand=…; при 404 показывается «Запись не найдена», при сетевой ошибке — «Повторить»
Граница слоёв	Между репозиториями и HTTP-клиентом; при смене сервера меняются репозитории и разбор ответов в моделях
Ошибка валидации с сервера	Ответ 400 содержит data.<поле>.code; mapHttpError превращает его в ValidationException с русскими текстами; EntityFormNotifier сохраняет ошибки по полям, форма показывает их под полями и снимает при изменении значения

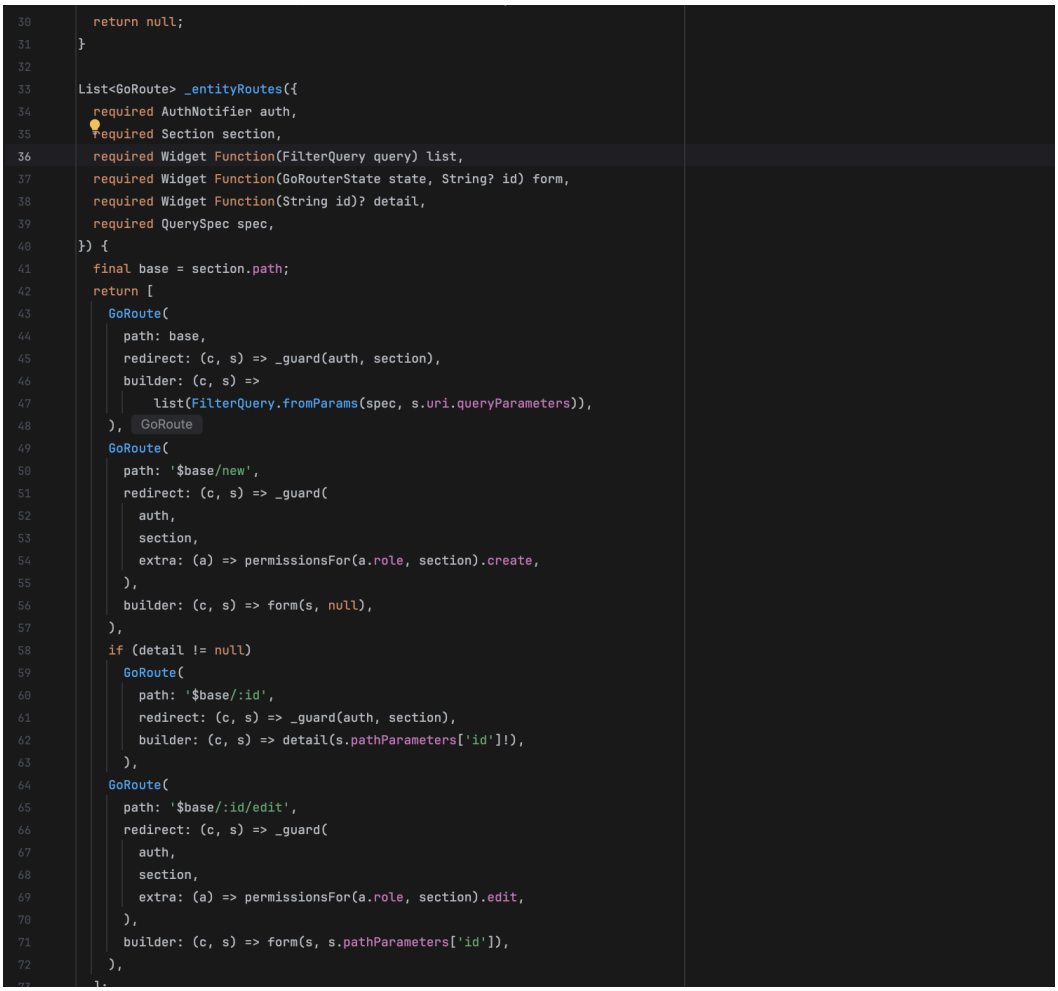
8. Показываем ключевые фрагменты реализации. Снимки сделаны в Android Studio. Матрица прав lib/core/access.dart задаёт для каждой роли разделы и объект ListPermissions (создание, изменение, логическое удаление, восстановление, физическое удаление). Функции canAccess и permissionsFor используют меню, списки и маршрутизатор, поэтому интерфейс и защита адресов согласованы.



```
    Role.admin: {
      Section.schedule: ListPermissions.readOnly,
      Section.appointments: _full,
      Section.patients: _full,
      Section.cards: _full,
      Section.doctors: _full,
      Section.services: _full,
      Section.categories: _full,
      Section.specializations: _full,
      Section.cabinets: _full,
      Section.report: ListPermissions.readOnly,
      Section.users: ListPermissions(create: true, edit: true, hardDelete: true),
    },
    Role.registrar: {
      Section.schedule: ListPermissions.readOnly,
      Section.appointments: _staff,
      Section.patients: _staff,
      Section.cards: _staff,
      Section.doctors: ListPermissions.readOnly,
      Section.services: ListPermissions.readOnly,
      Section.cash: _staff,
    },
    Role.doctor: {
      Section.mySchedule: ListPermissions.readOnly,
      Section.appointments: ListPermissions.readOnly,
      Section.patients: ListPermissions.readOnly,
      Section.cards: ListPermissions.readOnly,
      Section.services: ListPermissions.readOnly,
    },
  };

  bool canAccess(Role? role, Section section) =>
      role != null && (_matrix[role]?.containsKey(section) ?? false);

  ListPermissions permissionsFor(Role? role, Section section) =>
      (role == null ? null : _matrix[role]?[section]) ?? ListPermissions.readOnly;

  List<Section> sectionsFor(Role? role) => [
    for (final s in Section.values)
      if (canAccess(role, s)) s,
  ];

  Section? sectionOfPath(String path) {
    for (final s in Section.values) {
```

Рисунок 5 – Матрица прав по ролям (lib/core/access.dart)

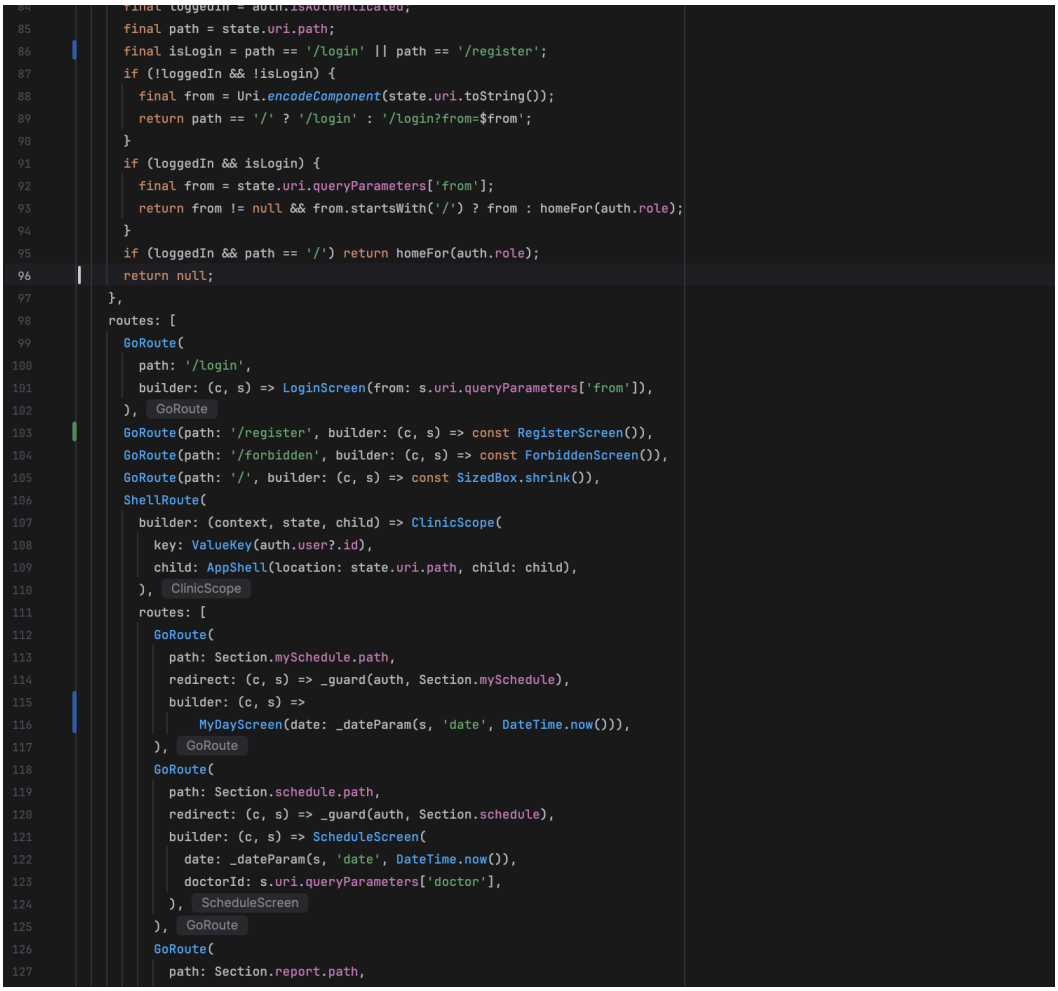
Для каждой сущности создаются четыре маршрута, маршруты создания и изменения дополнительно проверяют право create или edit. Недоступный раздел приводит на страницу 403.



```
  return null;
}

List<GoRoute> _entityRoutes({
  required AuthNotifier auth,
  required Section section,
  required Widget Function(FilterQuery query) list,
  required Widget Function(GoRouterState state, String? id) form,
  required Widget Function(String id)? detail,
  required QuerySpec spec,
}) {
  final base = section.path;
  return [
    GoRoute(
      path: base,
      redirect: (c, s) => _guard(auth, section),
      builder: (c, s) =>
          list(FilterQuery.fromParams(spec, s.uri.queryParameters)),
    ),
    GoRoute(
      path: '$base/new',
      redirect: (c, s) => _guard(
        auth,
        section,
        extra: (a) => permissionsFor(a.role, section).create,
      ),
      builder: (c, s) => form(s, null),
    ),
    if (detail != null)
      GoRoute(
        path: '$base/:id',
        redirect: (c, s) => _guard(auth, section),
        builder: (c, s) => detail(s.pathParameters['id']!),
      ),
    GoRoute(
      path: '$base/:id/edit',
      redirect: (c, s) => _guard(
        auth,
        section,
        extra: (a) => permissionsFor(a.role, section).edit,
      ),
      builder: (c, s) => form(s, s.pathParameters['id']),
    ),
  ];
```

Рисунок 6 – Функция _guard и маршруты сущностей (lib/core/router.dart)

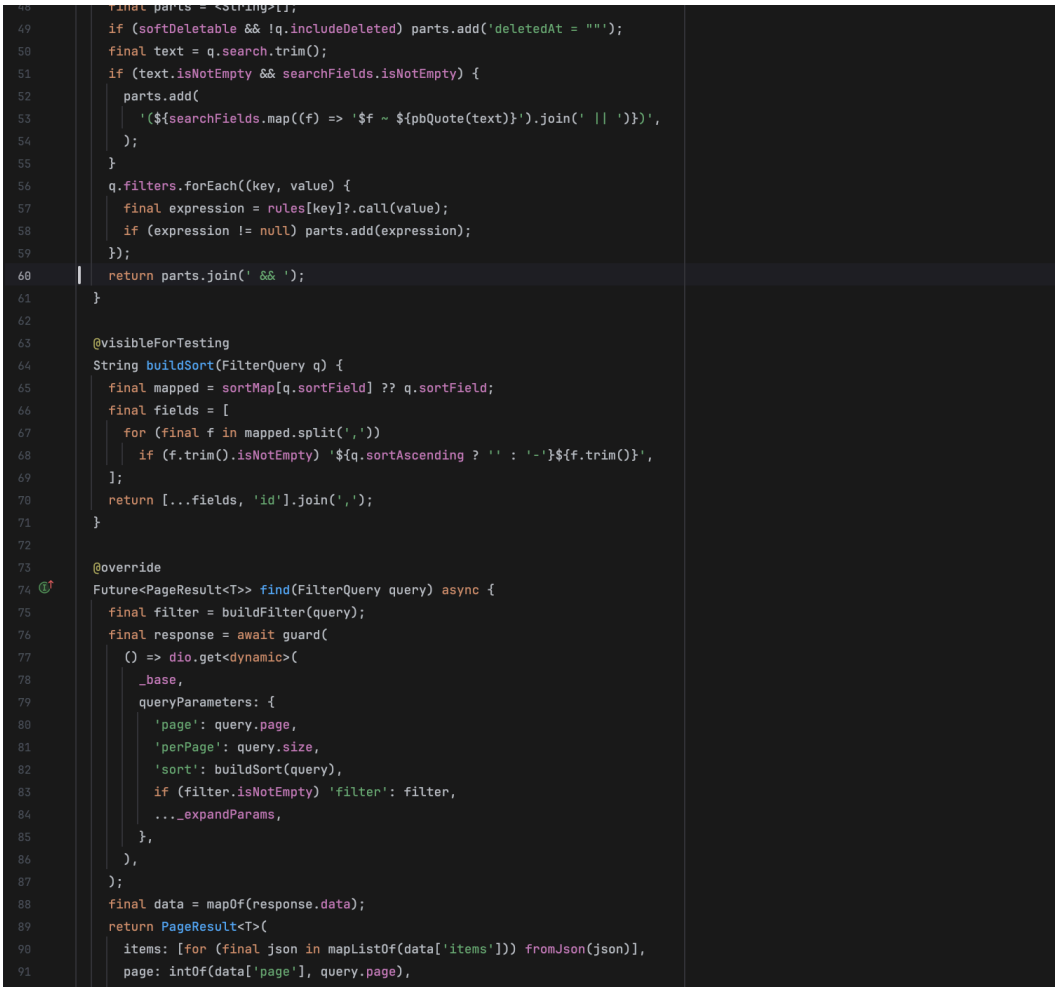
Глобальный redirect отправляет неавторизованного пользователя на /login с параметром from и возвращает по исходной ссылке после входа, а авторизованного с адреса входа — на главную страницу его роли.



```
    final loggedIn = auth.isAuthenticated;
    final path = state.uri.path;
    final isLogin = path == '/login' || path == '/register';
    if (!loggedIn && !isLogin) {
      final from = Uri.encodeComponent(state.uri.toString());
      return path == '/' ? '/login' : '/login?from=$from';
    }
    if (loggedIn && isLogin) {
      final from = state.uri.queryParameters['from'];
      return from != null && from.startsWith('/') ? from : homeFor(auth.role);
    }
    if (loggedIn && path == '/') return homeFor(auth.role);
    return null;
  },
  routes: [
    GoRoute(
      path: '/login',
      builder: (c, s) => LoginScreen(from: s.uri.queryParameters['from']),
    ), GoRoute
    GoRoute(path: '/register', builder: (c, s) => const RegisterScreen()),
    GoRoute(path: '/forbidden', builder: (c, s) => const ForbiddenScreen()),
    GoRoute(path: '/', builder: (c, s) => const SizedBox.shrink()),
    ShellRoute(
      builder: (context, state, child) => ClinicScope(
        key: ValueKey(auth.user?.id),
        child: AppShell(location: state.uri.path, child: child),
      ), ClinicScope
      routes: [
        GoRoute(
          path: Section.mySchedule.path,
          redirect: (c, s) => _guard(auth, Section.mySchedule),
          builder: (c, s) =>
              MyDayScreen(date: _dateParam(s, 'date', DateTime.now())),
        ), GoRoute
        GoRoute(
          path: Section.schedule.path,
          redirect: (c, s) => _guard(auth, Section.schedule),
          builder: (c, s) => ScheduleScreen(
            date: _dateParam(s, 'date', DateTime.now()),
            doctorId: s.uri.queryParameters['doctor'],
          ), ScheduleScreen
        ), GoRoute
        GoRoute(
          path: Section.report.path,
```

Рисунок 7 – Глобальный redirect маршрутизатора (lib/core/router.dart)

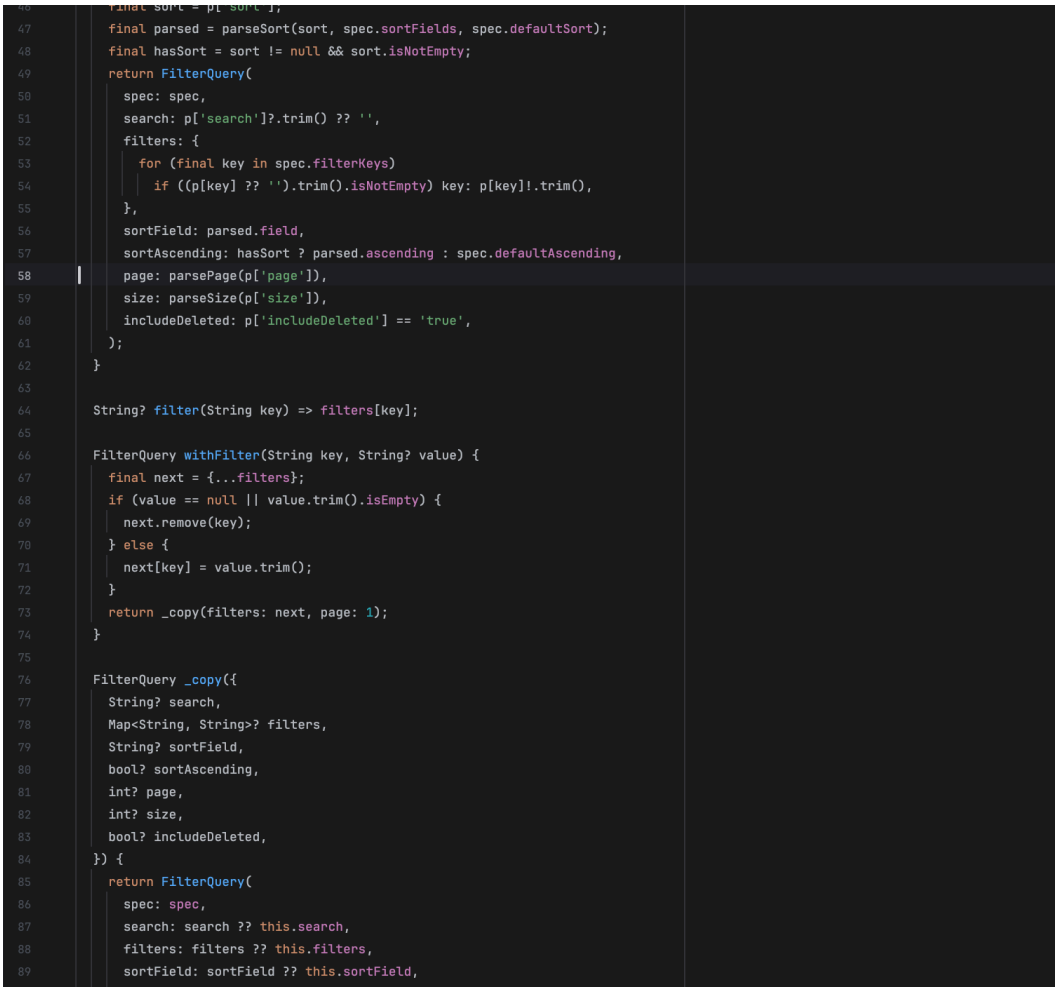
Базовый репозиторий PbRepository строит выражение filter из поиска и правил фильтров сущности, экранирует кавычки, добавляет условие неудалённых записей и запасной ключ сортировки. Выборку, сортировку и постраничный вывод выполняет сервер, клиент получает страницу и общее число записей.



```
    final parts = <String>[];
    if (softDeletable && !q.includeDeleted) parts.add('deletedAt = ""');
    final text = q.search.trim();
    if (text.isNotEmpty && searchFields.isNotEmpty) {
      parts.add(
        '(${searchFields.map((f) => '$f ~ ${pbQuote(text)}').join(' || ')})',
      );
    }
    q.filters.forEach((key, value) {
      final expression = rules[key]?.call(value);
      if (expression != null) parts.add(expression);
    });
    return parts.join(' && ');
  }

  @visibleForTesting
  String buildSort(FilterQuery q) {
    final mapped = sortMap[q.sortField] ?? q.sortField;
    final fields = [
      for (final f in mapped.split(','))
        if (f.trim().isNotEmpty) '${q.sortAscending ? '' : '-'}${f.trim()}',
    ];
    return [...fields, 'id'].join(',');
  }

  @override
  Future<PageResult<T>> find(FilterQuery query) async {
    final filter = buildFilter(query);
    final response = await guard(
      () => dio.get<dynamic>(
        _base,
        queryParameters: {
          'page': query.page,
          'perPage': query.size,
          'sort': buildSort(query),
          if (filter.isNotEmpty) 'filter': filter,
          ..._expandParams,
        },
      ),
    );
    final data = mapOf(response.data);
    return PageResult<T>(
      items: [for (final json in mapListOf(data['items'])) fromJson(json)],
      page: intOf(data['page'], query.page),
```

Рисунок 8 – PbRepository: построение filter и sort, запрос страницы списка

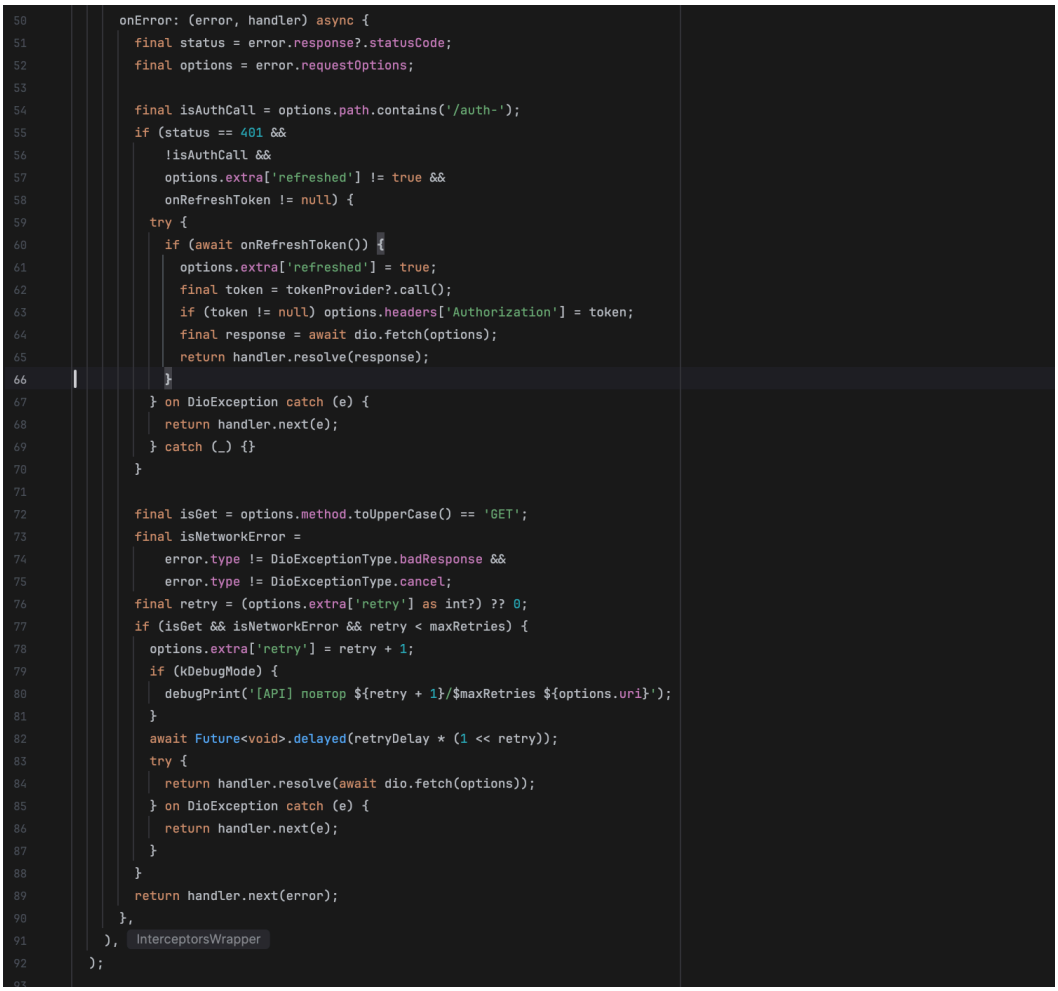
Класс FilterQuery разбирает параметры адреса, отбрасывает неизвестные поля сортировки и неверные значения страницы и собирает адрес обратно; именно он делает состояние списка ссылкой.



```
    final parsed = parseSort(sort, spec.sortFields, spec.defaultSort);
    final hasSort = sort != null && sort.isNotEmpty;
    return FilterQuery(
      spec: spec,
      search: p['search']?.trim() ?? '',
      filters: {
        for (final key in spec.filterKeys)
          if ((p[key] ?? '').trim().isNotEmpty) key: p[key]!.trim(),
      },
      sortField: parsed.field,
      sortAscending: hasSort ? parsed.ascending : spec.defaultAscending,
      page: parsePage(p['page']),
      size: parseSize(p['size']),
      includeDeleted: p['includeDeleted'] == 'true',
    );
  }

  String? filter(String key) => filters[key];

  FilterQuery withFilter(String key, String? value) {
    final next = {...filters};
    if (value == null || value.trim().isEmpty) {
      next.remove(key);
    } else {
      next[key] = value.trim();
    }
    return _copy(filters: next, page: 1);
  }

  FilterQuery _copy({
    String? search,
    Map<String, String>? filters,
    String? sortField,
    bool? sortAscending,
    int? page,
    int? size,
    bool? includeDeleted,
  }) {
    return FilterQuery(
      spec: spec,
      search: search ?? this.search,
      filters: filters ?? this.filters,
      sortField: sortField ?? this.sortField,
```

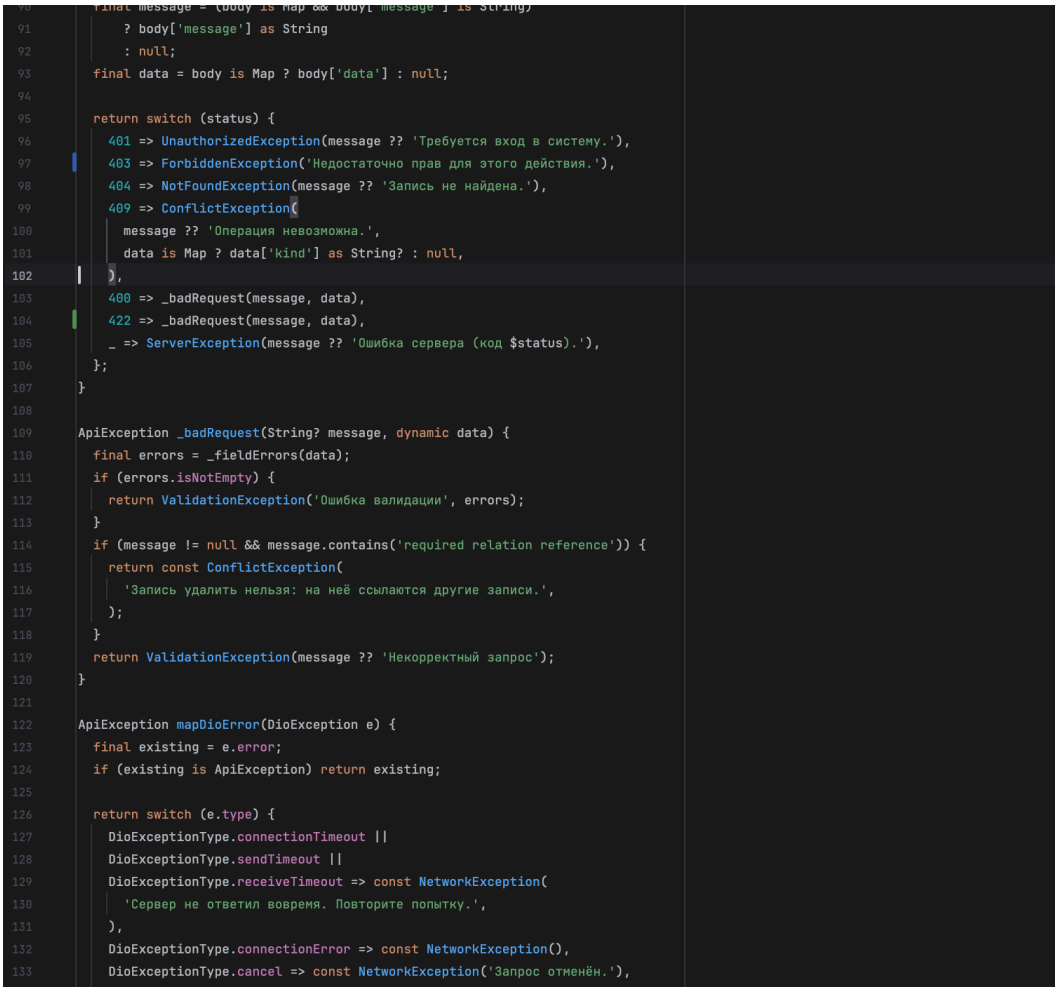
Рисунок 9 – FilterQuery: разбор и сборка параметров адреса (lib/models/filter_query.dart)

Перехватчик Dio подставляет токен, при ответе 401 один раз обновляет его и повторяет запрос, а запросы чтения при сетевых ошибках повторяет с нарастающей задержкой. Ответы с кодами 400 и выше превращаются в типизированные исключения.



```
      onError: (error, handler) async {
        final status = error.response?.statusCode;
        final options = error.requestOptions;

        final isAuthCall = options.path.contains('/auth-');
        if (status == 401 &&
            !isAuthCall &&
            options.extra['refreshed'] != true &&
            onRefreshToken != null) {
          try {
            if (await onRefreshToken()) {
              options.extra['refreshed'] = true;
              final token = tokenProvider?.call();
              if (token != null) options.headers['Authorization'] = token;
              final response = await dio.fetch(options);
              return handler.resolve(response);
            }
          } on DioException catch (e) {
            return handler.next(e);
          } catch (_) {}
        }

        final isGet = options.method.toUpperCase() == 'GET';
        final isNetworkError =
            error.type != DioExceptionType.badResponse &&
            error.type != DioExceptionType.cancel;
        final retry = (options.extra['retry'] as int?) ?? 0;
        if (isGet && isNetworkError && retry < maxRetries) {
          options.extra['retry'] = retry + 1;
          if (kDebugMode) {
            debugPrint('[API] повтор ${retry + 1}/$maxRetries ${options.uri}');
          }
          await Future<void>.delayed(retryDelay * (1 << retry));
          try {
            return handler.resolve(await dio.fetch(options));
          } on DioException catch (e) {
            return handler.next(e);
          }
        }
        return handler.next(error);
      },
    ), InterceptorsWrapper
  );
```

Рисунок 10 – Перехватчик Dio: токен, обновление при 401, повторы (lib/core/api_client.dart)



```
    final message = (body is Map && body['message'] is String)
        ? body['message'] as String
        : null;
    final data = body is Map ? body['data'] : null;

    return switch (status) {
      401 => UnauthorizedException(message ?? 'Требуется вход в систему.'),
      403 => ForbiddenException('Недостаточно прав для этого действия.'),
      404 => NotFoundException(message ?? 'Запись не найдена.'),
      409 => ConflictException(
        message ?? 'Операция невозможна.',
        data is Map ? data['kind'] as String? : null,
      ),
      400 => _badRequest(message, data),
      422 => _badRequest(message, data),
      _ => ServerException(message ?? 'Ошибка сервера (код $status).'),
    };
  }

  ApiException _badRequest(String? message, dynamic data) {
    final errors = _fieldErrors(data);
    if (errors.isNotEmpty) {
      return ValidationException('Ошибка валидации', errors);
    }
    if (message != null && message.contains('required relation reference')) {
      return const ConflictException(
        'Запись удалить нельзя: на неё ссылаются другие записи.',
      );
    }
    return ValidationException(message ?? 'Некорректный запрос');
  }

  ApiException mapDioError(DioException e) {
    final existing = e.error;
    if (existing is ApiException) return existing;

    return switch (e.type) {
      DioExceptionType.connectionTimeout ||
      DioExceptionType.sendTimeout ||
      DioExceptionType.receiveTimeout => const NetworkException(
        'Сервер не ответил вовремя. Повторите попытку.',
      ),
      DioExceptionType.connectionError => const NetworkException(),
      DioExceptionType.cancel => const NetworkException('Запрос отменён.'),
```

Рисунок 11 – Преобразование ответов сервера в исключения клиента (mapHttpError)

AuthNotifier при запуске читает сессию из хранилища, проверяет срок токена и запрашивает у сервера актуальные данные пользователя, поэтому подменённая роль исправляется. Повреждённая запись сессии игнорируется.

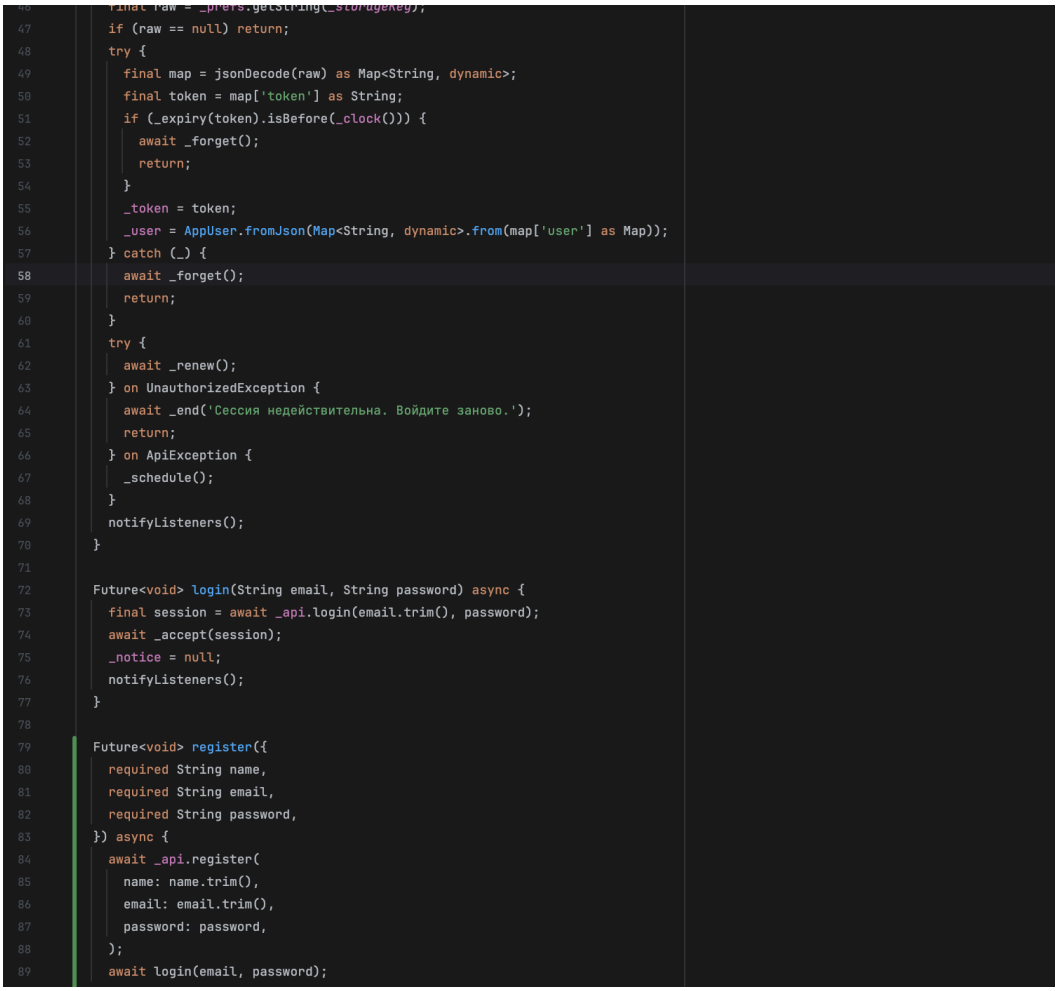
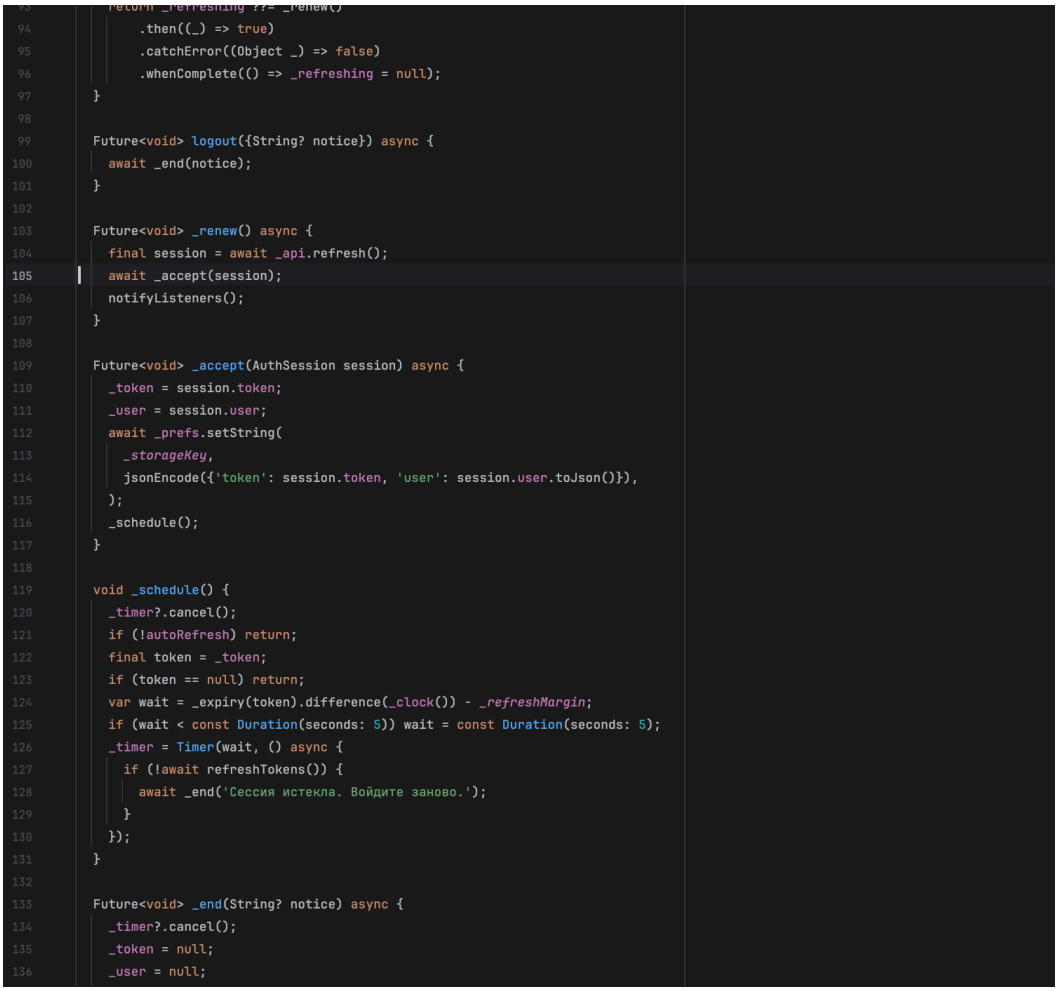


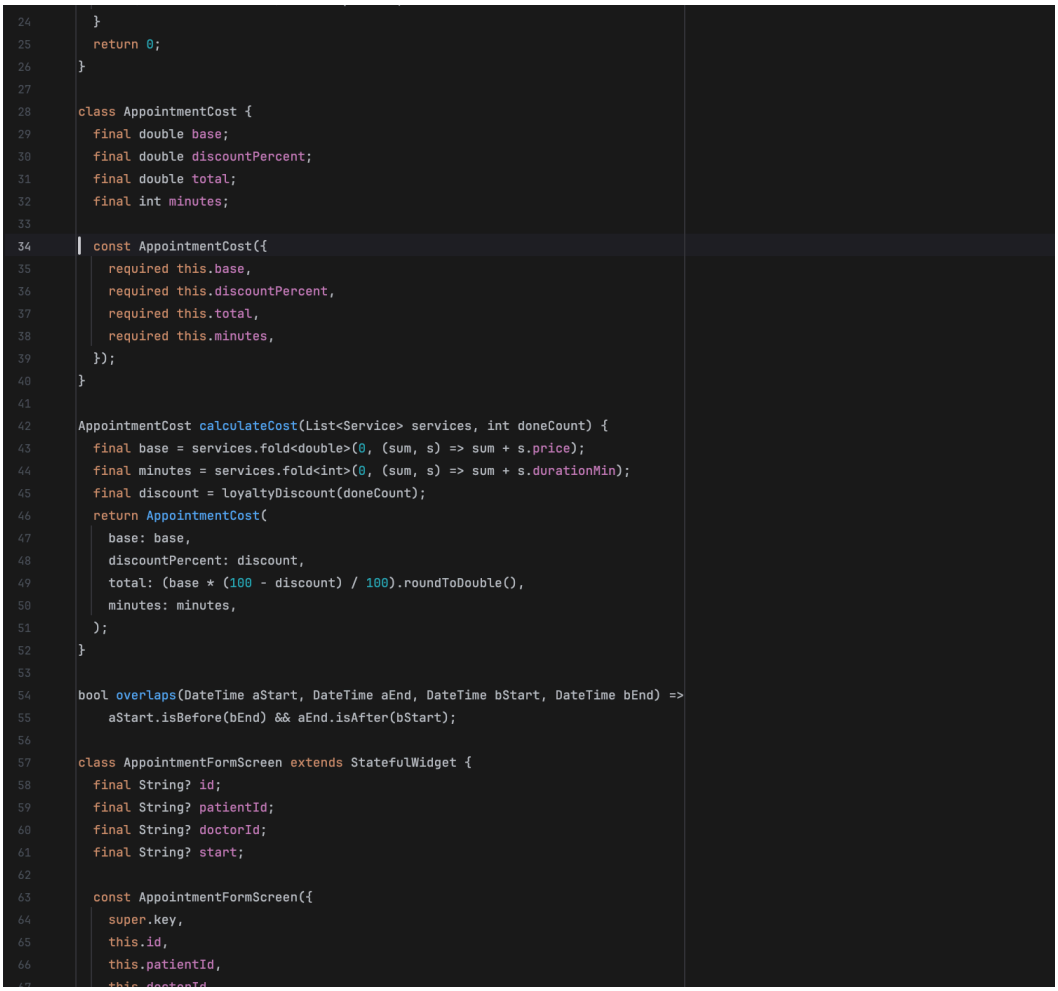
Рисунок 12 – AuthNotifier: восстановление сессии после перезагрузки страницы



```
      .then((_) => true)
      .catchError((Object _) => false)
      .whenComplete(() => _refreshing = null);
  }

  Future<void> logout({String? notice}) async {
    await _end(notice);
  }

  Future<void> _renew() async {
    final session = await _api.refresh();
    await _accept(session);
    notifyListeners();
  }

  Future<void> _accept(AuthSession session) async {
    _token = session.token;
    _user = session.user;
    await _prefs.setString(
      _storageKey,
      jsonEncode({'token': session.token, 'user': session.user.toJson()}),
    );
    _schedule();
  }

  void _schedule() {
    _timer?.cancel();
    if (!autoRefresh) return;
    final token = _token;
    if (token == null) return;
    var wait = _expiry(token).difference(_clock()) - _refreshMargin;
    if (wait < const Duration(seconds: 5)) wait = const Duration(seconds: 5);
    _timer = Timer(wait, () async {
      if (!await refreshTokens()) {
        await _end('Сессия истекла. Войдите заново.');
      }
    });
  }

  Future<void> _end(String? notice) async {
    _timer?.cancel();
    _token = null;
    _user = null;
```

Рисунок 13 – AuthNotifier: единый запрос обновления токена и автоматическое продление

Функции loyaltyDiscount, calculateCost и overlaps дают пользователю расчёт до отправки формы. Окончательный расчёт выполняет сервер, поэтому клиентский результат — подсказка, а не источник истины.



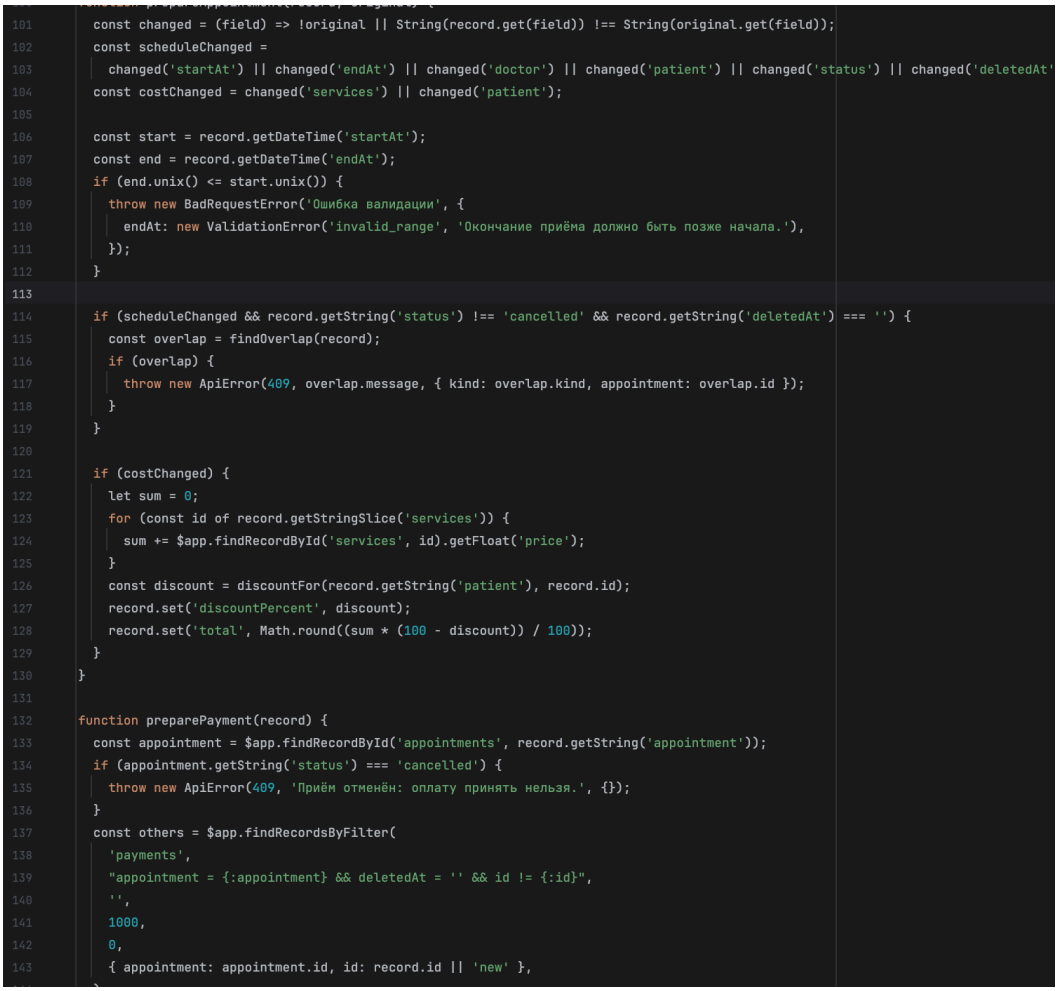
```
  }
  return 0;
}

class AppointmentCost {
  final double base;
  final double discountPercent;
  final double total;
  final int minutes;

  const AppointmentCost({
    required this.base,
    required this.discountPercent,
    required this.total,
    required this.minutes,
  });
}

AppointmentCost calculateCost(List<Service> services, int doneCount) {
  final base = services.fold<double>(0, (sum, s) => sum + s.price);
  final minutes = services.fold<int>(0, (sum, s) => sum + s.durationMin);
  final discount = loyaltyDiscount(doneCount);
  return AppointmentCost(
    base: base,
    discountPercent: discount,
    total: (base * (100 - discount) / 100).roundToDouble(),
    minutes: minutes,
  );
}

bool overlaps(DateTime aStart, DateTime aEnd, DateTime bStart, DateTime bEnd) =>
    aStart.isBefore(bEnd) && aEnd.isAfter(bStart);

class AppointmentFormScreen extends StatefulWidget {
  final String? id;
  final String? patientId;
  final String? doctorId;
  final String? start;

  const AppointmentFormScreen({
    super.key,
    this.id,
    this.patientId,
```

Рисунок 14 – Расчёт скидки, длительности и стоимости приёма (appointment_form_screen.dart)

Серверный хук ищет пересечения приёмов врача, пациента и кабинета, пересчитывает скидку постоянного пациента и итоговую стоимость и отвечает 409 или 400 при нарушениях. Клиентская проверка не может заменить эту серверную.

```
  const start = record.getDateTime('startAt').string();
  const end = record.getDateTime('endAt').string();
  const doctorId = record.getString('doctor');
  const patientId = record.getString('patient');
  const doctor = $app.findRecordById('doctors', doctorId);
  const cabinetId = doctor.getString('cabinet');

  const base =
    "status != 'cancelled' && deletedAt = '' && id != {:id} && startAt < {:end} && endAt > {:start}";
  const params = { id: record.id || 'new', start, end, doctor: doctorId, patient: patientId, cabinet: cabinetId };

  const checks = [
    ['doctor', 'Врач занят в это время', `${base} && doctor = {:doctor}`],
    ['patient', 'Пациент уже записан на это время', `${base} && patient = {:patient}`],
  ];
  if (cabinetId) {
    checks.push(['cabinet', 'Кабинет занят в это время', `${base} && doctor.cabinet = {:cabinet}`]);
  }

  for (const [kind, text, filter] of checks) {
    const found = $app.findRecordsByFilter('appointments', filter, 'startAt', 1, 0, params);
    if (found.length > 0) {
      return { kind, message: `${text}: существует приём ${timeLabel(found[0])}.`, id: found[0].id };
    }
  }
  return null;
}

function discountFor(patientId, excludeId) {
  const done = $app.findRecordsByFilter(
    'appointments',
    "patient = {:patient} && status = 'done' && deletedAt = '' && id != {:id}",
    '',
    1000,
    0,
    { patient: patientId, id: excludeId || 'new' },
  );
  if (done.length >= 10) return 15;
  if (done.length >= 5) return 10;
  return 0;
}

function prepareAppointment(record, original) {
```

Рисунок 15 – Хук: поиск пересечения приёмов (pocketbase/pb_hooks/lib.js)



```
  const changed = (field) => !original || String(record.get(field)) !== String(original.get(field));
  const scheduleChanged =
    changed('startAt') || changed('endAt') || changed('doctor') || changed('patient') || changed('status') || changed('deletedAt'
  const costChanged = changed('services') || changed('patient');

  const start = record.getDateTime('startAt');
  const end = record.getDateTime('endAt');
  if (end.unix() <= start.unix()) {
    throw new BadRequestError('Ошибка валидации', {
      endAt: new ValidationError('invalid_range', 'Окончание приёма должно быть позже начала.'),
    });
  }

  if (scheduleChanged && record.getString('status') !== 'cancelled' && record.getString('deletedAt') === '') {
    const overlap = findOverlap(record);
    if (overlap) {
      throw new ApiError(409, overlap.message, { kind: overlap.kind, appointment: overlap.id });
    }
  }

  if (costChanged) {
    let sum = 0;
    for (const id of record.getStringSlice('services')) {
      sum += $app.findRecordById('services', id).getFloat('price');
    }
    const discount = discountFor(record.getString('patient'), record.id);
    record.set('discountPercent', discount);
    record.set('total', Math.round((sum * (100 - discount)) / 100));
  }
}

function preparePayment(record) {
  const appointment = $app.findRecordById('appointments', record.getString('appointment'));
  if (appointment.getString('status') === 'cancelled') {
    throw new ApiError(409, 'Приём отменён: оплату принять нельзя.', {});
  }
  const others = $app.findRecordsByFilter(
    'payments',
    "appointment = {:appointment} && deletedAt = '' && id != {:id}",
    '',
    1000,
    0,
    { appointment: appointment.id, id: record.id || 'new' },
```

Рисунок 16 – Хук: скидка постоянного пациента и подготовка приёма к сохранению

9. Демонстрируем работу программы. Снимки получены на опубликованной сборке с эмулятором API и на сборке с сервером PocketBase. Для проверки подготовлены учётные записи: администратор admin@clinic.test / Admin123!, регистратор registrar@clinic.test / Registrar123!, врач doctor@clinic.test / Doctor123!. Открываем страницу входа: в демо-режиме под формой есть кнопки заполнения учётных записей.

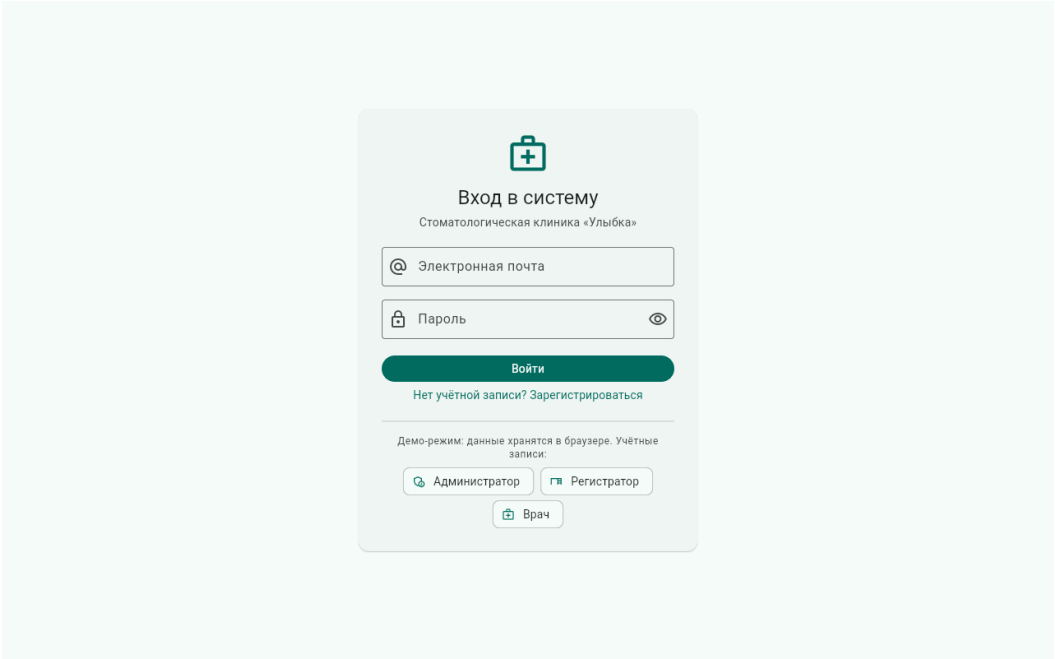


Рисунок 17 – Страница входа

Вводим неверный пароль. Сервер отвечает ошибкой, сообщение выводится над формой.

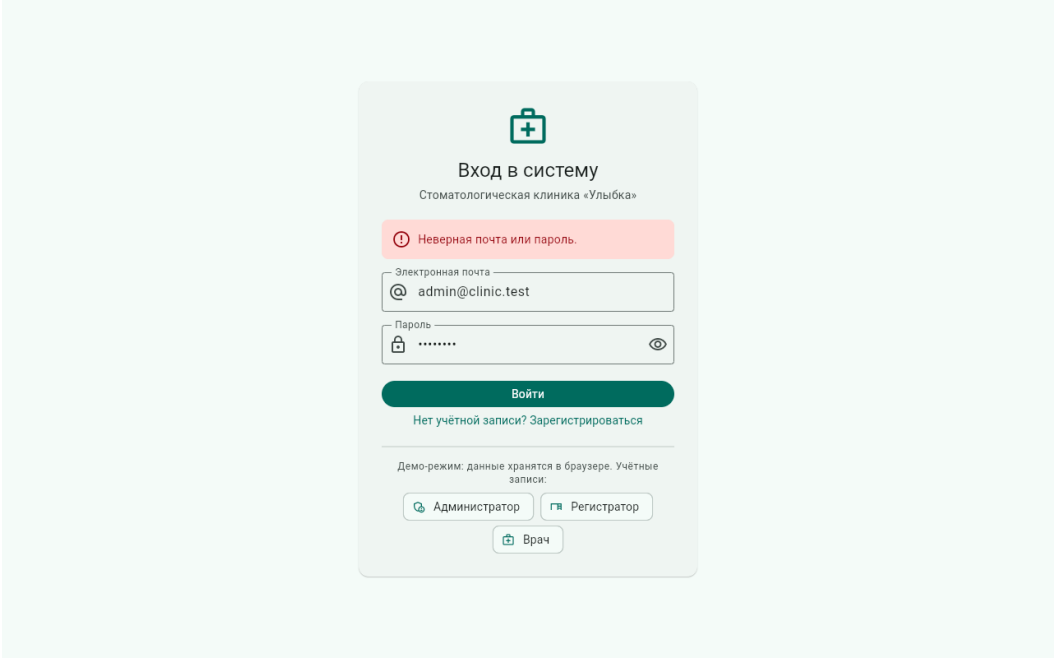


Рисунок 18 – Неверный пароль

Открываем регистрацию. Новая учётная запись получает роль регистратора, пароль проверяется на длину и совпадение.

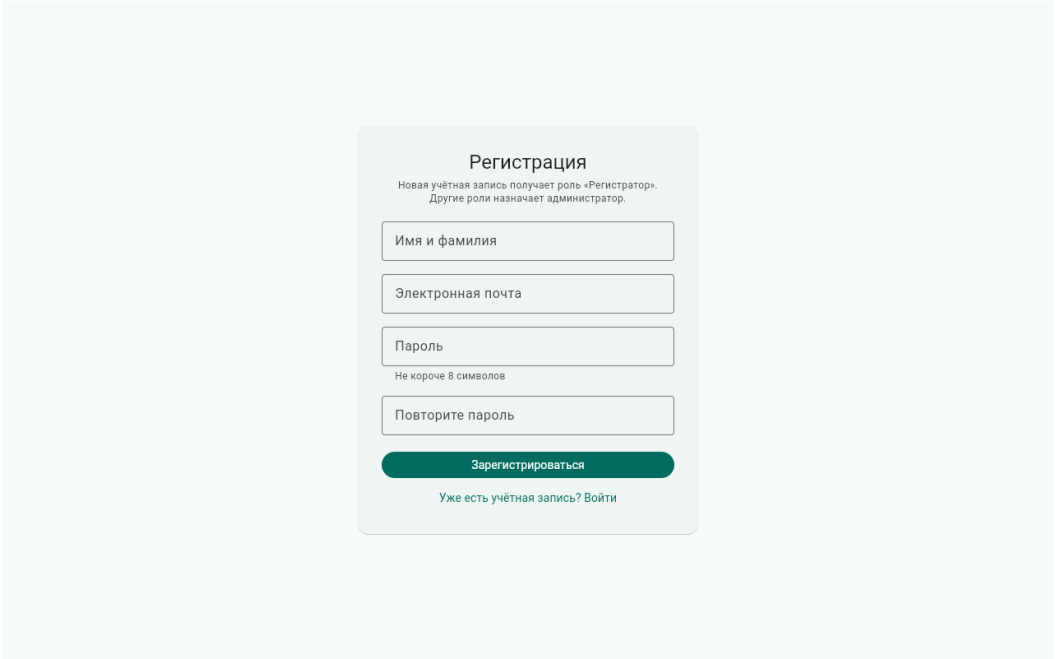


Рисунок 19 – Регистрация

Входим администратором. Раздел «Отчёт» показывает число проведённых приёмов, начисленную и полученную выручку, средний чек, долю отмен и диаграммы по врачам, категориям услуг, способам оплаты и дням.

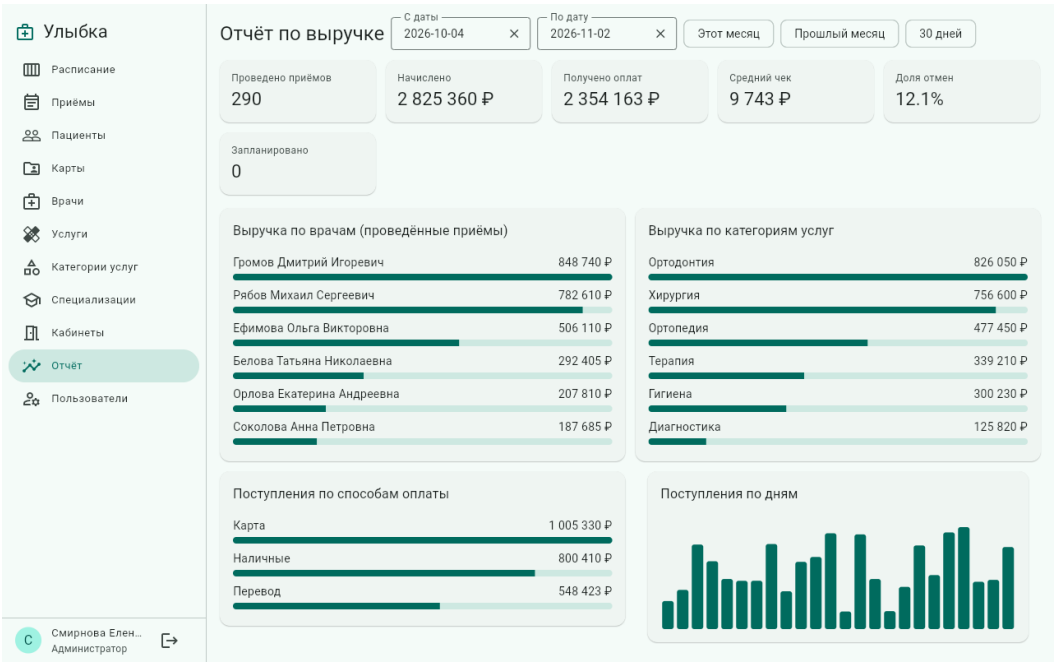


Рисунок 20 – Отчёт по выручке

Открываем список услуг с условиями отбора: категория «Терапия», цена от 2000 рублей, сортировка по цене. Поиск, фильтры, сортировка и страницы выполняются на сервере, а условия попадают в адрес /services?category=…&priceMin=2000&sort=price,desc.

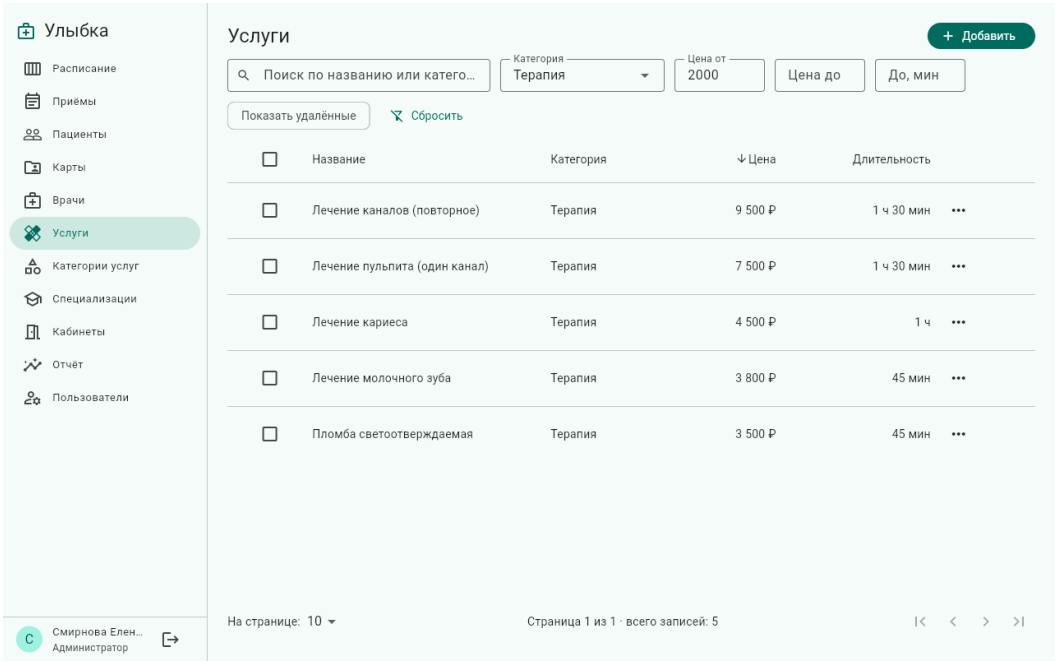


Рисунок 21 – Список услуг с фильтрами и сортировкой

Открываем список пациентов с фильтрами по возрасту и наличию карты.

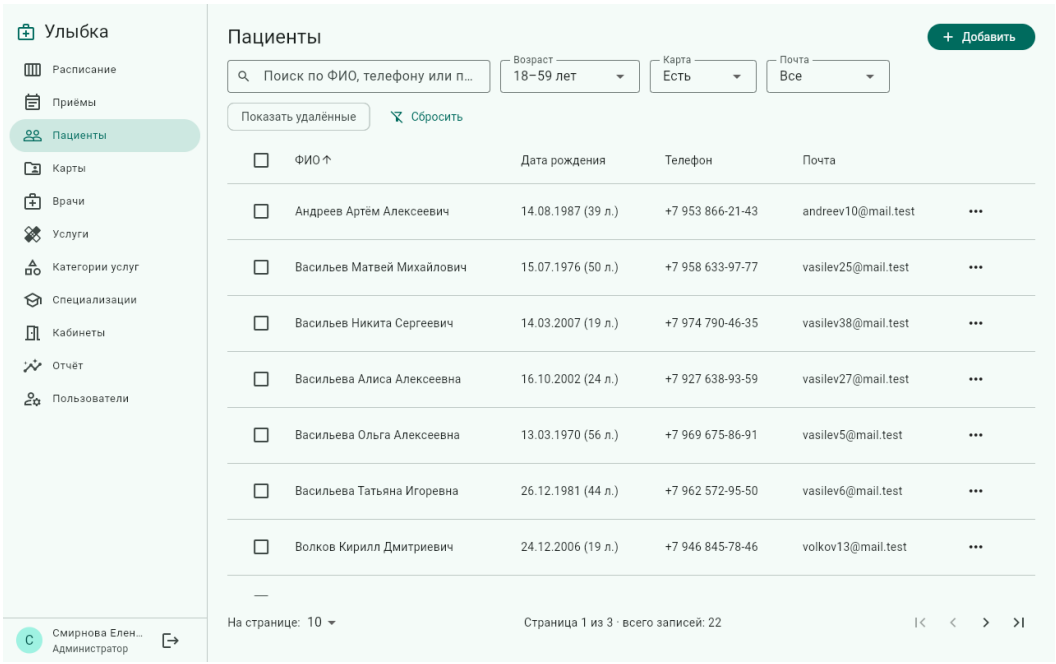


Рисунок 22 – Список пациентов с фильтрами

Раздел «Пользователи» доступен только администратору. Отправляем пустую форму нового пользователя: ошибки обязательных полей показываются у полей.

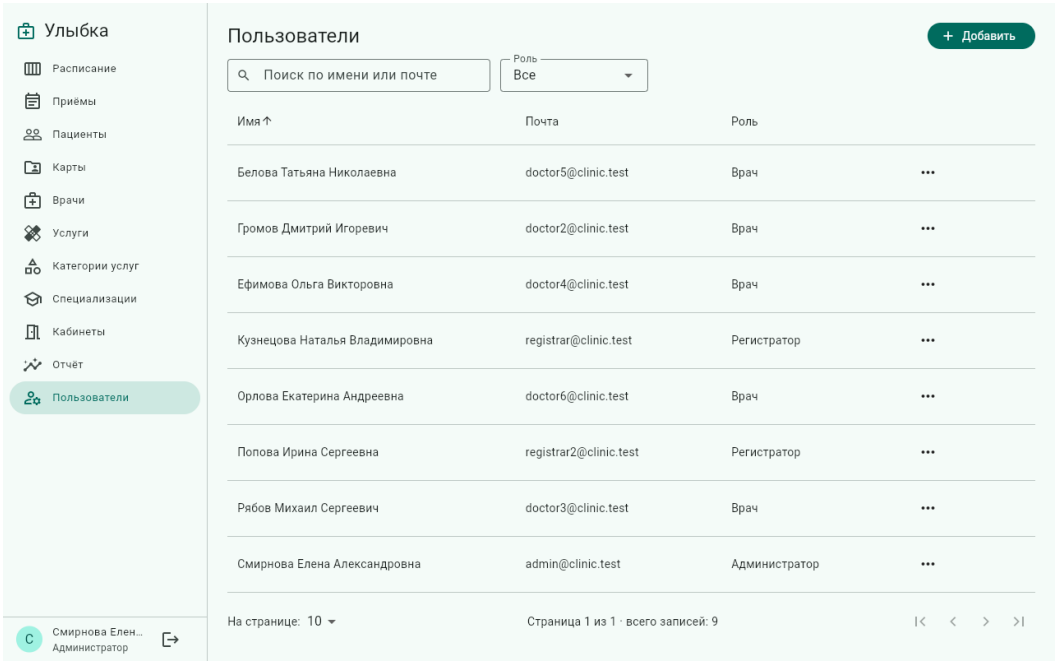


Рисунок 23 – Раздел «Пользователи»

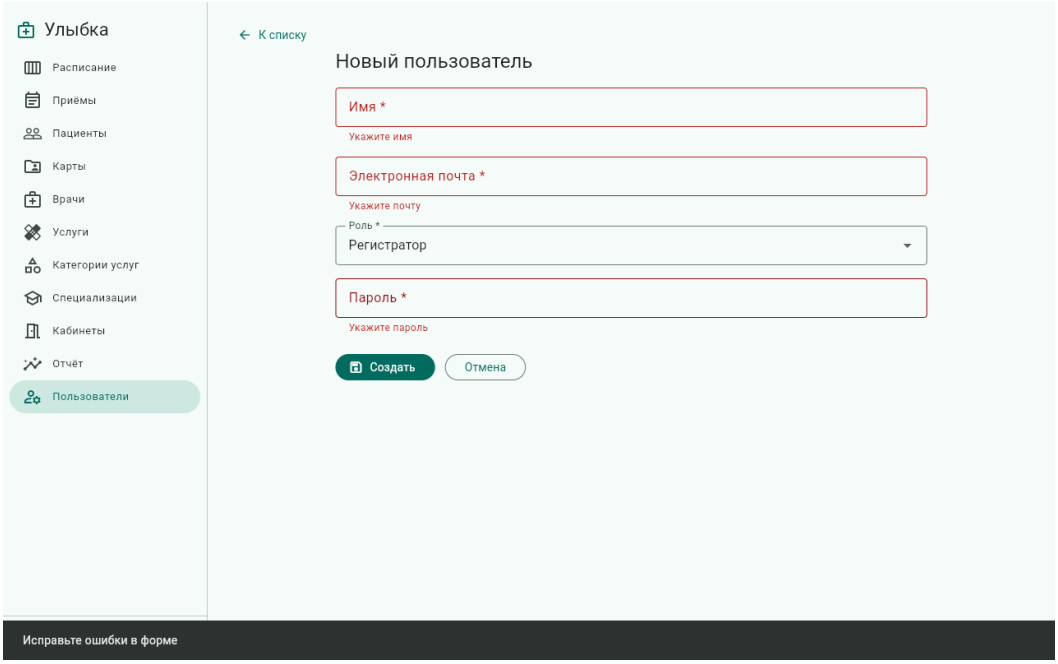


Рисунок 24 – Ошибки валидации в форме пользователя

Пробуем удалить специализацию, у которой есть врачи. Сначала подтверждаем логическое удаление, затем сервер отвечает конфликтом 409 и перечисляет зависимости.

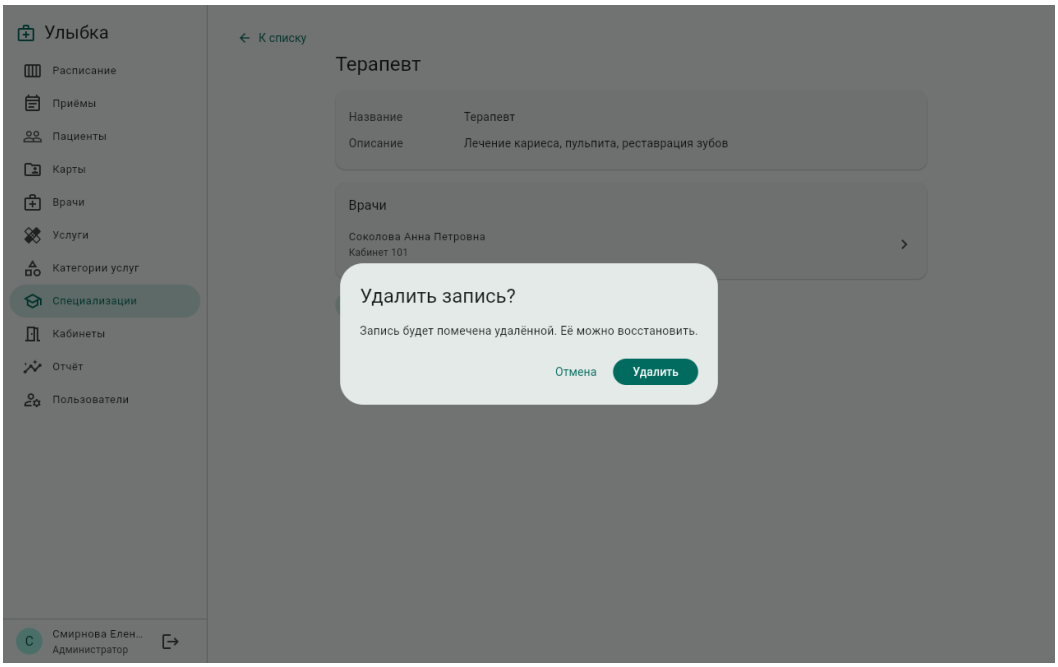


Рисунок 25 – Подтверждение логического удаления

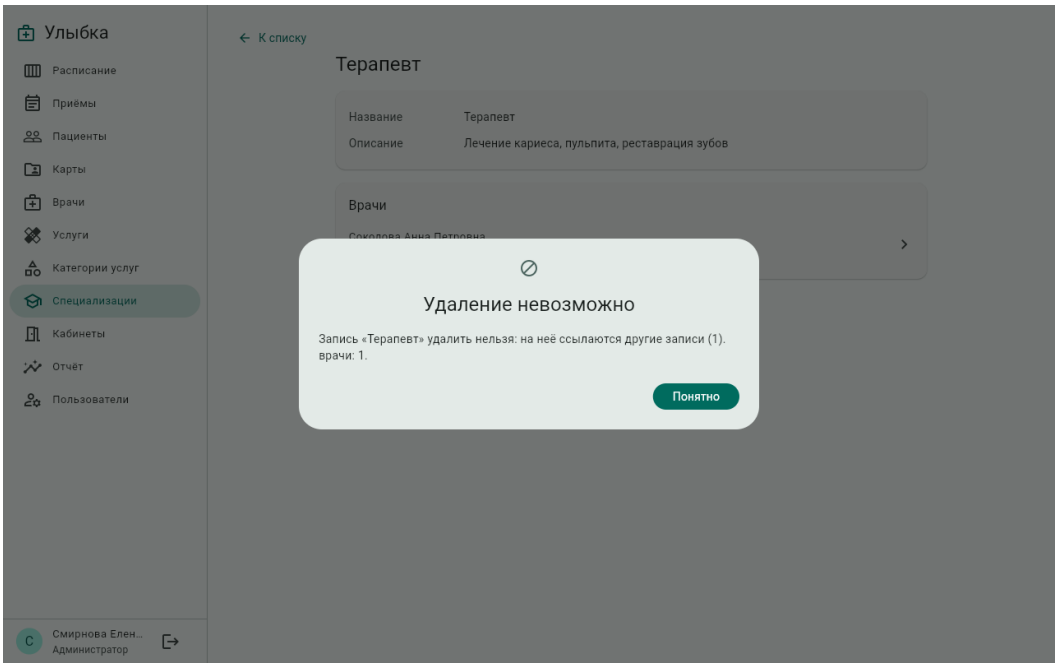


Рисунок 26 – Конфликт 409: запись с зависимостями удалить нельзя

Включаем показ удалённых записей (includeDeleted=true). Удалённая запись помечена, для неё доступны восстановление и физическое удаление, а выделенные записи удаляются сразу несколькими.

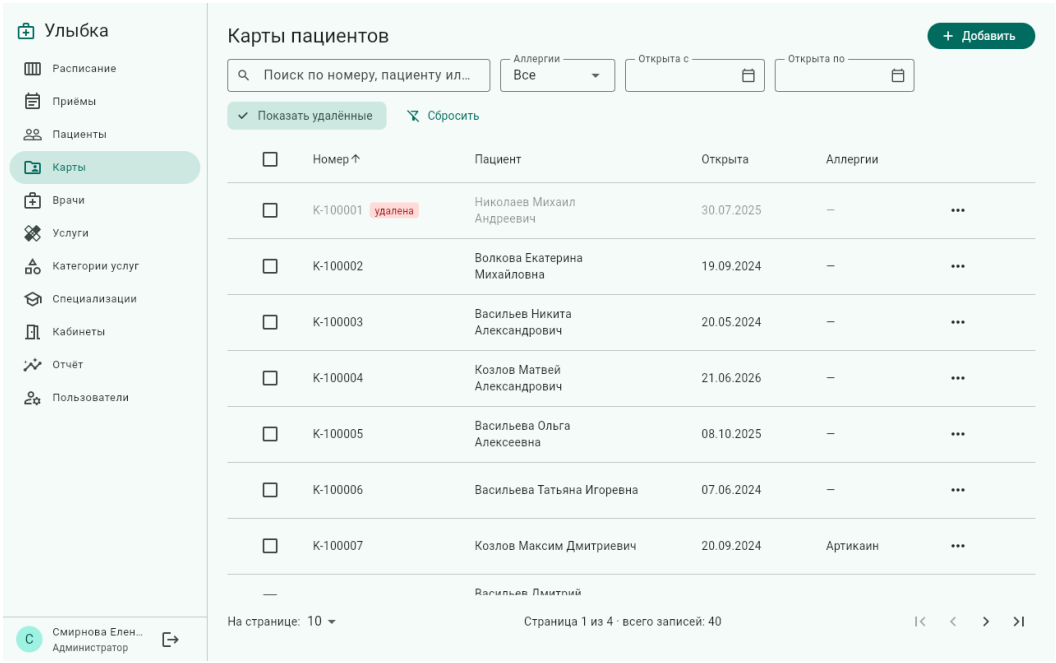


Рисунок 27 – Показ удалённых записей

Входим регистратором. Расписание показывает занятость врачей по часам, щелчок по свободному времени открывает форму нового приёма с подставленными врачом и временем.

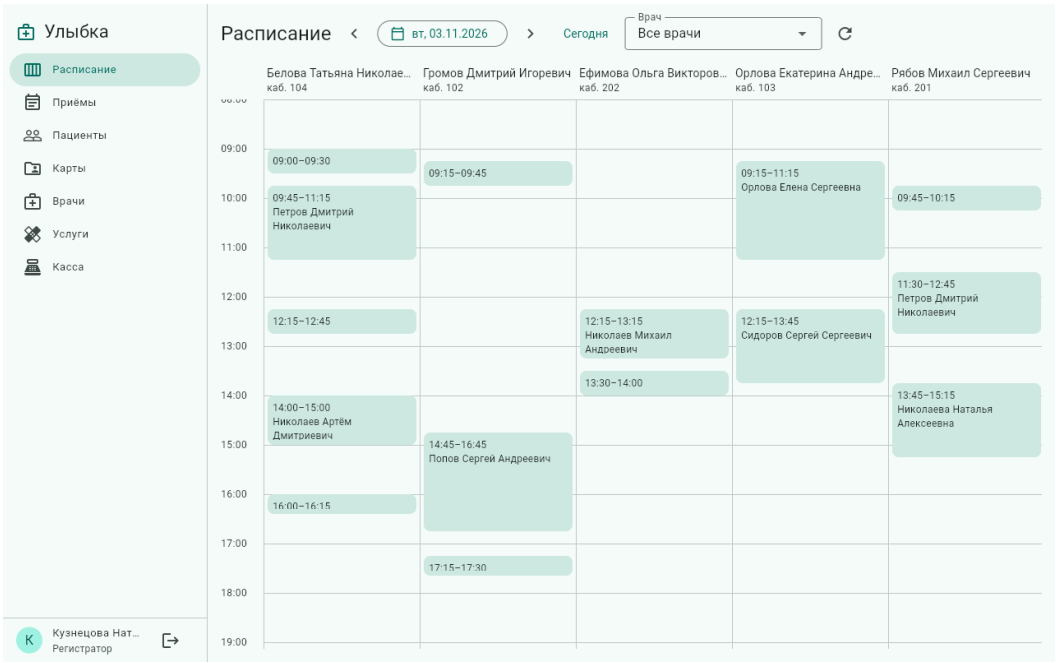


Рисунок 28 – Расписание на день

Открываем список приёмов с фильтрами по статусу, врачу, категории услуг и датам.

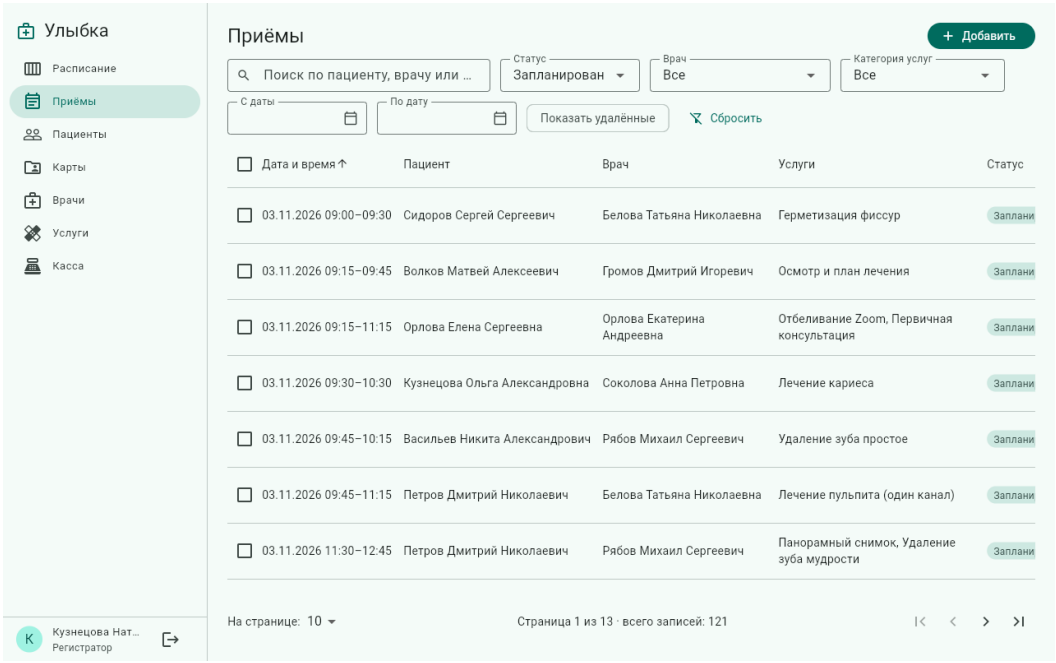


Рисунок 29 – Список приёмов

Открываем форму приёма: пациент выбирается поиском на сервере, врач — из списка, услуги — множественным выбором, под формой считаются длительность и стоимость со скидкой постоянного пациента.

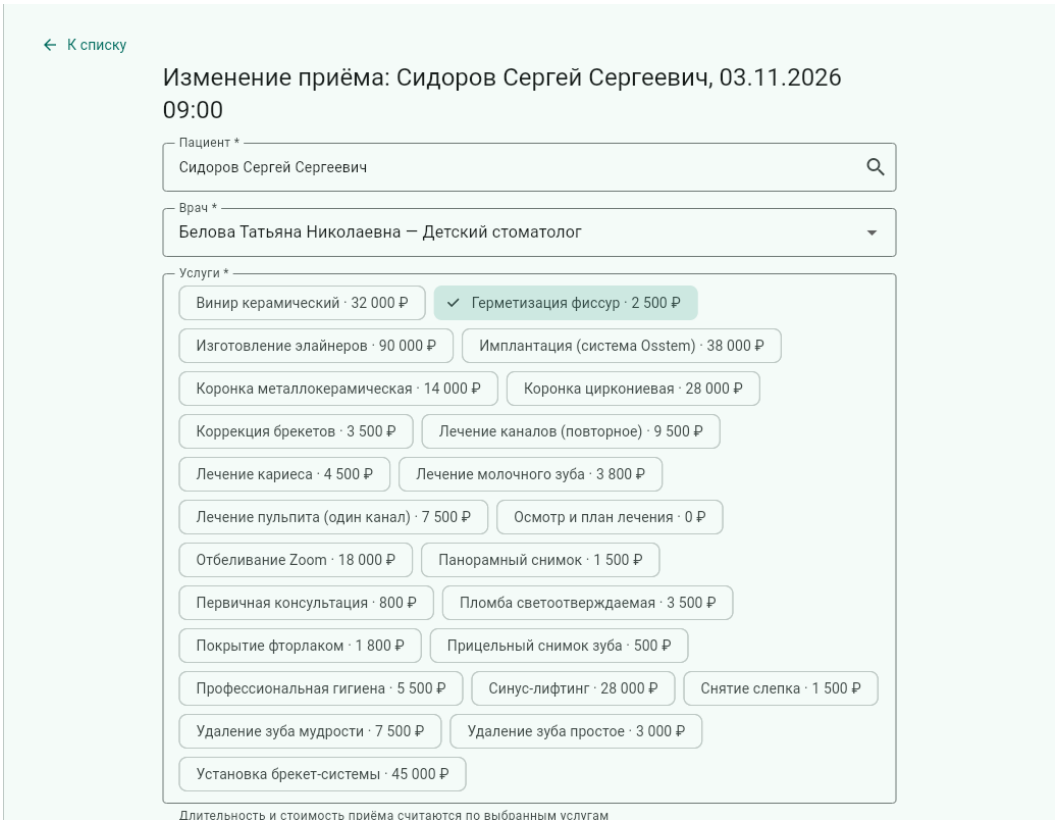


Рисунок 30 – Форма приёма

Выбираем время, пересекающееся с другим приёмом врача. Расчёт под формой показывает занятость врача и предупреждение.

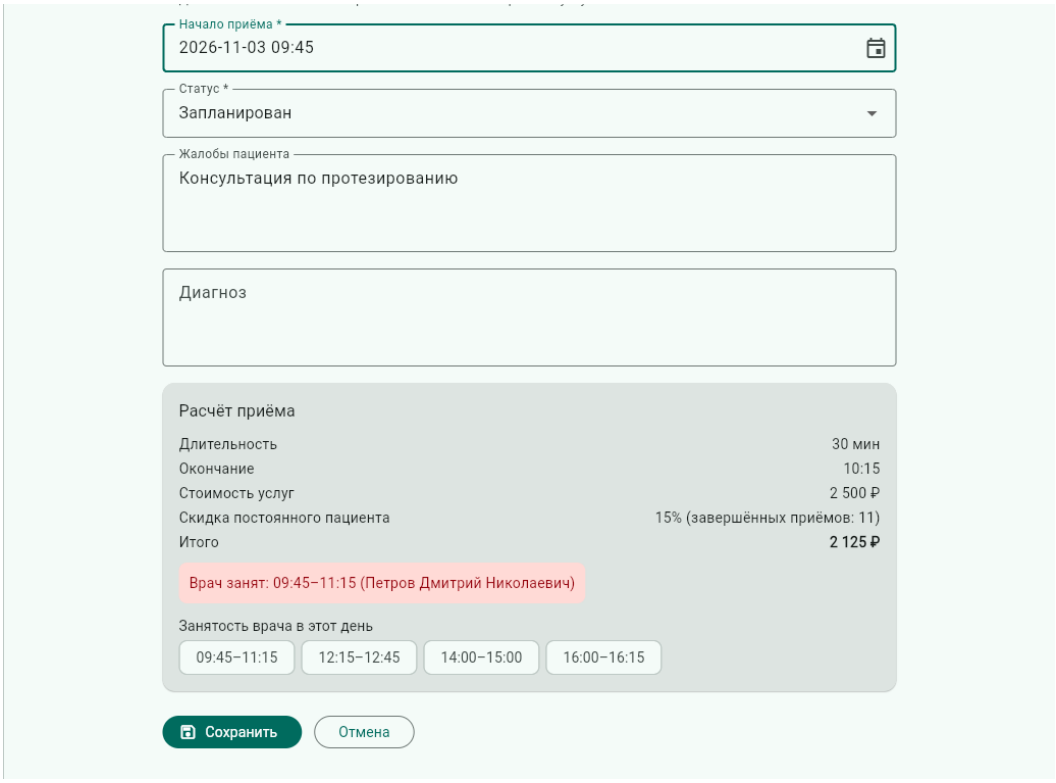


Рисунок 31 – Предупреждение о занятости врача

Всё равно сохраняем форму. Сервер проверяет пересечение сам и отвечает 409, сообщение выводится в верхней части формы.

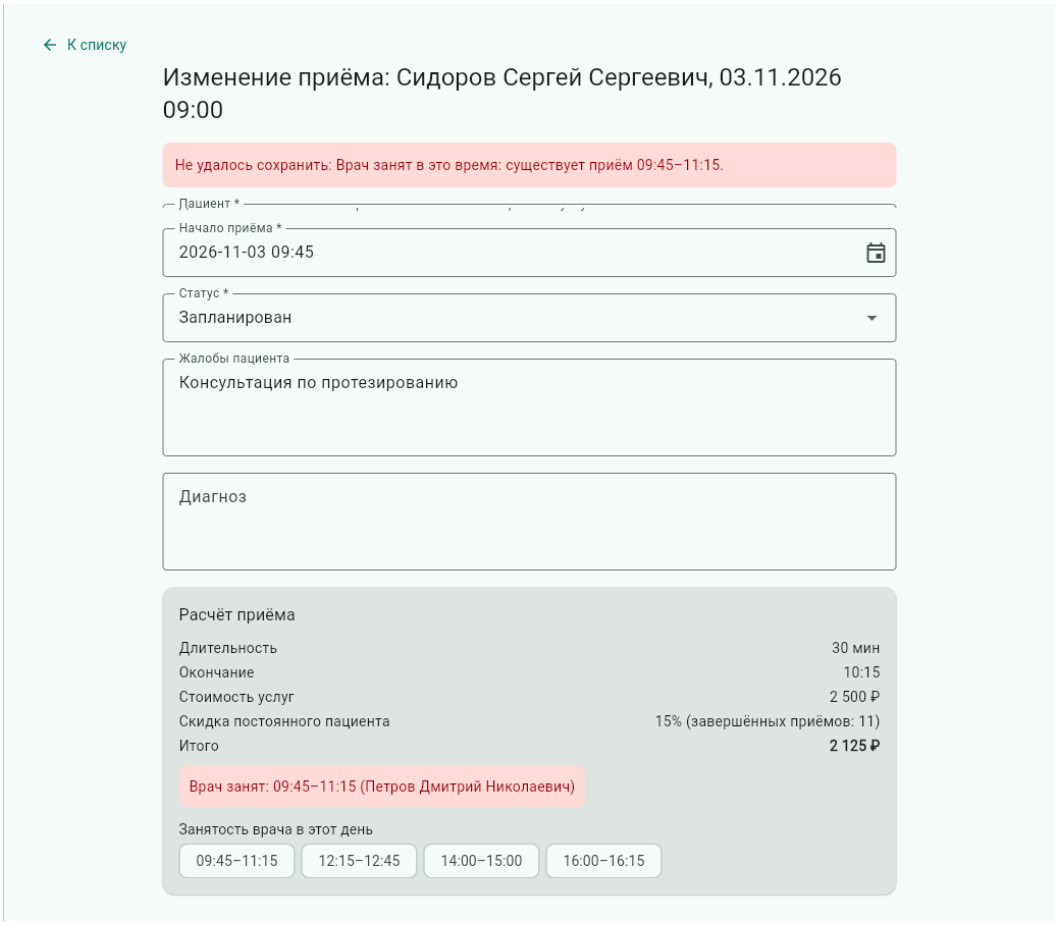


Рисунок 32 – Ошибка 409 при сохранении приёма

Открываем кассу для приёма. Сумма платежа предлагается по остатку.

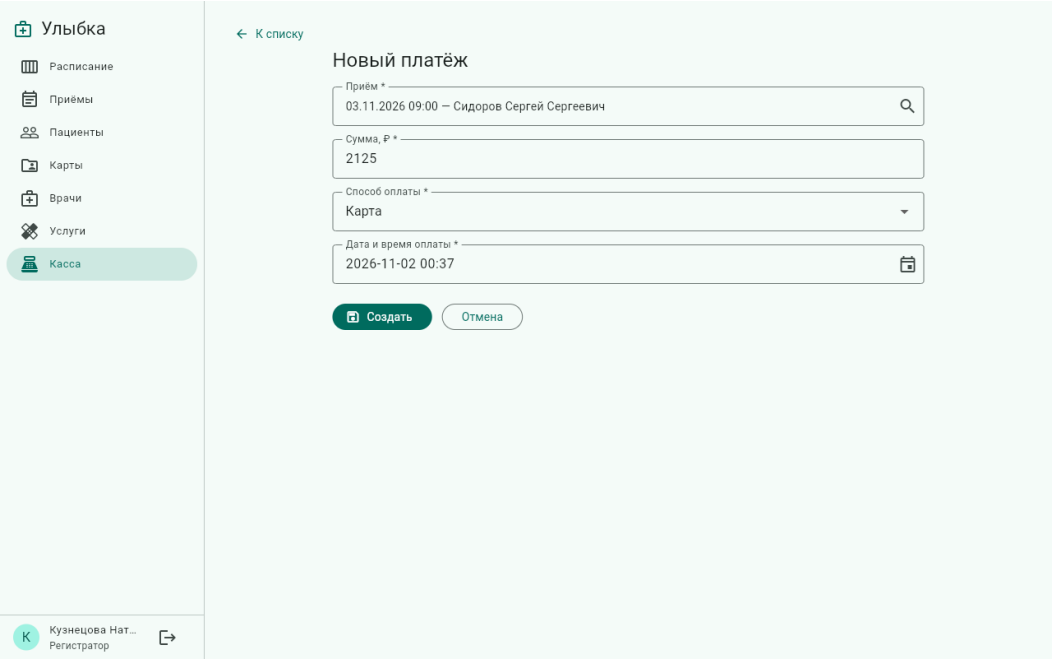


Рисунок 33 – Касса: платёж по приёму

Увеличиваем сумму больше остатка. Сервер отвечает 409 и сообщает, сколько осталось оплатить.

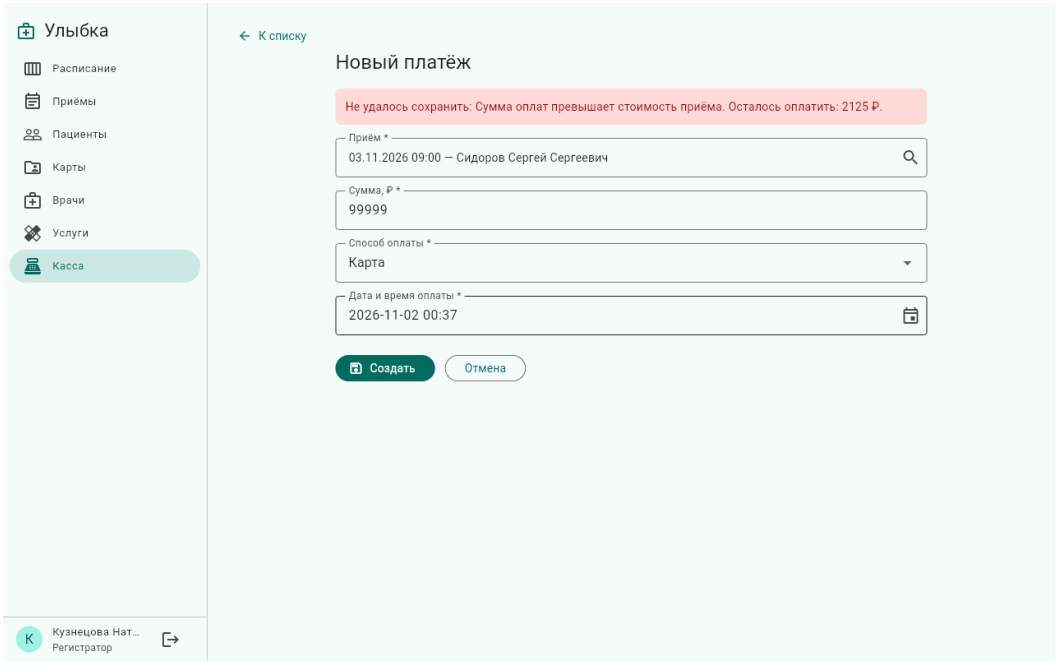


Рисунок 34 – Сумма больше остатка

Открываем по прямой ссылке раздел отчёта, недоступный регистратору. Маршрутизатор перехватывает переход и показывает страницу 403.

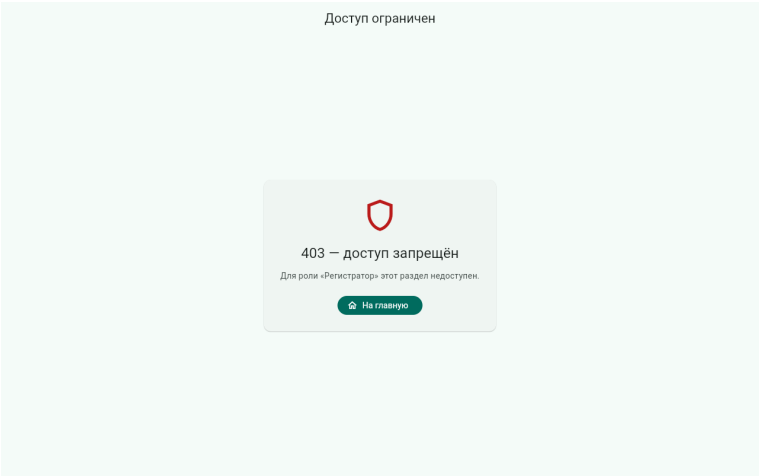


Рисунок 35 – Страница 403

Входим врачом. «Мой приём» показывает приёмы дня; кнопка «Провести приём» открывает окно со статусом и диагнозом.

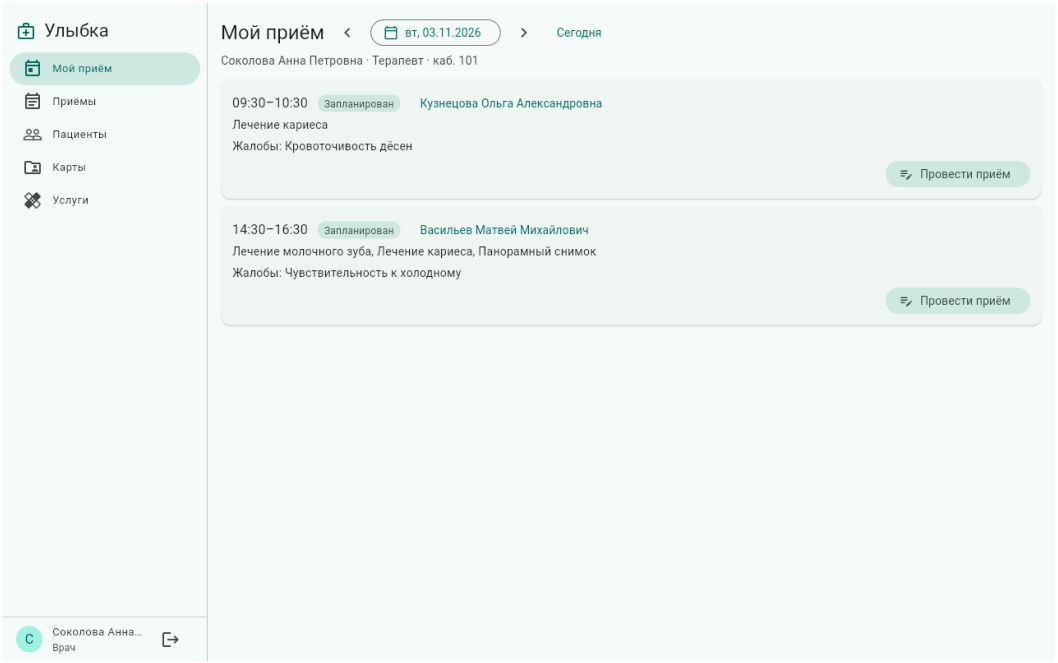


Рисунок 36 – Мой приём

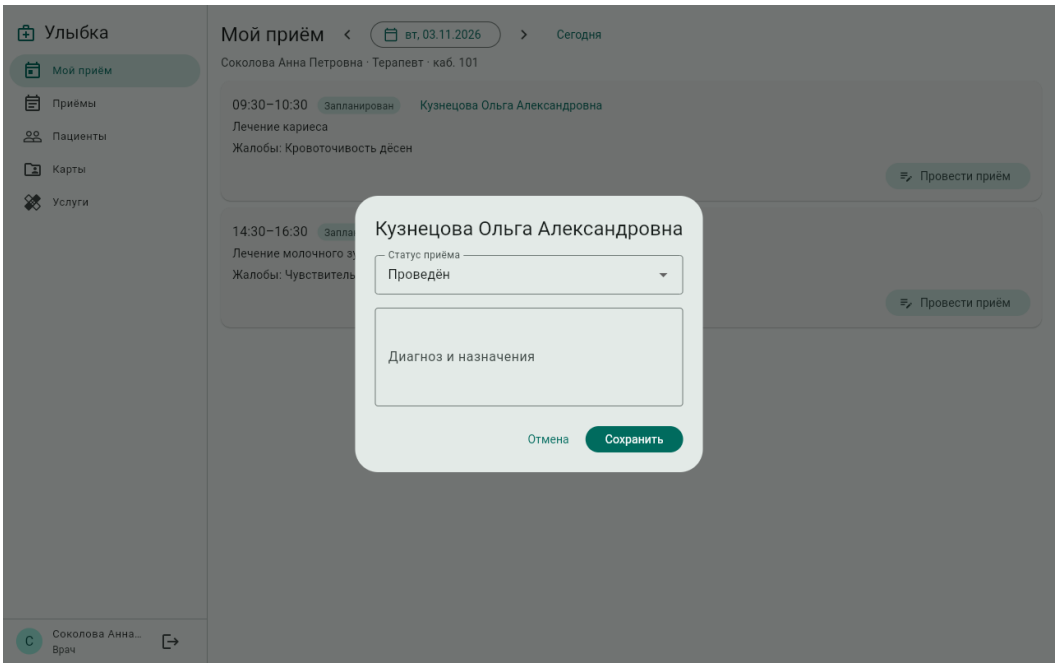


Рисунок 37 – Проведение приёма

Открываем список пациентов врача. Он доступен только для чтения: кнопок добавления и удаления нет, а по прямой ссылке на раздел администратора врач получает страницу 403.

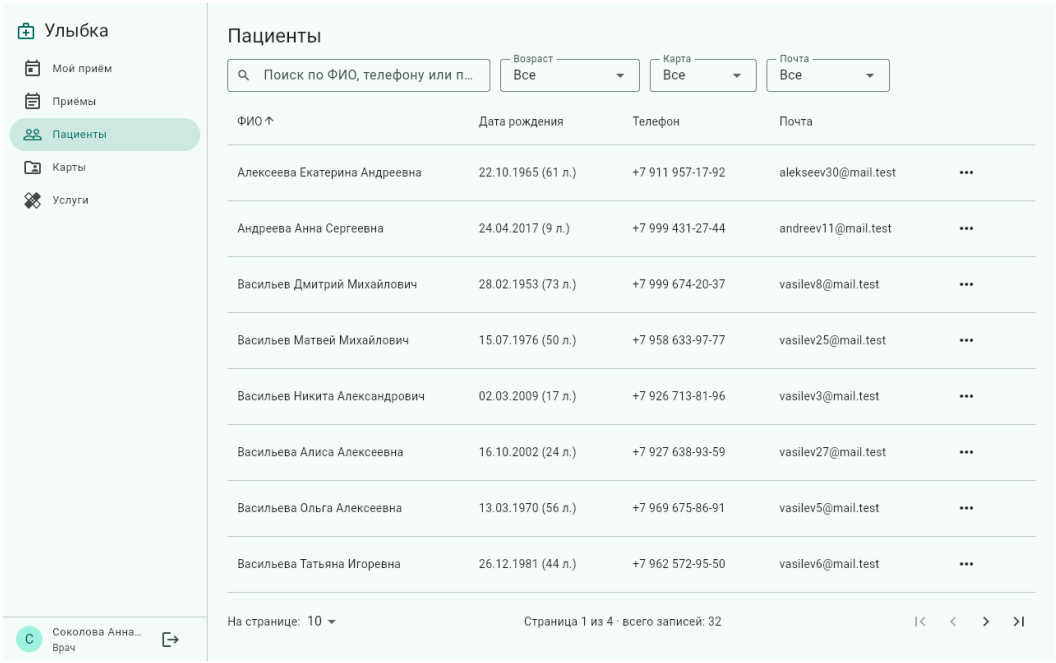


Рисунок 38 – Пациенты врача

Проверяем адаптивную вёрстку. При ширине 360 пикселей список превращается в карточки, навигация вынесена в выдвижное меню, форма занимает всю ширину.

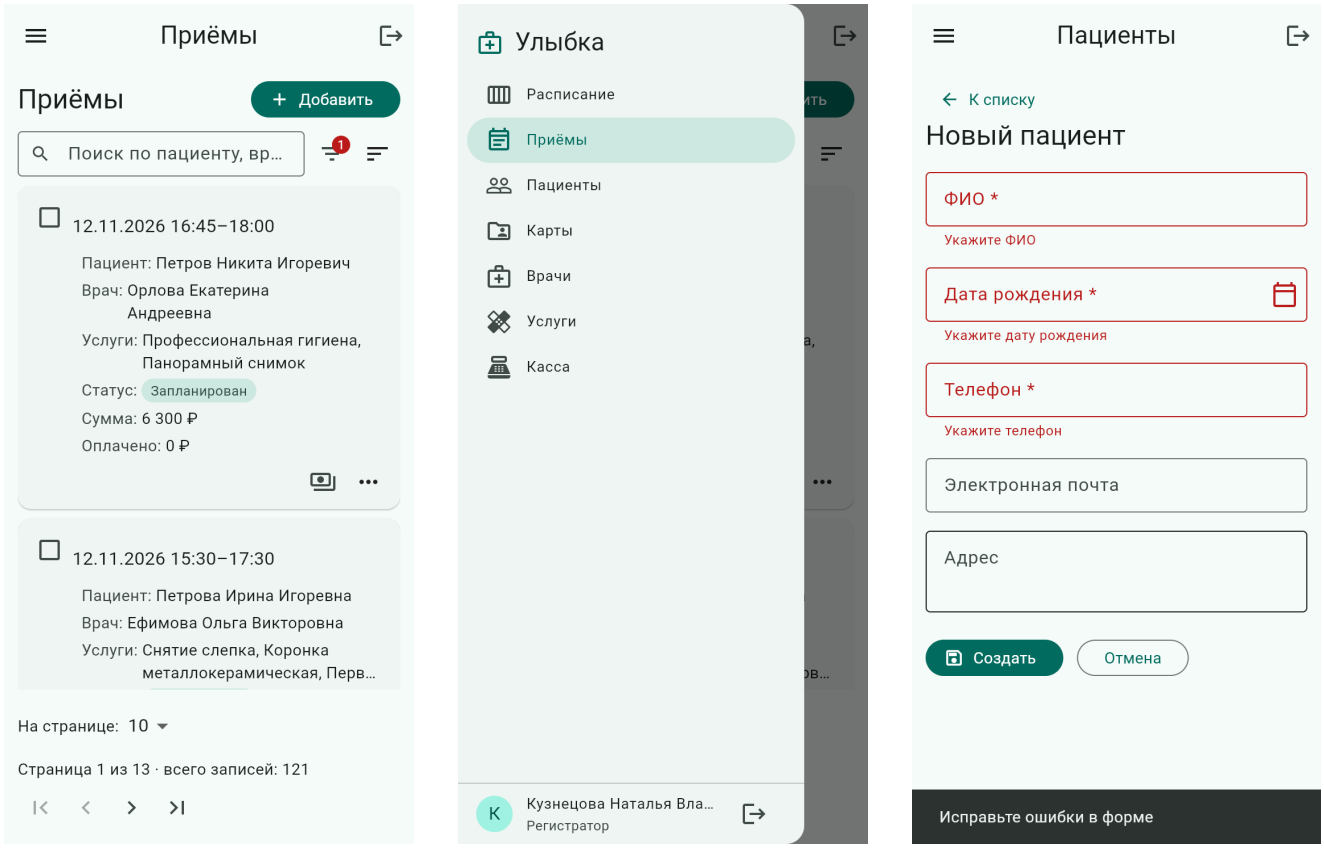


Рисунок 39 – Ширина 360 пикселей: список, меню, форма с ошибками

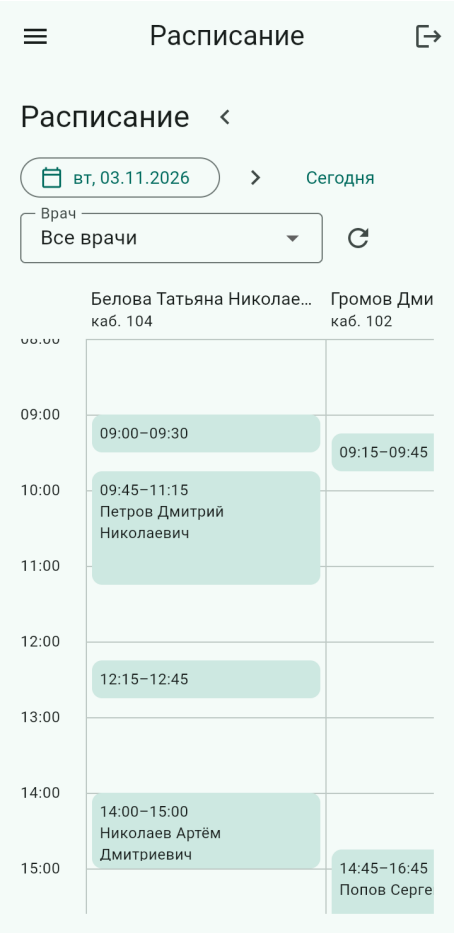


Рисунок 40 – Расписание при ширине 360 пикселей

При ширине 768 и 1920 пикселей отображается боковая панель и таблица.

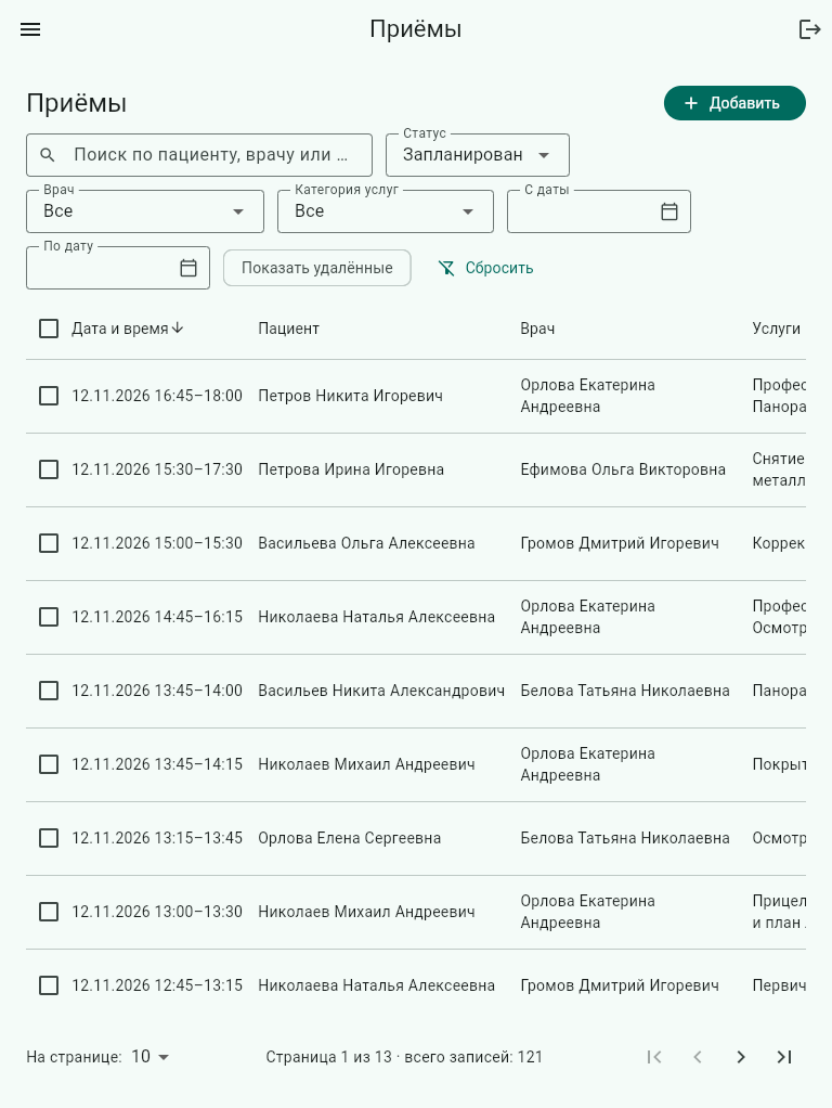


Рисунок 41 – Ширина 768 пикселей

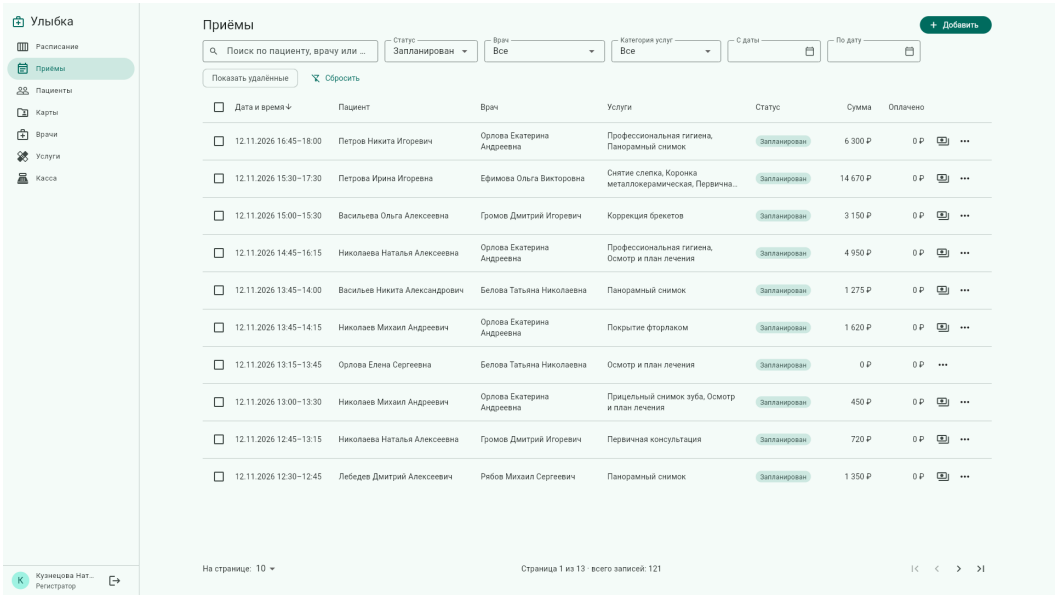


Рисунок 42 – Ширина 1920 пикселей

Проверяем обработку сетевых ошибок на сборке с сервером PocketBase. Сервер выключен, пробуем войти: появляется понятное сообщение.

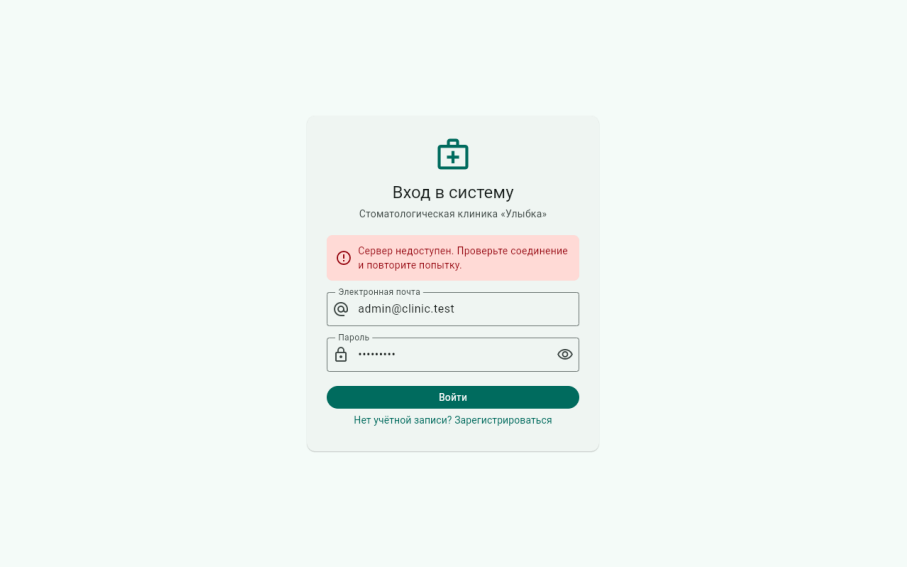


Рисунок 43 – Сервер выключен: вход

Открываем список при остановленном сервере. Запрос повторяется автоматически, затем выводятся сообщение и кнопка «Повторить». После запуска сервера нажимаем её, и данные загружаются.

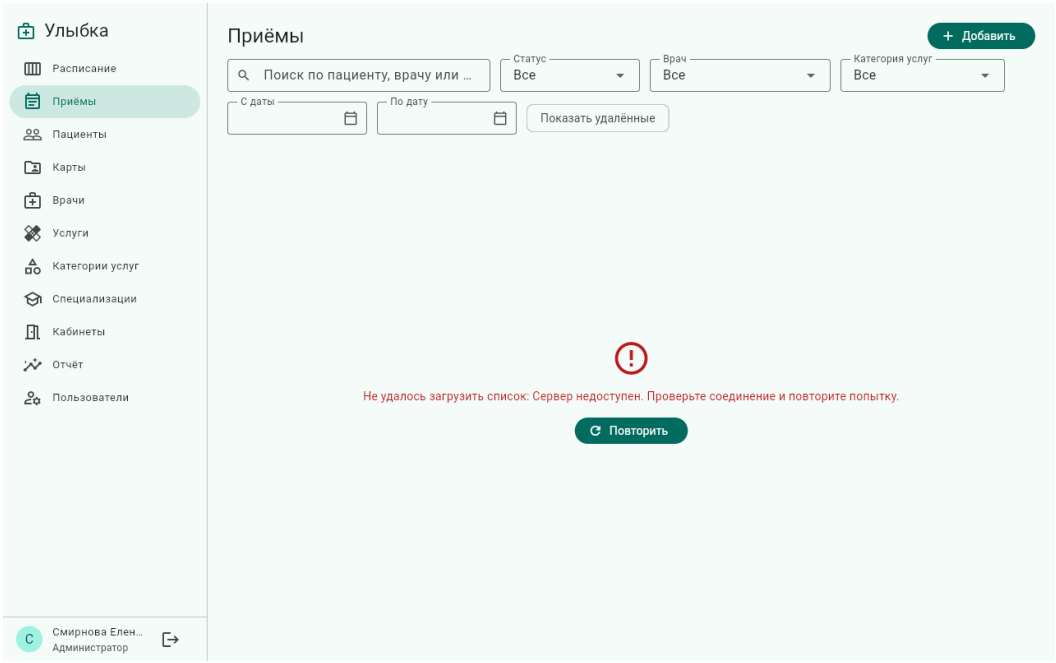


Рисунок 44 – Сервер недоступен при загрузке списка

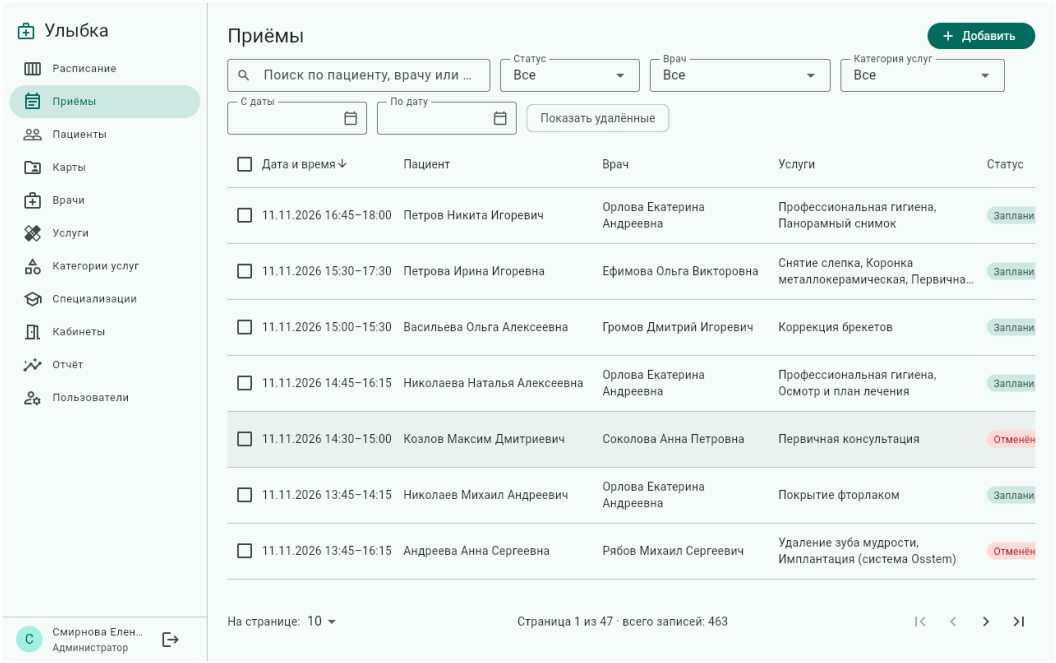


Рисунок 45 – После запуска сервера и нажатия «Повторить»

10. Покрываем проект тестами. Модульные и виджетные тесты запускаются командой flutter test, все 57 тестов проходят, flutter analyze замечаний не выдаёт. Серверная часть в тестах заменена эмулятором, который реализует правила доступа, проверки и хуки PocketBase на тех же данных; сами правила PocketBase проверяет скрипт verify.mjs (21 проверка).

Рисунок 46 – Результат выполнения flutter analyze и flutter test

Таблица 25 – Модульные тесты (unit_test.dart, demo_backend_test.dart)

Группа	Что проверяется
FilterQuery	разбор параметров адреса и обратная сборка без потерь; замена неизвестного поля сортировки и неверных страницы и размера; сброс страницы при смене фильтра
PbRepository	построение filter с экранированием кавычек и условием deletedAt; показ удалённых; префикс сортировки по убыванию и запасной ключ; игнорирование неверной даты фильтра
Ошибки API	ответ 400 с полями превращается в ValidationException; 409 сохраняет вид конфликта; 401, 403, 404 и 5xx имеют собственные типы; таймаут и обрыв соединения дают NetworkException
Права доступа	у каждой роли есть собственный раздел; права не наследуются; главная страница зависит от роли
Расчёт приёма	скидка по числу проведённых приёмов; стоимость и длительность по услугам; касание границ интервалов не считается пересечением

Группа	Что проверяется
Отчёт	выручка по врачам, категориям, способам оплаты и дням, средний чек, доля отмен, задолженность
Валидаторы и формат	телефон, почта, номер карты, дата и время, сумма; форматирование денег, минут и дат
Фильтры сервера	сравнения, регистронезависимый поиск, скобки и логические операторы; некорректный фильтр вызывает исключение
Разбор приёма	оплаты и вложенные записи берутся из expand, считаются долг и длительность
Сервер (эмулятор)	врач видит только свои приёмы и не видит оплат; поиск, фильтры и сортировка на сервере; 409 при пересечении; сумма и скидка считаются на сервере; удаление записи со связями даёт 409; ошибка поля номера карты приходит как 400; платёж не больше остатка; обновление токена; неверный пароль

Таблица 26 – Тесты виджетов (widget_test.dart)

№	Тест
1	Адаптивность: для каждой роли (администратор, регистратор, врач) и ширин 360, 768 и 1920 px проходятся все доступные экраны, ни на одном нет переполнения (9 тестов)

Вывод: разработана информационная система стоматологической клиники. Схема данных из десяти связанных сущностей с тремя типами связей спроектирована и развёрнута на платформе PocketBase, для неё описаны правила доступа, индексы и серверные хуки, подготовлены скрипты создания, наполнения и проверки, экспорт схемы в JSON и SQL. Клиент на Flutter Web разделён на слои моделей, репозиториев, состояния, экранов и виджетов, десять сущностей обслуживаются универсальными виджетами. Реализованы регистрация и вход, три роли с собственным разделом у каждой, защита маршрутов с отдельной страницей отказа, скрытие недоступных элементов, серверные поиск, фильтры по трём и более критериям, сортировка и пагинация с состоянием в адресе, сессия, переживающая перезагрузку, логическое и физическое удаление, восстановление и множественное удаление. Сетевой слой обрабатывает недоступность сервера, таймаут, ошибки полей (400 и 422), конфликты 409 и истечение токена с автоматическим обновлением. Содержательная особенность предметной области — запись на приём с расчётом длительности и стоимости, скидкой постоянного пациента и проверкой пересечения времени врача, кабинета и пациента на сервере и в форме, а также расписание и отчёт по выручке. Серверная

валидация обязательна для всех операций и показана на снимках. Адрес сервера задаётся параметром сборки, приложение опубликовано на GitHub Pages, прямые ссылки на внутренние экраны открываются корректно. Проект проверен статическим анализатором без замечаний и покрыт 57 тестами: 35 модульными и 22 виджетными.

Контрольные адреса проекта:

Исходный код репозитория: https://github.com/Fiven4/dental_clinic

Опубликованное веб-приложение: https://fiven4.github.io/dental_clinic/